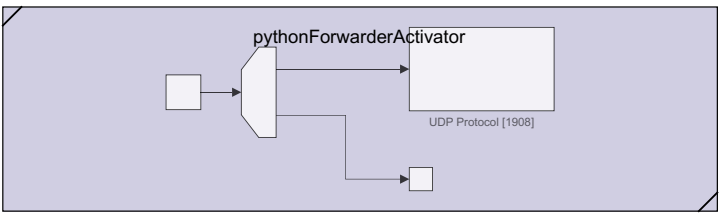
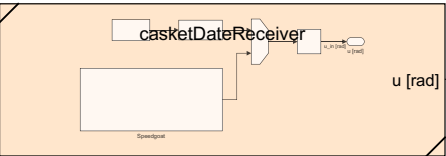


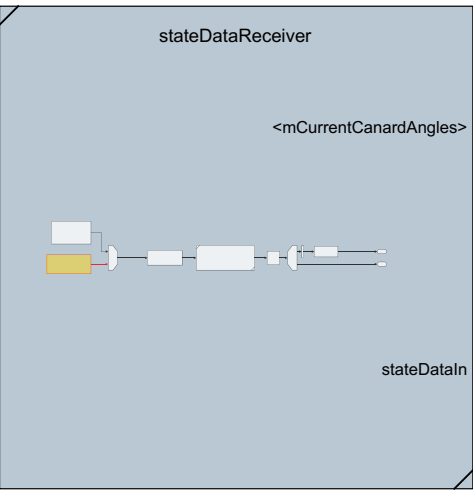
Speedgoat Configuration



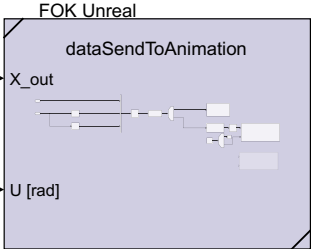
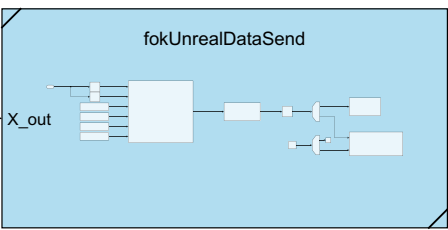
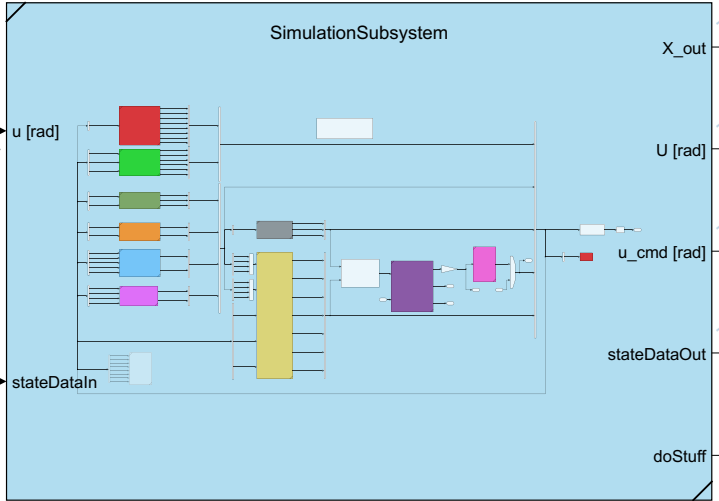
Python Forwarder Activator



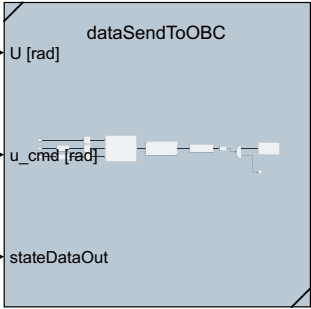
Casket



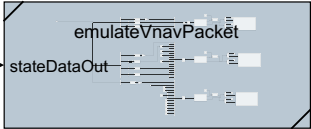
stateData receiver



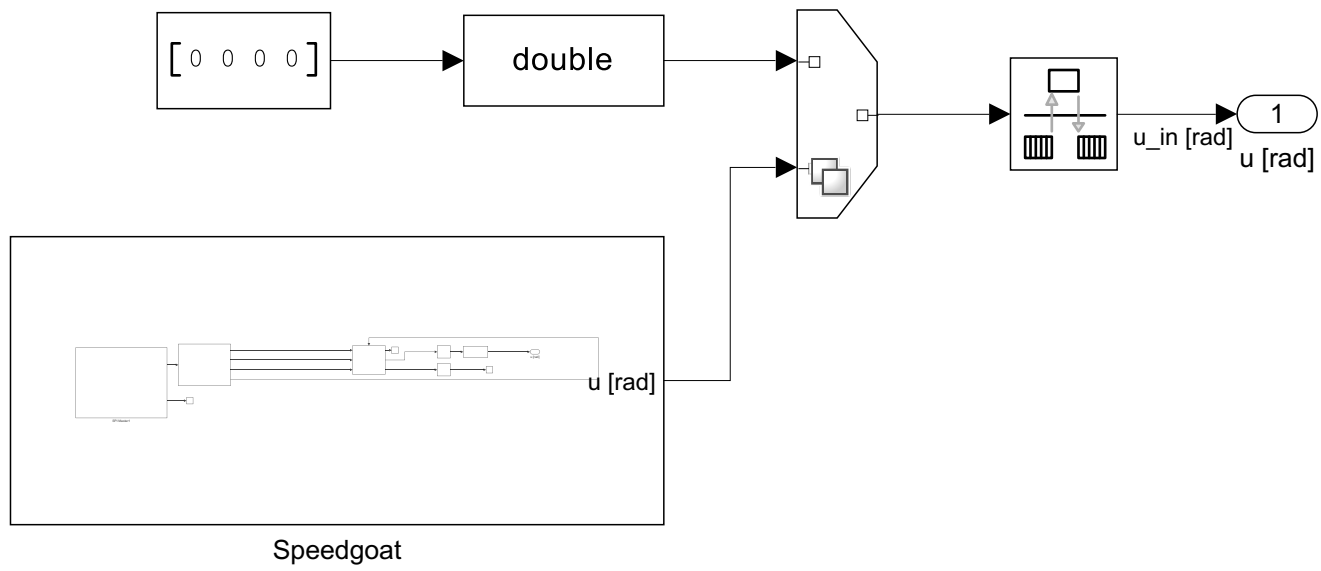
Data send to animation

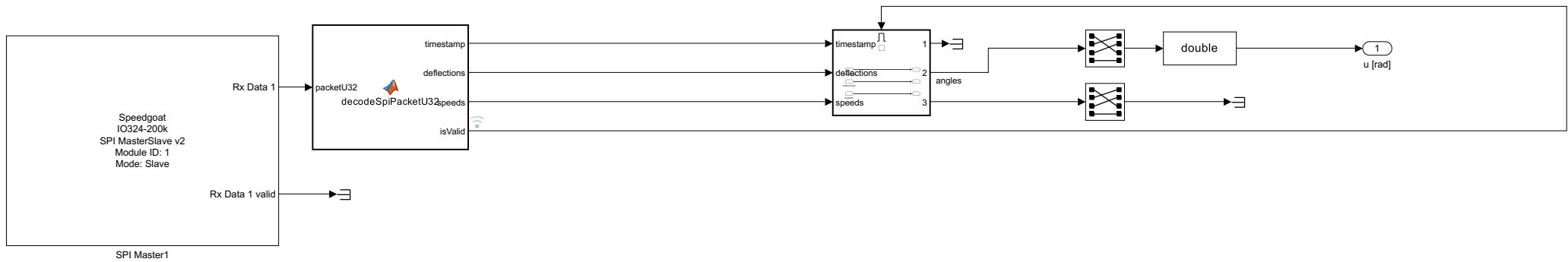


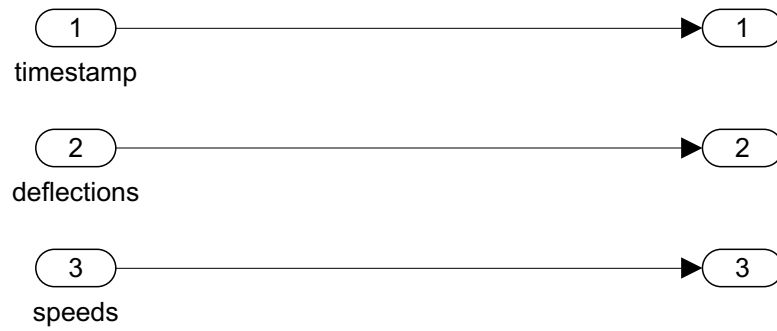
Data send



Emulate VNAV Packet







```

function [timestamp, deflections, speeds, isValid] = decodeSpiPacketU32(packetU32)
%#codegen
% Decode one 42-byte packet carried as 21 uint32 words (lower 16 bits used).
% Packet layout (big-endian):
%   bytes 1..8 : uint64 timestamp
%   bytes 9..24 : 4x float32 deflections
%   bytes 25..40 : 4x float32 speeds
%   byte 41 : marker 0xFA
%   byte 42 : XOR checksum over bytes 1..41

% Fixed-size typed outputs for codegen
timestamp = uint64(0);
deflections = zeros(4,1,'double');
speeds = zeros(4,1,'double');
isValid = false;

% % Input contract: uint32 vector, 21 elements, only low 16 bits meaningful
% assert(isa(packetU32,'uint32'));
% assert(numel(packetU32) == 21);

% Rebuild raw packet bytes from 16-bit words (MSB first in each word)
bytes = zeros(42,1,'uint8');
for k = 1:21
    w = uint16(bitand(packetU32(k), uint32(65535)));
    bytes(2*k-1) = uint8(bitshift(w, -8)); % high byte
    bytes(2*k) = uint8(bitand(w, uint16(255))); % low byte
end

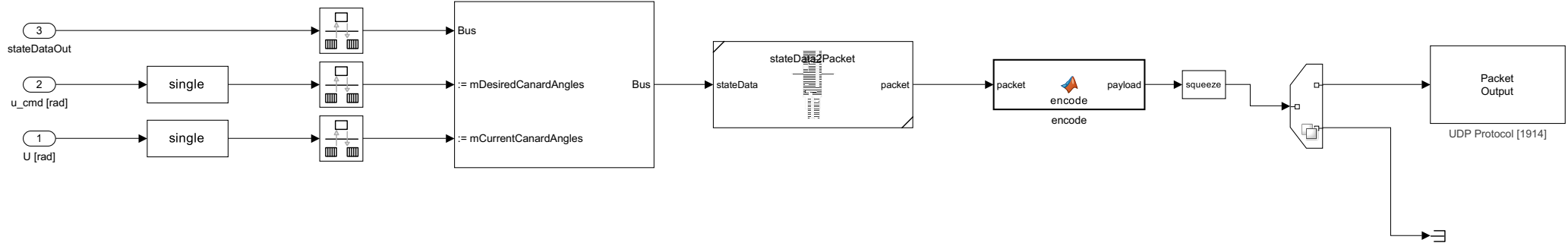
% Marker + checksum check
rxChecksum = bytes(42);
calc = uint8(0);
for i = 1:41
    calc = bitxor(calc, bytes(i));
end
isValid = (bytes(41) == uint8(250)) && (calc == rxChecksum);

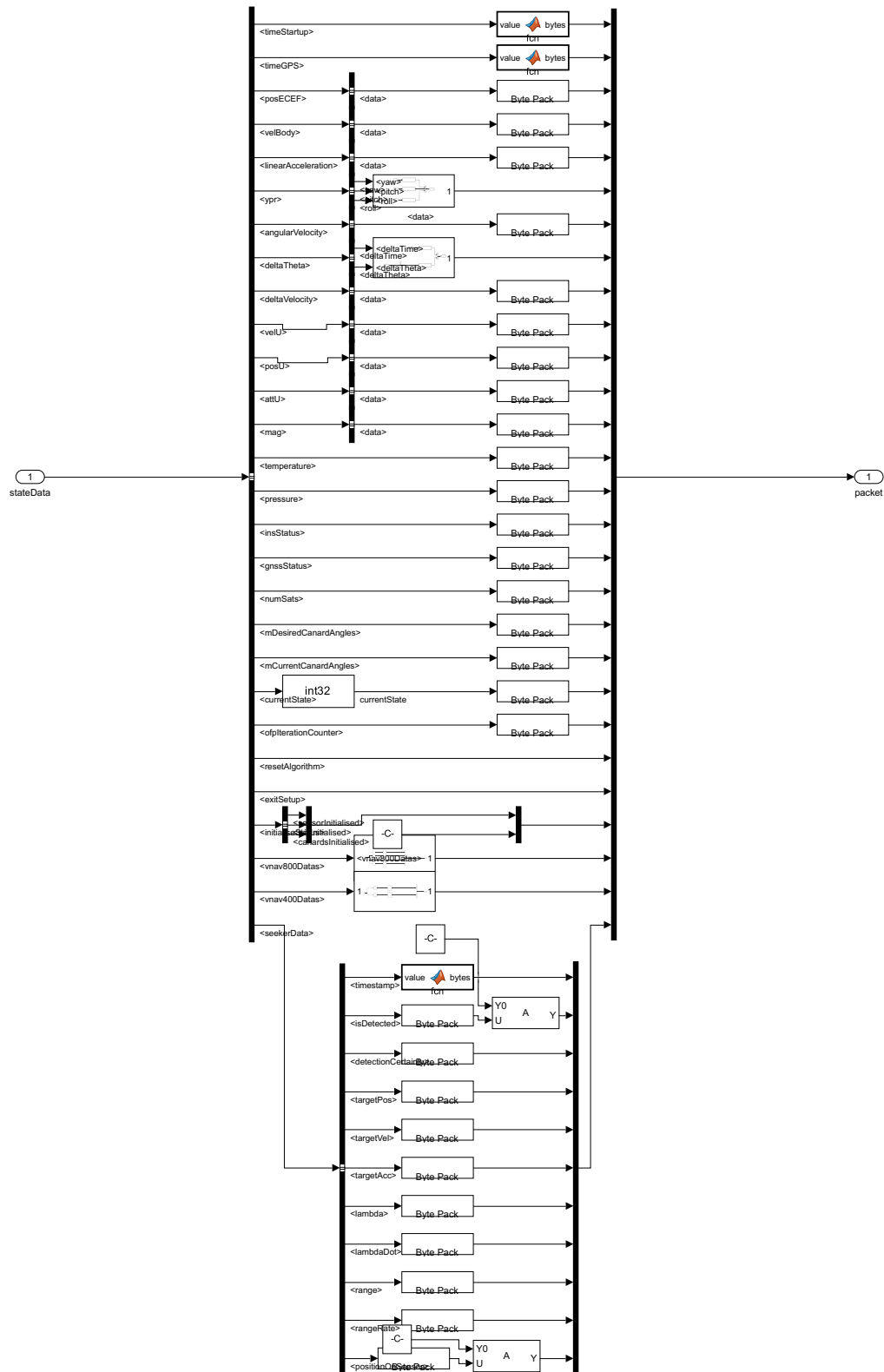
% Timestamp (uint64, big-endian)
t = uint64(0);
for i = 1:8
    t = bitor(bitshift(t, 8), uint64(bytes(i)));
end
timestamp = t;

% 8x float32 values (big-endian), convert to double
vals = zeros(1,8,'double');
for i = 1:8
    idx = 9 + (i-1)*4;
    u = bitor( ...
        bitor(bitshift(uint32(bytes(idx)), 24), bitshift(uint32(bytes(idx+1)), 16)), ...
        bitor(bitshift(uint32(bytes(idx+2)), 8), uint32(bytes(idx+3))) ...
    );
    s = typecast(u, 'single');
    vals(i) = double(s);
end

deflections(:,1) = vals(1:4).'; % 4x1
speeds(:,1) = vals(5:8); % 1x4
end

```

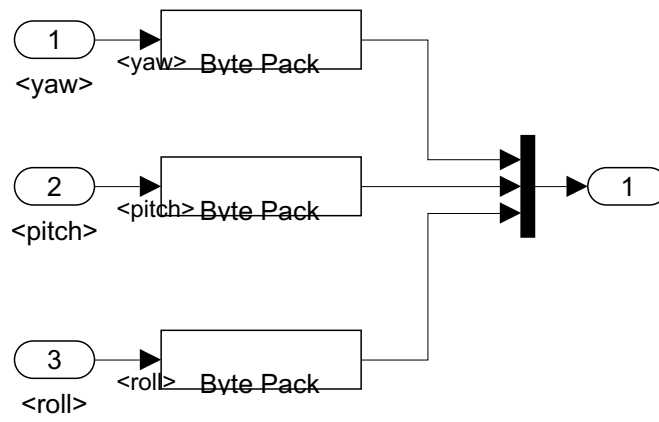


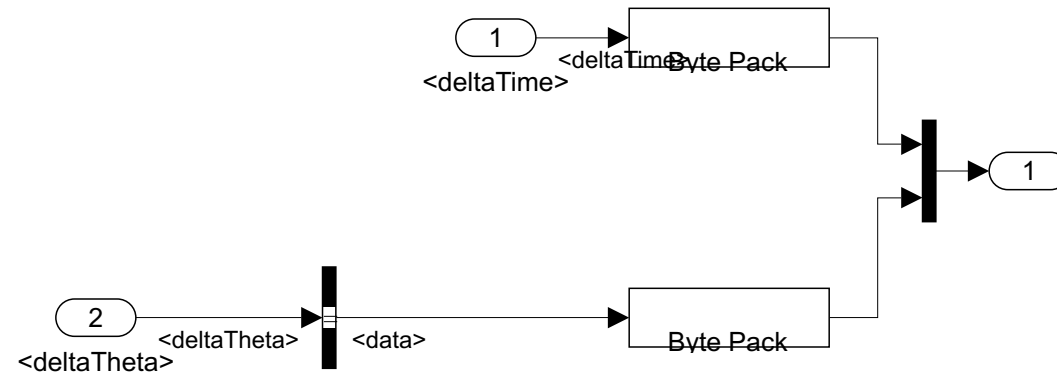


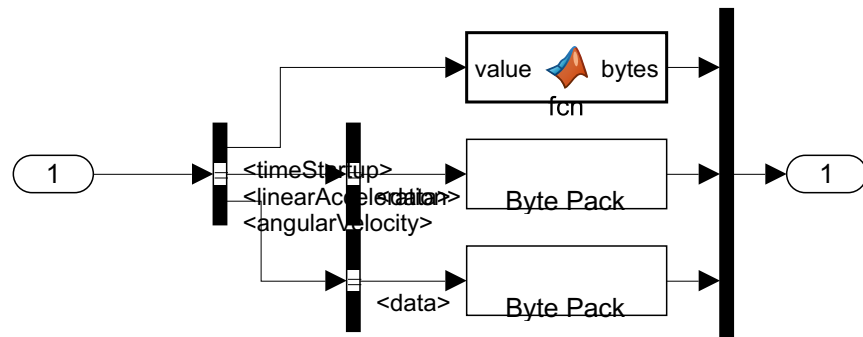
```
function bytes = fcn(value)
bytes = packUint64(value);
```

```
function bytes = fcn(value)
bytes = packUint64(value);
```

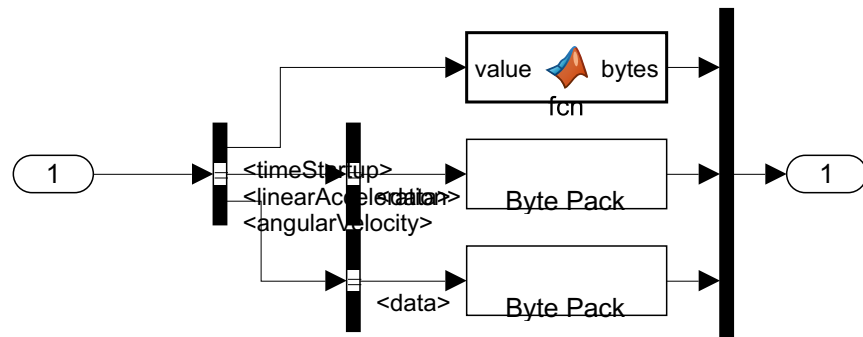
```
function bytes = fcn(value)
bytes = packUint64(value);
```



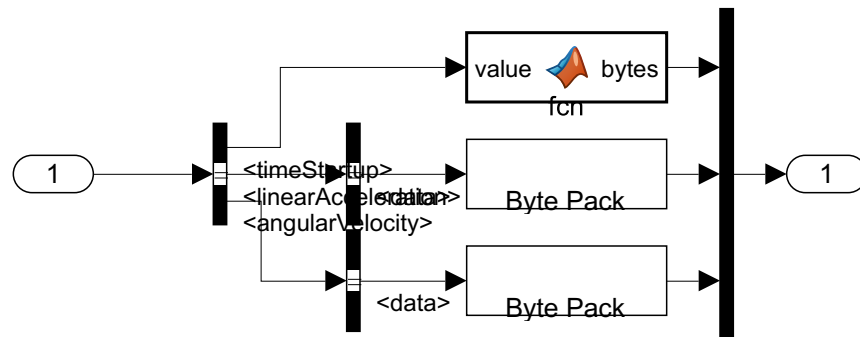




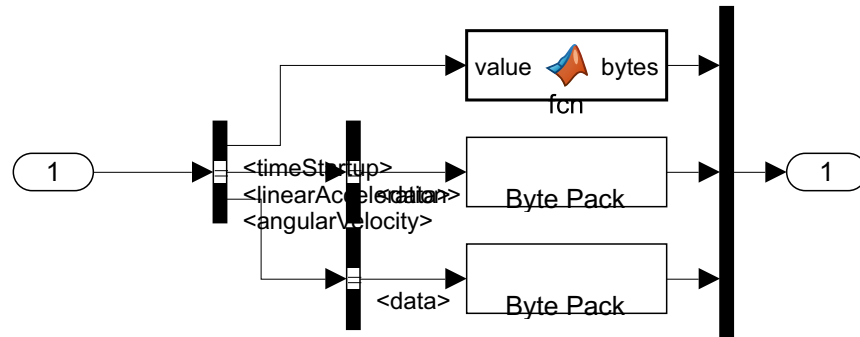
```
function bytes = fcn(value)
bytes = packUint64(value);
```



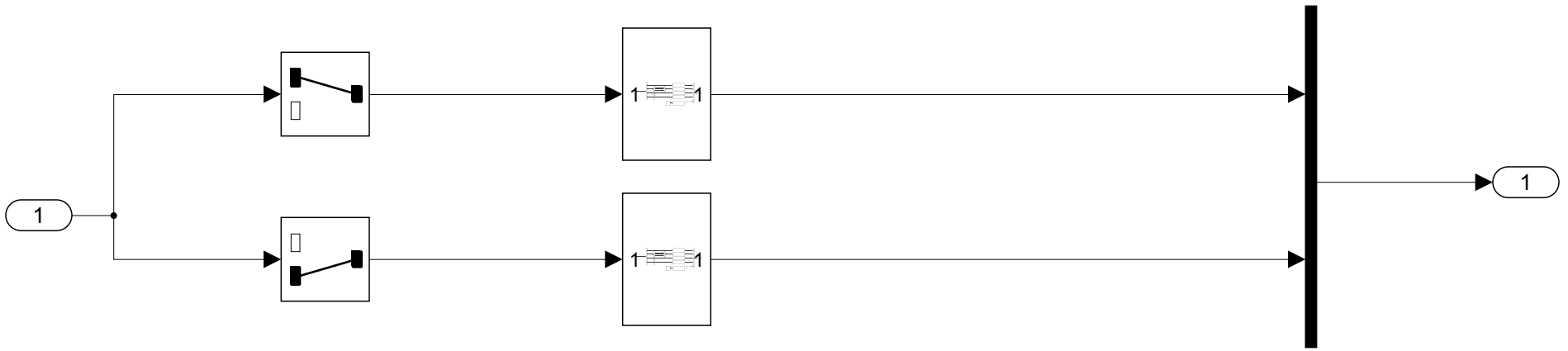
```
function bytes = fcn(value)
bytes = packUint64(value);
```



```
function bytes = fcn(value)
bytes = packUint64(value);
```



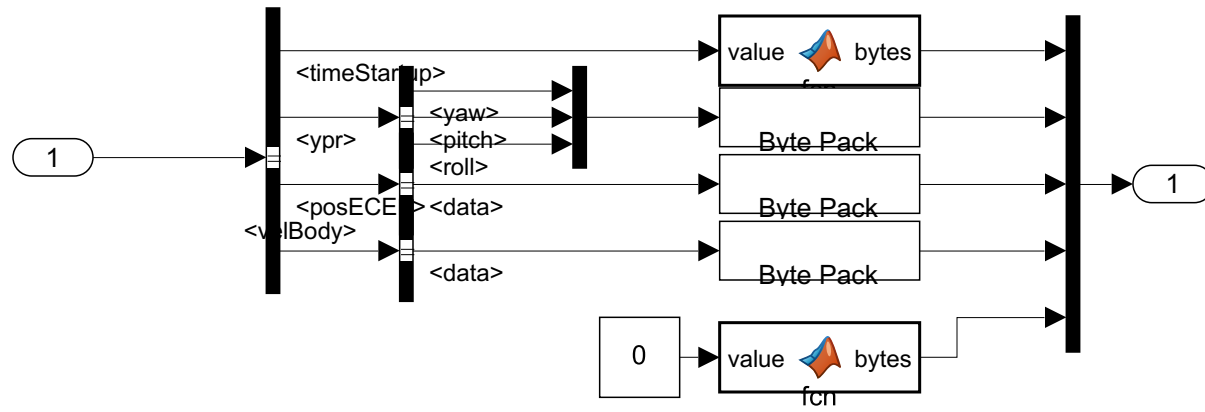
```
function bytes = fcn(value)
bytes = packUint64(value);
```





```
function bytes = fcn(value)
bytes = packUint64(value);
```

```
function bytes = fcn(value)
bytes = packUint64(value);
```



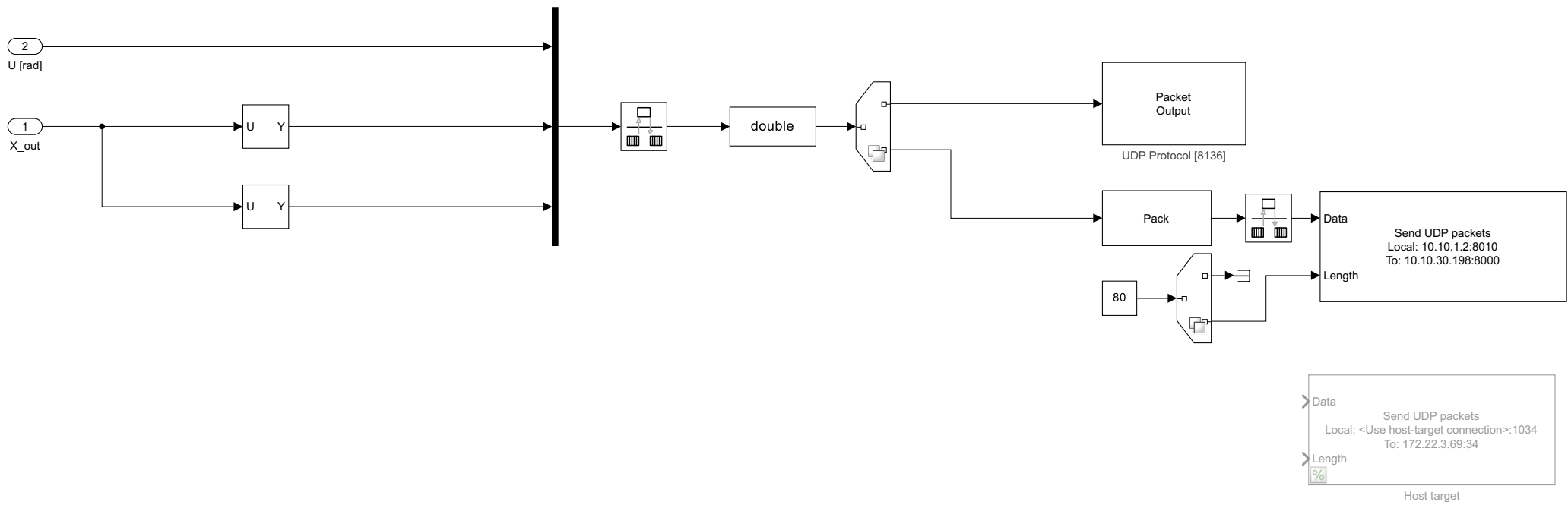
```
function bytes = fcn(value)
bytes = packUint64(value);
```

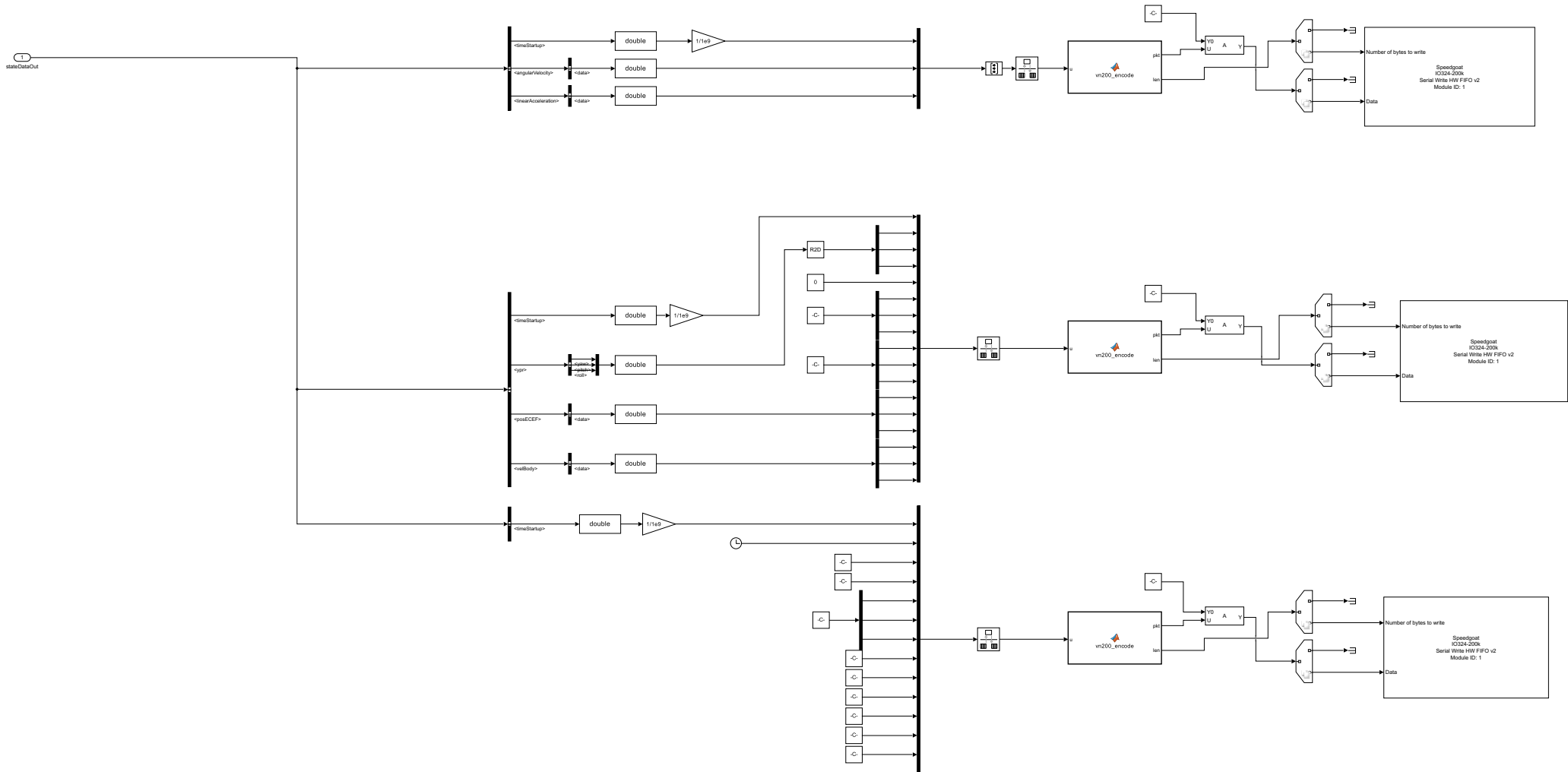
```
function bytes = fcn(value)
bytes = packUint64(value);
```

```
function payload = encode(packet)
%#codegen
PAYLOAD_SIZE = coder.const(1024);
payload = zeros(1, PAYLOAD_SIZE, 'uint8');

payload(1:PAYLOAD_SIZE) = encodePacket(packet);

end
```





```

function [pkt, len] = vn200_encode(u, packetType) %#codegen
%VN200_ENCODE   Symulator ramek binarnych VectorNav VN-200 (HIL / Speedgoat).
%
%   [pkt, len] = vn200_encode(u, packetType)
%
% Jeden blok MATLAB Function o konfigurowalnym parametrze packetType.
% W bloku MATLAB Function ustaw 'packetType' jako PARAMETER (Ports and
% Data Manager -> Scope: Parameter, Tunable: OFF). Dzieki temu rozmiary
% wejscia/wyjscia sa stale w czasie kompilacji (wymog Simulink Coder /
% Speedgoat).
%
% packetType = 1  -> BinaryOutput1, 800 Hz, ramka 38 B
%   u (7x1)  = [timeStartup_s;           % czas od startu [s]
%               angularRate_radps(3);    % rad/s, body XYZ
%               accel_mps2(3)]           % m/s^2, body XYZ
%
% packetType = 2  -> BinaryOutput2, 400 Hz, ramka 94 B
%   u (17x1) = [timeStartup_s;
%               ypr_deg(3);              % Yaw,Pitch,Roll [DEG]!
%               deltaTime_s;             % okres calkowania (0.0025)
%               deltaTheta_deg(3);       % [deg]
%               deltaVel_mps(3);         % [m/s]
%               posEcef_m(3);            % [m] (kodowane jako double)
%               velBody_mps(3)]          % [m/s]
%
% packetType = 3  -> BinaryOutput3, 200 Hz, ramka 69 B
%   u (13x1) = [timeStartup_s;
%               timeGps_s;               % sekundy od epoki GPS 1980
%               temperature_C;
%               pressure_kPa;
%               mag_G(3);                % [Gauss], body XYZ
%               numSats;                 % uint8
%               gnssStatus;              % bitfield uint16
%               attU_deg;                % [deg]
%               insStatus;               % bitfield uint16
%               posU_m;                 % [m]
%               velU_mps]               % [m/s]
%
% pkt - uint8, kolumna o stalej dlugosci (38/94/69 zaleznie od packetType)
% len - uint16, dlugosc ramki (rowna numel(pkt); wygodne dla Serial Send)
%
% Format wg VN-200 ICD v2.1.0.0 sek. 2.1.3:
% [0xFA][group byte][type words LE][payload LE][CRC16-CCITT BE]
% CRC po headerze (bez 0xFA) + payloadzie; poly 0x1021, init 0.
% UWAGA: serializacja float/double przez typecast = kolejnosc bajtow hosta.
% Speedgoat (x86) i Windows/Linux sa little-endian, czyli zgodnie z ICD.

```

```

switch packetType
    case 1
        assert(numel(u) == 7);
        pkt = encodeBO1(u(:));
    case 2
        assert(numel(u) == 17);
        pkt = encodeBO2(u(:));
    otherwise
        assert(numel(u) == 13);
        pkt = encodeBO3(u(:));
end
len = uint16(numel(pkt));

```

end

```
%=====
% BinaryOutput1 (800 Hz): Common{TimeStartup, AngularRate, Accel}
% group 0x01, Common word 0x0121, payload 32 B, total 38 B
%=====
```

```
function pkt = encodeBO1(u)
pkt = zeros(38, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA sync
pkt(2) = uint8(1);             % group: Common
pkt(3:4) = u16le(uint16(289)); % 0x0121
pkt(5:12) = u64le_ns(u(1));    % TimeStartup [ns]
pkt(13:24) = f32le3(u(2:4));   % AngularRate [rad/s]
pkt(25:36) = f32le3(u(5:7));   % Accel [m/s^2]
pkt(37:38) = crcBE(pkt(2:36));
end
```

```
%=====
% BinaryOutput2 (400 Hz):
% Common{TimeStartup,Ypr} + IMU{DeltaTheta,DeltaVel} + INS{PosEcef,VelBody}
% group 0x25, words: 0x0009, 0x00C0, 0x000C, payload 84 B, total 94 B
%=====
```

```
function pkt = encodeBO2(u)
pkt = zeros(94, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(37);            % 0x25: Common|IMU|INS
pkt(3:4) = u16le(uint16(9));    % Common: 0x0009
pkt(5:6) = u16le(uint16(192));  % IMU: 0x00C0
pkt(7:8) = u16le(uint16(12));   % INS: 0x000C
pkt(9:16) = u64le_ns(u(1));    % TimeStartup
pkt(17:28) = f32le3(u(2:4));    % Ypr [deg]
pkt(29:32) = f32le(u(5));       % DeltaTheta.dtime [s]
pkt(33:44) = f32le3(u(6:8));    % DeltaTheta [deg]
pkt(45:56) = f32le3(u(9:11));   % DeltaVel [m/s]
pkt(57:80) = f64le3(u(12:14));  % PosEcef [m] (double)
pkt(81:92) = f32le3(u(15:17));  % VelBody [m/s]
pkt(93:94) = crcBE(pkt(2:92));
end
```

```
%=====
% BinaryOutput3 (200 Hz):
% Common{TimeStartup,TimeGps} + IMU{Temp,Pres,Mag} + GNSS{NumSats,GnssStatus}
% + Attitude{AttU} + INS{InsStatus,PosU,VelU}
% group 0x3D, words: 0x0003, 0x0130, 0x8008+0x0004 (ext), 0x2000, 0x0601
% payload 53 B, total 69 B
%=====
```

```
function pkt = encodeBO3(u)
pkt = zeros(69, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(61);            % 0x3D: Common|IMU|GNSS|Att|INS
pkt(3:4) = u16le(uint16(3));    % Common: 0x0003
pkt(5:6) = u16le(uint16(304));  % IMU: 0x0130
pkt(7:8) = u16le(uint16(32776)); % GNSS 1: 0x8008 (NumSats + ext bit)
pkt(9:10) = u16le(uint16(4));   % GNSS 2: 0x0004 (GnssStatus, offset 17)
pkt(11:12) = u16le(uint16(8192)); % Att: 0x2000 (AttU)
pkt(13:14) = u16le(uint16(1537)); % INS: 0x0601 (InsStatus|PosU|VelU)
pkt(15:22) = u64le_ns(u(1));    % TimeStartup
pkt(23:30) = u64le_ns(u(2));    % TimeGps
pkt(31:34) = f32le(u(3));       % Temperature [degC]
```

```

pkt(35:38) = f32le(u(4));           % Pressure [kPa]
pkt(39:50) = f32le3(u(5:7));        % Mag [G]
pkt(51)     = uint8(u(8));           % NumSats
pkt(52:53)  = u16le(uint16(u(9)));  % GnssStatus
pkt(54:57)  = f32le(u(10));         % AttU [deg]
pkt(58:59)  = u16le(uint16(u(11))); % InsStatus
pkt(60:63)  = f32le(u(12));         % PosU [m]
pkt(64:67)  = f32le(u(13));         % VelU [m/s]
pkt(68:69)  = crcBE(pkt(2:67));
end

%=====
% Serializacja little-endian
%=====

function b = u16le(v)
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitand(v, uint16(255)));
b(2) = uint8(bitshift(v, -8));
end

function b = u64le_ns(t_s)
% sekundy (double) -> uint64 nanosekundy -> 8 bajtow LE
t = max(t_s, 0);
v = uint64(t * 1e9);           % uint64() zaokragla do najblizszej
b = zeros(8, 1, 'uint8');
for i = 1:8
    b(i) = uint8(bitand(v, uint64(255)));
    v = bitshift(v, -8);
end
end

function b = f32le(v)
b = reshape(typecast(single(v), 'uint8'), 4, 1); % host LE (x86)
end

function b = f32le3(v)
b = reshape(typecast(single(v(:)'), 'uint8'), 12, 1);
end

function b = f64le3(v)
b = reshape(typecast(double(v(:)'), 'uint8'), 24, 1);
end

%=====
% CRC16-CCITT (poly 0x1021, init 0) - ICD sek. 1.4.3; zwraca 2 bajty BE
%=====

function b = crcBE(data)
crc = uint16(0);
for i = 1:numel(data)
    crc = bitor(bitshift(crc, -8), bitshift(crc, 8));
    crc = bitxor(crc, uint16(data(i)));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), -4));
    crc = bitxor(crc, bitshift(crc, 12));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), 5));
end
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitshift(crc, -8)); % big-endian!
b(2) = uint8(bitand(crc, uint16(255)));
end

```



```

function [pkt, len] = vn200_encode(u, packetType) %#codegen
%VN200_ENCODE   Symulator ramek binarnych VectorNav VN-200 (HIL / Speedgoat).
%
%   [pkt, len] = vn200_encode(u, packetType)
%
%   Jeden blok MATLAB Function o konfigurowalnym parametrze packetType.
%   W bloku MATLAB Function ustaw 'packetType' jako PARAMETER (Ports and
%   Data Manager -> Scope: Parameter, Tunable: OFF). Dzieki temu rozmiary
%   wejscia/wyjscia sa stale w czasie kompilacji (wymog Simulink Coder /
%   Speedgoat).
%
%   packetType = 1  -> BinaryOutput1, 800 Hz, ramka 38 B
%       u (7x1) = [timeStartup_s;           % czas od startu [s]
%                 angularRate_radps(3);    % rad/s, body XYZ
%                 accel_mps2(3)]           % m/s^2, body XYZ
%
%   packetType = 2  -> BinaryOutput2, 400 Hz, ramka 94 B
%       u (17x1) = [timeStartup_s;
%                  ypr_deg(3);             % Yaw,Pitch,Roll [DEG]!
%                  deltaTime_s;           % okres calkowania (0.0025)
%                  deltaTheta_deg(3);     % [deg]
%                  deltaVel_mps(3);       % [m/s]
%                  posEcef_m(3);          % [m] (kodowane jako double)
%                  velBody_mps(3)]        % [m/s]
%
%   packetType = 3  -> BinaryOutput3, 200 Hz, ramka 69 B
%       u (13x1) = [timeStartup_s;
%                  timeGps_s;             % sekundy od epoki GPS 1980
%                  temperature_C;
%                  pressure_kPa;
%                  mag_G(3);              % [Gauss], body XYZ
%                  numSats;               % uint8
%                  gnssStatus;            % bitfield uint16
%                  attU_deg;              % [deg]
%                  insStatus;             % bitfield uint16
%                  posU_m;                % [m]
%                  velU_mps]              % [m/s]
%
%   pkt - uint8, kolumna o stalej dlugosci (38/94/69 zaleznie od packetType)
%   len - uint16, dlugosc ramki (rowna numel(pkt); wygodne dla Serial Send)
%
%   Format wg VN-200 ICD v2.1.0.0 sek. 2.1.3:
%   [0xFA][group byte][type words LE][payload LE][CRC16-CCITT BE]
%   CRC po headerze (bez 0xFA) + payloadzie; poly 0x1021, init 0.
%   UWAGA: serializacja float/double przez typecast = kolejnosc bajtow hosta.
%   Speedgoat (x86) i Windows/Linux sa little-endian, czyli zgodnie z ICD.

```

```

switch packetType
    case 1
        assert(numel(u) == 7);
        pkt = encodeBO1(u(:));
    case 2
        assert(numel(u) == 17);
        pkt = encodeBO2(u(:));
    otherwise
        assert(numel(u) == 13);
        pkt = encodeBO3(u(:));
end
len = uint16(numel(pkt));

```

end

```
%=====
% BinaryOutput1 (800 Hz): Common{TimeStartup, AngularRate, Accel}
% group 0x01, Common word 0x0121, payload 32 B, total 38 B
%=====
```

```
function pkt = encodeBO1(u)
pkt = zeros(38, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA sync
pkt(2) = uint8(1);             % group: Common
pkt(3:4) = u16le(uint16(289)); % 0x0121
pkt(5:12) = u64le_ns(u(1));    % TimeStartup [ns]
pkt(13:24) = f32le3(u(2:4));   % AngularRate [rad/s]
pkt(25:36) = f32le3(u(5:7));   % Accel [m/s^2]
pkt(37:38) = crcBE(pkt(2:36));
end
```

```
%=====
% BinaryOutput2 (400 Hz):
% Common{TimeStartup,Ypr} + IMU{DeltaTheta,DeltaVel} + INS{PosEcef,VelBody}
% group 0x25, words: 0x0009, 0x00C0, 0x000C, payload 84 B, total 94 B
%=====
```

```
function pkt = encodeBO2(u)
pkt = zeros(94, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(37);            % 0x25: Common|IMU|INS
pkt(3:4) = u16le(uint16(9));   % Common: 0x0009
pkt(5:6) = u16le(uint16(192)); % IMU: 0x00C0
pkt(7:8) = u16le(uint16(12));  % INS: 0x000C
pkt(9:16) = u64le_ns(u(1));    % TimeStartup
pkt(17:28) = f32le3(u(2:4));   % Ypr [deg]
pkt(29:32) = f32le(u(5));      % DeltaTheta.dtime [s]
pkt(33:44) = f32le3(u(6:8));   % DeltaTheta [deg]
pkt(45:56) = f32le3(u(9:11));  % DeltaVel [m/s]
pkt(57:80) = f64le3(u(12:14)); % PosEcef [m] (double)
pkt(81:92) = f32le3(u(15:17)); % VelBody [m/s]
pkt(93:94) = crcBE(pkt(2:92));
end
```

```
%=====
% BinaryOutput3 (200 Hz):
% Common{TimeStartup,TimeGps} + IMU{Temp,Pres,Mag} + GNSS{NumSats,GnssStatus}
% + Attitude{AttU} + INS{InsStatus,PosU,VelU}
% group 0x3D, words: 0x0003, 0x0130, 0x8008+0x0004 (ext), 0x2000, 0x0601
% payload 53 B, total 69 B
%=====
```

```
function pkt = encodeBO3(u)
pkt = zeros(69, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(61);            % 0x3D: Common|IMU|GNSS|Att|INS
pkt(3:4) = u16le(uint16(3));    % Common: 0x0003
pkt(5:6) = u16le(uint16(304));  % IMU: 0x0130
pkt(7:8) = u16le(uint16(32776)); % GNSS 1: 0x8008 (NumSats + ext bit)
pkt(9:10) = u16le(uint16(4));   % GNSS 2: 0x0004 (GnssStatus, offset 17)
pkt(11:12) = u16le(uint16(8192)); % Att: 0x2000 (AttU)
pkt(13:14) = u16le(uint16(1537)); % INS: 0x0601 (InsStatus|PosU|VelU)
pkt(15:22) = u64le_ns(u(1));    % TimeStartup
pkt(23:30) = u64le_ns(u(2));    % TimeGps
pkt(31:34) = f32le(u(3));       % Temperature [degC]
```

```

pkt(35:38) = f32le(u(4));           % Pressure [kPa]
pkt(39:50) = f32le3(u(5:7));        % Mag [G]
pkt(51)     = uint8(u(8));           % NumSats
pkt(52:53) = u16le(uint16(u(9)));    % GnssStatus
pkt(54:57) = f32le(u(10));           % AttU [deg]
pkt(58:59) = u16le(uint16(u(11)));   % InsStatus
pkt(60:63) = f32le(u(12));           % PosU [m]
pkt(64:67) = f32le(u(13));           % VelU [m/s]
pkt(68:69) = crcBE(pkt(2:67));
end

%=====
% Serializacja little-endian
%=====

function b = u16le(v)
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitand(v, uint16(255)));
b(2) = uint8(bitshift(v, -8));
end

function b = u64le_ns(t_s)
% sekundy (double) -> uint64 nanosekundy -> 8 bajtow LE
t = max(t_s, 0);
v = uint64(t * 1e9);                % uint64() zaokragla do najblizszej
b = zeros(8, 1, 'uint8');
for i = 1:8
    b(i) = uint8(bitand(v, uint64(255)));
    v = bitshift(v, -8);
end
end

function b = f32le(v)
b = reshape(typecast(single(v), 'uint8'), 4, 1); % host LE (x86)
end

function b = f32le3(v)
b = reshape(typecast(single(v(:)'), 'uint8'), 12, 1);
end

function b = f64le3(v)
b = reshape(typecast(double(v(:)'), 'uint8'), 24, 1);
end

%=====
% CRC16-CCITT (poly 0x1021, init 0) - ICD sek. 1.4.3; zwraca 2 bajty BE
%=====

function b = crcBE(data)
crc = uint16(0);
for i = 1:numel(data)
    crc = bitor(bitshift(crc, -8), bitshift(crc, 8));
    crc = bitxor(crc, uint16(data(i)));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), -4));
    crc = bitxor(crc, bitshift(crc, 12));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), 5));
end
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitshift(crc, -8));      % big-endian!
b(2) = uint8(bitand(crc, uint16(255)));
end

```



```

function [pkt, len] = vn200_encode(u, packetType) %#codegen
%VN200_ENCODE   Symulator ramek binarnych VectorNav VN-200 (HIL / Speedgoat).
%
%   [pkt, len] = vn200_encode(u, packetType)
%
%   Jeden blok MATLAB Function o konfigurowalnym parametrze packetType.
%   W bloku MATLAB Function ustaw 'packetType' jako PARAMETER (Ports and
%   Data Manager -> Scope: Parameter, Tunable: OFF). Dzieki temu rozmiary
%   wejscia/wyjscia sa stale w czasie kompilacji (wymog Simulink Coder /
%   Speedgoat).
%
%   packetType = 1  -> BinaryOutput1, 800 Hz, ramka 38 B
%       u (7x1) = [timeStartup_s;           % czas od startu [s]
%                 angularRate_radps(3);    % rad/s, body XYZ
%                 accel_mps2(3)]           % m/s^2, body XYZ
%
%   packetType = 2  -> BinaryOutput2, 400 Hz, ramka 94 B
%       u (17x1) = [timeStartup_s;
%                  ypr_deg(3);              % Yaw,Pitch,Roll [DEG]!
%                  deltaTime_s;             % okres calkowania (0.0025)
%                  deltaTheta_deg(3);       % [deg]
%                  deltaVel_mps(3);         % [m/s]
%                  posEcef_m(3);            % [m] (kodowane jako double)
%                  velBody_mps(3)]         % [m/s]
%
%   packetType = 3  -> BinaryOutput3, 200 Hz, ramka 69 B
%       u (13x1) = [timeStartup_s;
%                  timeGps_s;               % sekundy od epoki GPS 1980
%                  temperature_C;
%                  pressure_kPa;
%                  mag_G(3);                % [Gauss], body XYZ
%                  numSats;                 % uint8
%                  gnssStatus;              % bitfield uint16
%                  attU_deg;                % [deg]
%                  insStatus;               % bitfield uint16
%                  posU_m;                  % [m]
%                  velU_mps]                % [m/s]
%
%   pkt - uint8, kolumna o stalej dlugosci (38/94/69 zaleznie od packetType)
%   len - uint16, dlugosc ramki (rowna numel(pkt); wygodne dla Serial Send)
%
%   Format wg VN-200 ICD v2.1.0.0 sek. 2.1.3:
%   [0xFA][group byte][type words LE][payload LE][CRC16-CCITT BE]
%   CRC po headerze (bez 0xFA) + payloadzie; poly 0x1021, init 0.
%   UWAGA: serializacja float/double przez typecast = kolejnosc bajtow hosta.
%   Speedgoat (x86) i Windows/Linux sa little-endian, czyli zgodnie z ICD.

```

```

switch packetType
    case 1
        assert(numel(u) == 7);
        pkt = encodeBO1(u(:));
    case 2
        assert(numel(u) == 17);
        pkt = encodeBO2(u(:));
    otherwise
        assert(numel(u) == 13);
        pkt = encodeBO3(u(:));
end
len = uint16(numel(pkt));

```

end

```
%=====
% BinaryOutput1 (800 Hz): Common{TimeStartup, AngularRate, Accel}
% group 0x01, Common word 0x0121, payload 32 B, total 38 B
%=====
```

```
function pkt = encodeBO1(u)
pkt = zeros(38, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA sync
pkt(2) = uint8(1);             % group: Common
pkt(3:4) = u16le(uint16(289)); % 0x0121
pkt(5:12) = u64le_ns(u(1));    % TimeStartup [ns]
pkt(13:24) = f32le3(u(2:4));   % AngularRate [rad/s]
pkt(25:36) = f32le3(u(5:7));   % Accel [m/s^2]
pkt(37:38) = crcBE(pkt(2:36));
end
```

```
%=====
% BinaryOutput2 (400 Hz):
% Common{TimeStartup,Ypr} + IMU{DeltaTheta,DeltaVel} + INS{PosEcef,VelBody}
% group 0x25, words: 0x0009, 0x00C0, 0x000C, payload 84 B, total 94 B
%=====
```

```
function pkt = encodeBO2(u)
pkt = zeros(94, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(37);            % 0x25: Common|IMU|INS
pkt(3:4) = u16le(uint16(9));    % Common: 0x0009
pkt(5:6) = u16le(uint16(192));  % IMU: 0x00C0
pkt(7:8) = u16le(uint16(12));   % INS: 0x000C
pkt(9:16) = u64le_ns(u(1));     % TimeStartup
pkt(17:28) = f32le3(u(2:4));    % Ypr [deg]
pkt(29:32) = f32le(u(5));       % DeltaTheta.dtime [s]
pkt(33:44) = f32le3(u(6:8));    % DeltaTheta [deg]
pkt(45:56) = f32le3(u(9:11));   % DeltaVel [m/s]
pkt(57:80) = f64le3(u(12:14));  % PosEcef [m] (double)
pkt(81:92) = f32le3(u(15:17));  % VelBody [m/s]
pkt(93:94) = crcBE(pkt(2:92));
end
```

```
%=====
% BinaryOutput3 (200 Hz):
% Common{TimeStartup,TimeGps} + IMU{Temp,Pres,Mag} + GNSS{NumSats,GnssStatus}
% + Attitude{AttU} + INS{InsStatus,PosU,VelU}
% group 0x3D, words: 0x0003, 0x0130, 0x8008+0x0004 (ext), 0x2000, 0x0601
% payload 53 B, total 69 B
%=====
```

```
function pkt = encodeBO3(u)
pkt = zeros(69, 1, 'uint8');
pkt(1) = uint8(250);           % 0xFA
pkt(2) = uint8(61);            % 0x3D: Common|IMU|GNSS|Att|INS
pkt(3:4) = u16le(uint16(3));    % Common: 0x0003
pkt(5:6) = u16le(uint16(304));  % IMU: 0x0130
pkt(7:8) = u16le(uint16(32776)); % GNSS 1: 0x8008 (NumSats + ext bit)
pkt(9:10) = u16le(uint16(4));   % GNSS 2: 0x0004 (GnssStatus, offset 17)
pkt(11:12) = u16le(uint16(8192)); % Att: 0x2000 (AttU)
pkt(13:14) = u16le(uint16(1537)); % INS: 0x0601 (InsStatus|PosU|VelU)
pkt(15:22) = u64le_ns(u(1));    % TimeStartup
pkt(23:30) = u64le_ns(u(2));    % TimeGps
pkt(31:34) = f32le(u(3));       % Temperature [degC]
```

```

pkt(35:38) = f32le(u(4));           % Pressure [kPa]
pkt(39:50) = f32le3(u(5:7));        % Mag [G]
pkt(51)     = uint8(u(8));           % NumSats
pkt(52:53)  = u16le(uint16(u(9)));   % GnssStatus
pkt(54:57)  = f32le(u(10));          % AttU [deg]
pkt(58:59)  = u16le(uint16(u(11)));  % InsStatus
pkt(60:63)  = f32le(u(12));          % PosU [m]
pkt(64:67)  = f32le(u(13));          % VelU [m/s]
pkt(68:69)  = crcBE(pkt(2:67));
end

%=====
% Serializacja little-endian
%=====

function b = u16le(v)
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitand(v, uint16(255)));
b(2) = uint8(bitshift(v, -8));
end

function b = u64le_ns(t_s)
% sekundy (double) -> uint64 nanosekundy -> 8 bajtow LE
t = max(t_s, 0);
v = uint64(t * 1e9);           % uint64() zaokragla do najblizszej
b = zeros(8, 1, 'uint8');
for i = 1:8
    b(i) = uint8(bitand(v, uint64(255)));
    v = bitshift(v, -8);
end
end

function b = f32le(v)
b = reshape(typecast(single(v), 'uint8'), 4, 1); % host LE (x86)
end

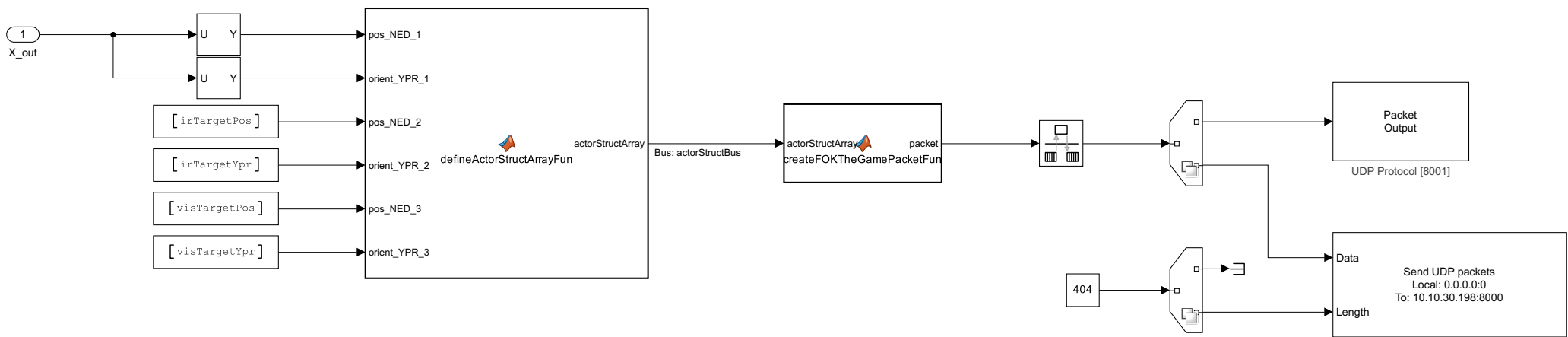
function b = f32le3(v)
b = reshape(typecast(single(v(:)'), 'uint8'), 12, 1);
end

function b = f64le3(v)
b = reshape(typecast(double(v(:)'), 'uint8'), 24, 1);
end

%=====
% CRC16-CCITT (poly 0x1021, init 0) - ICD sek. 1.4.3; zwraca 2 bajty BE
%=====

function b = crcBE(data)
crc = uint16(0);
for i = 1:numel(data)
    crc = bitor(bitshift(crc, -8), bitshift(crc, 8));
    crc = bitxor(crc, uint16(data(i)));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), -4));
    crc = bitxor(crc, bitshift(crc, 12));
    crc = bitxor(crc, bitshift(bitand(crc, uint16(255)), 5));
end
b = zeros(2, 1, 'uint8');
b(1) = uint8(bitshift(crc, -8)); % big-endian!
b(2) = uint8(bitand(crc, uint16(255)));
end

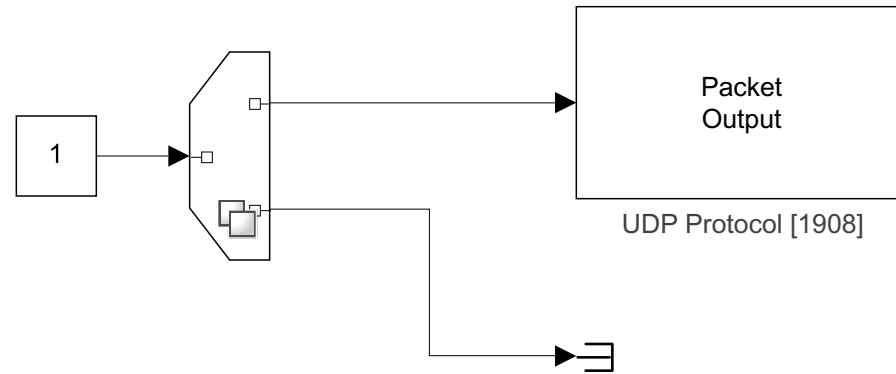
```

```
function packet = createFOKTheGamePacketFun(actorStructArray)
%#codegen
packet=zeros([512,1],"uint8");
packet(1:512) = createFOKTheGamePacket(actorStructArray);

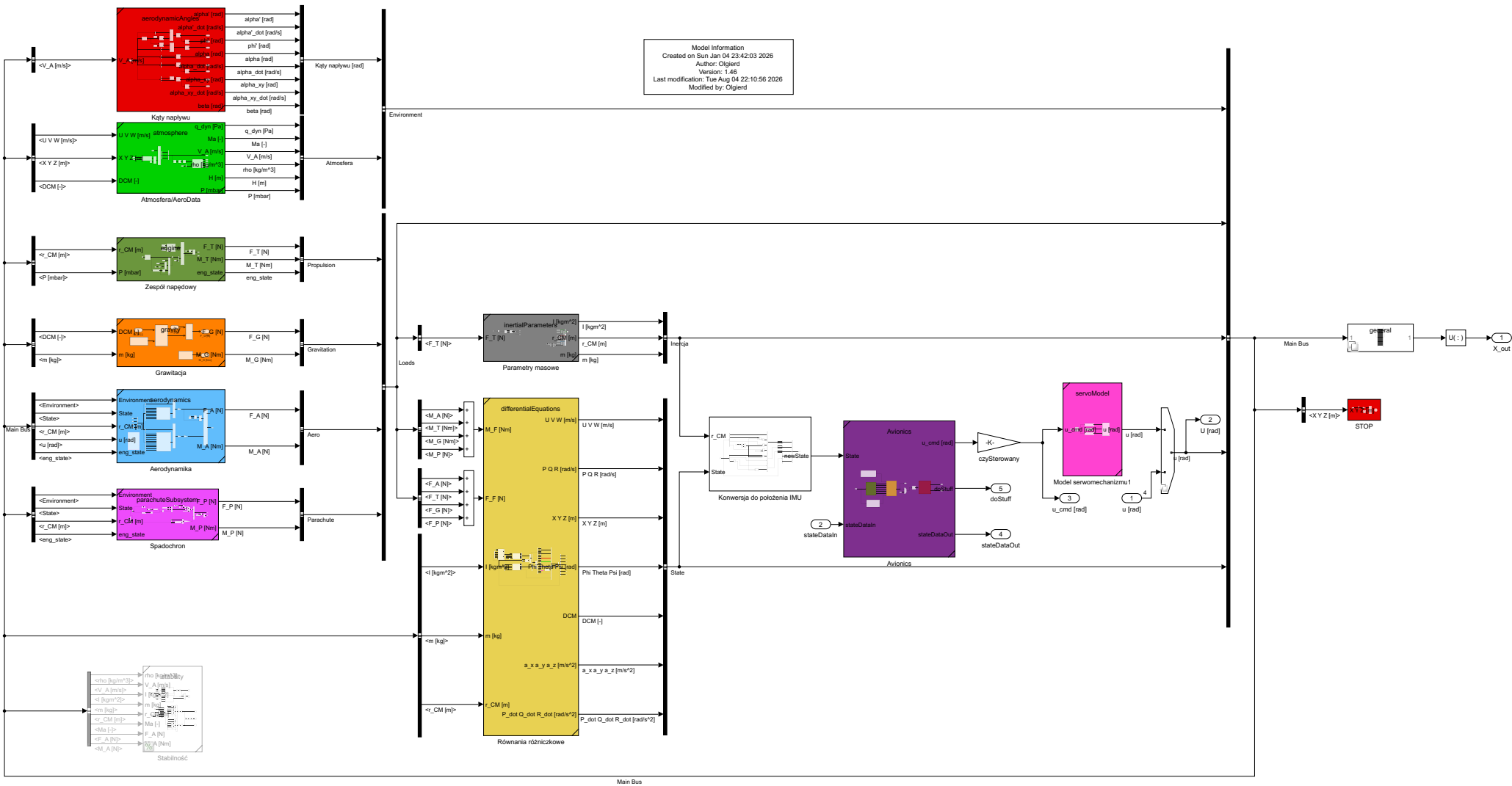
end
```

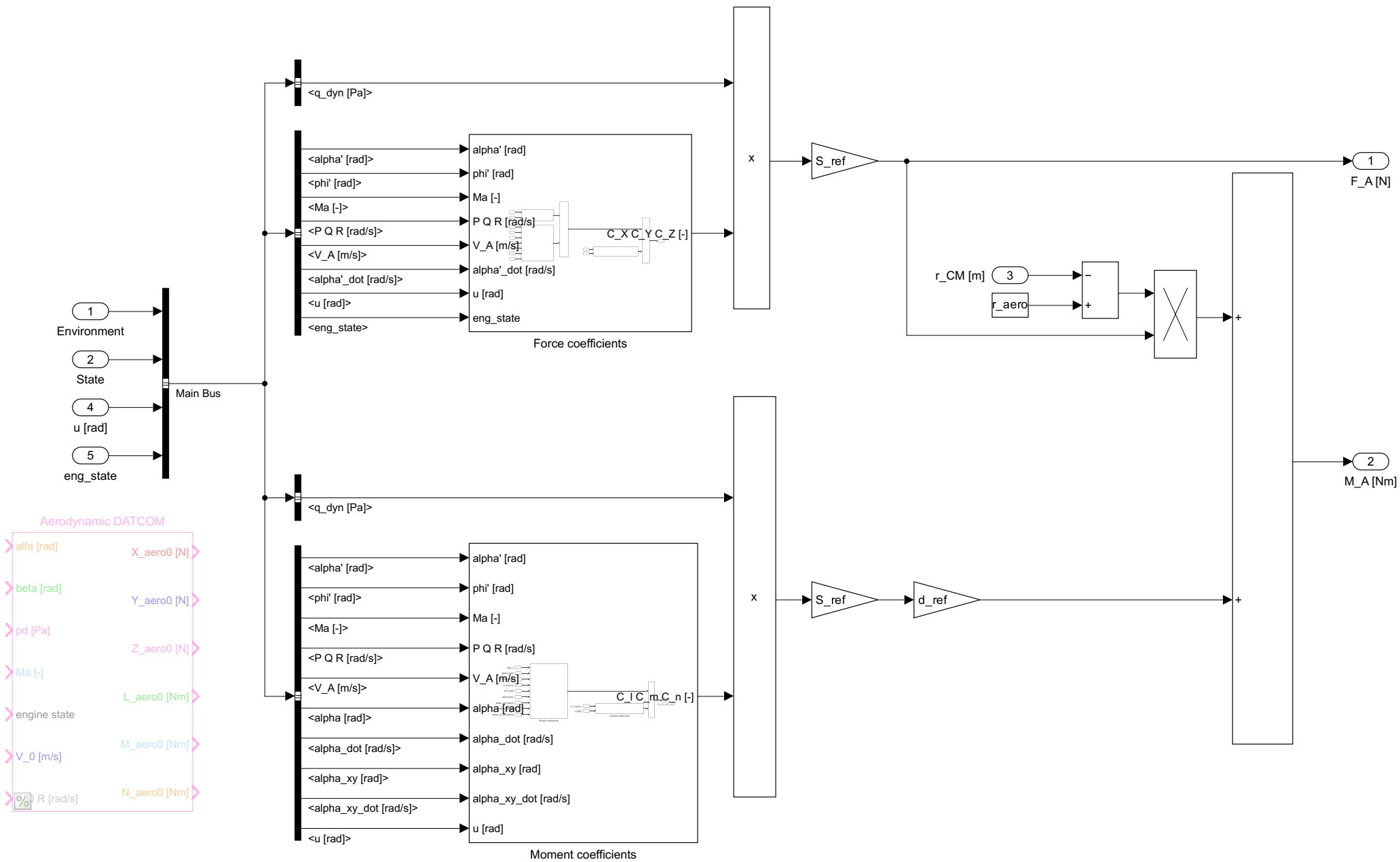
```
function actorStructArray = defineActorStructArrayFun(pos_NED_1, orient_YPR_1, pos_NED_2, orient_YPR_2, pos_NED_3, orient_YPR_3)
actorStructArray = defineActorStructArray(pos_NED_1, orient_YPR_1, pos_NED_2, orient_YPR_2, pos_NED_3, orient_YPR_3);
end
```

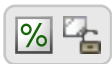
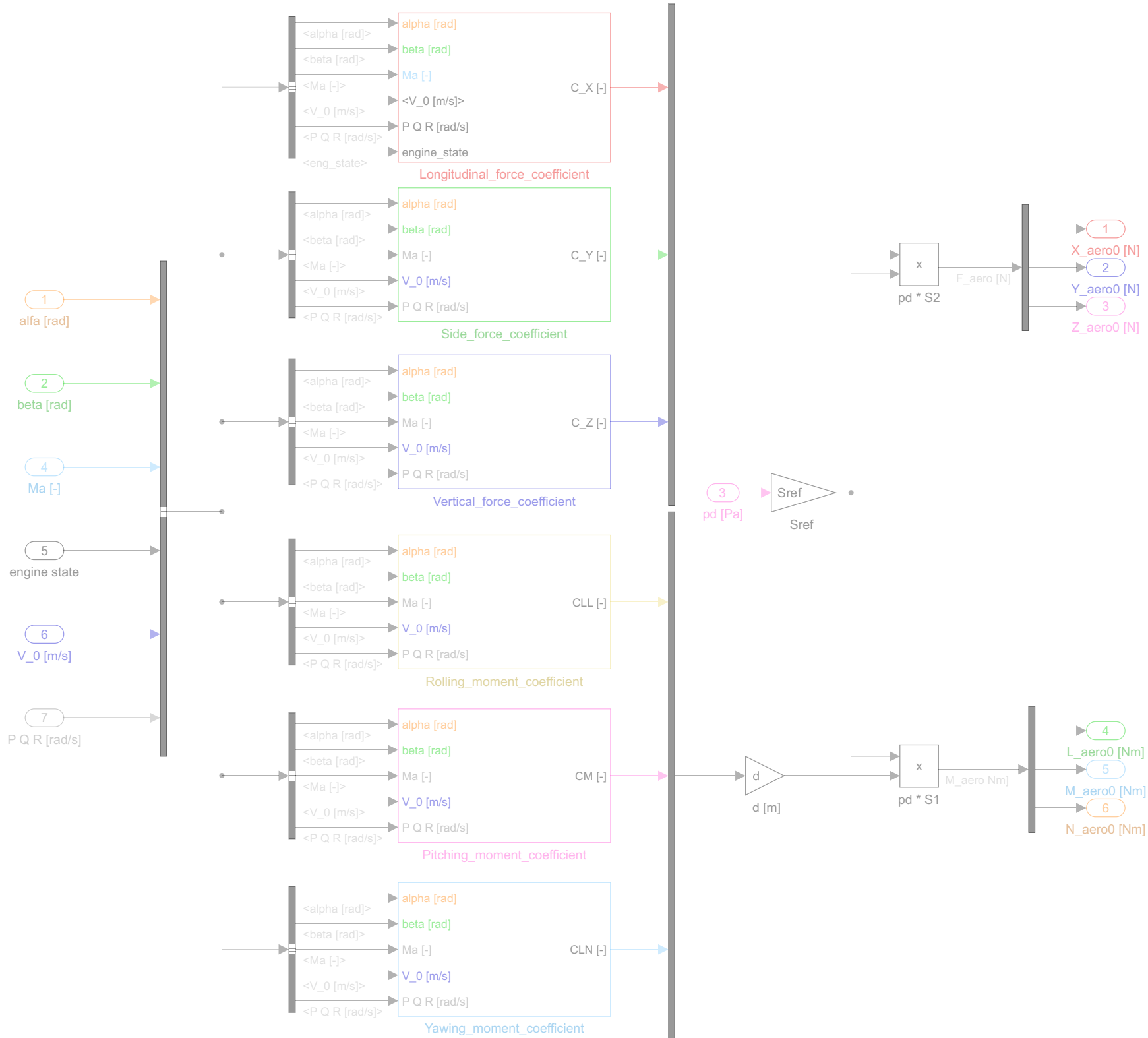


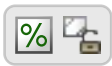
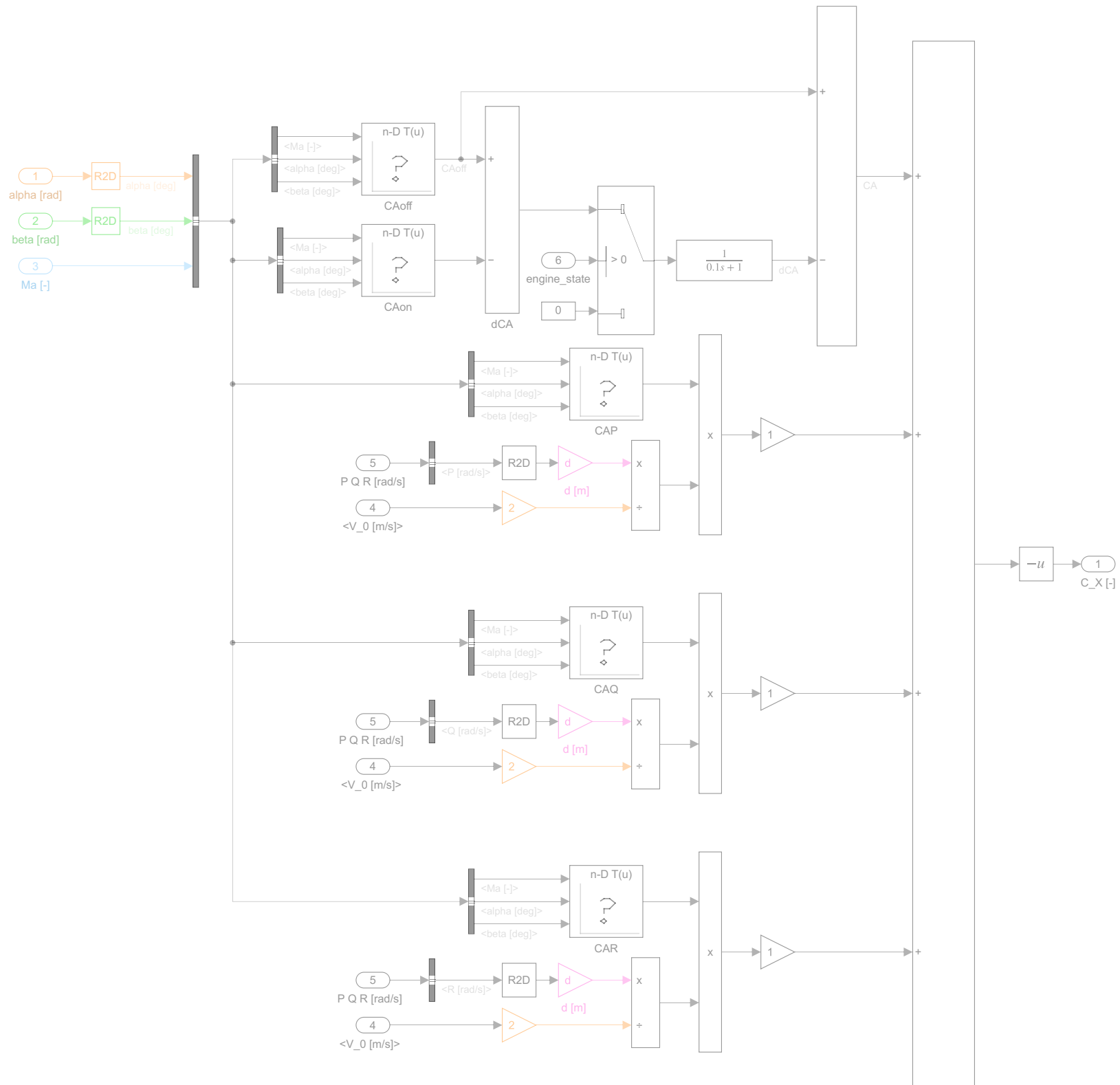
Speedgoat
IO324-200k
Serial Setup v2
Module ID: 1

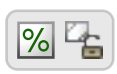
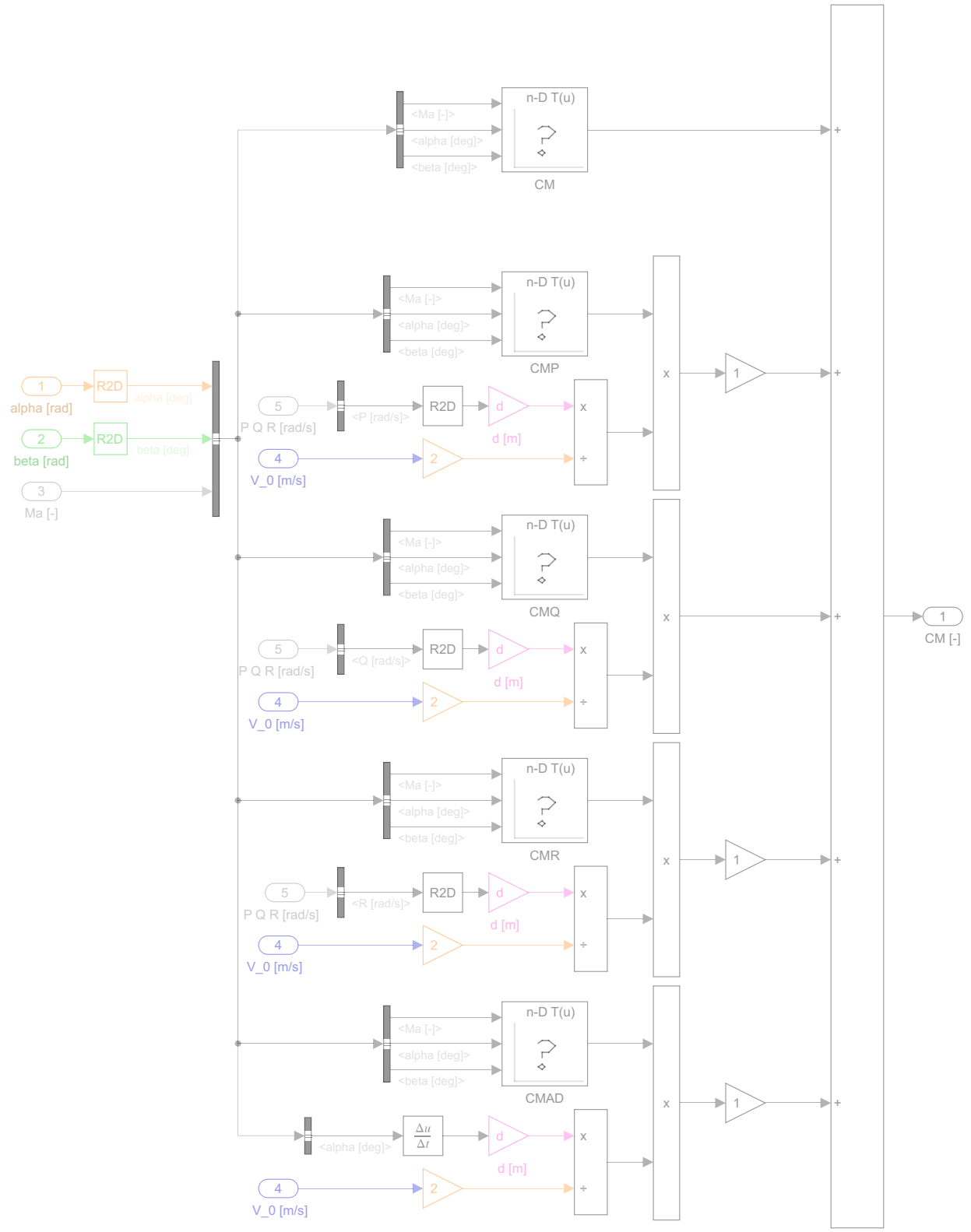
Speedgoat
IO324-200k
Setup v3
Module ID: 1

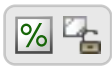
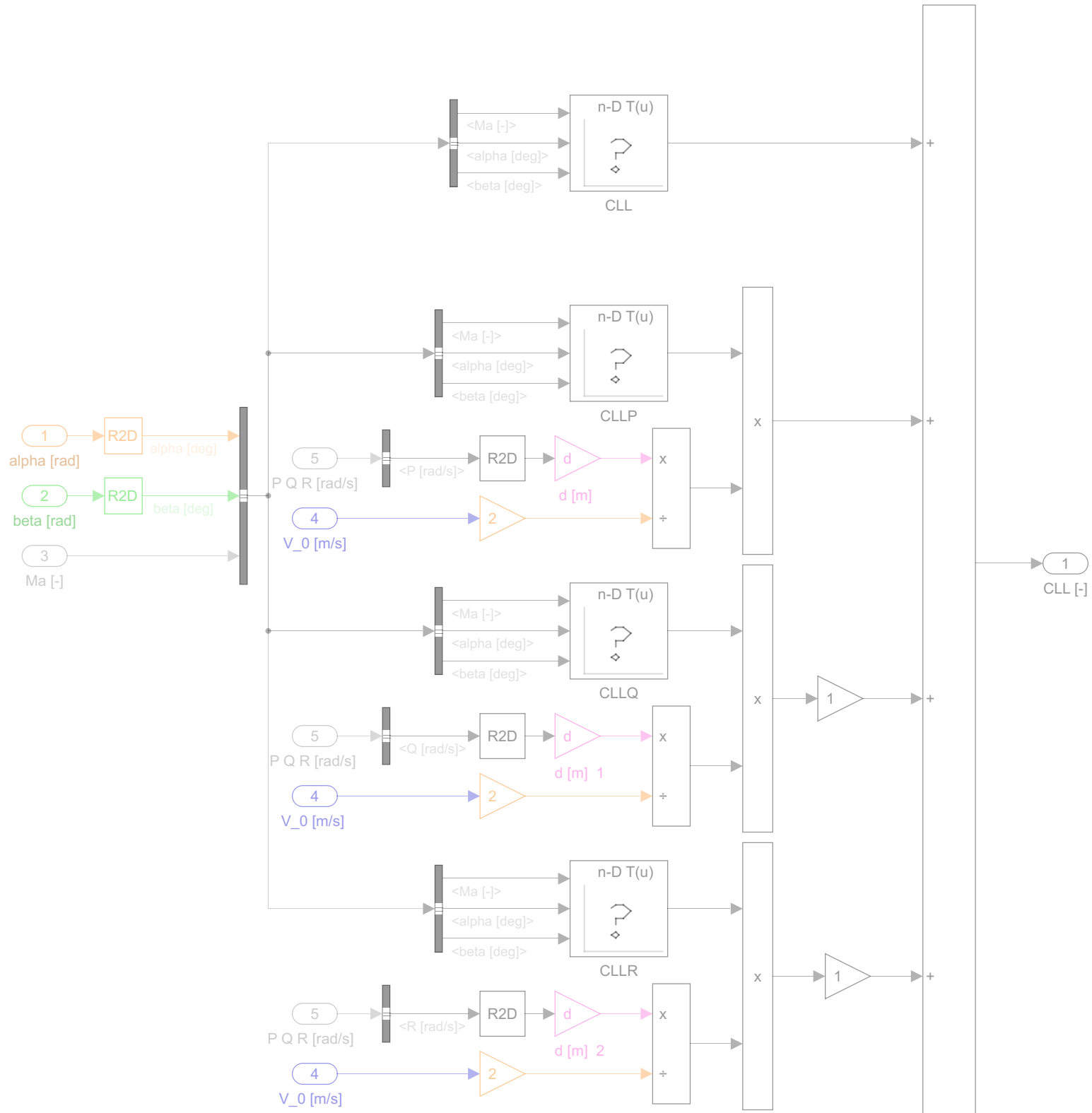


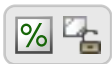
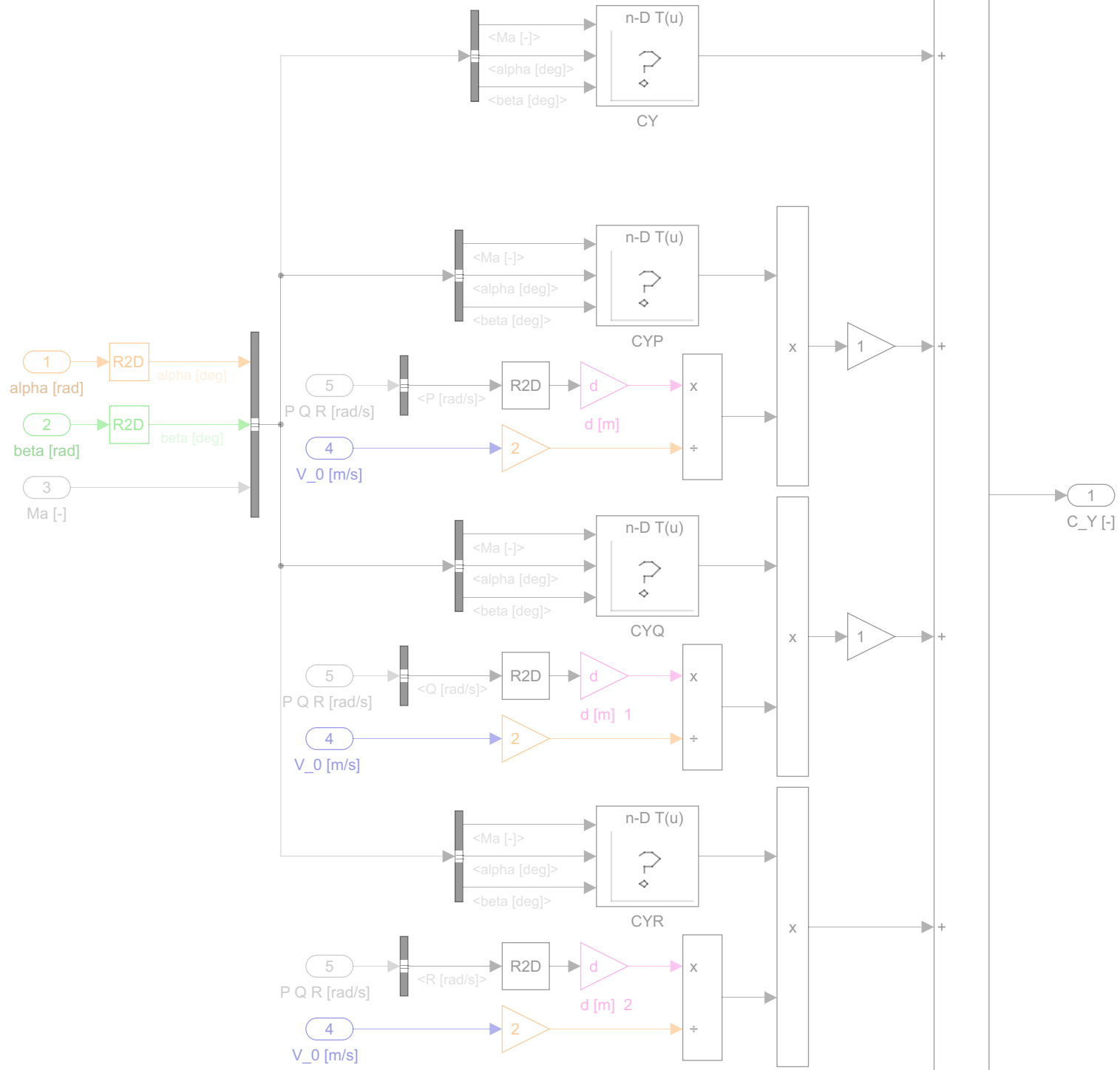


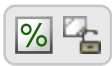
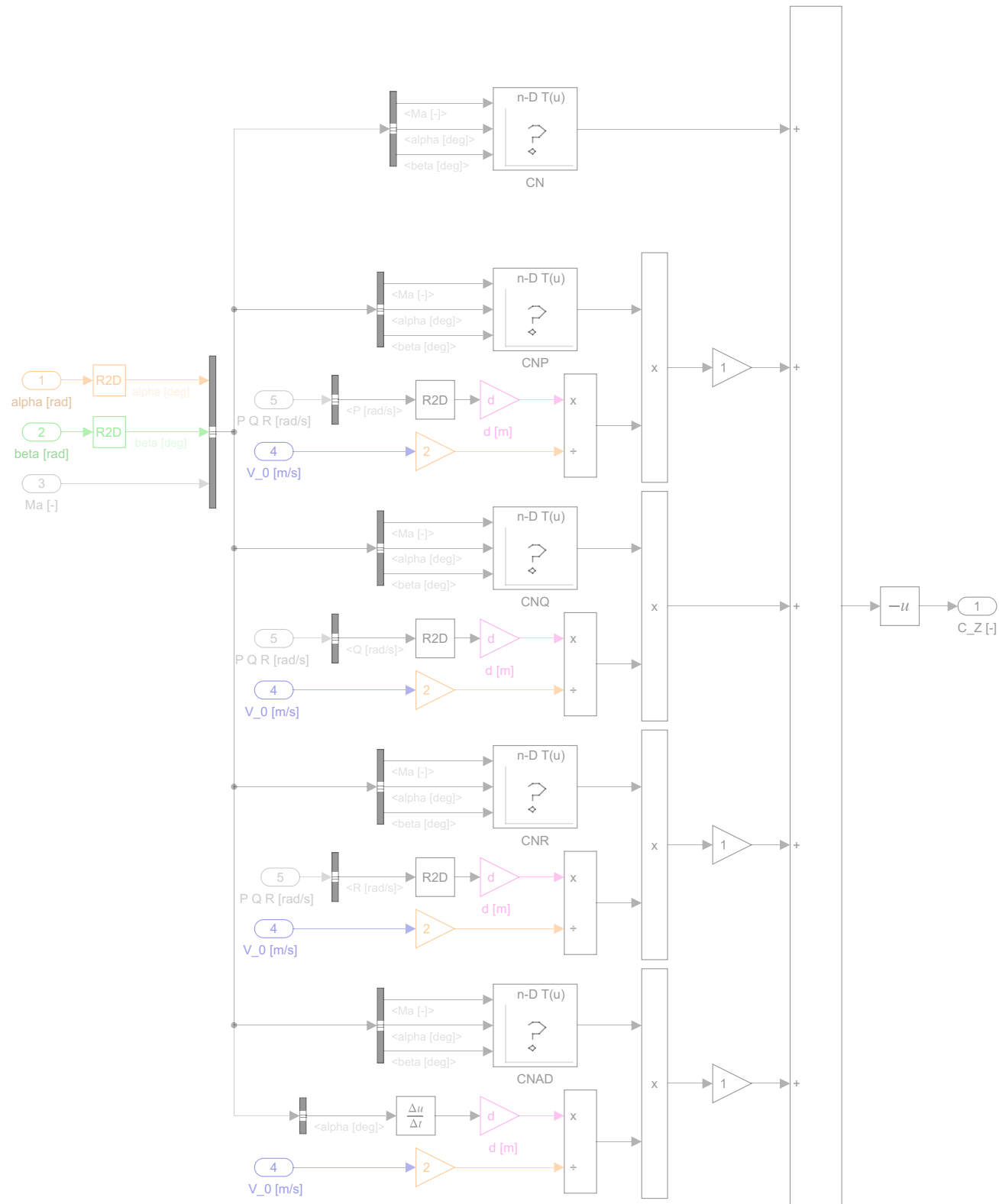


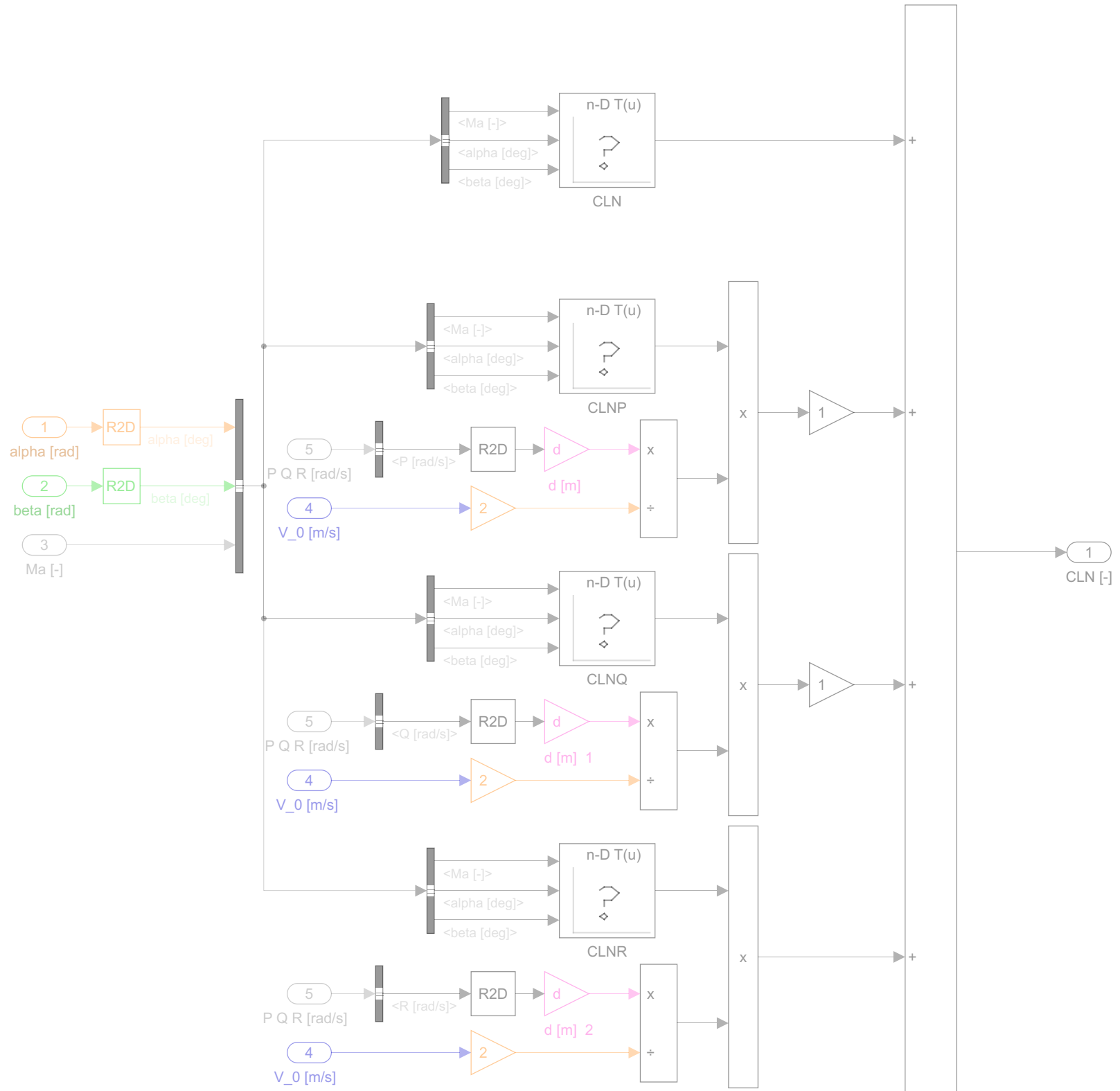


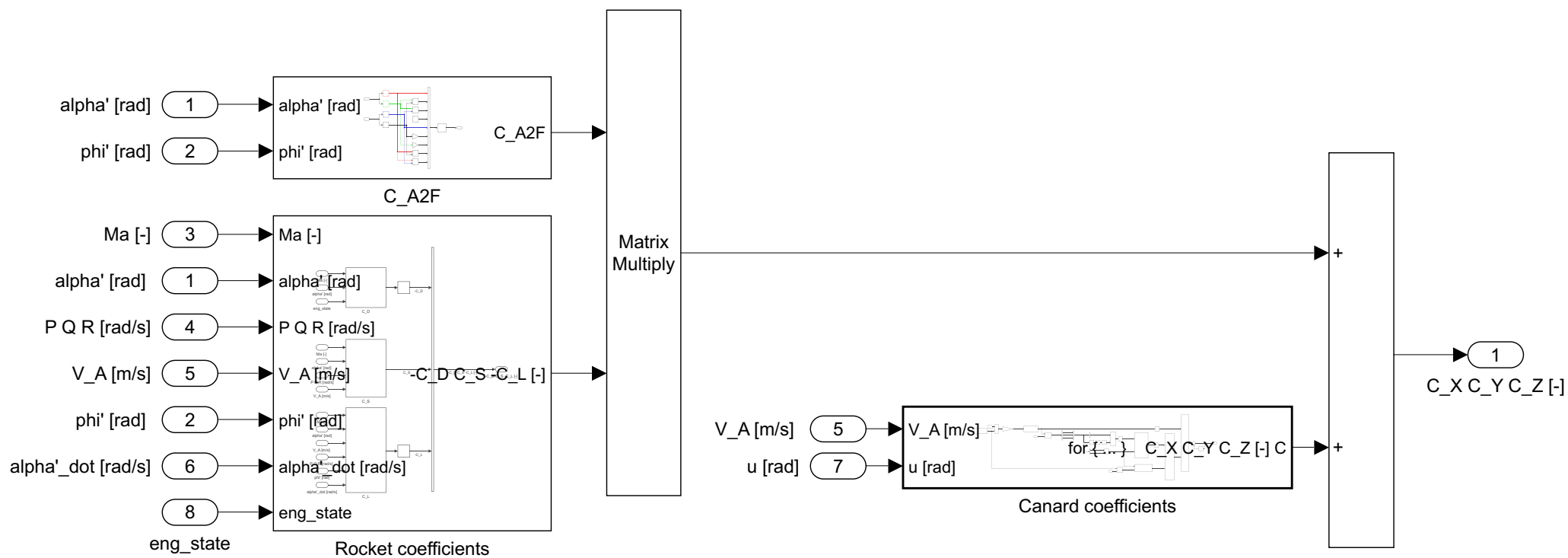




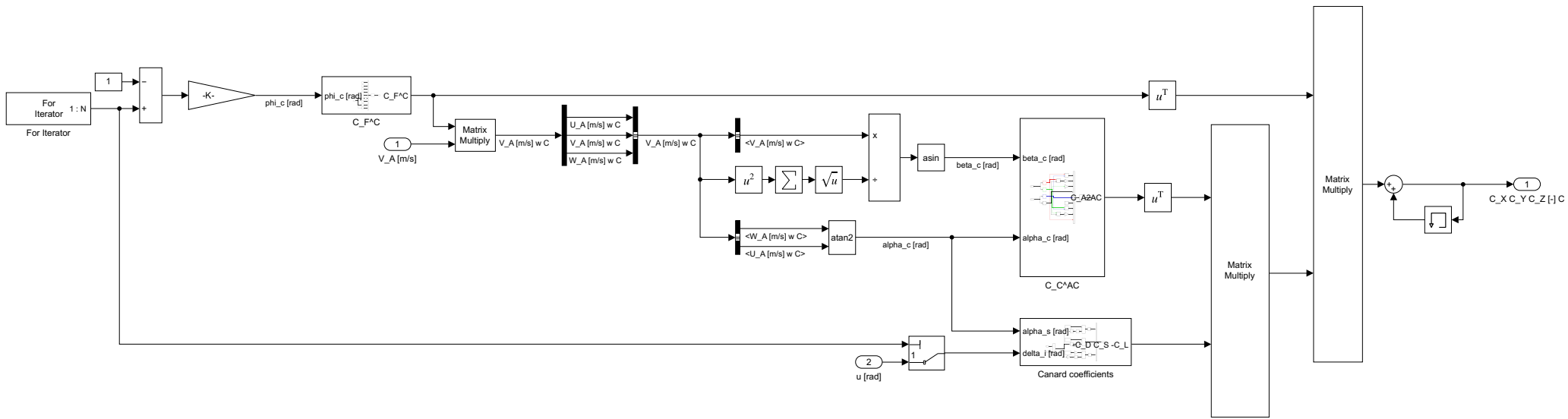


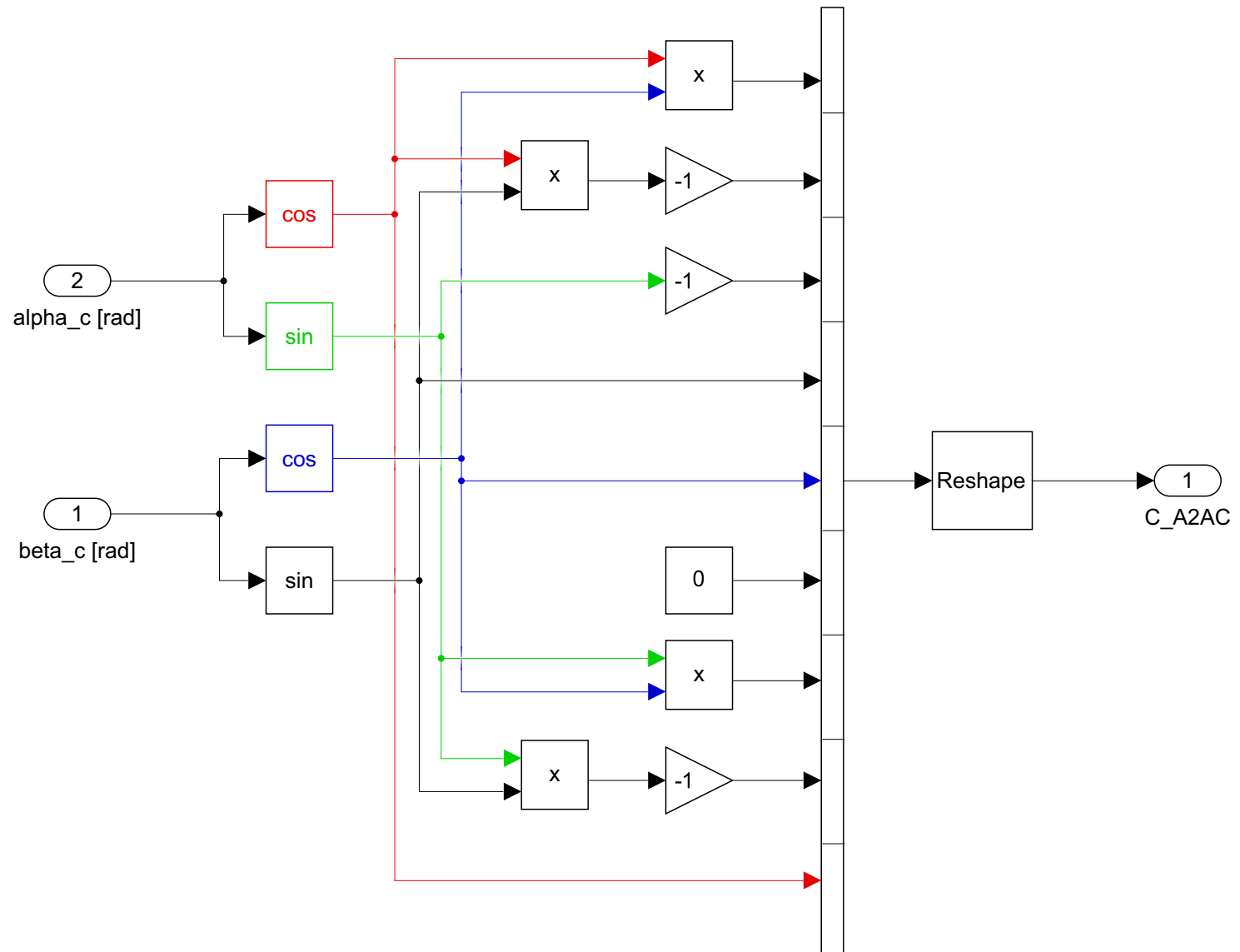


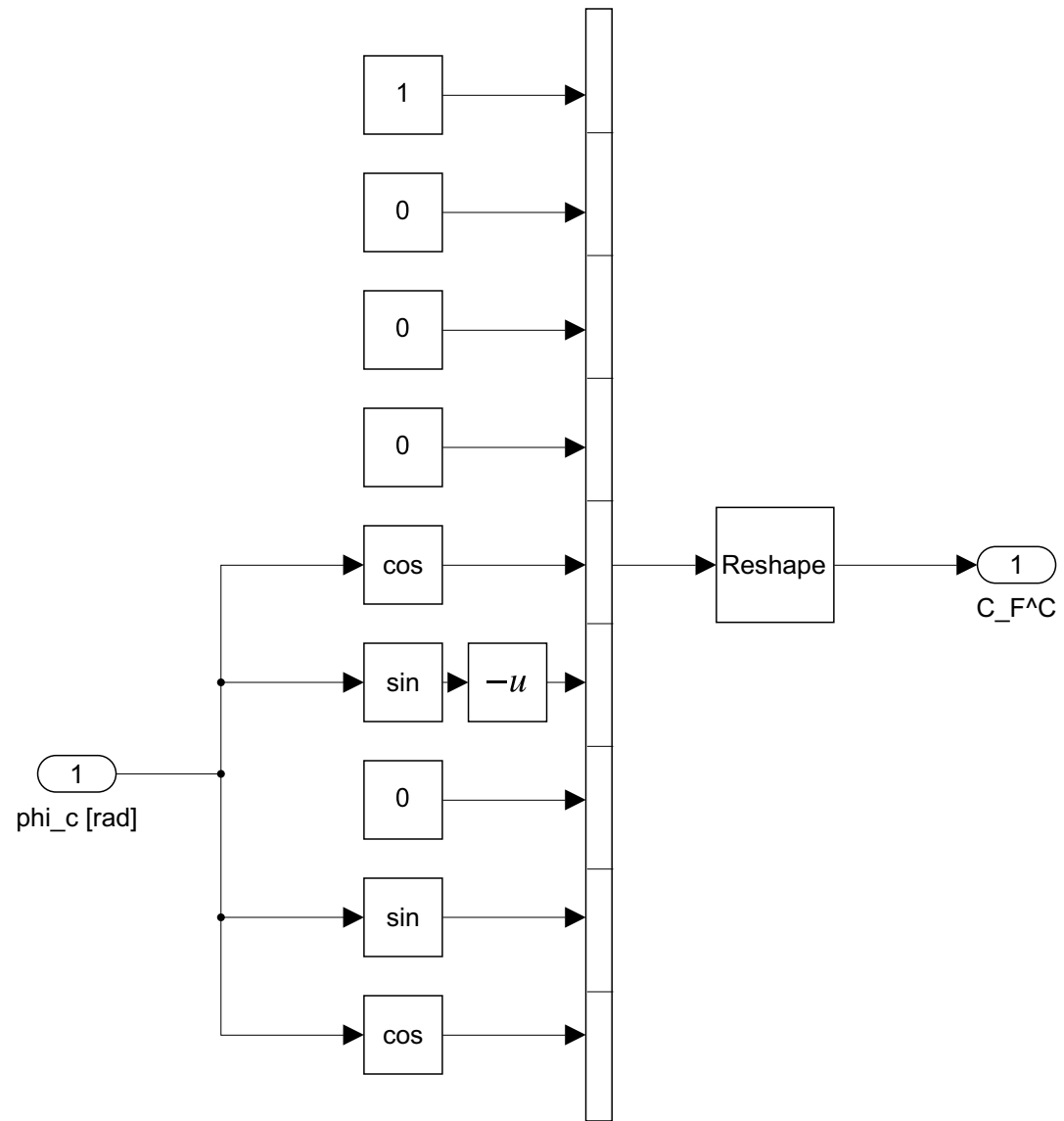


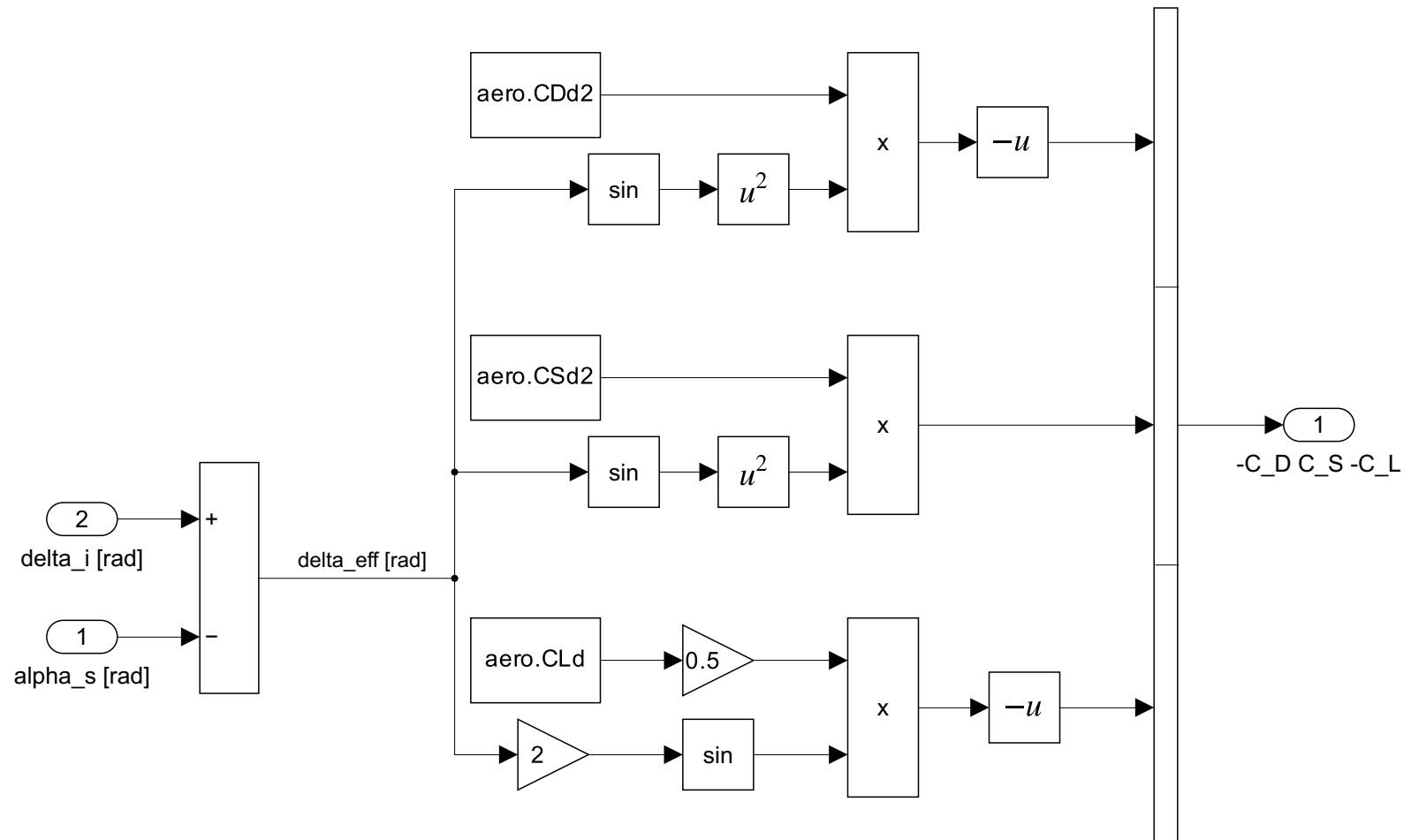


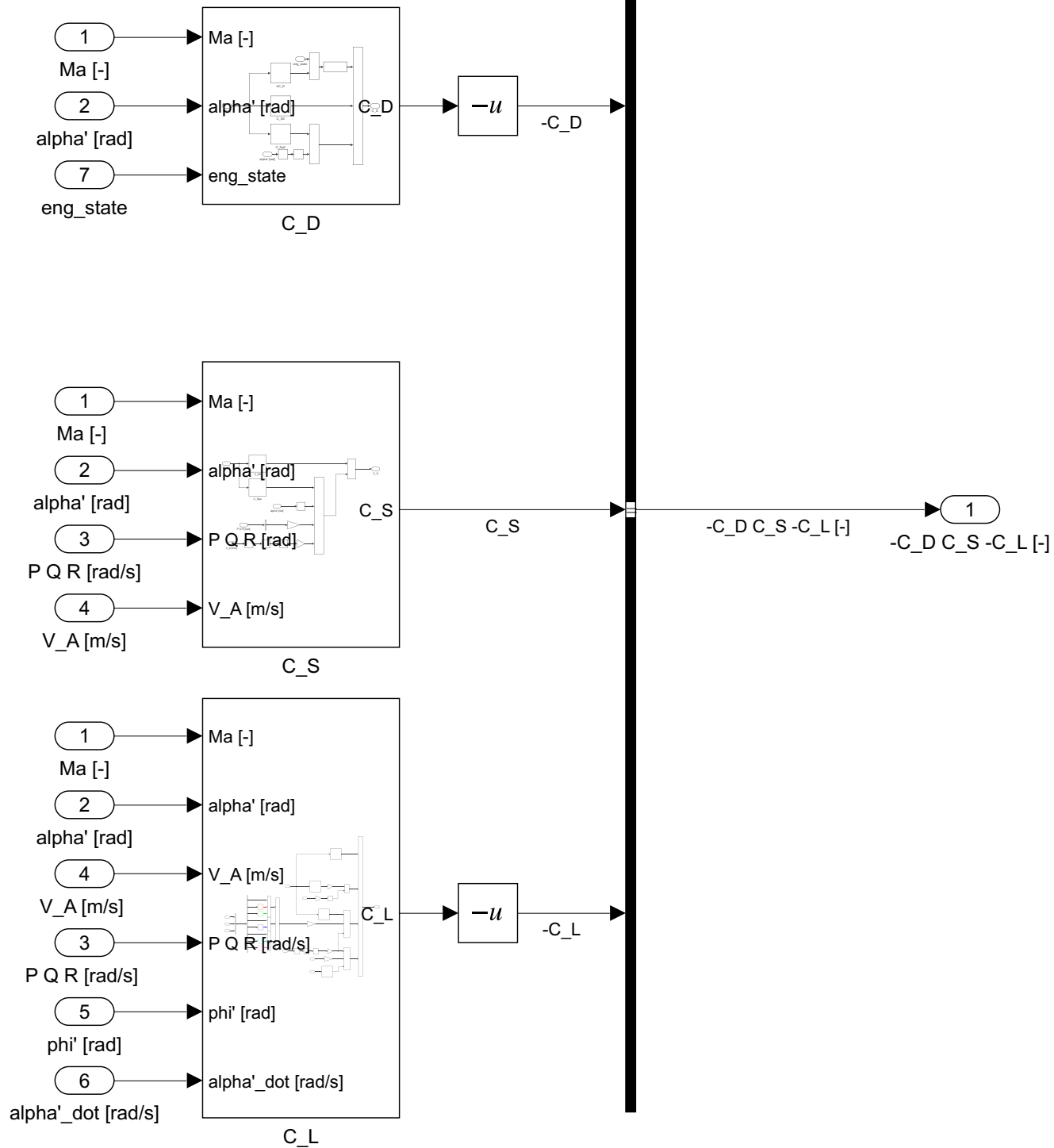


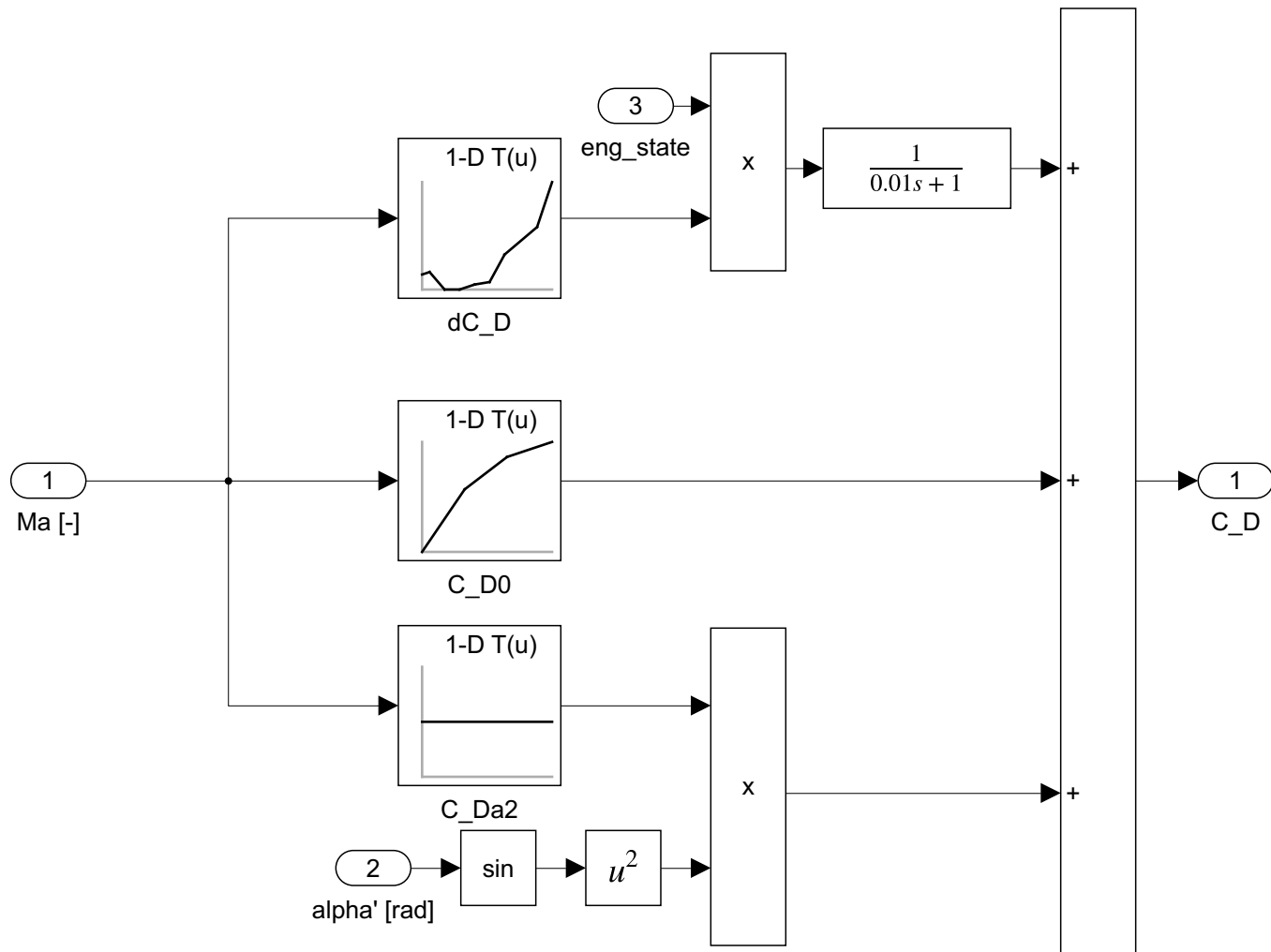


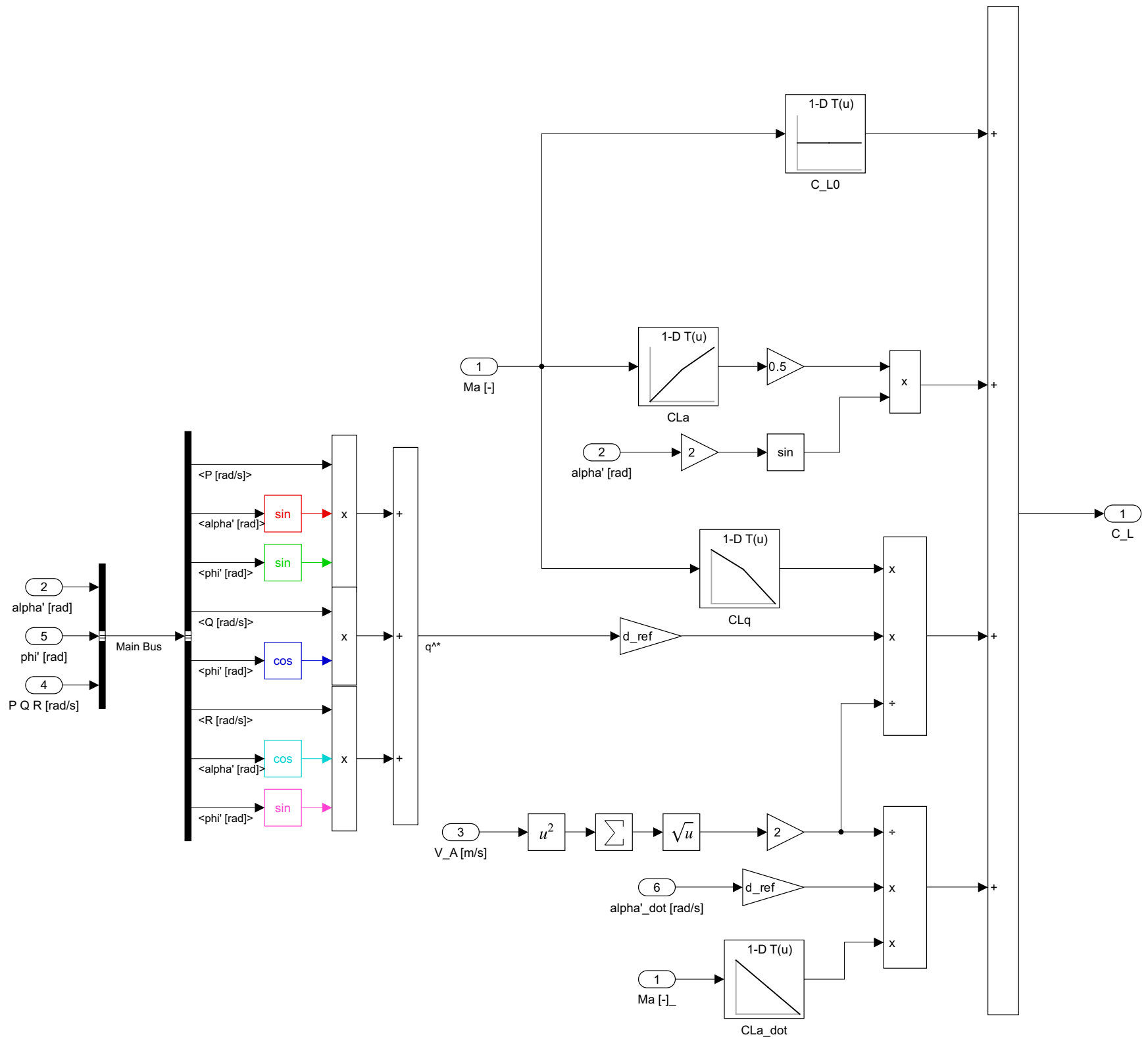


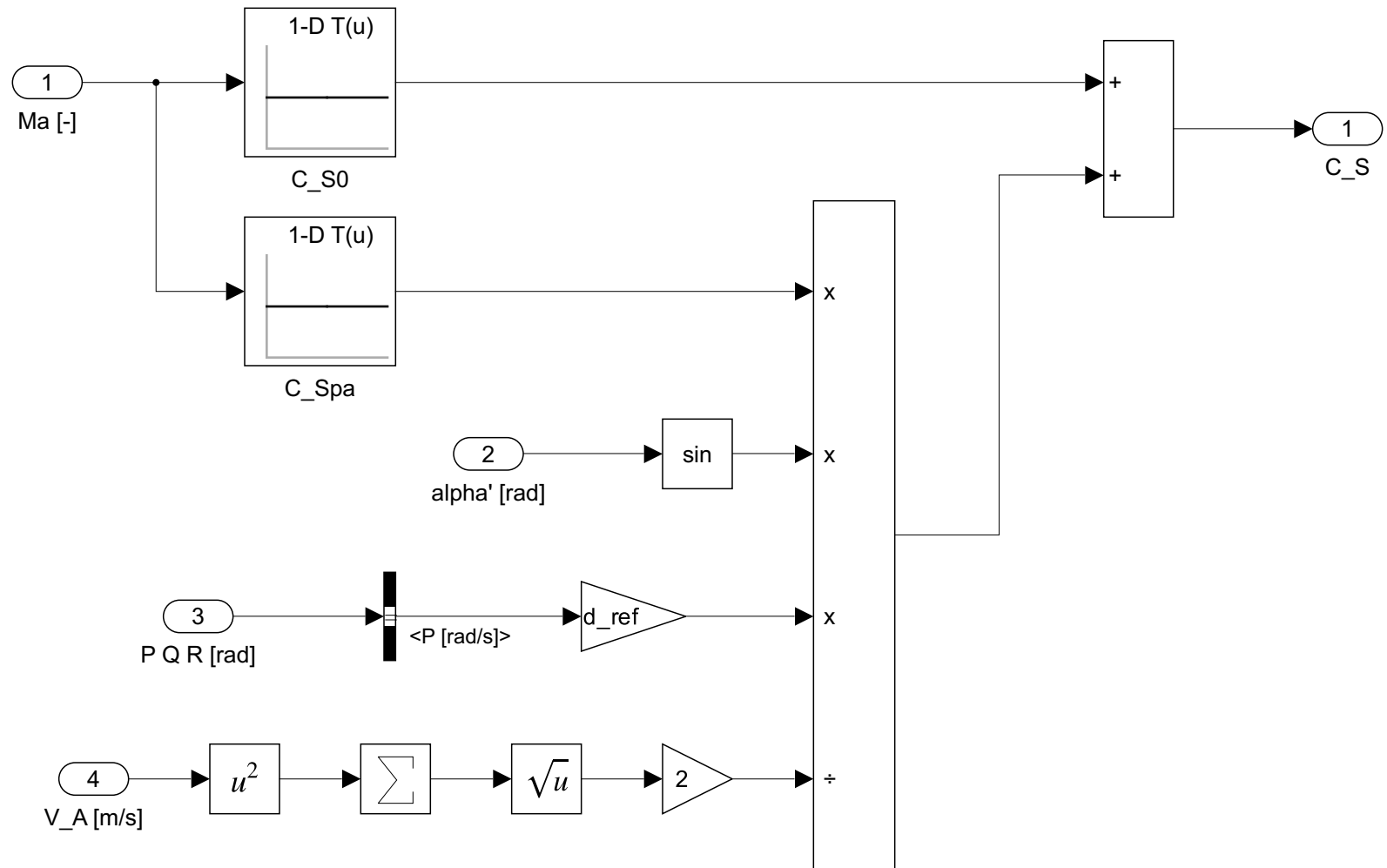


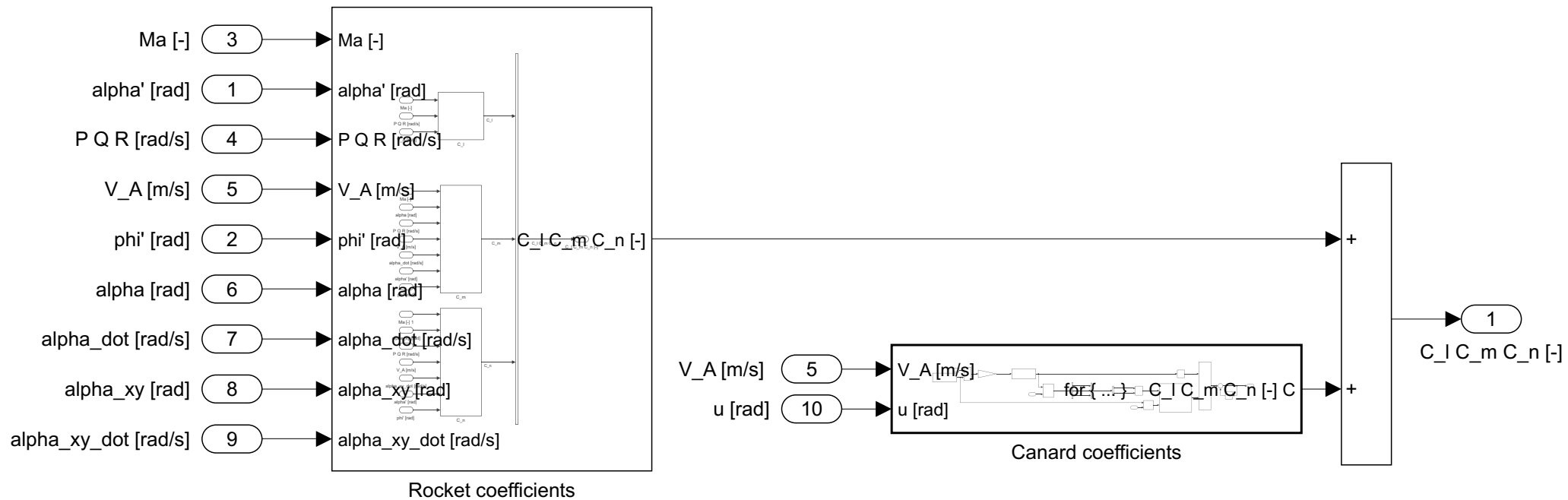




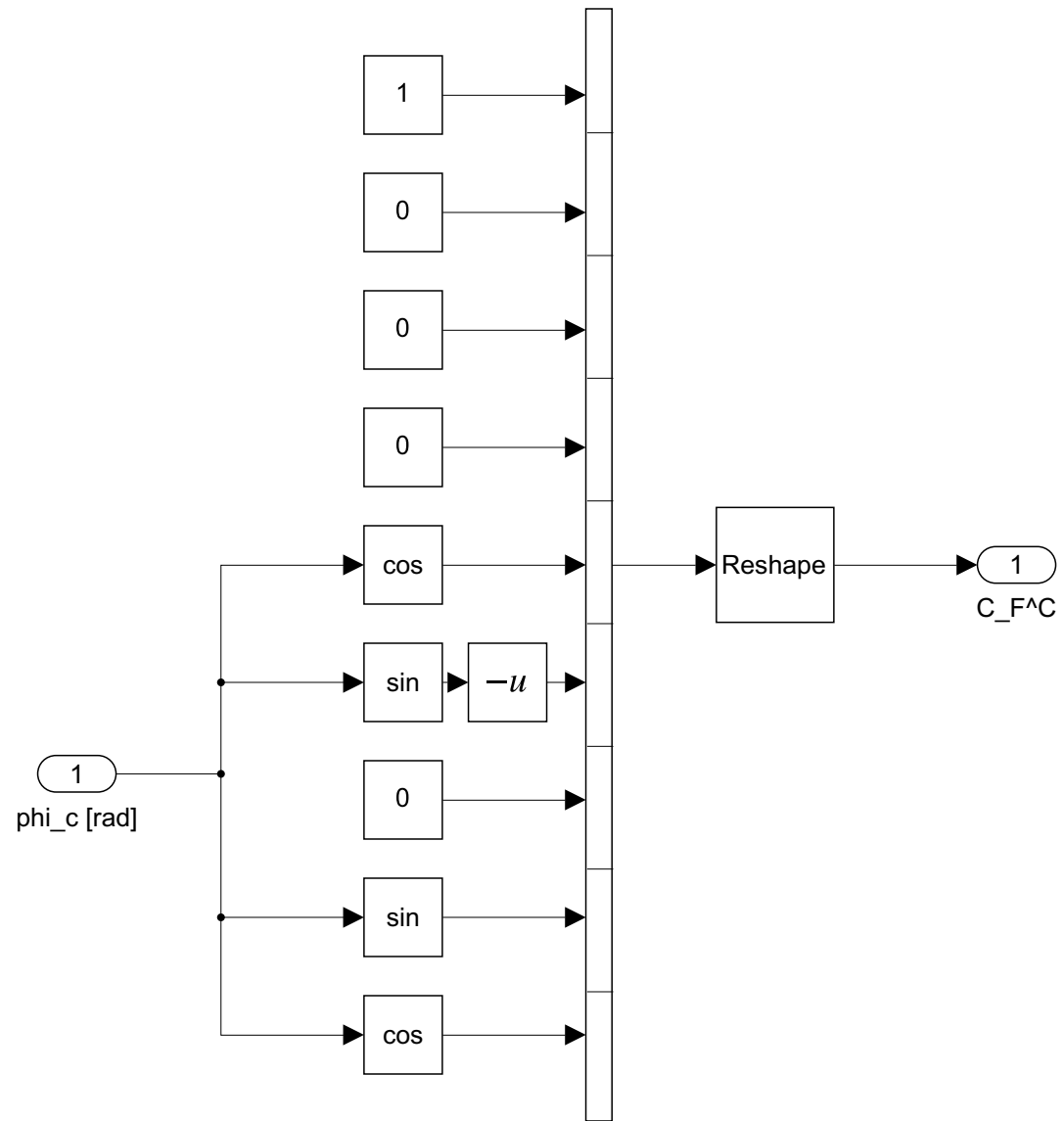


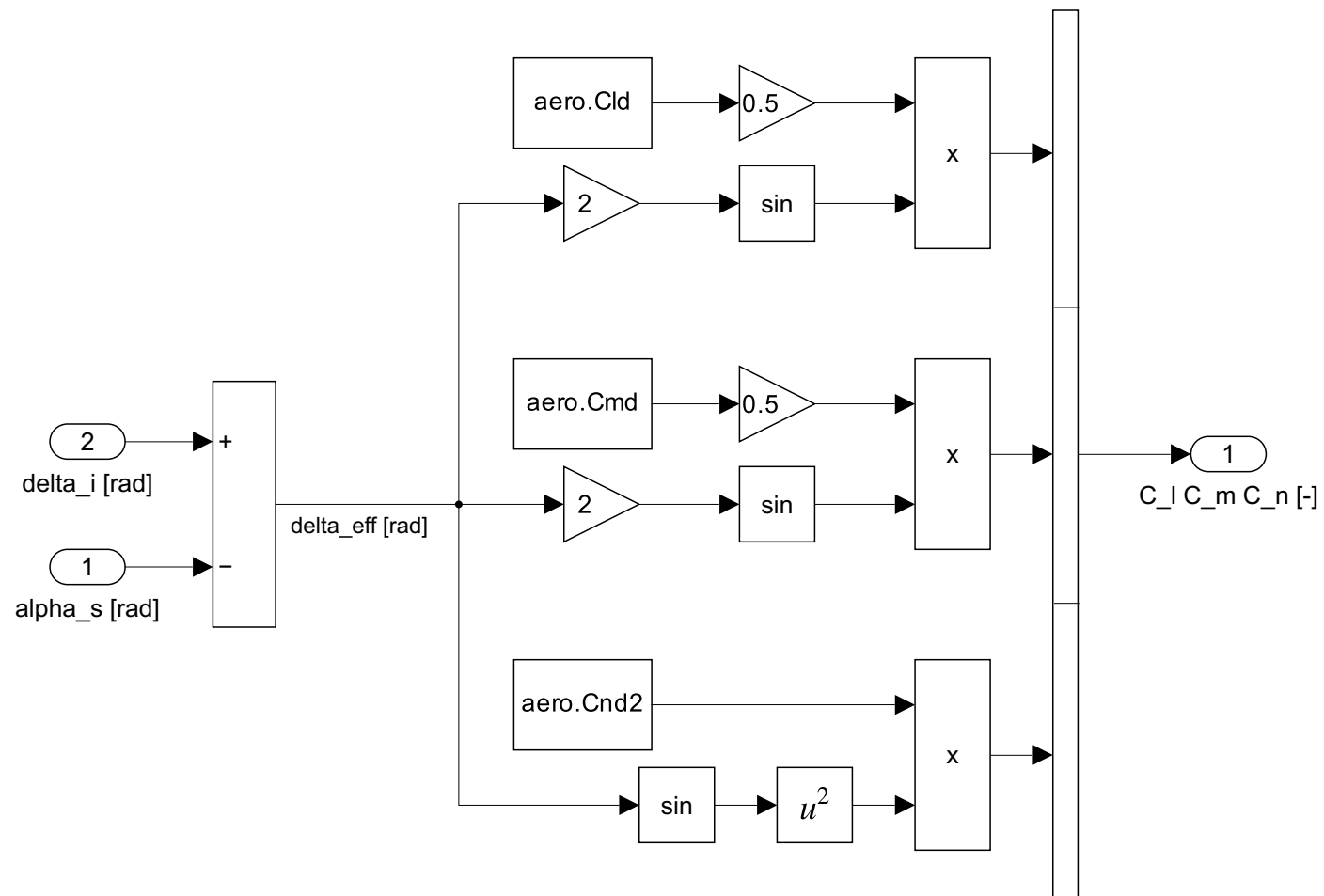


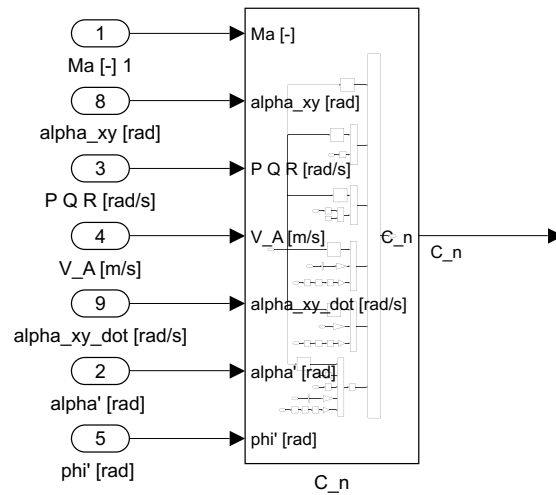
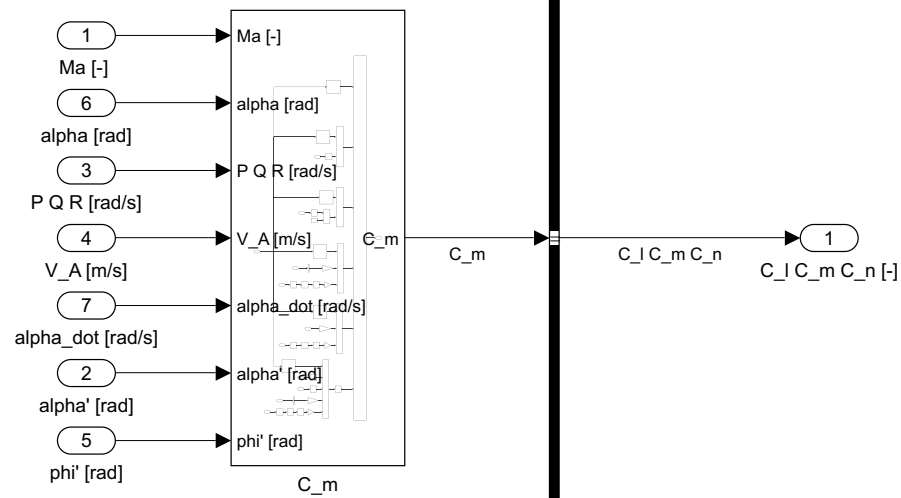
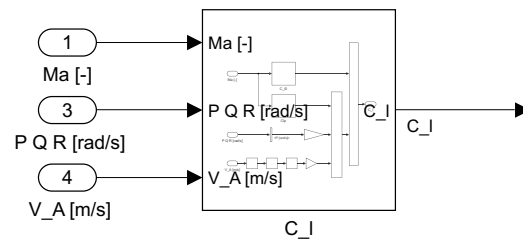






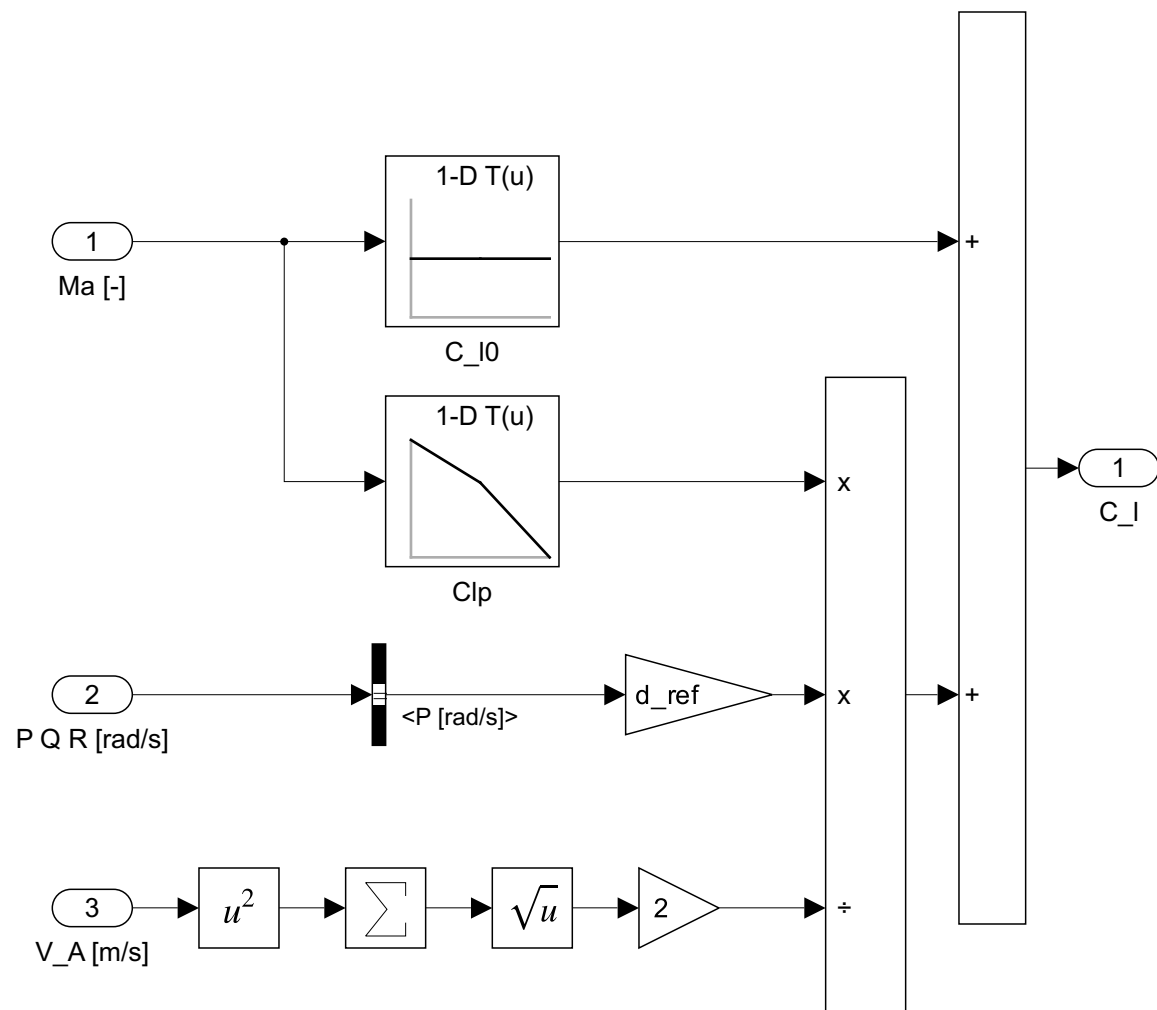


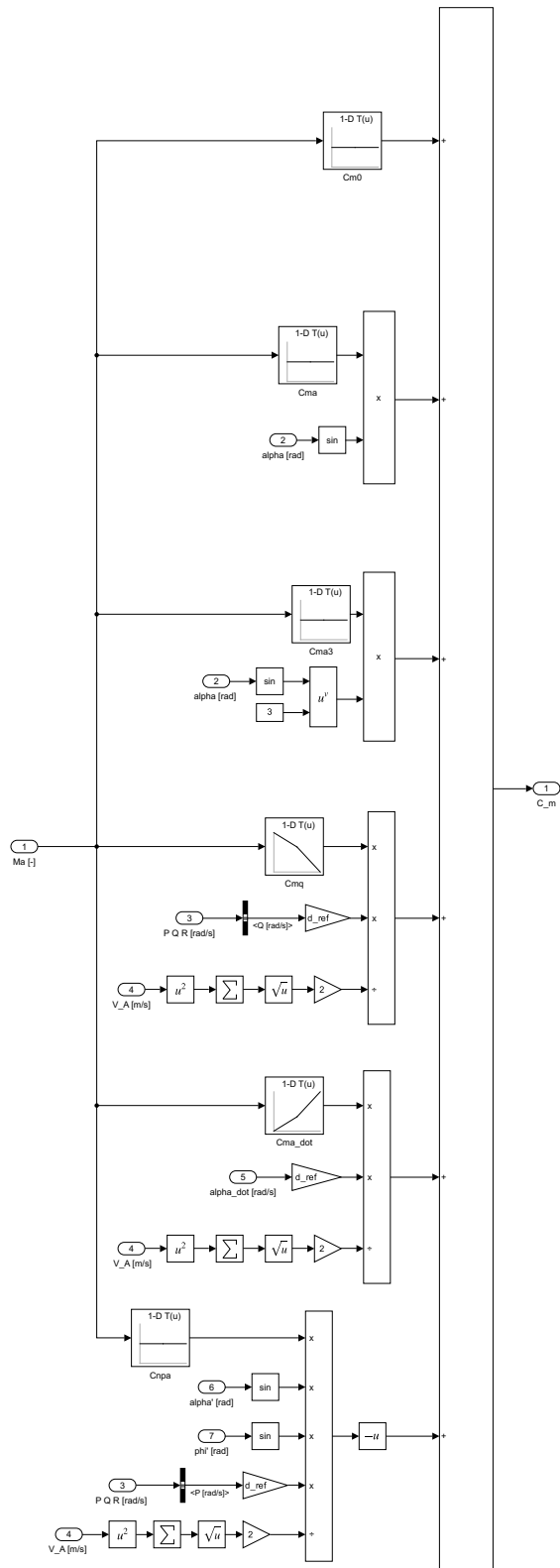


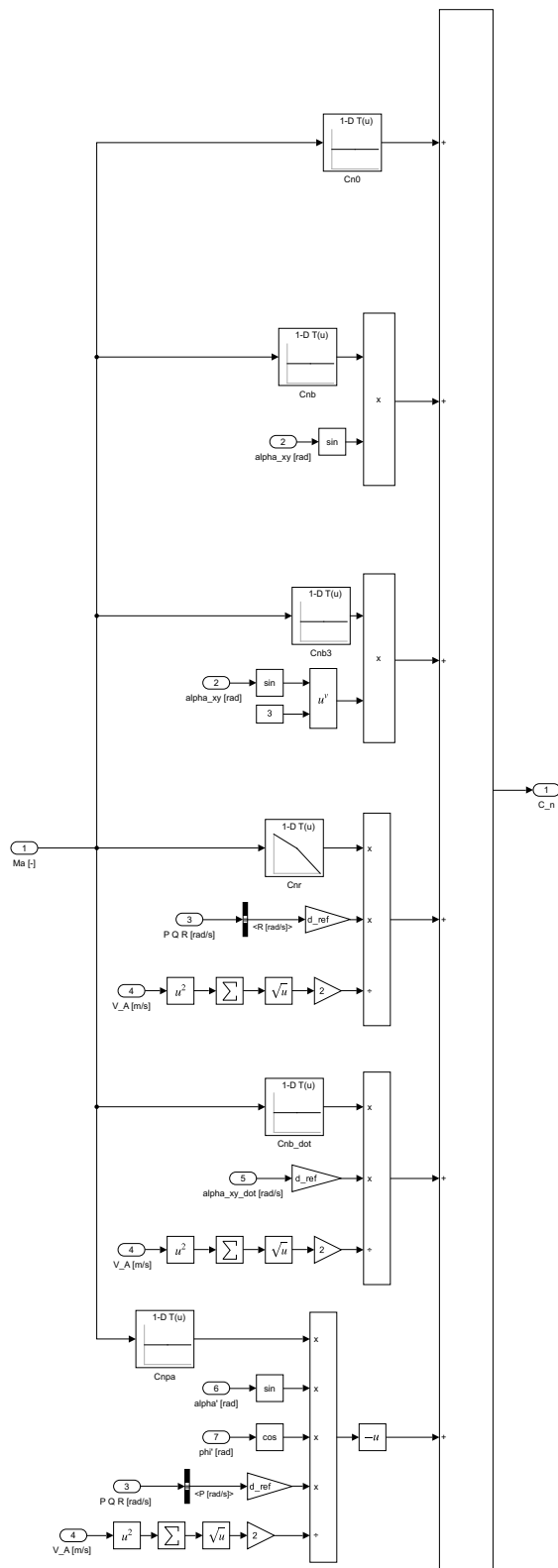


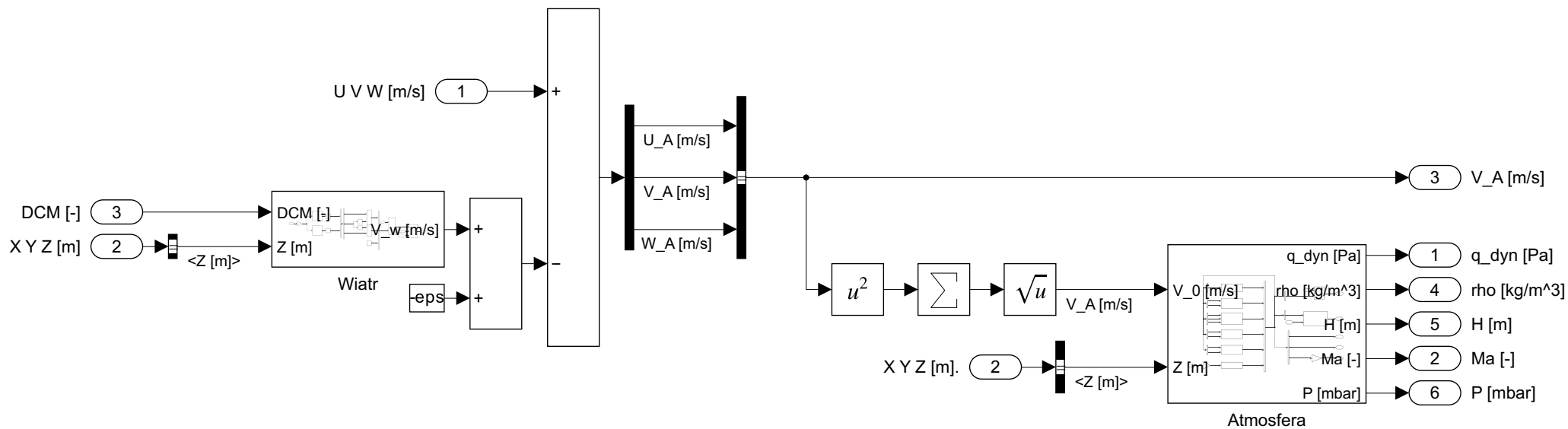
$C_I\ C_m\ C_n$
 $C_I\ C_m\ C_n$ [-]

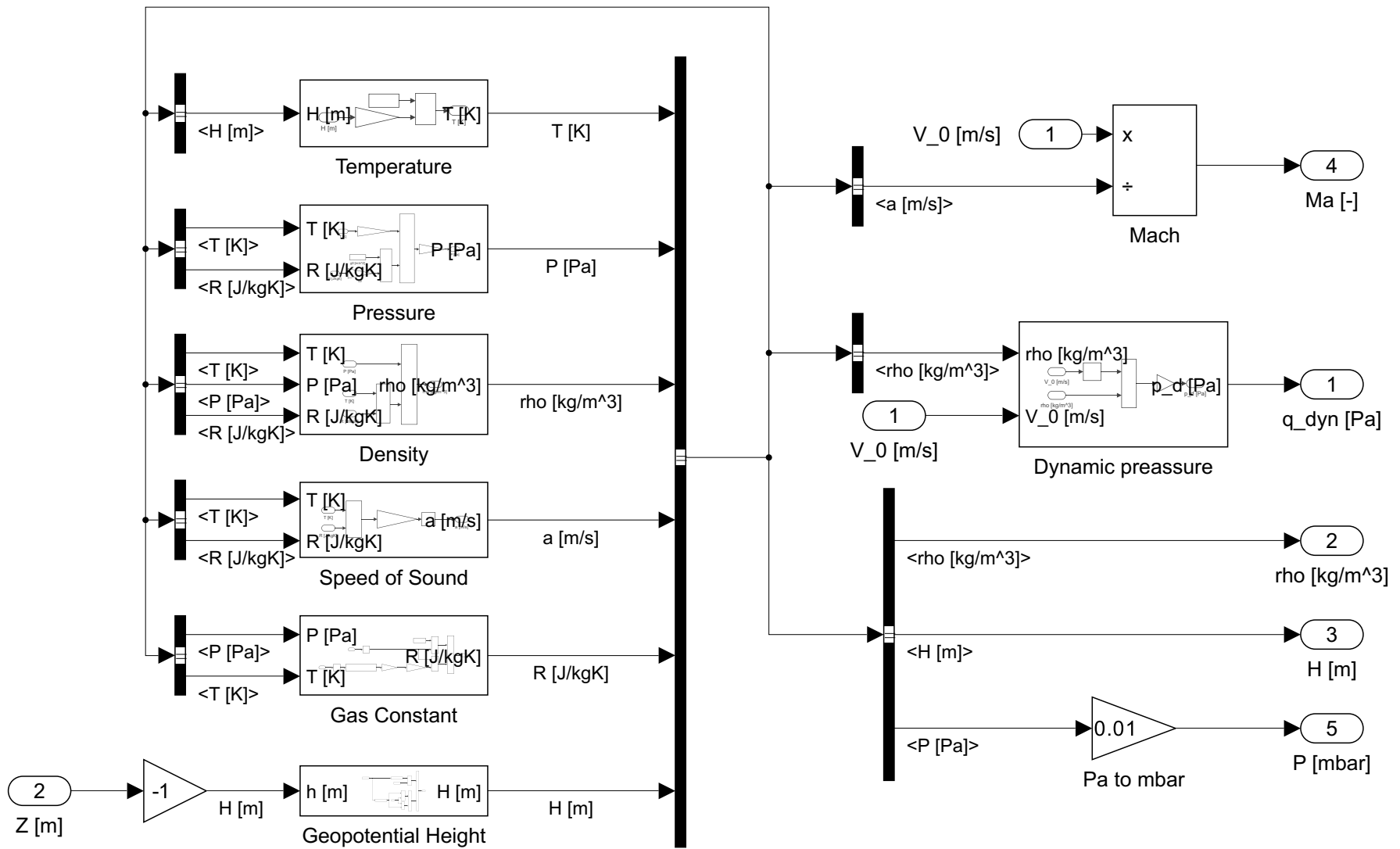


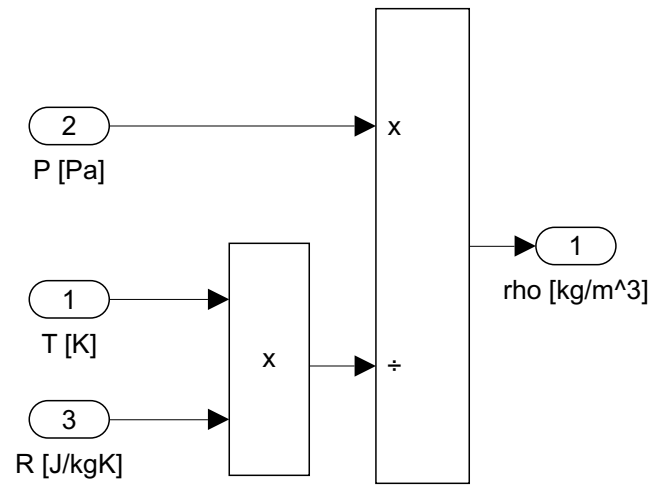


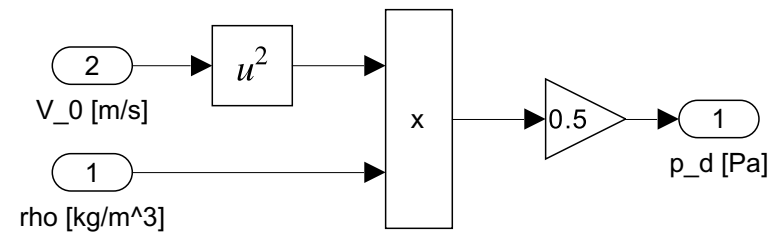


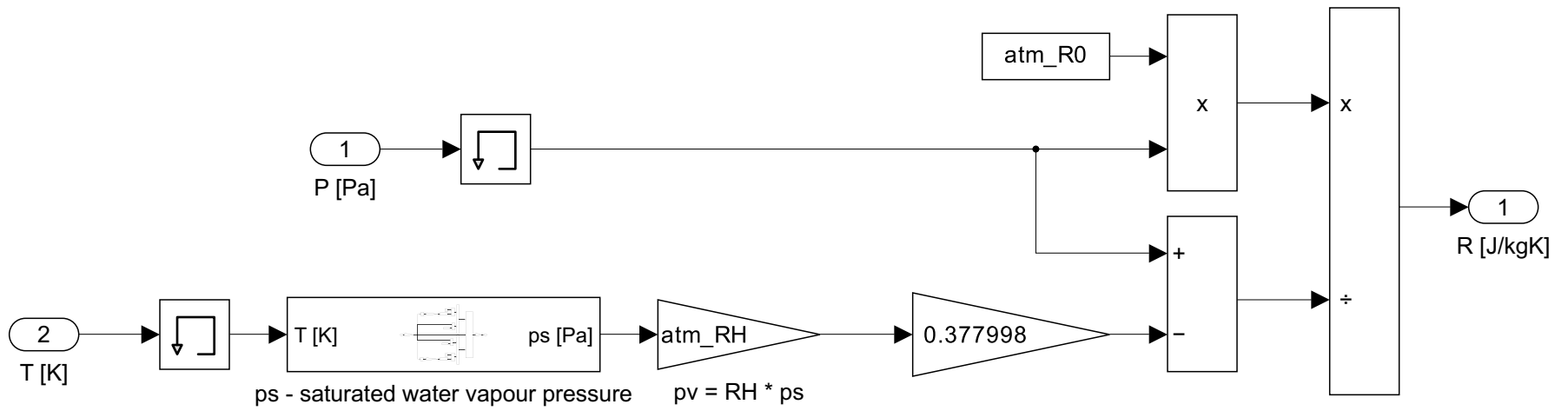


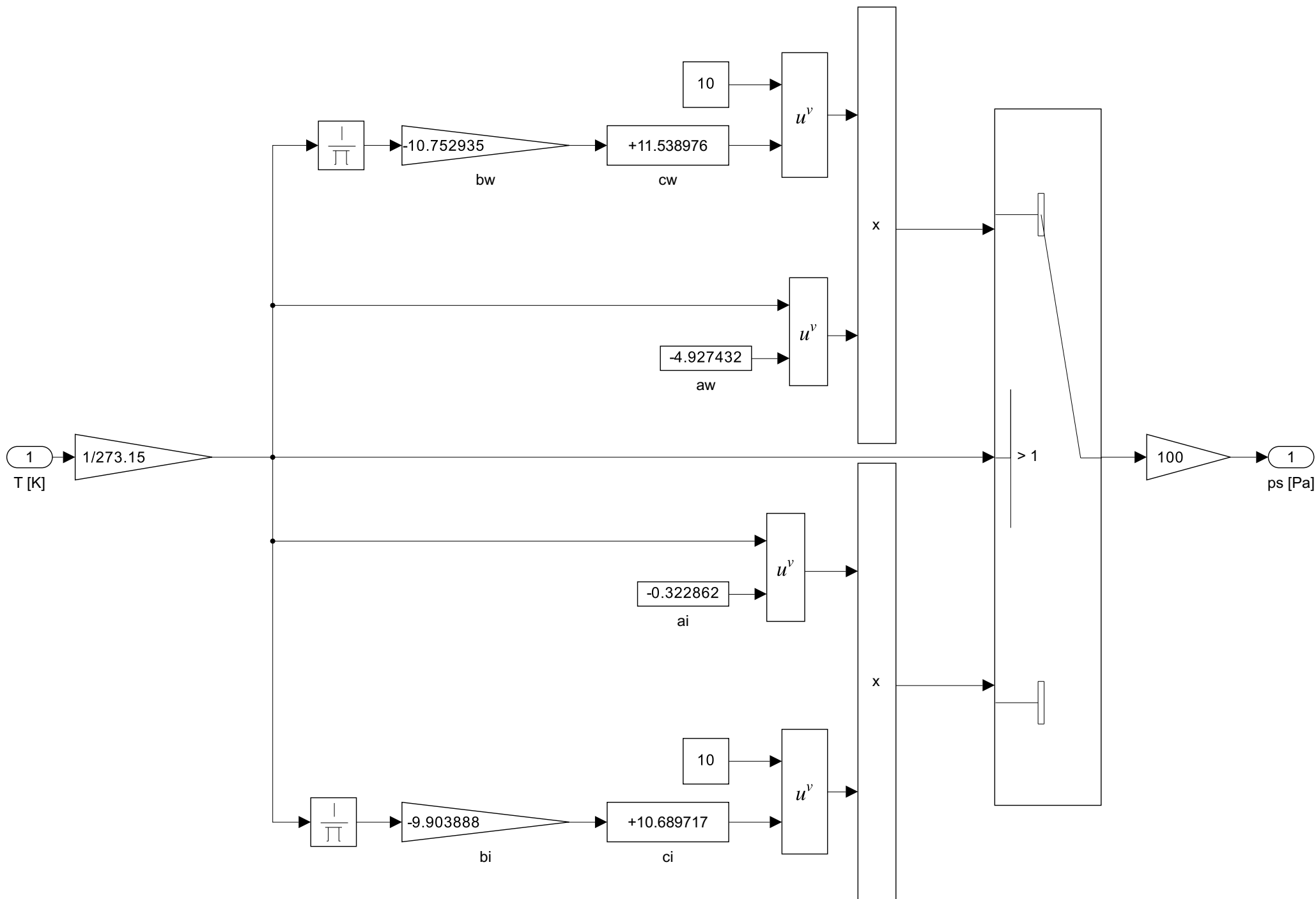


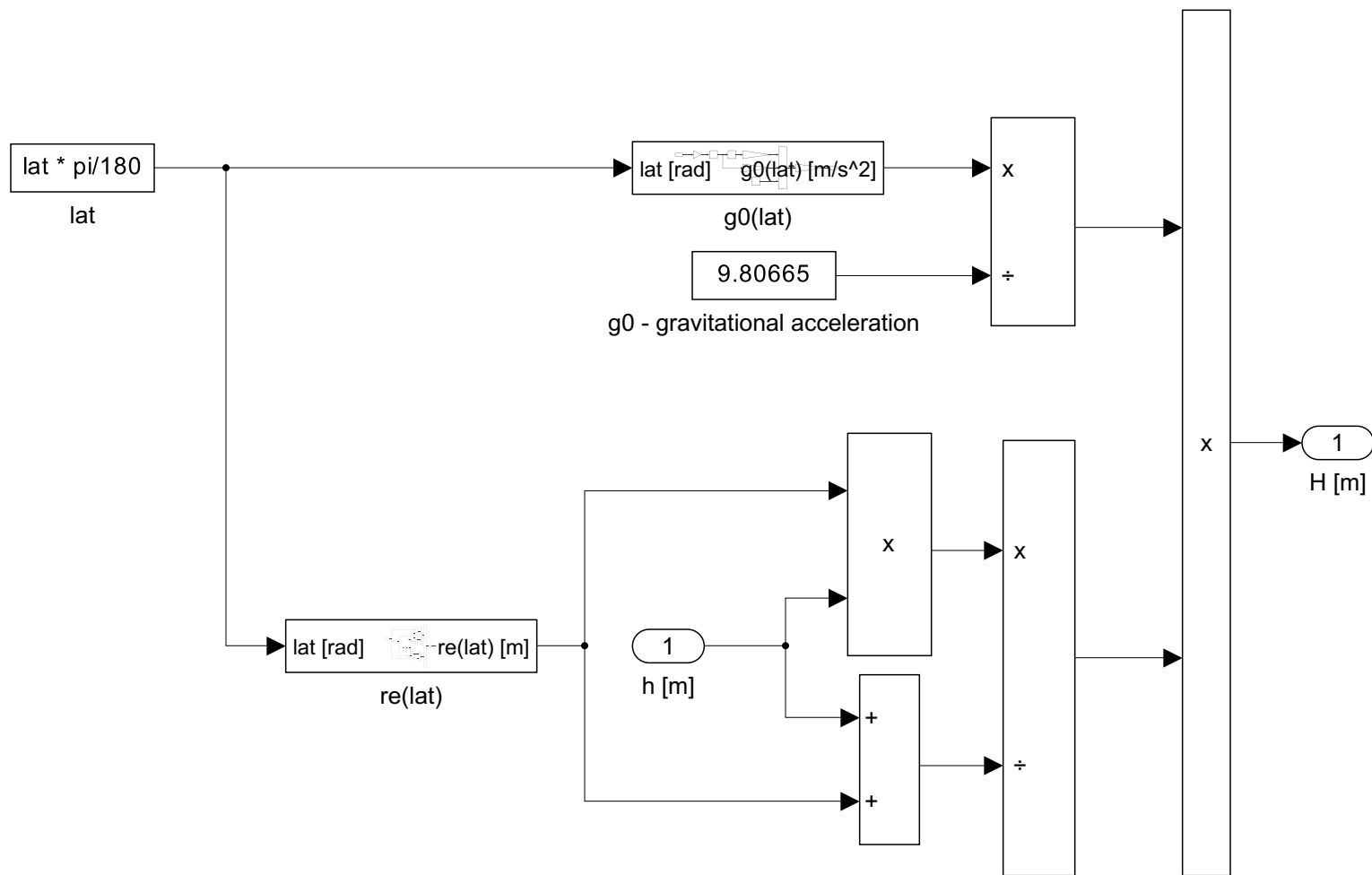


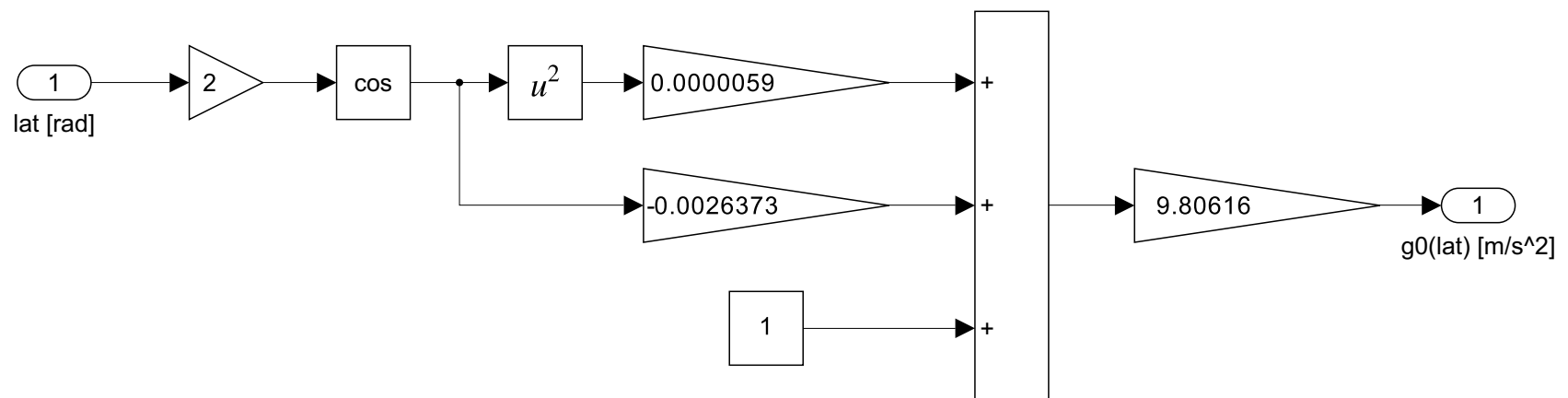












6378137
a - equatorial radius

6356752.3
c - polar radius

1
lat [rad]

tan

u^2

4

u^v

4

u^v

x

u^v

x

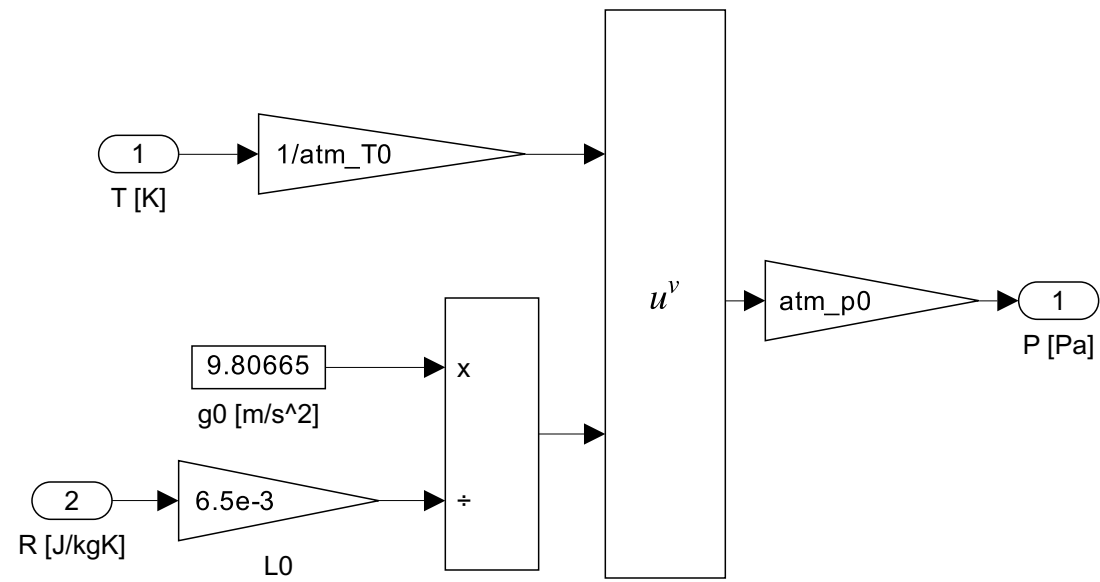
x

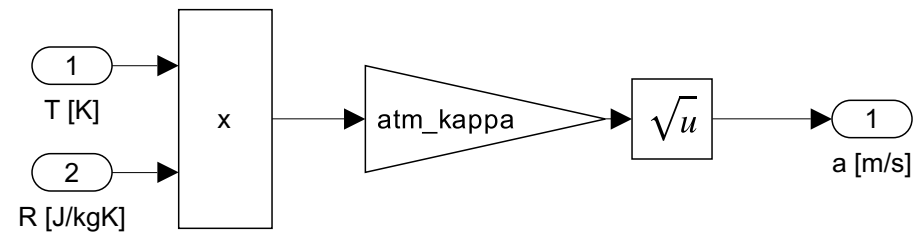
u^2

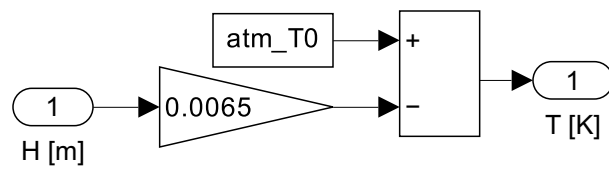
$\sqrt{u}$

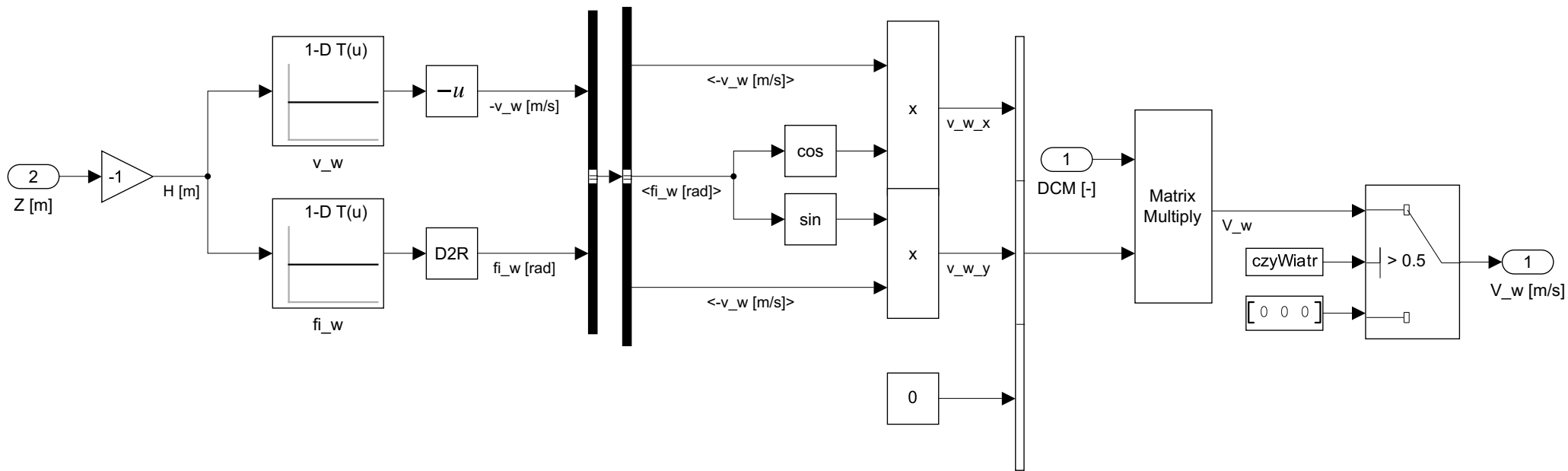
1
re(lat) [m]

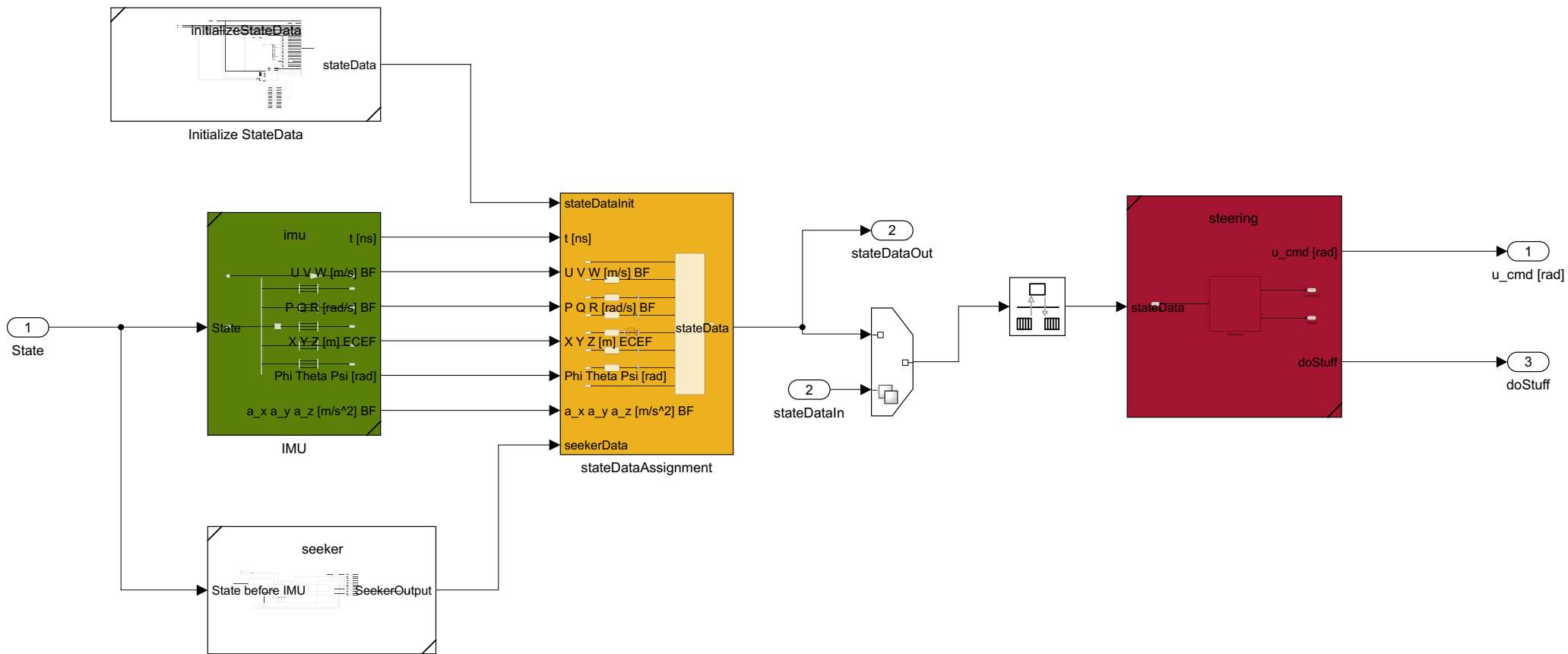


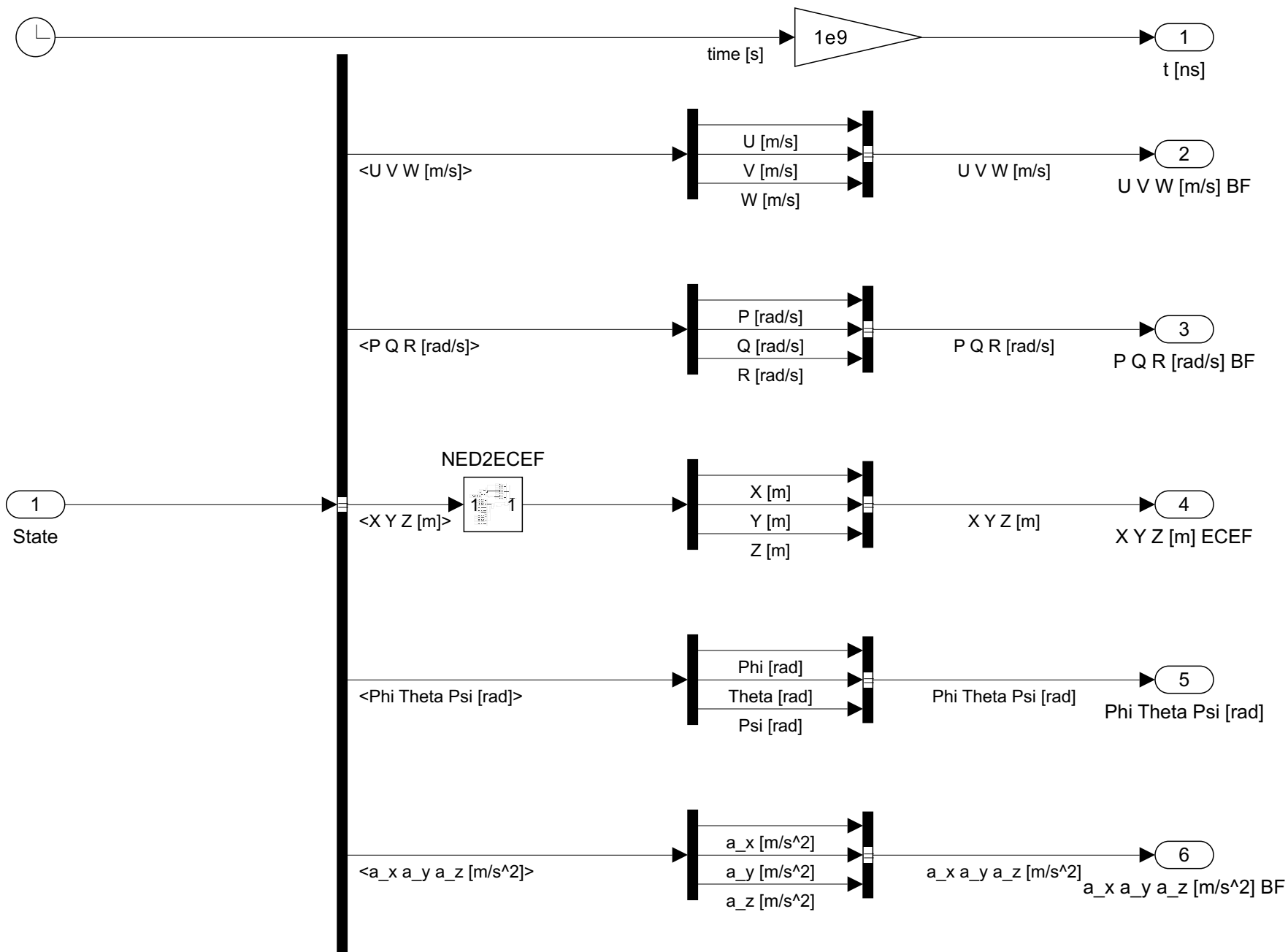


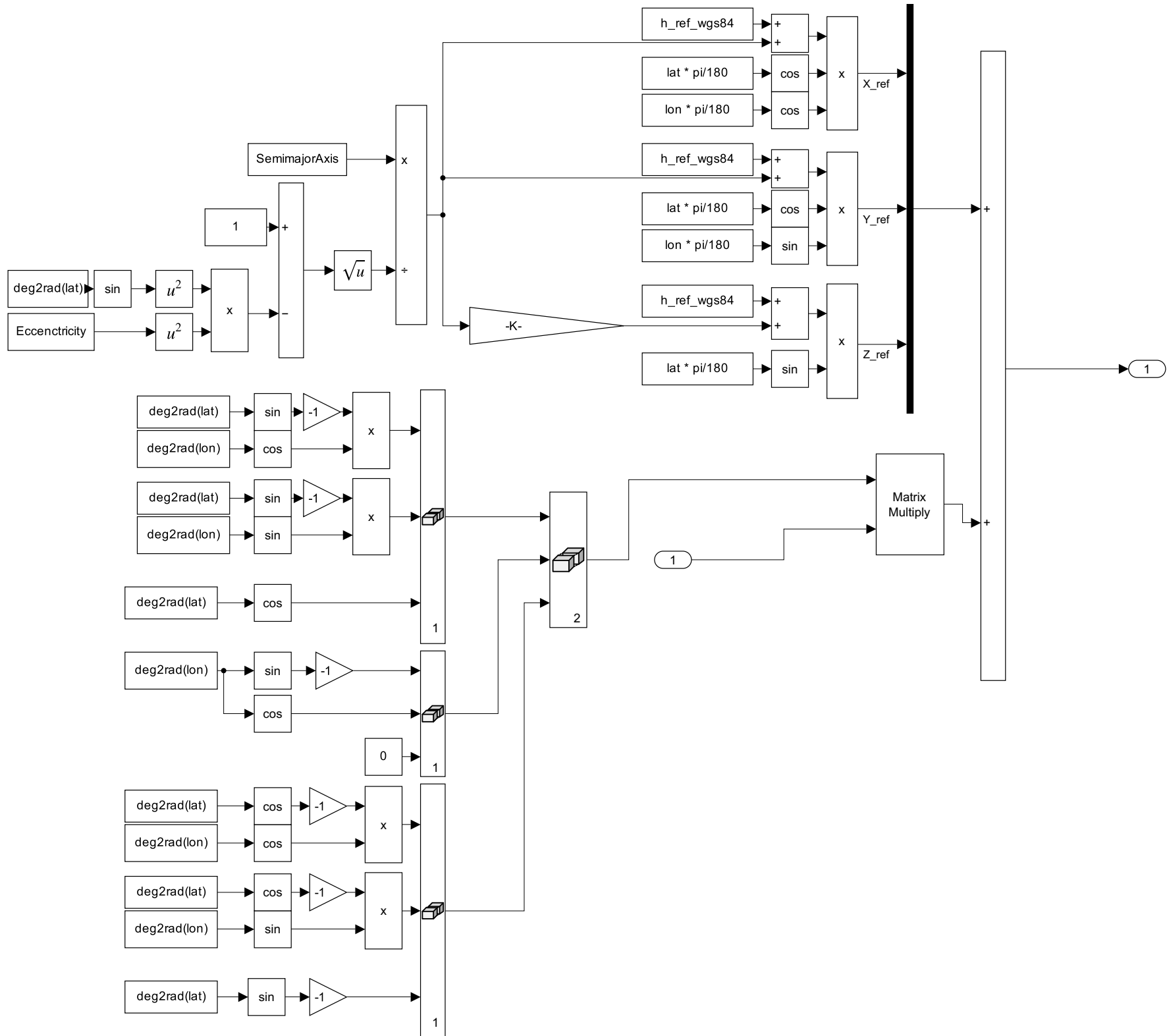


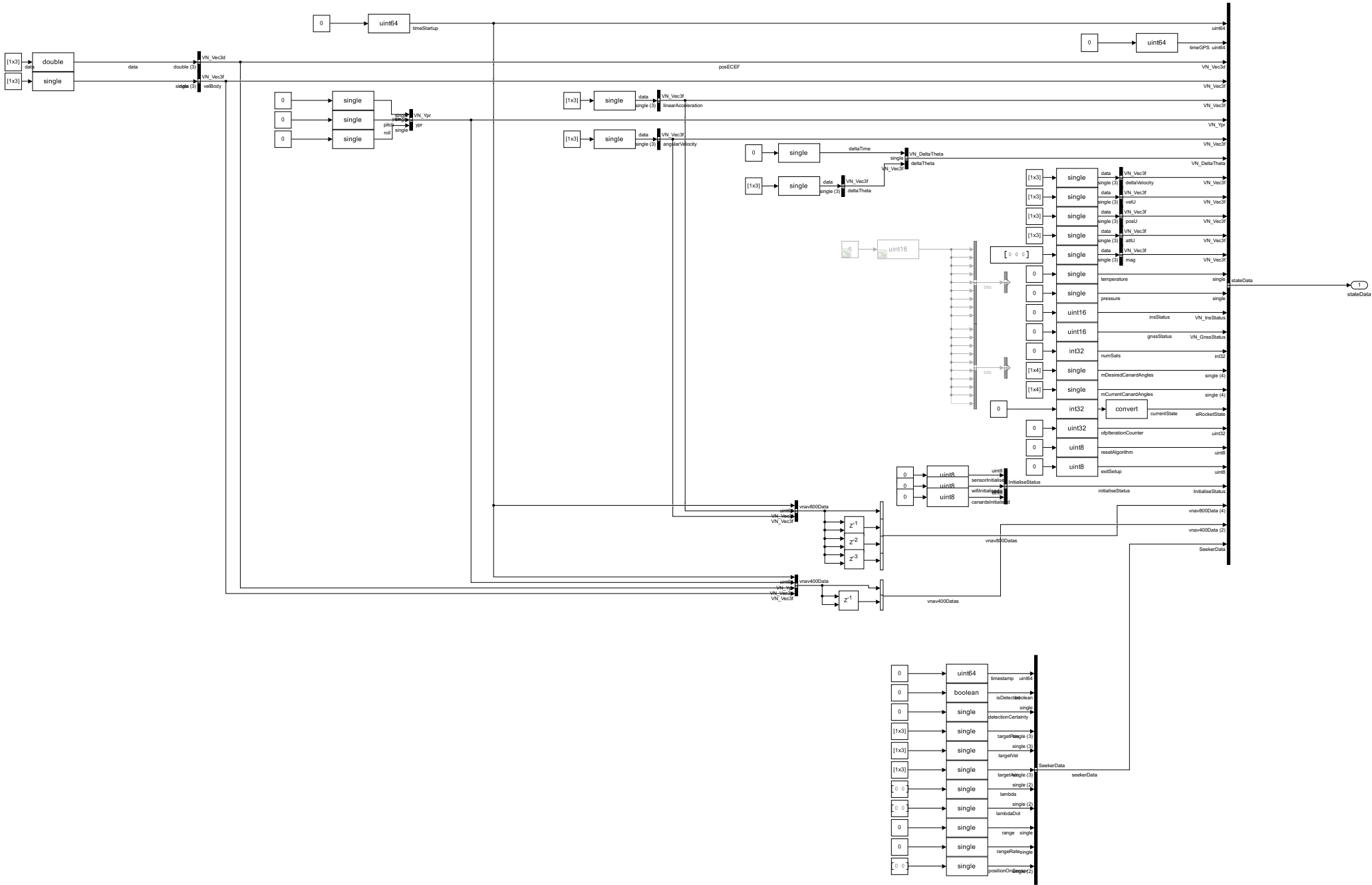


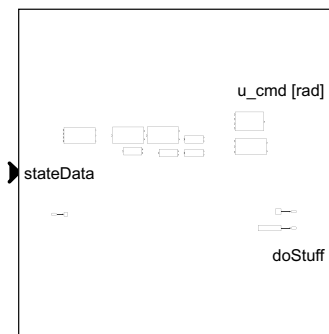




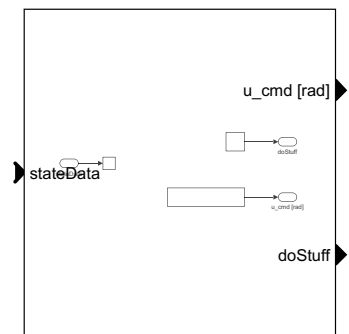




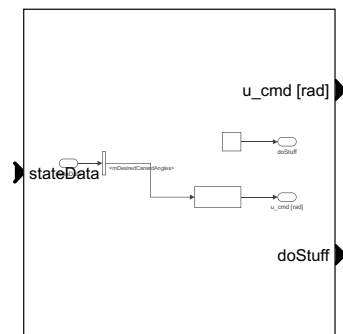




Archive



doNothing

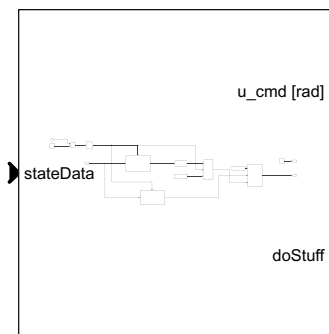


extractDesiredCanardAngles

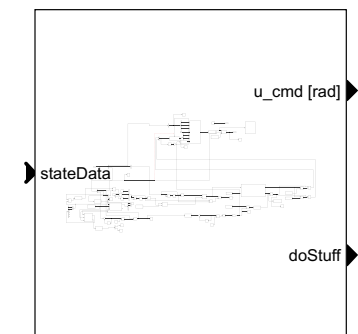
1
u_cmd [rad]

1
stateData

2
doStuff

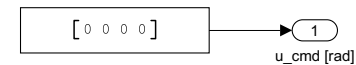
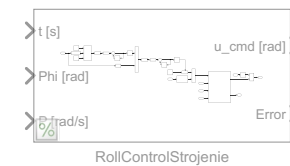
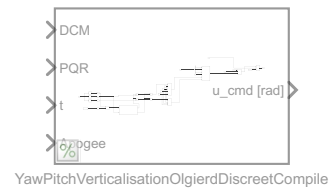
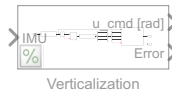
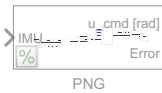
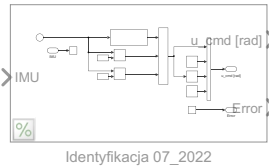
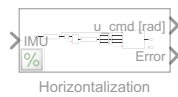
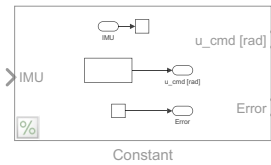
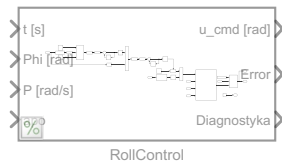


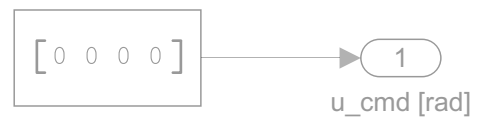
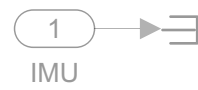
Fokyo Drift

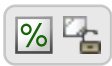
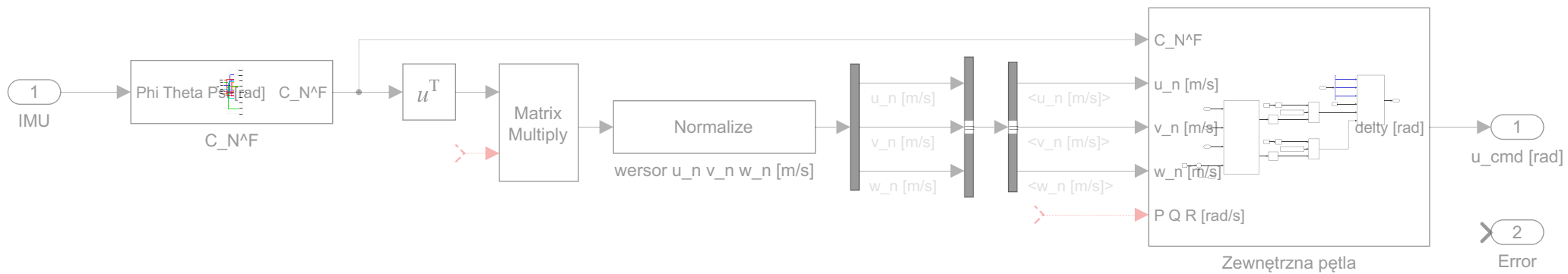


Pure pursuit









1
Phi Theta Psi [rad]



cos

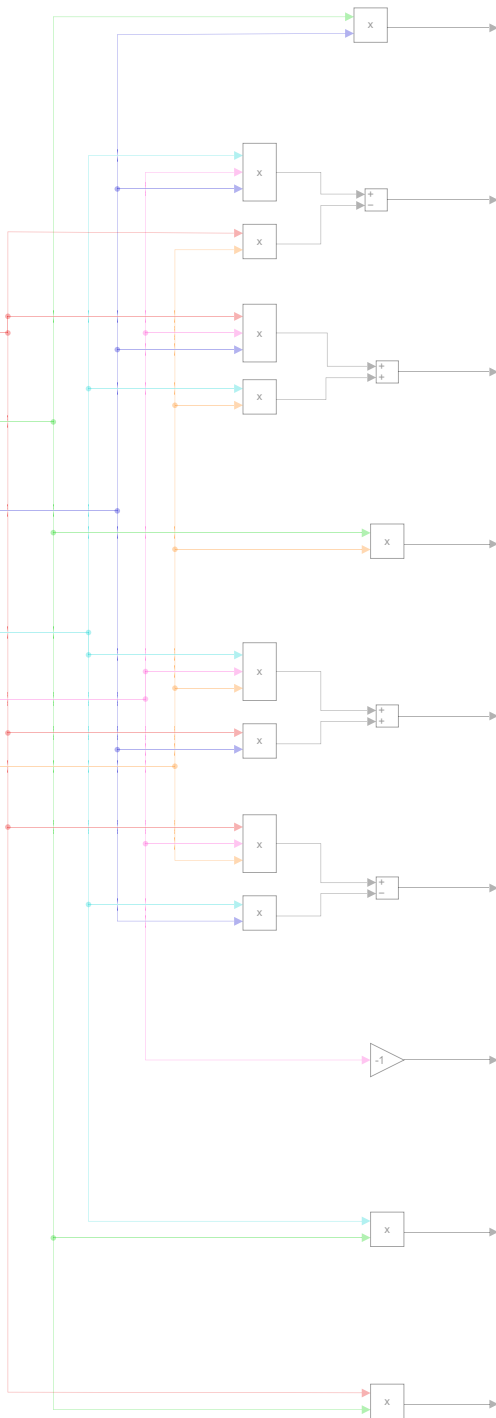
cos

cos

sin

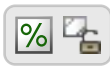
sin

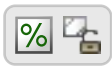
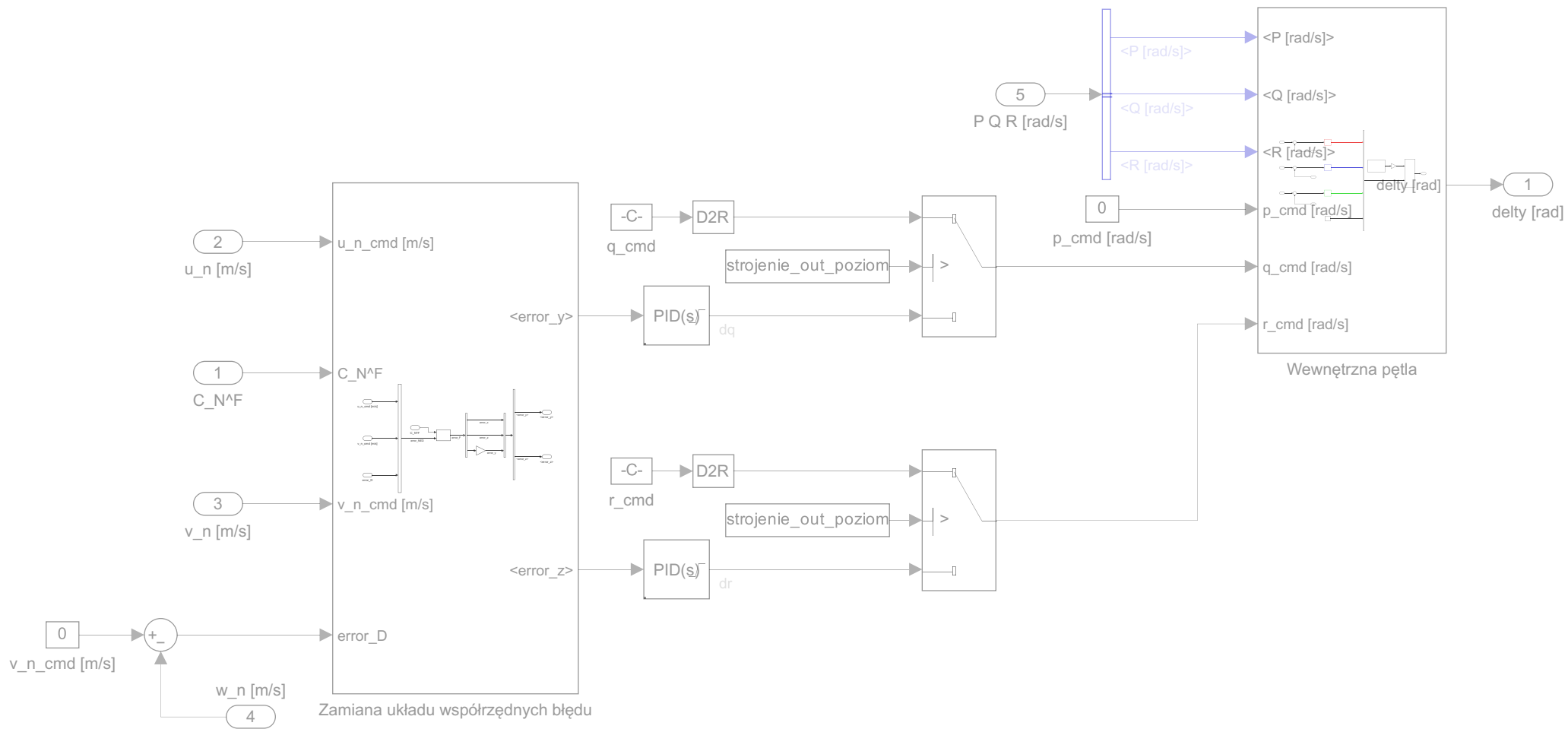
sin

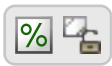
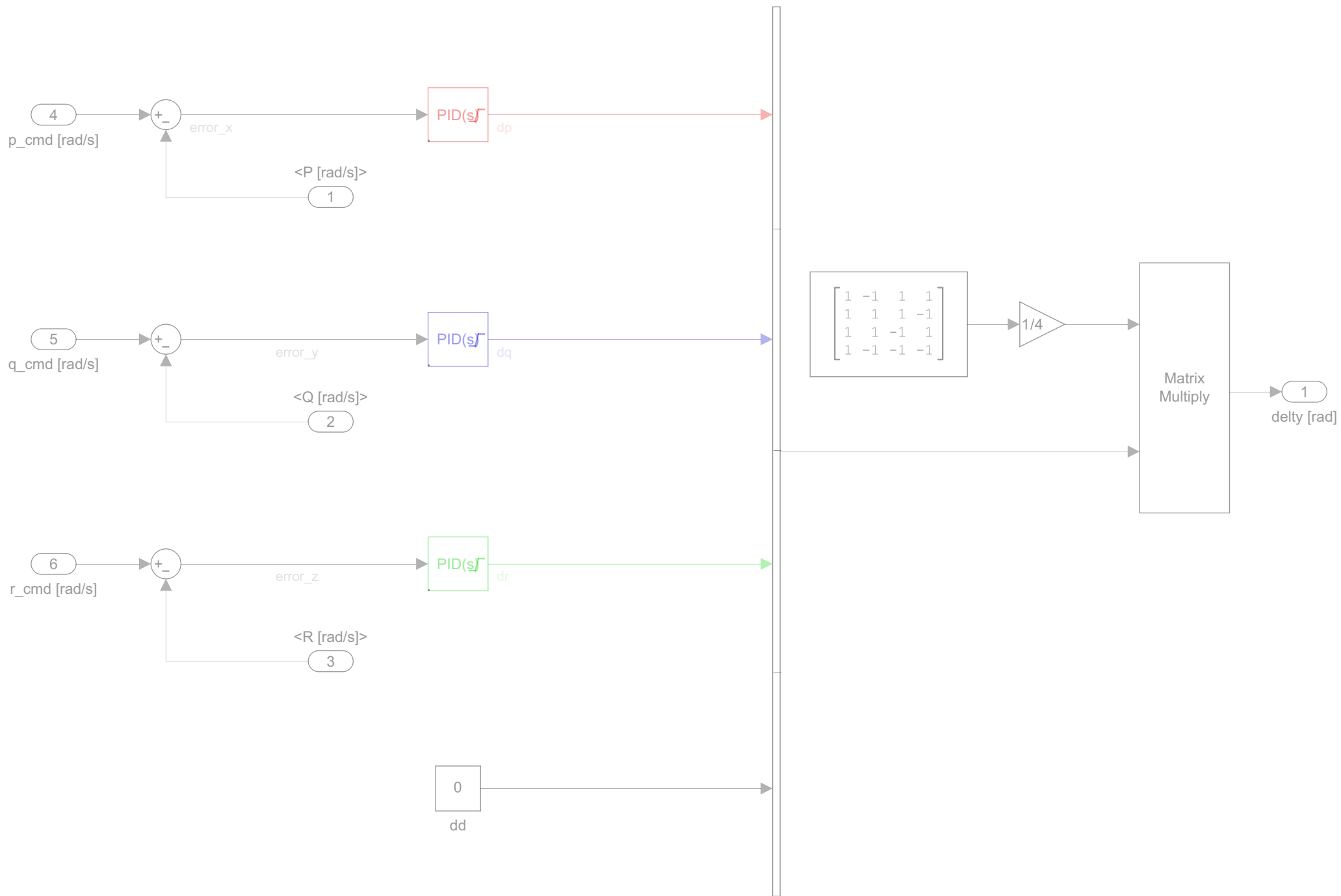


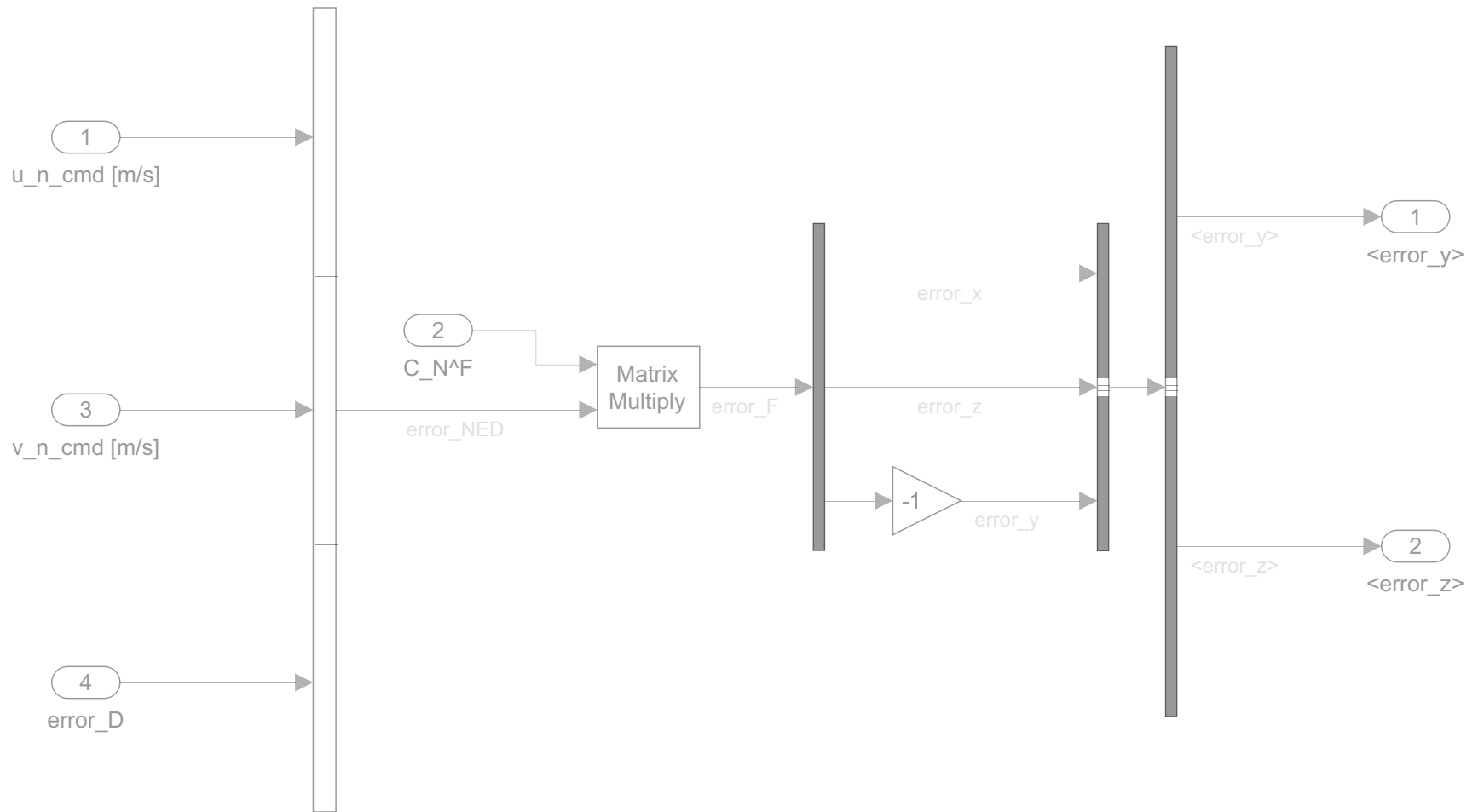
Reshape

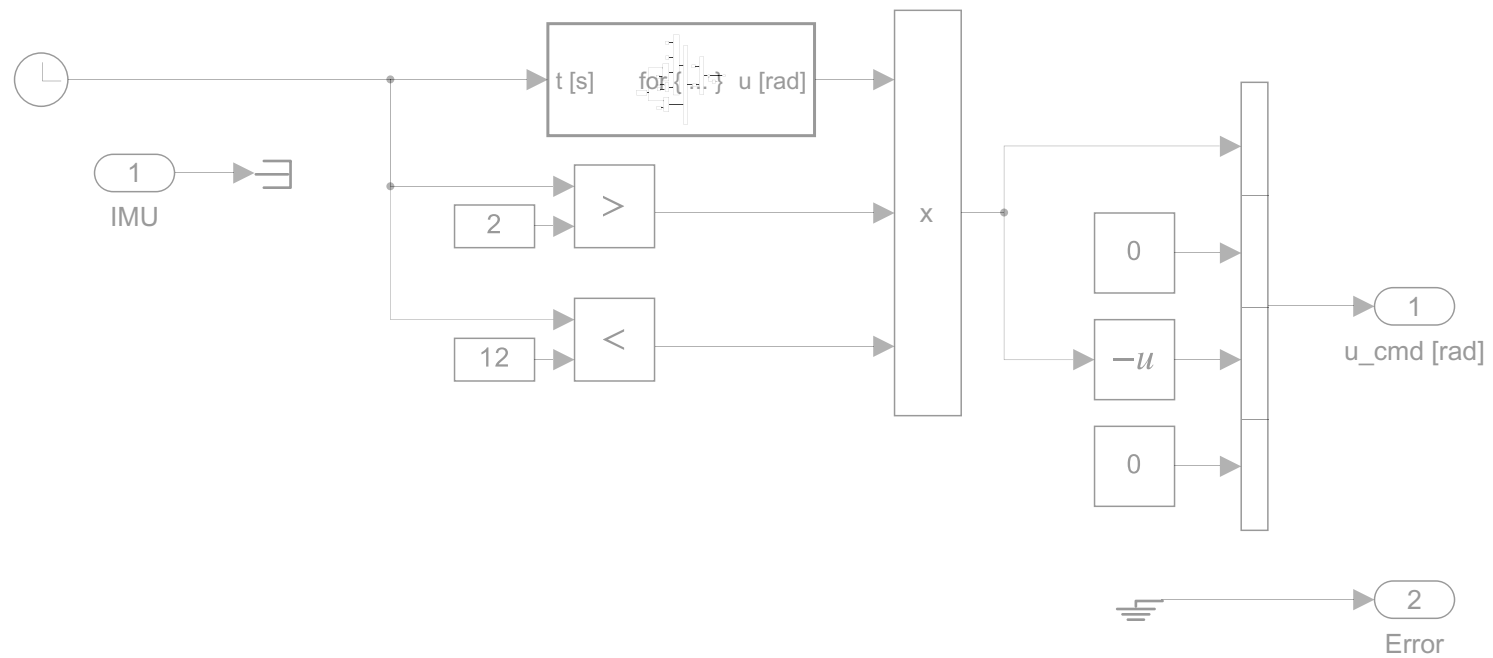
1
C_N*F

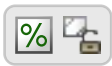
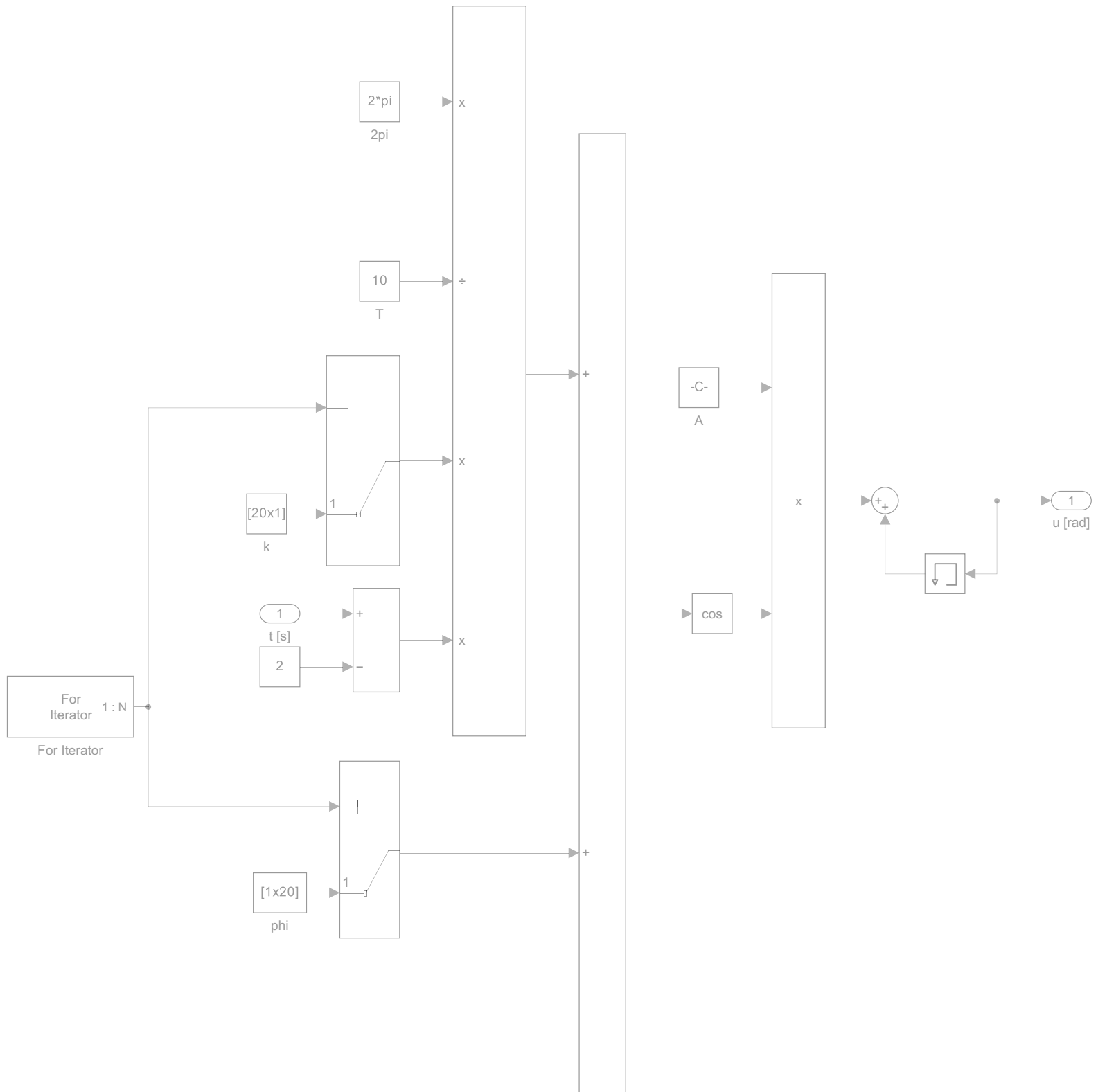


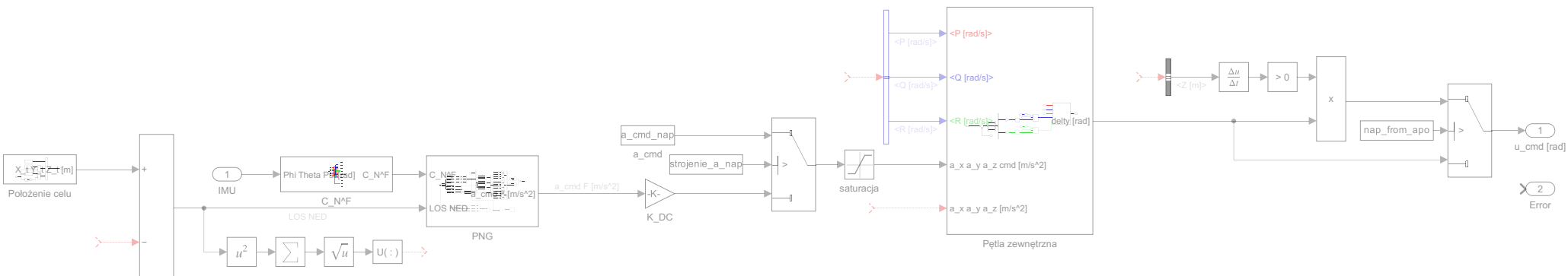
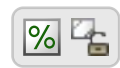












1
Phi Theta Psi [rad]



cos

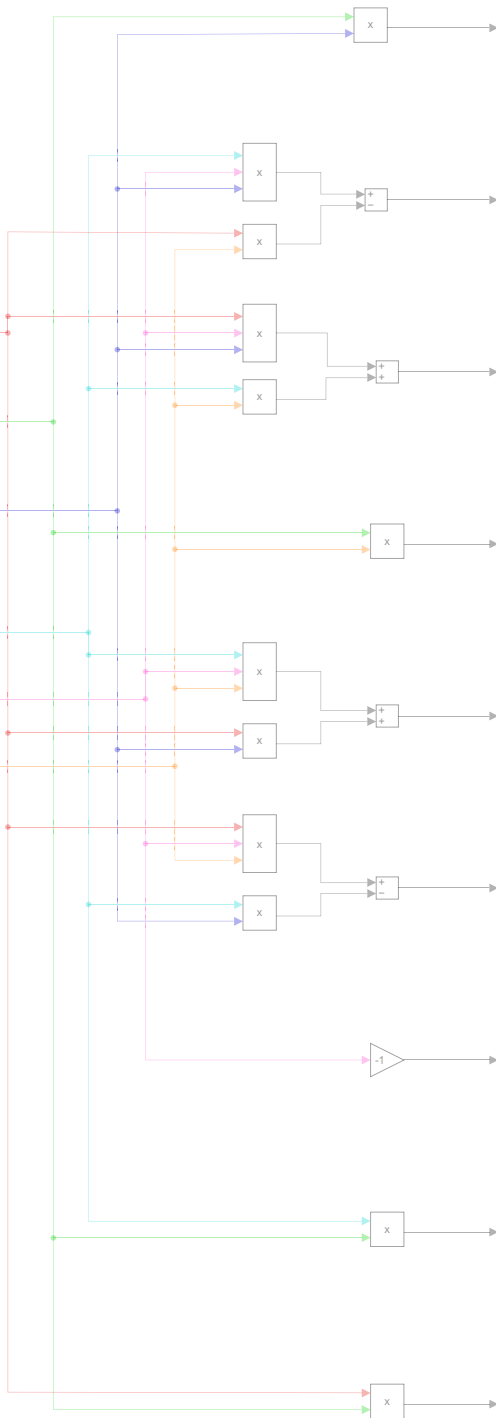
cos

cos

sin

sin

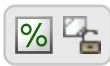
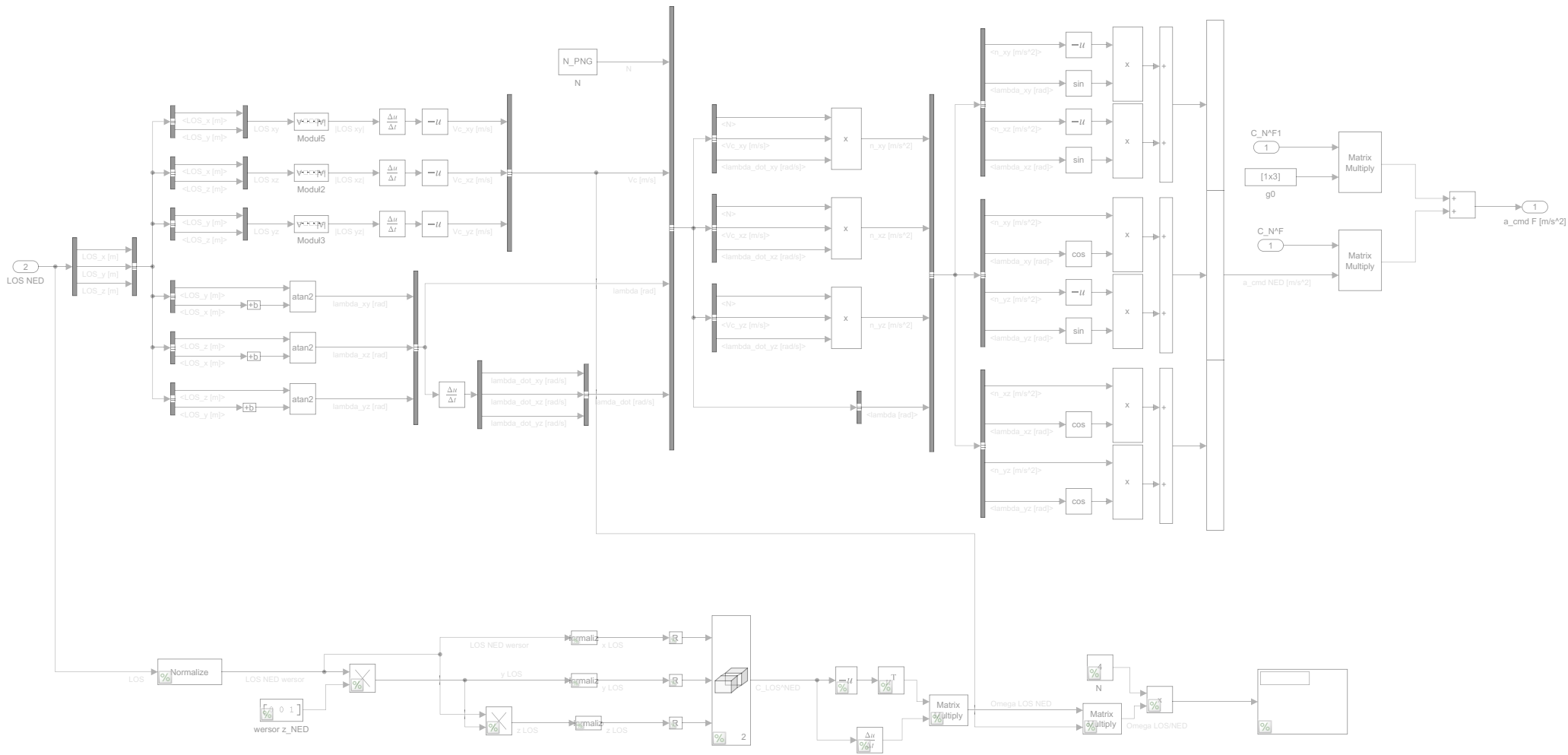
sin

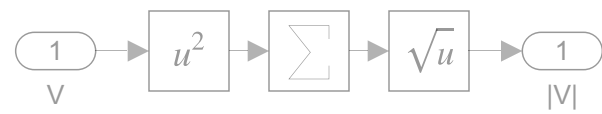


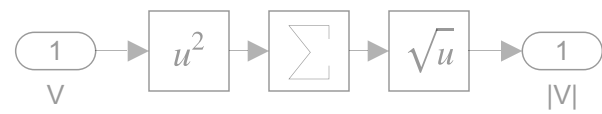
Reshape

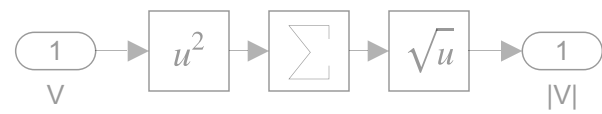
1
C_N^F

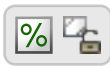
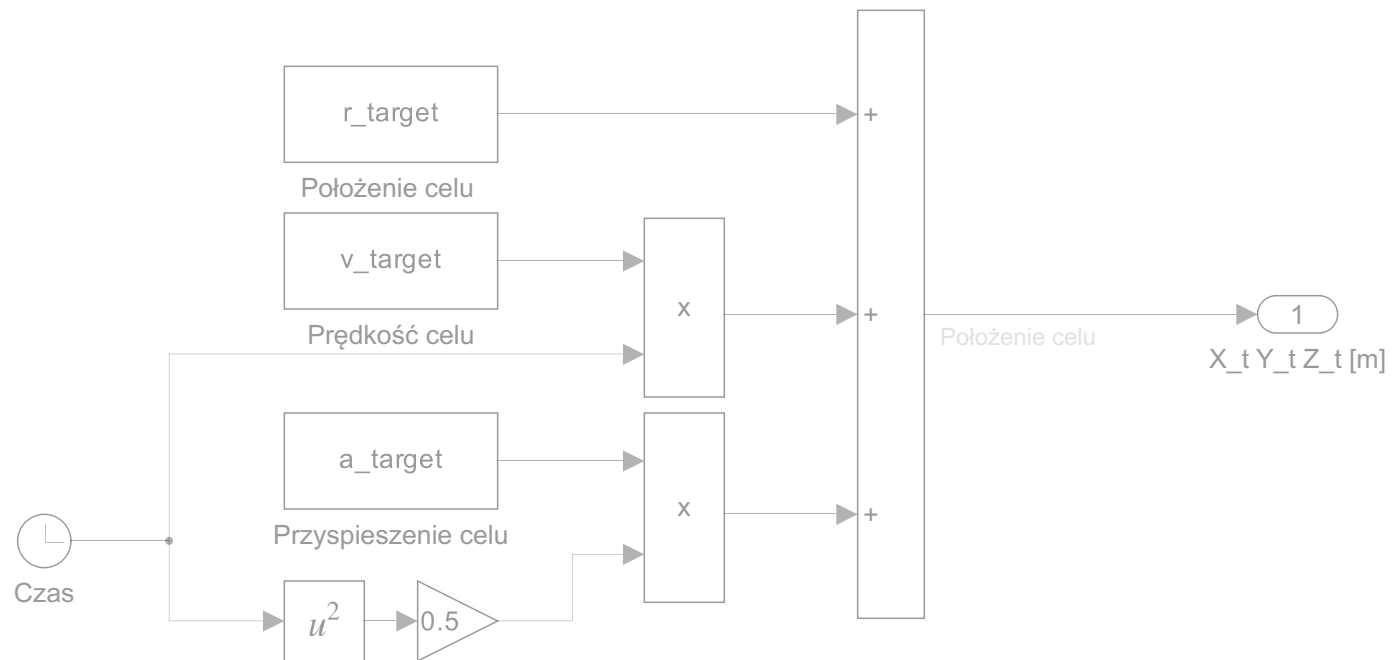


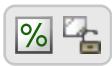
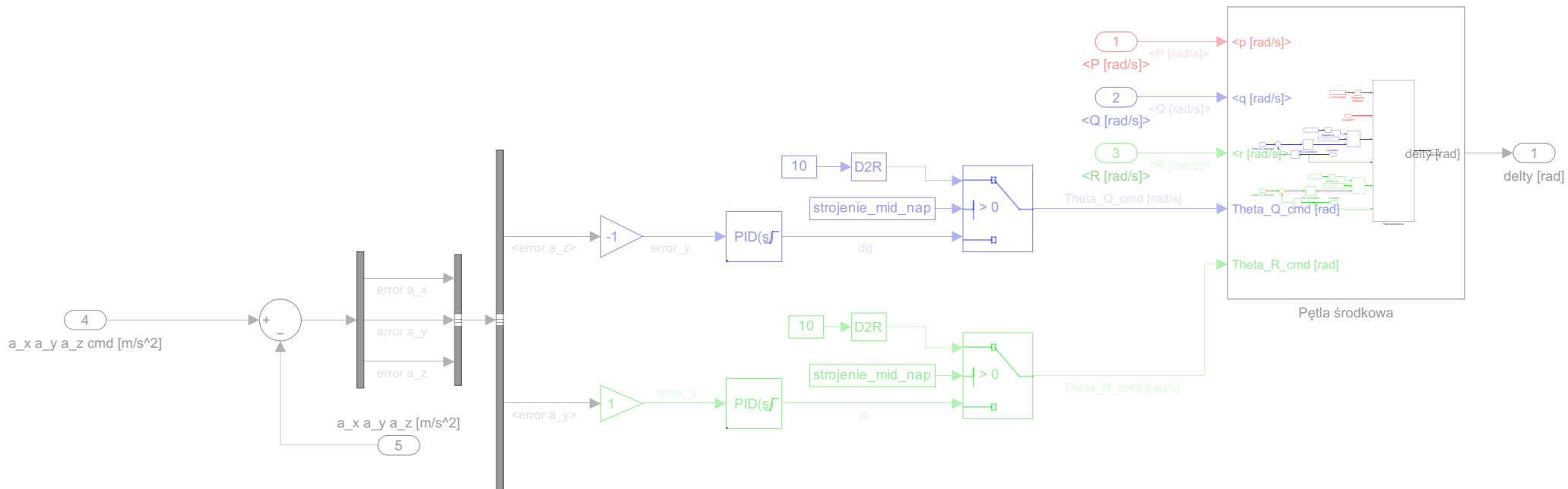


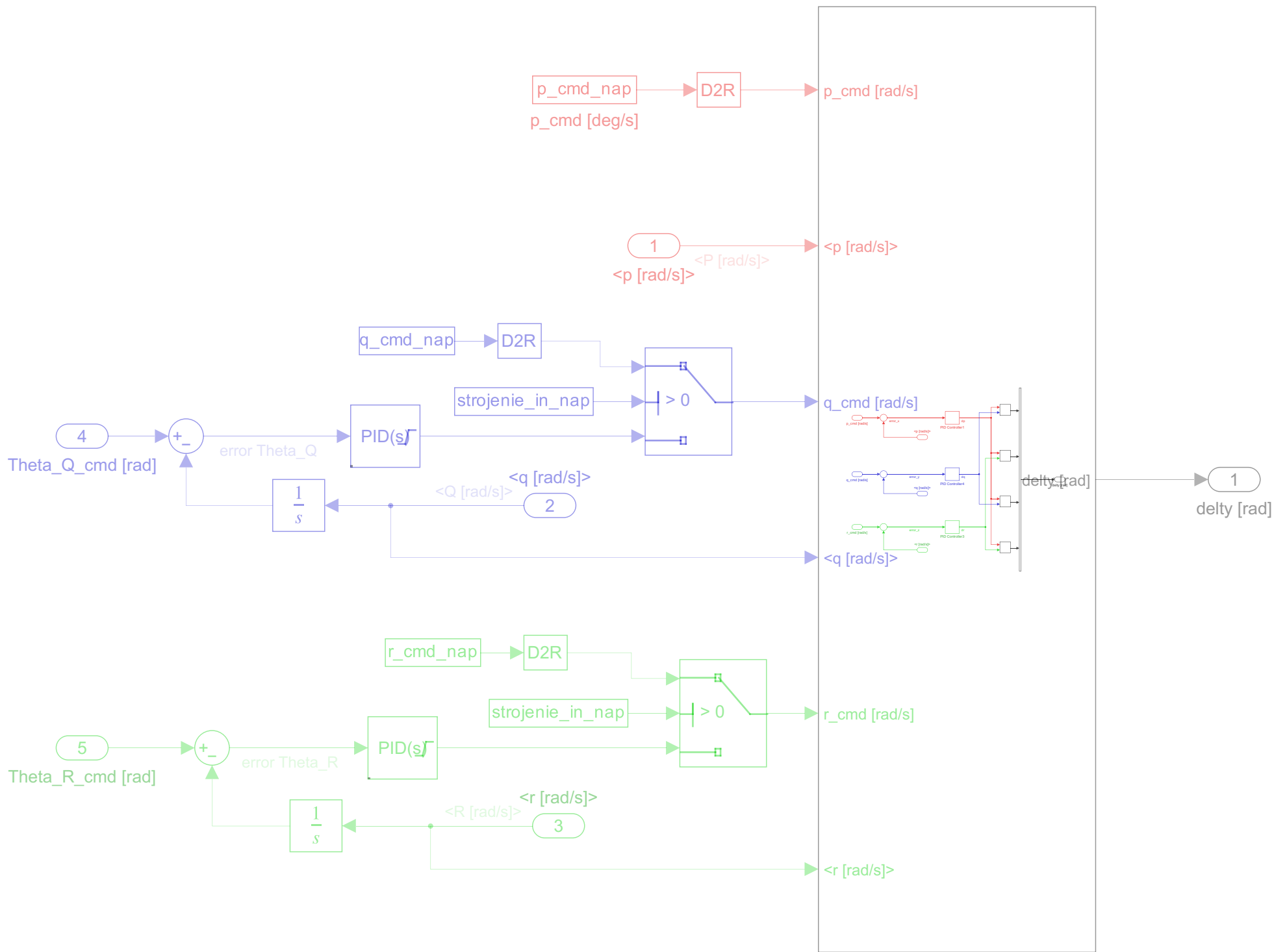




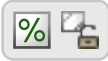


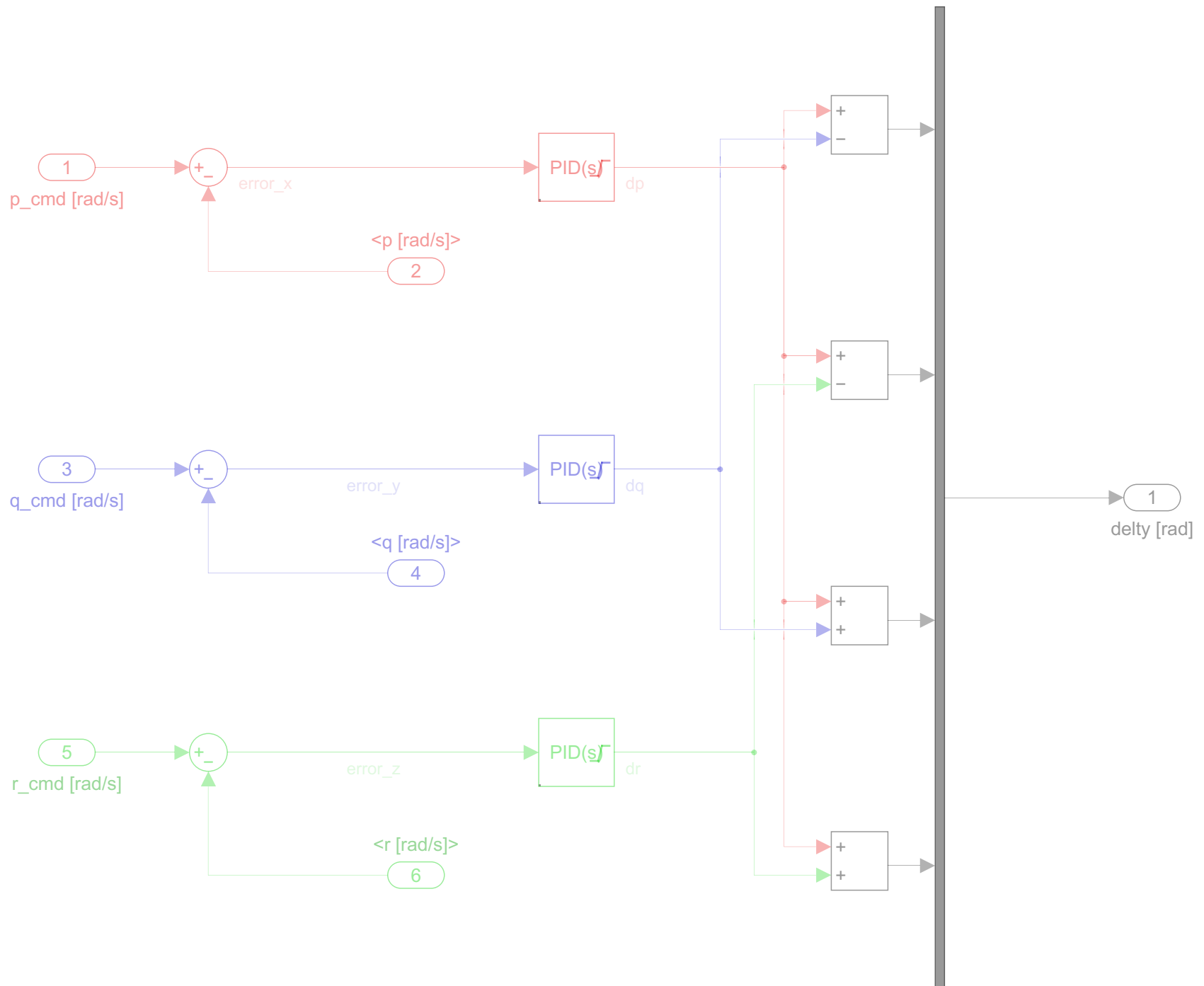


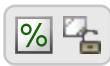
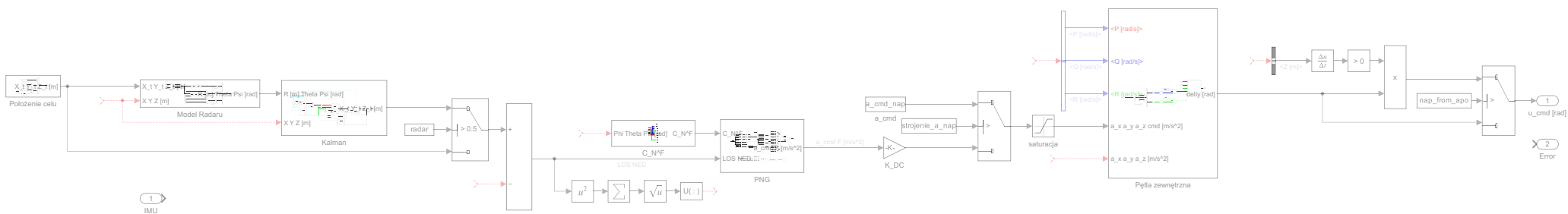




Pętla wewnętrzna







1
Phi Theta Psi [rad]



cos

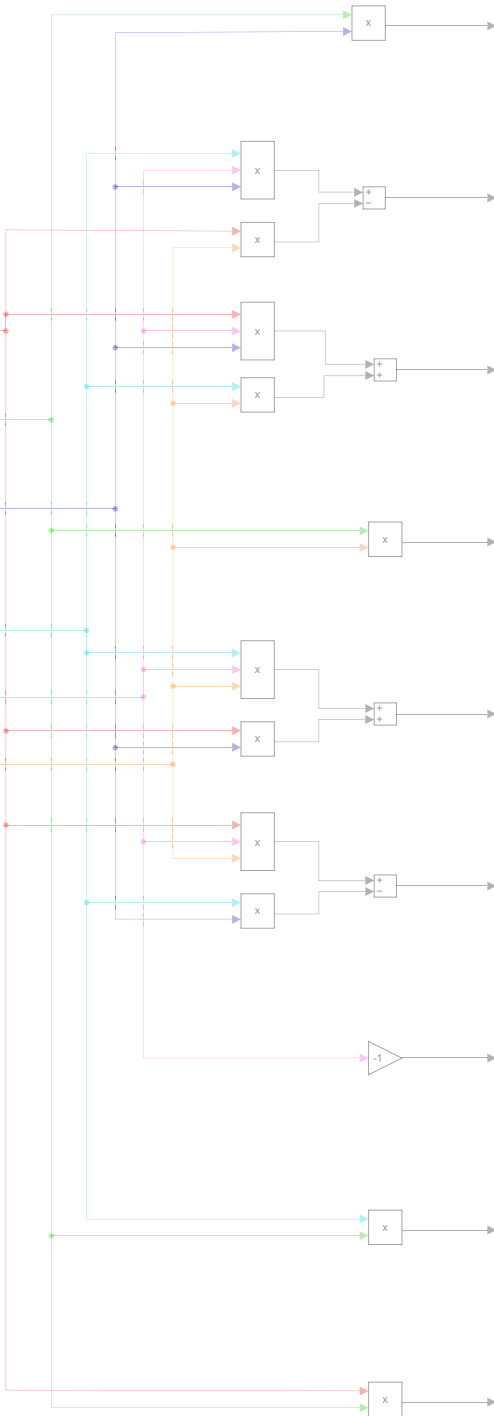
cos

cos

sin

sin

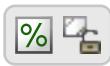
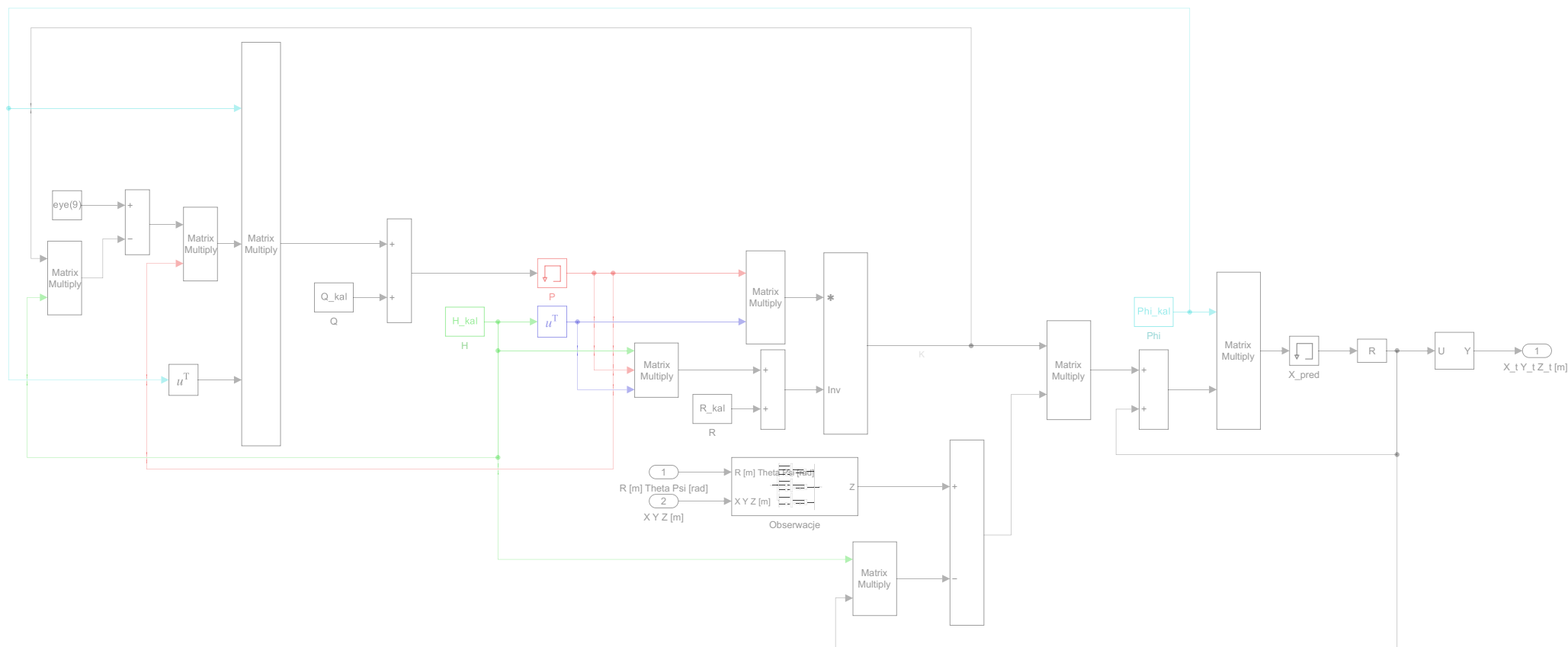
sin

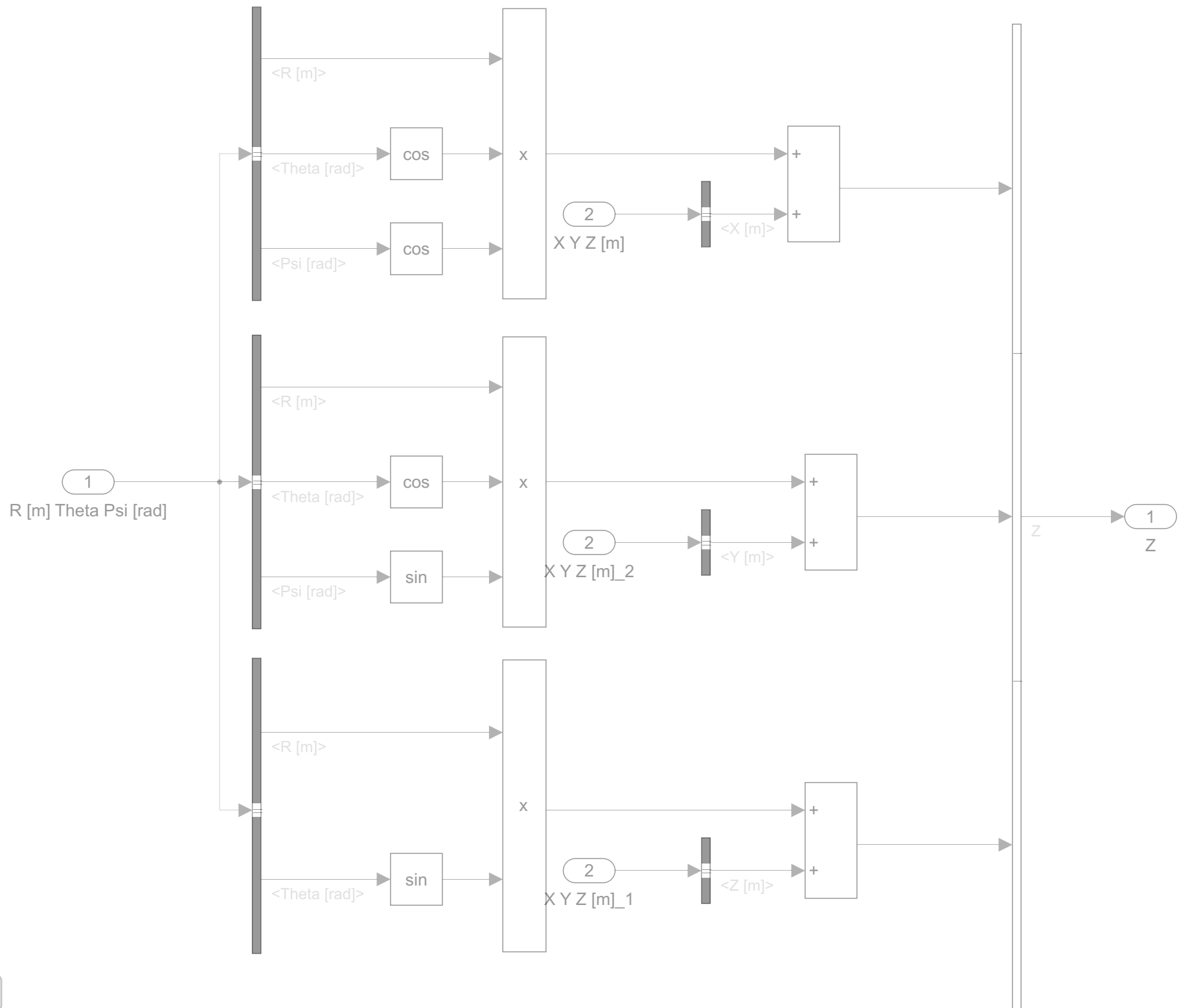


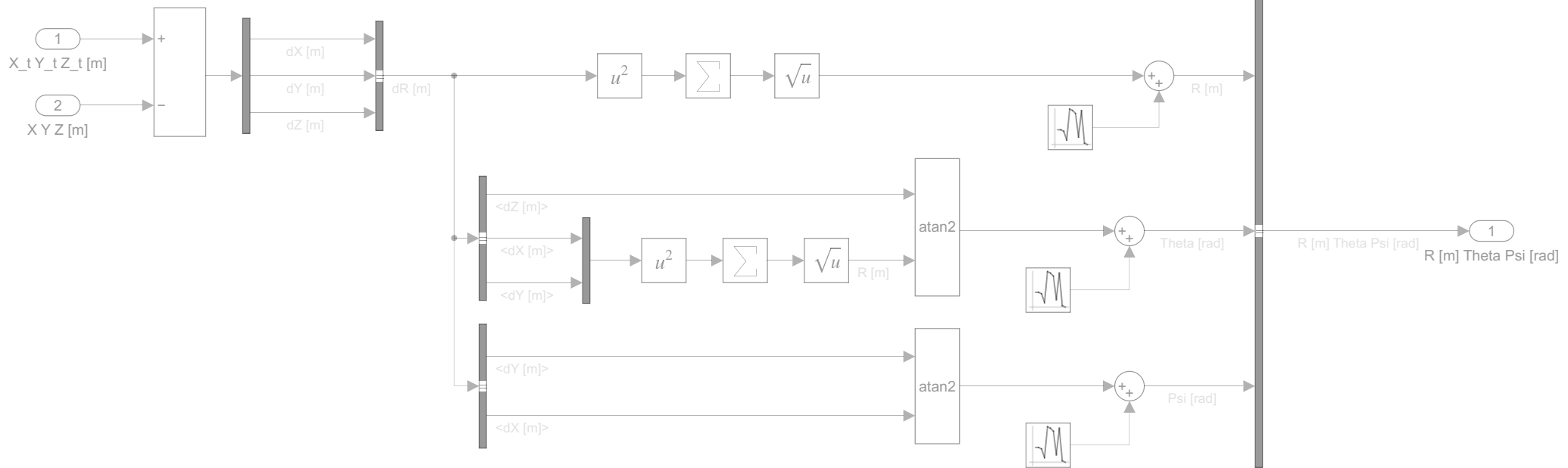
Reshape

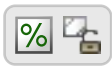
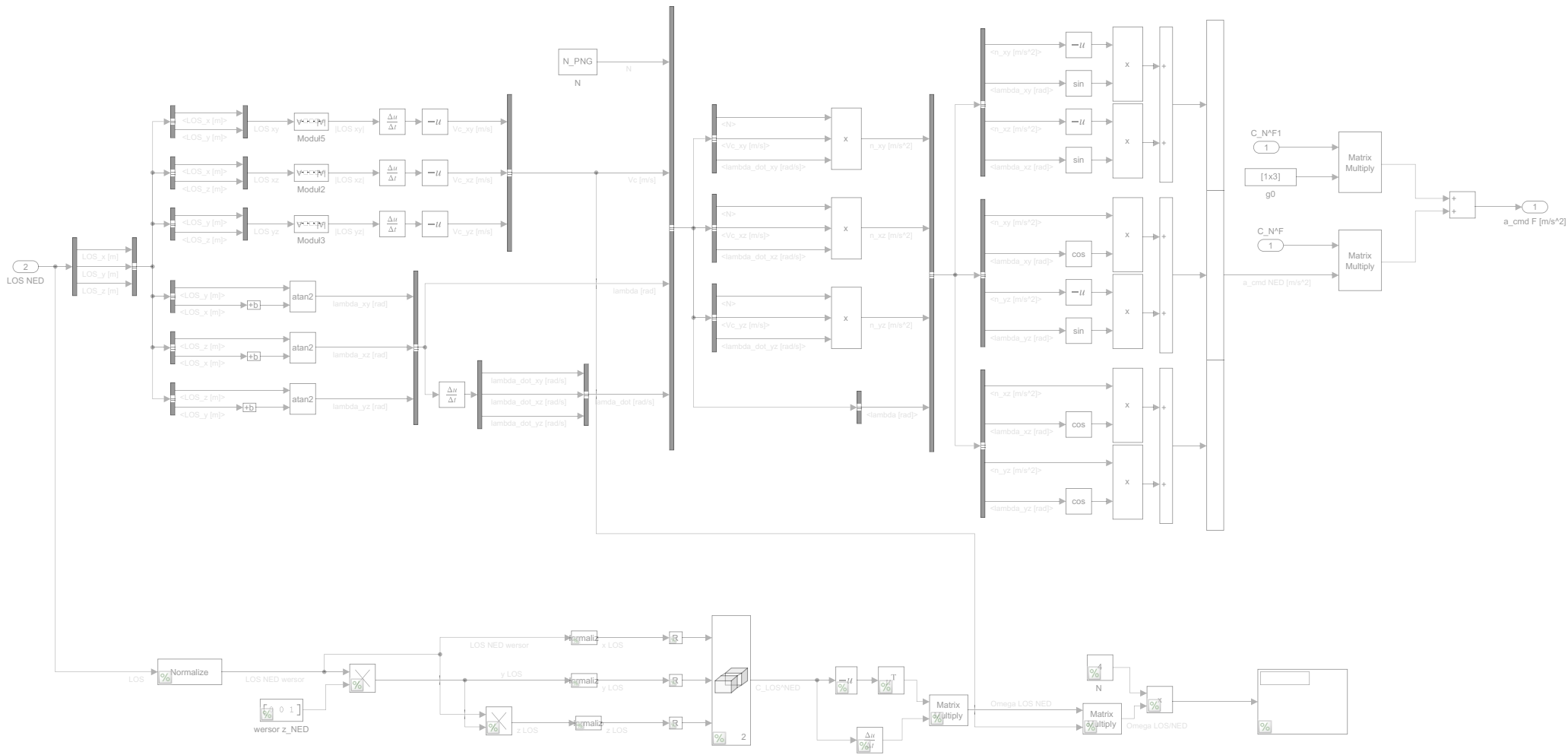
1
C_N^F

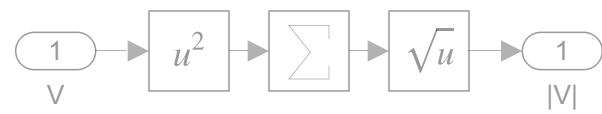


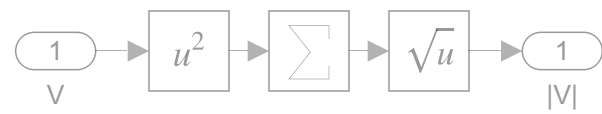


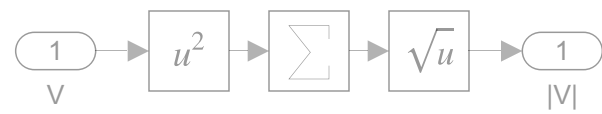


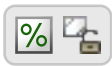
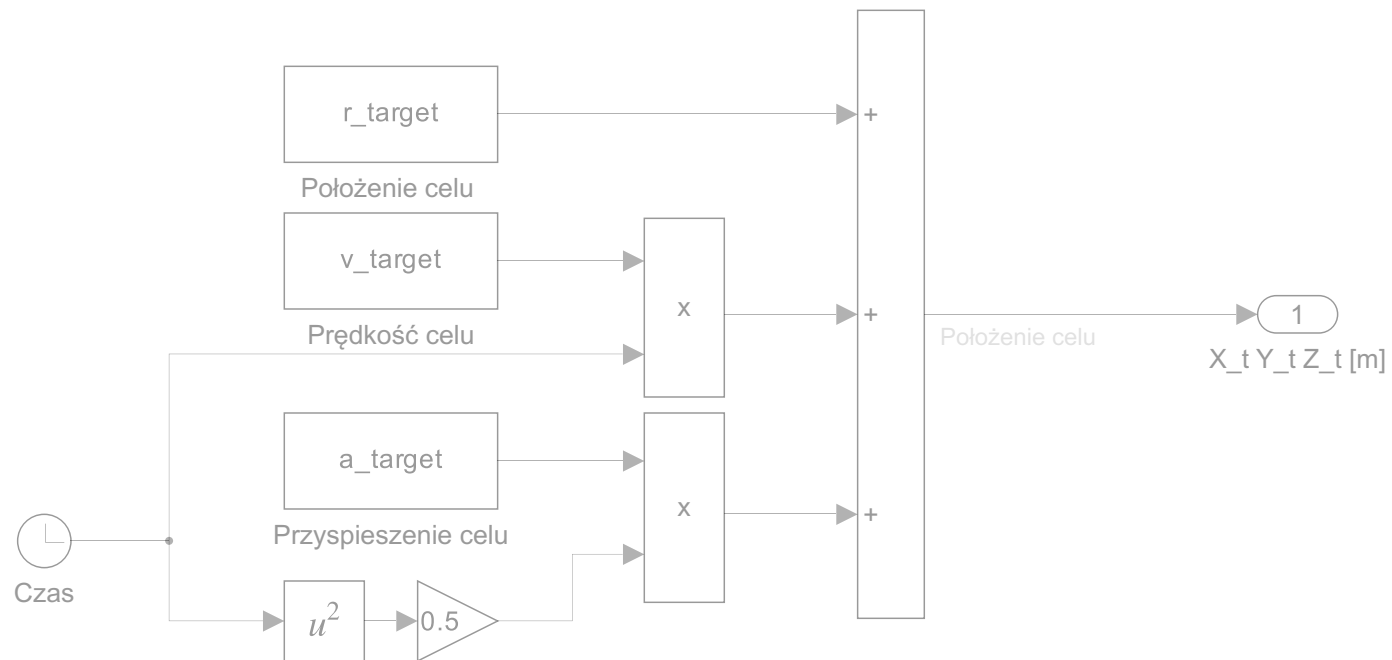


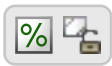
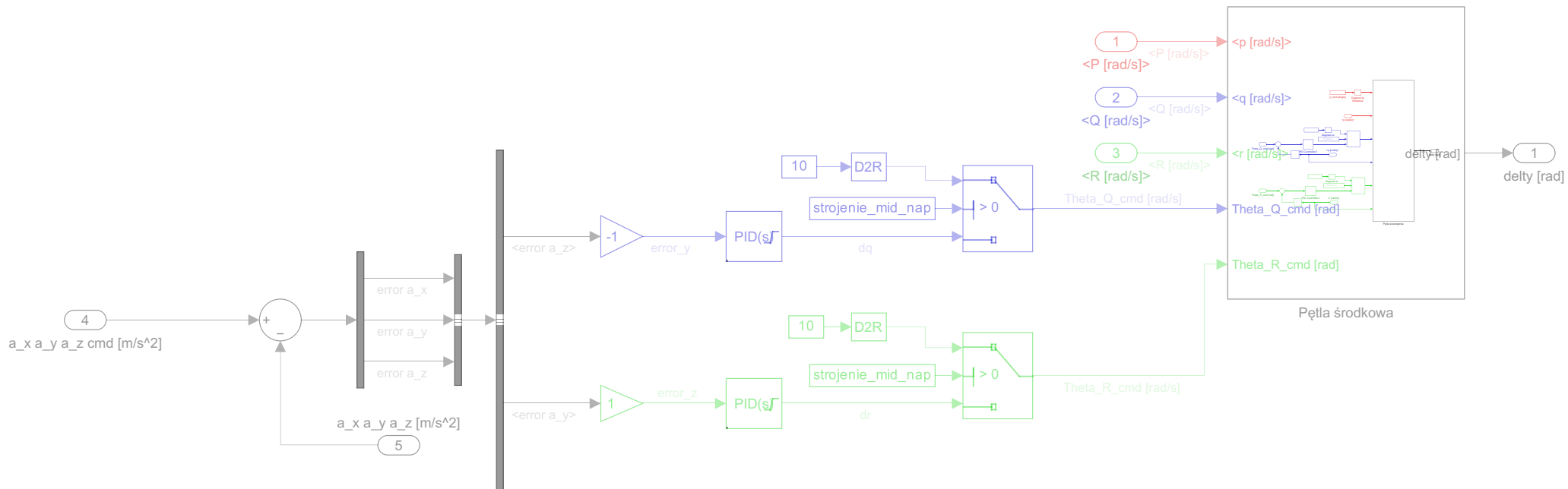


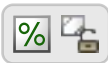
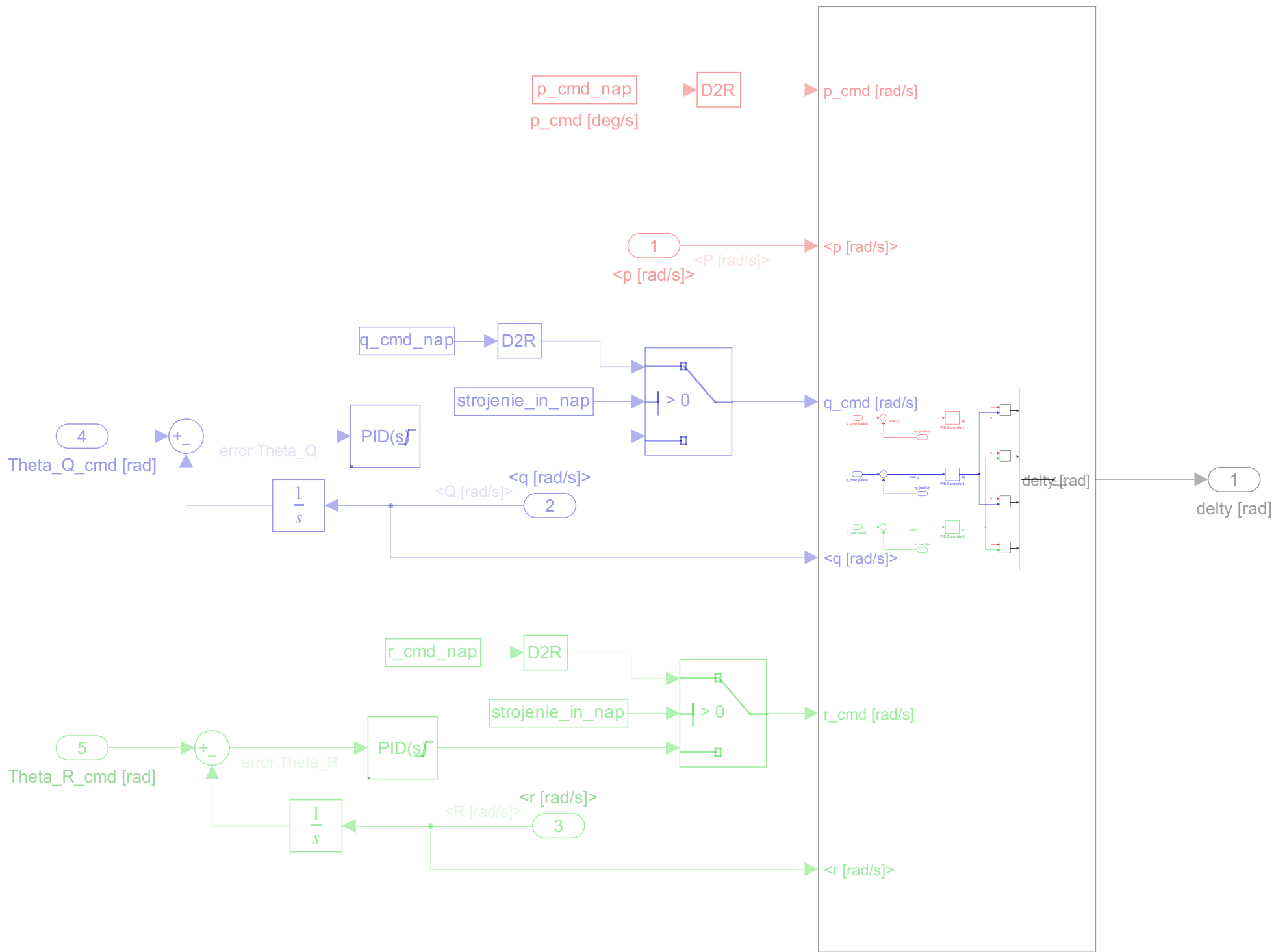


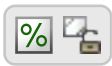
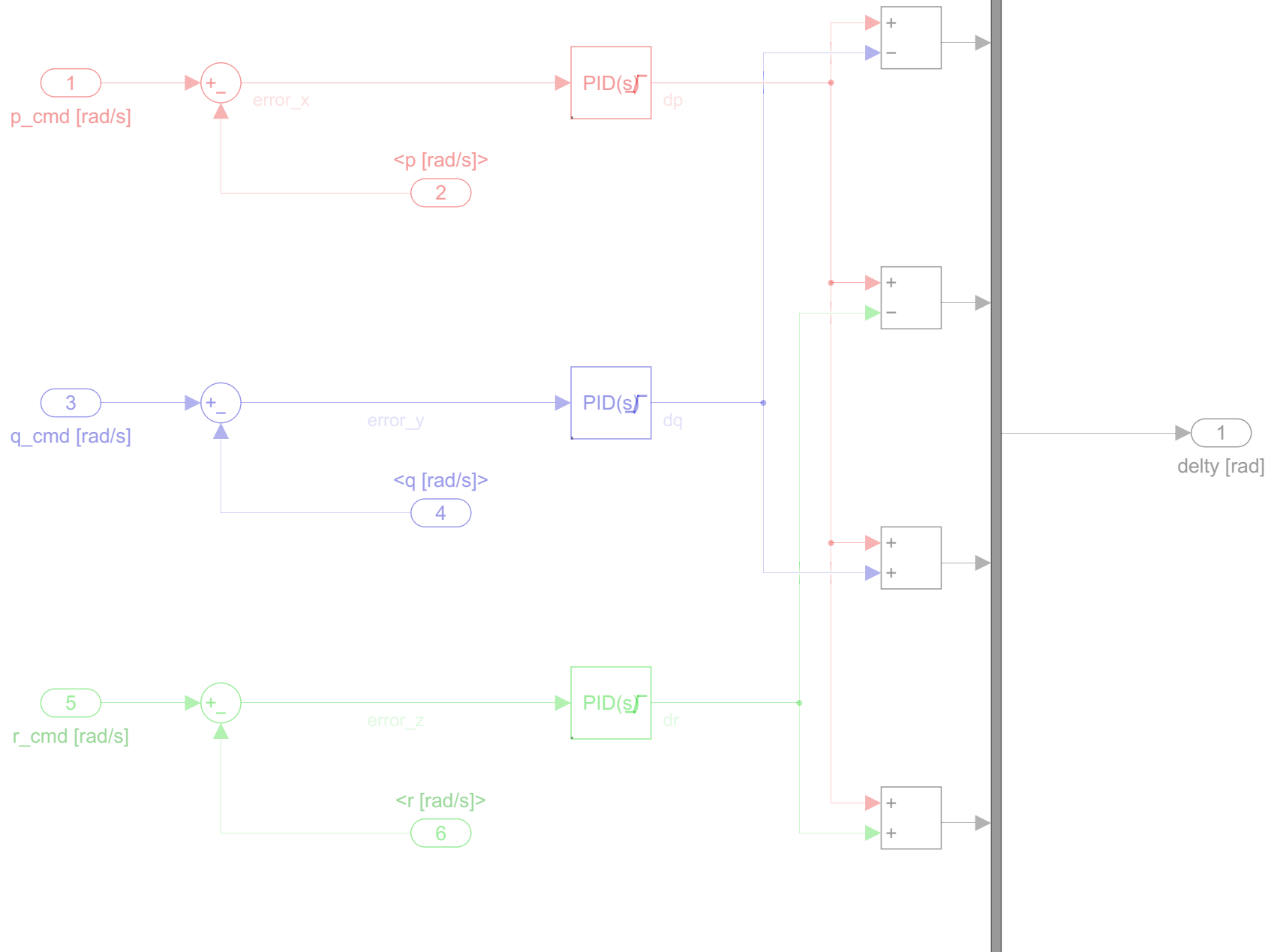


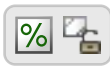
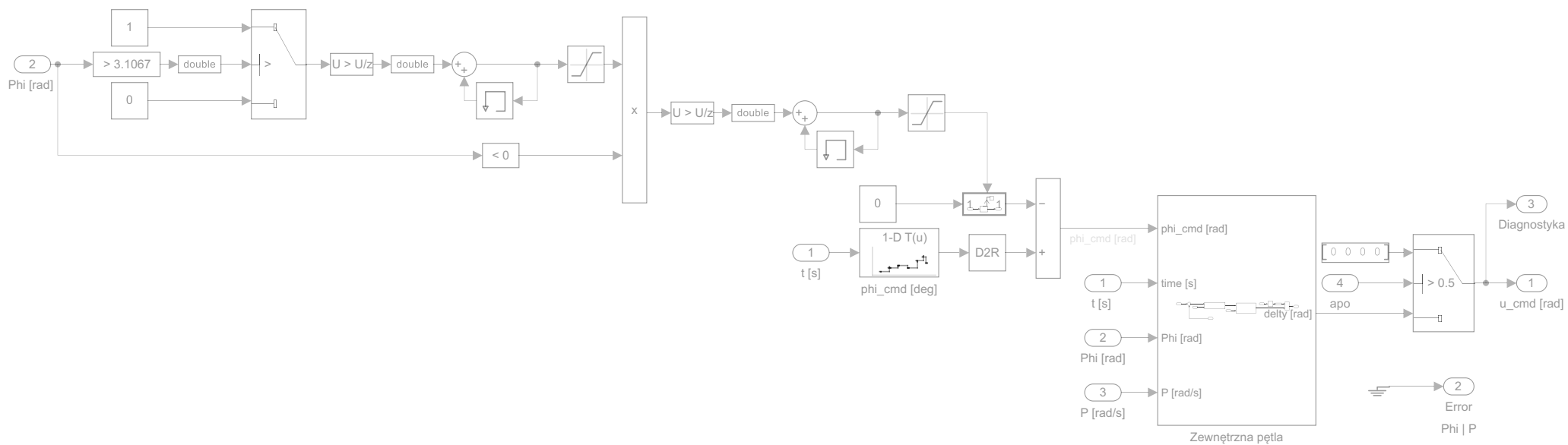




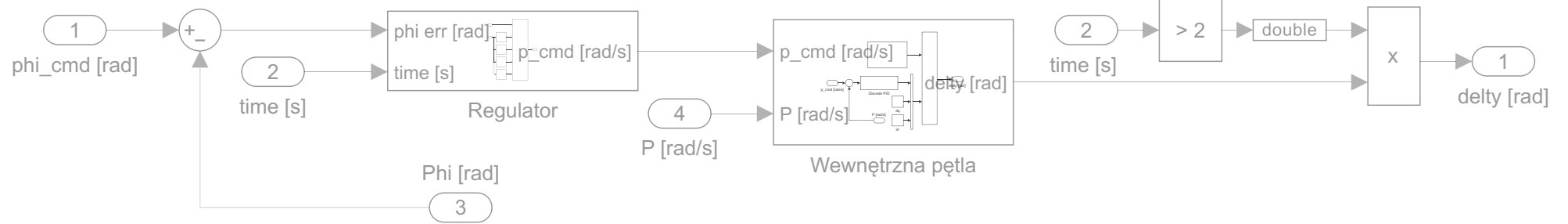


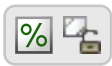
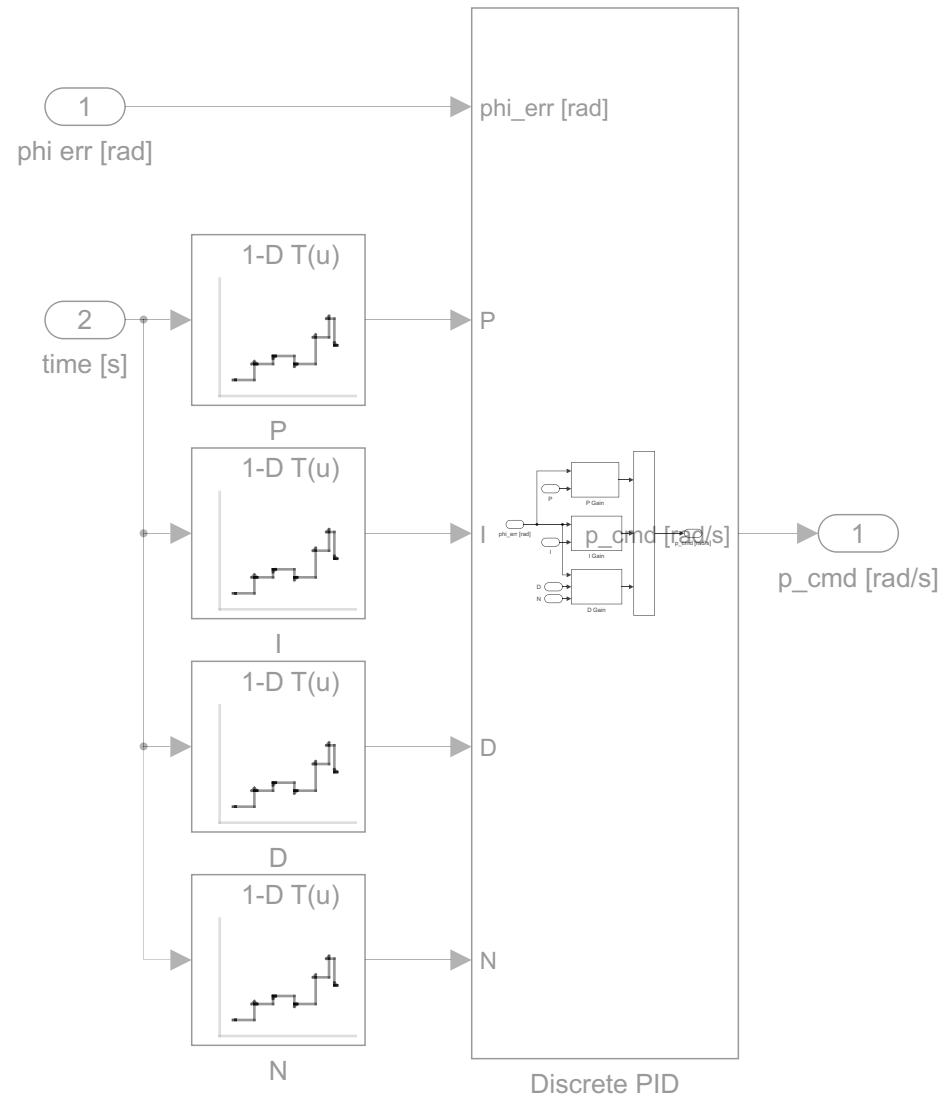


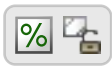
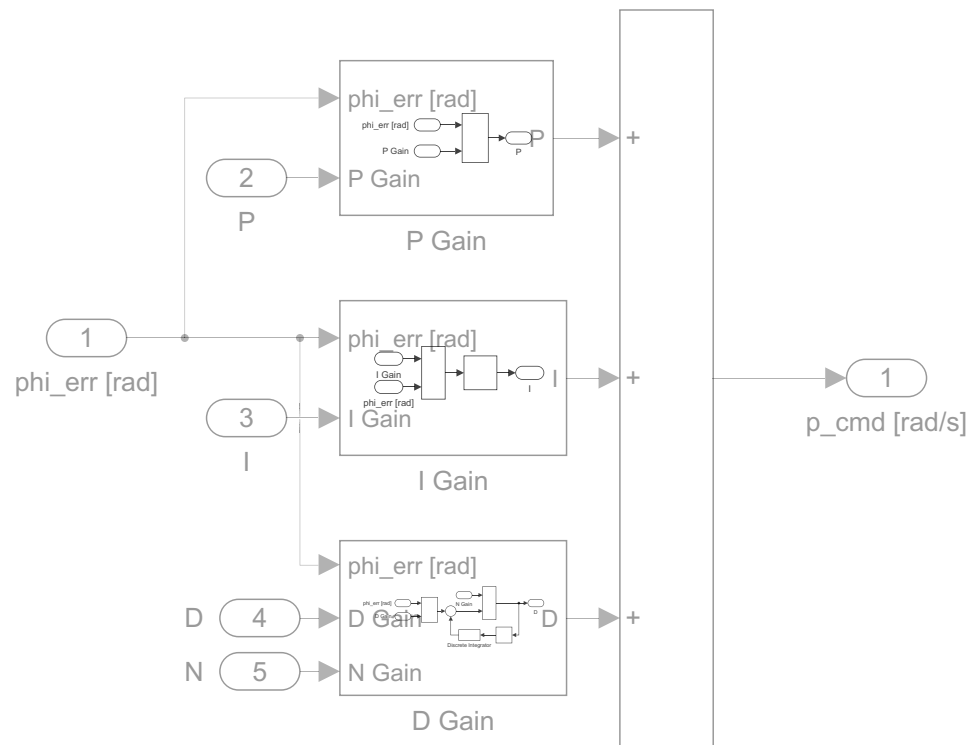


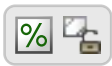
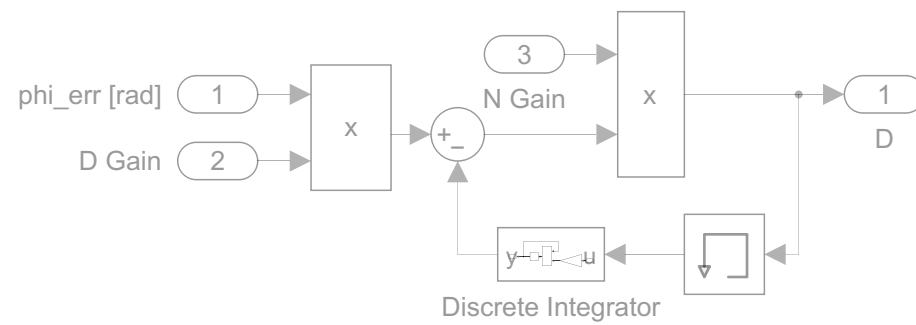


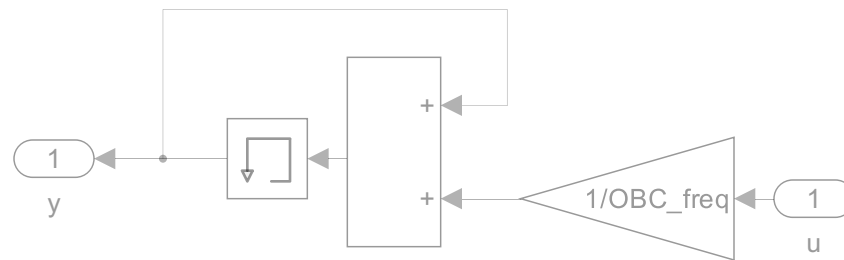


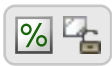
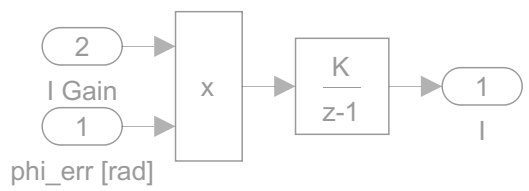


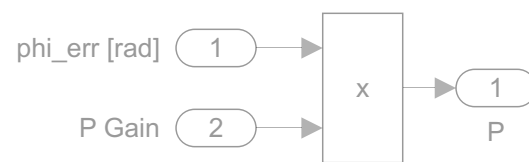








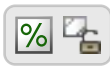
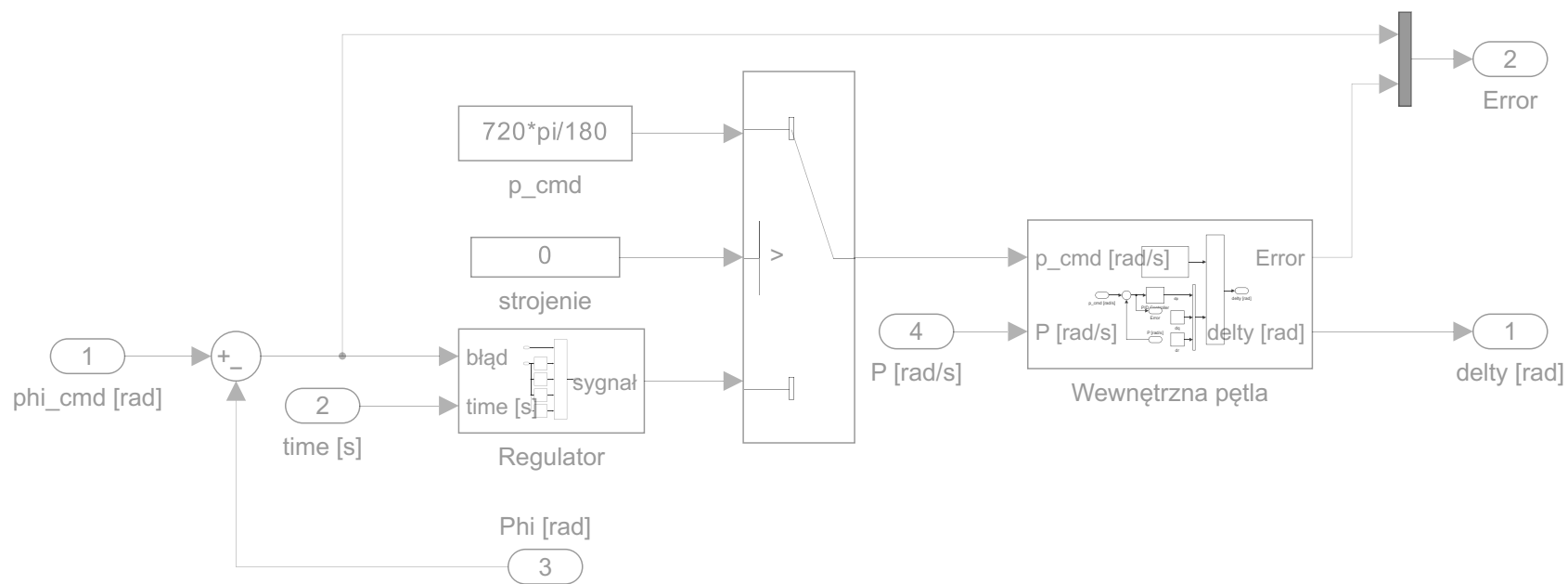


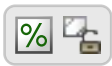
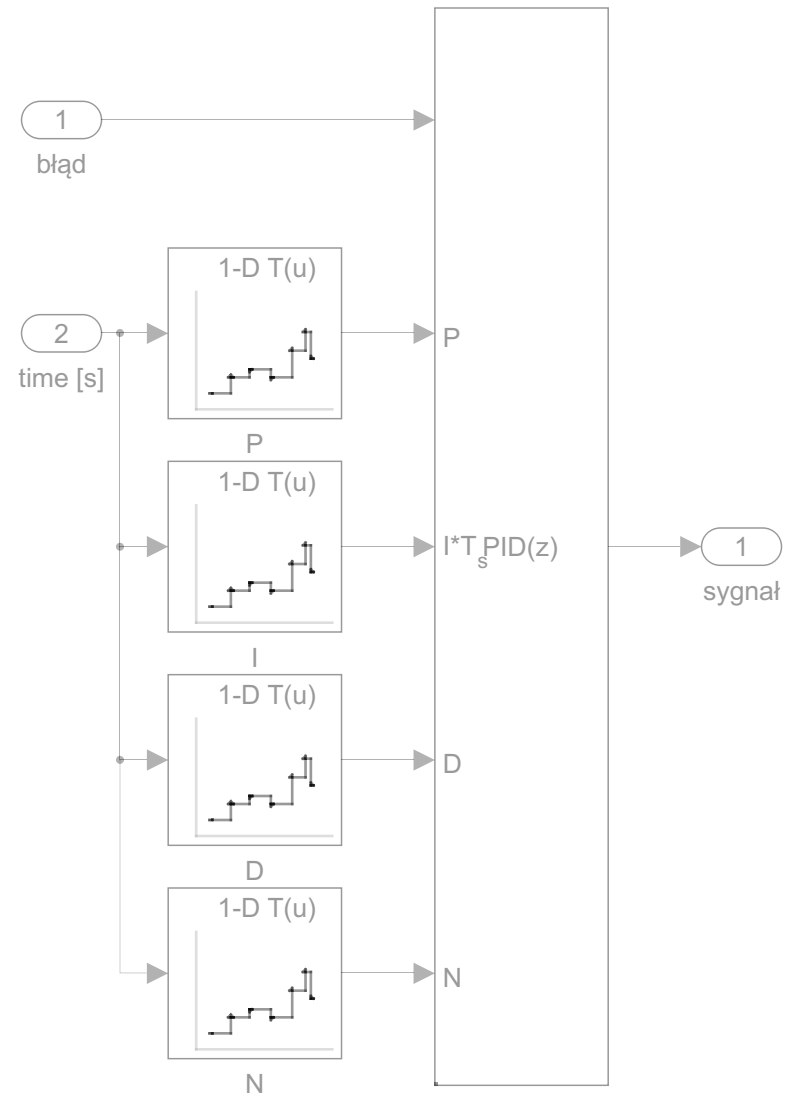


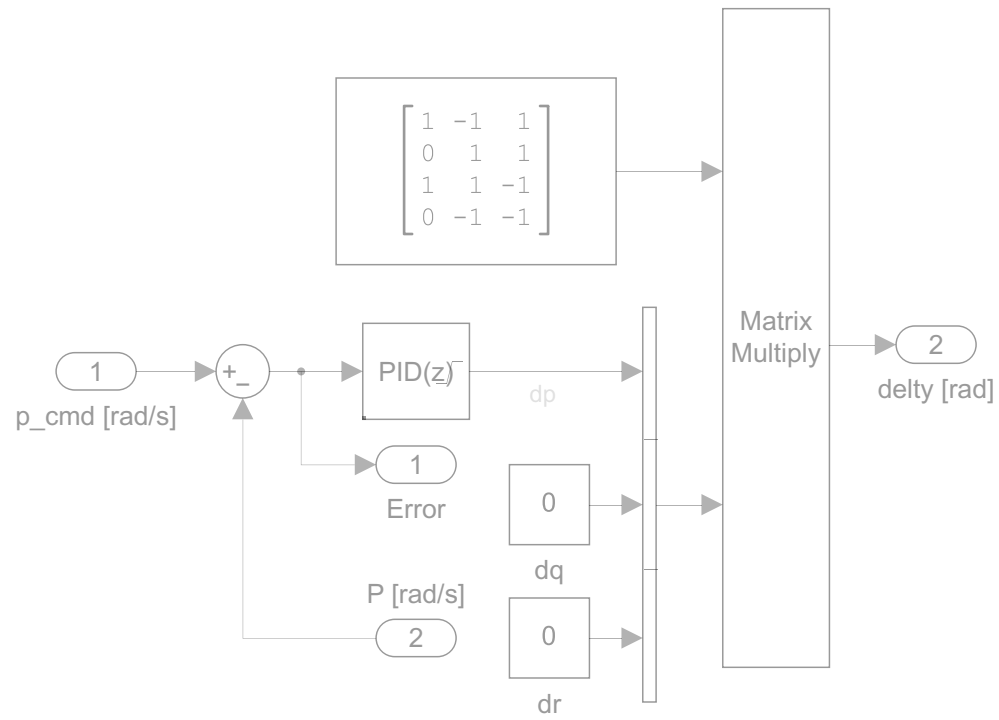


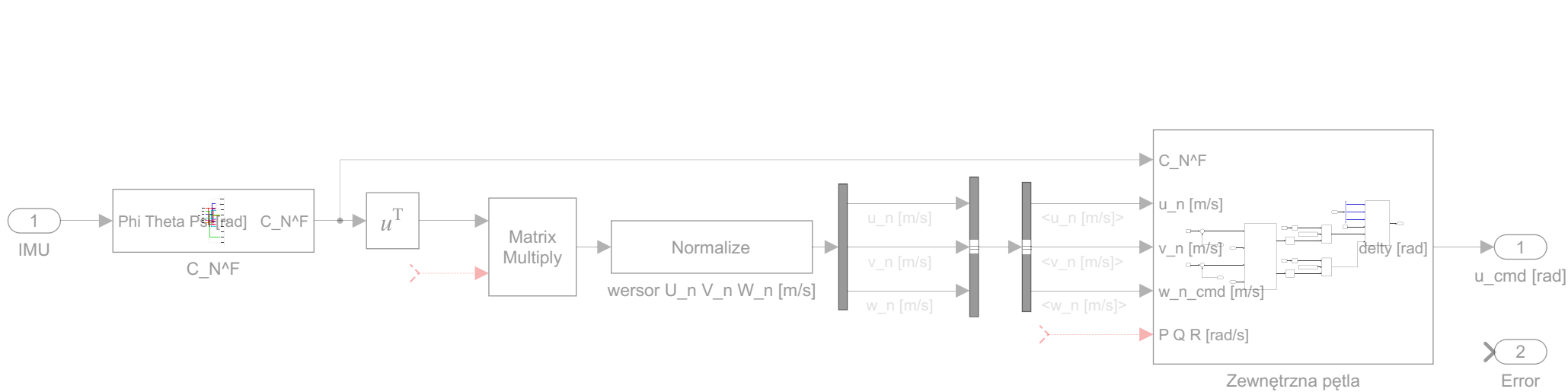












1
Phi Theta Psi [rad]



cos

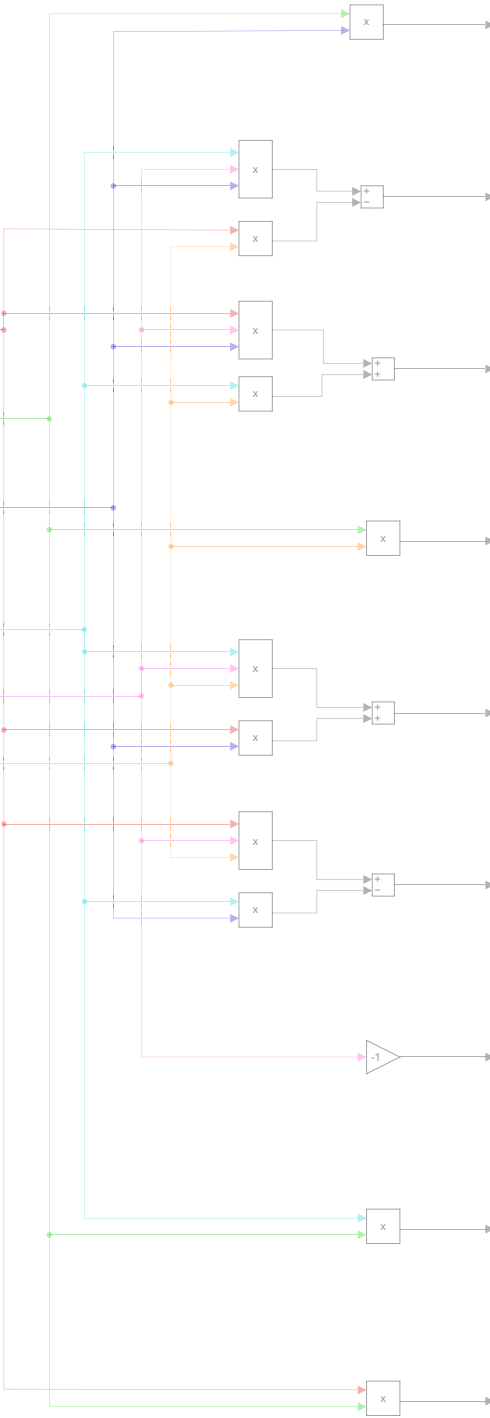
cos

cos

sin

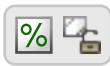
sin

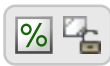
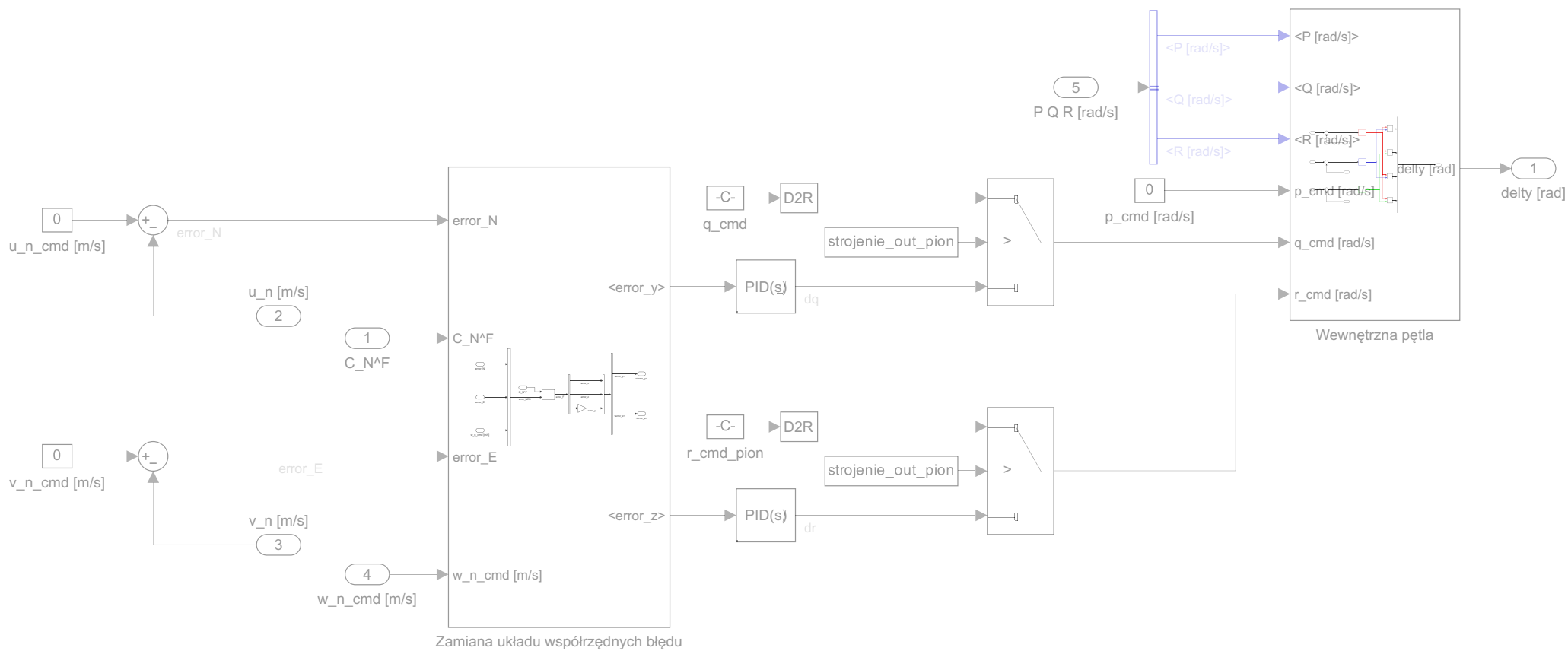
sin

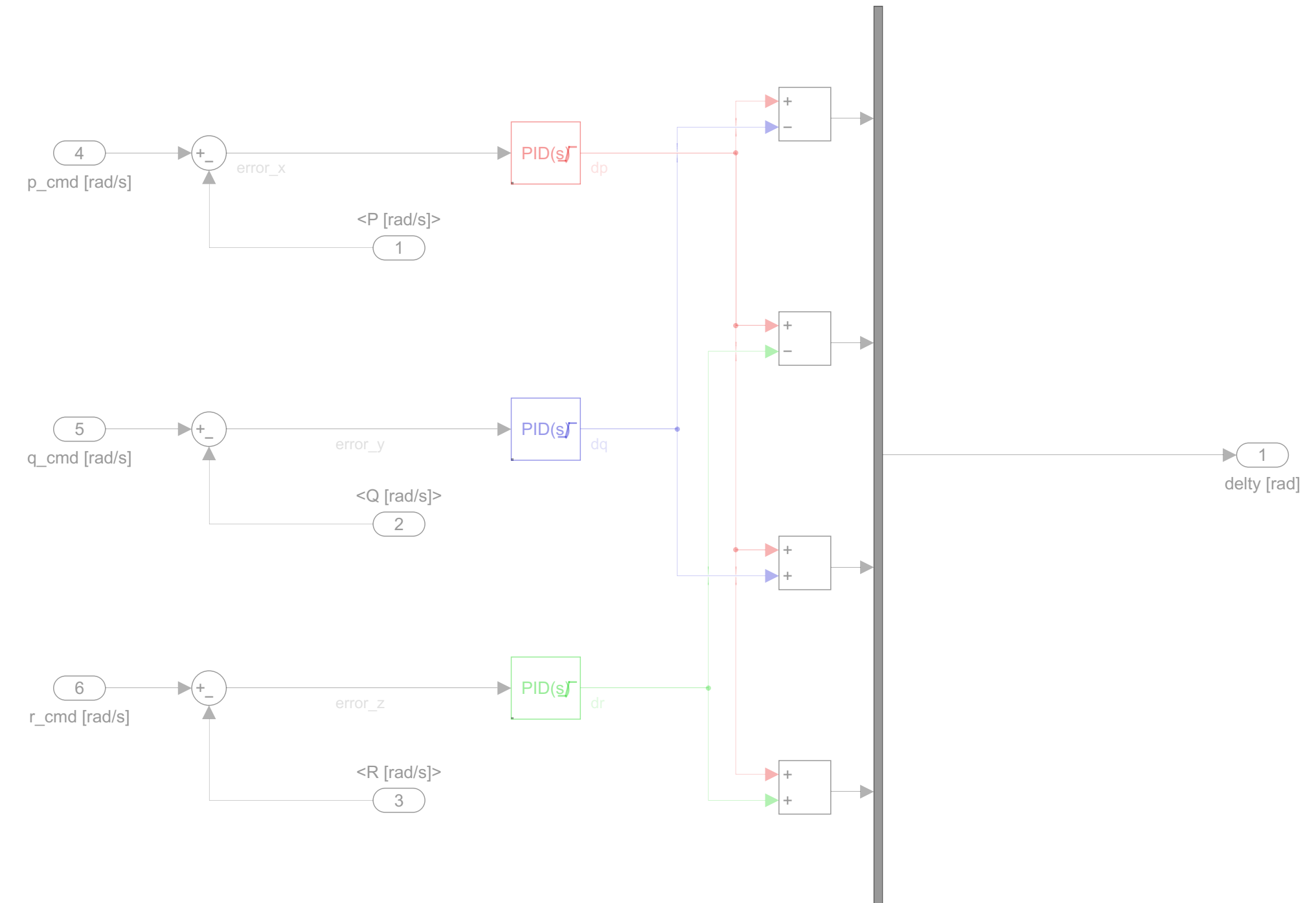


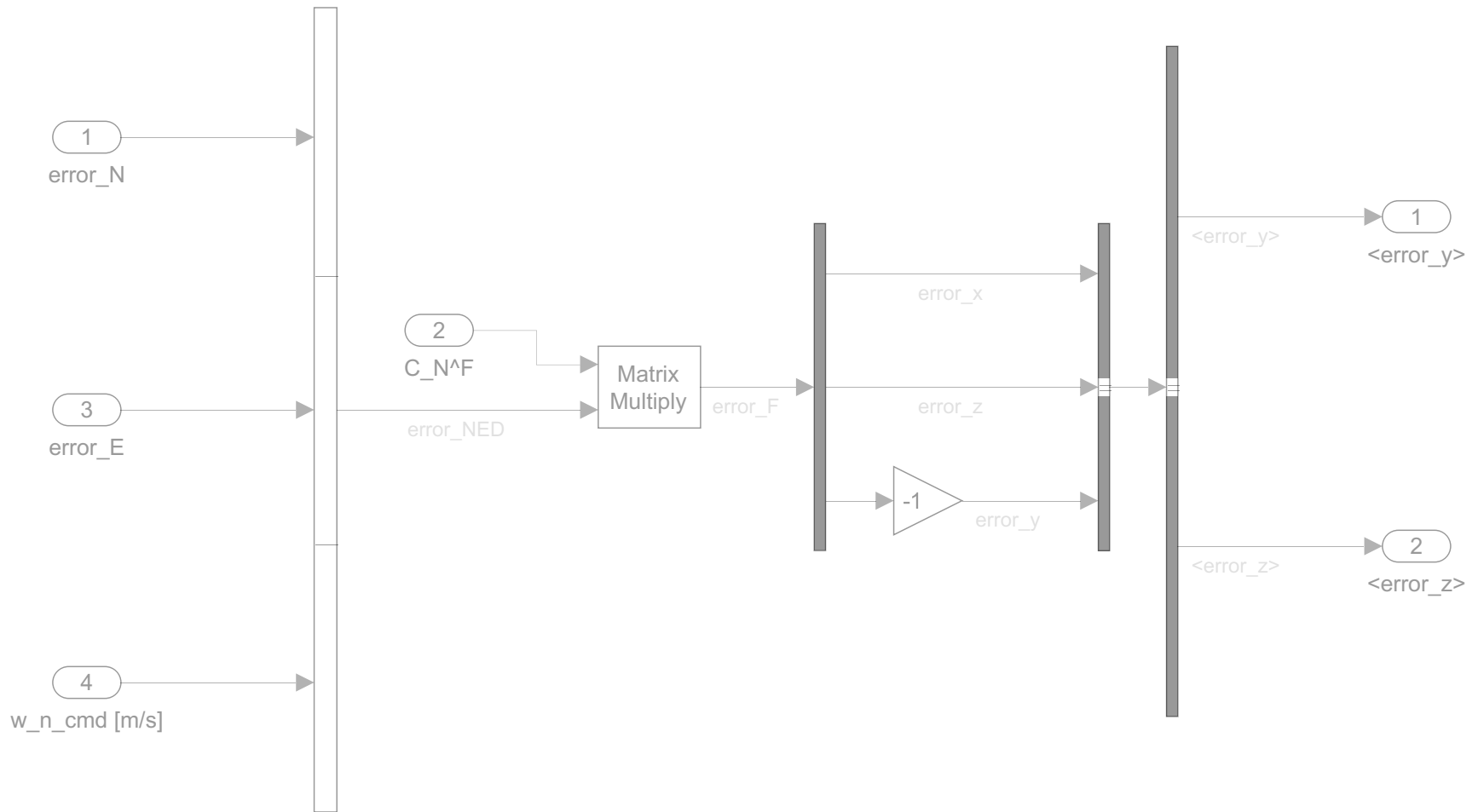
Reshape

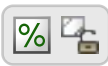
1
 $C_N \cdot F$



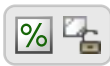
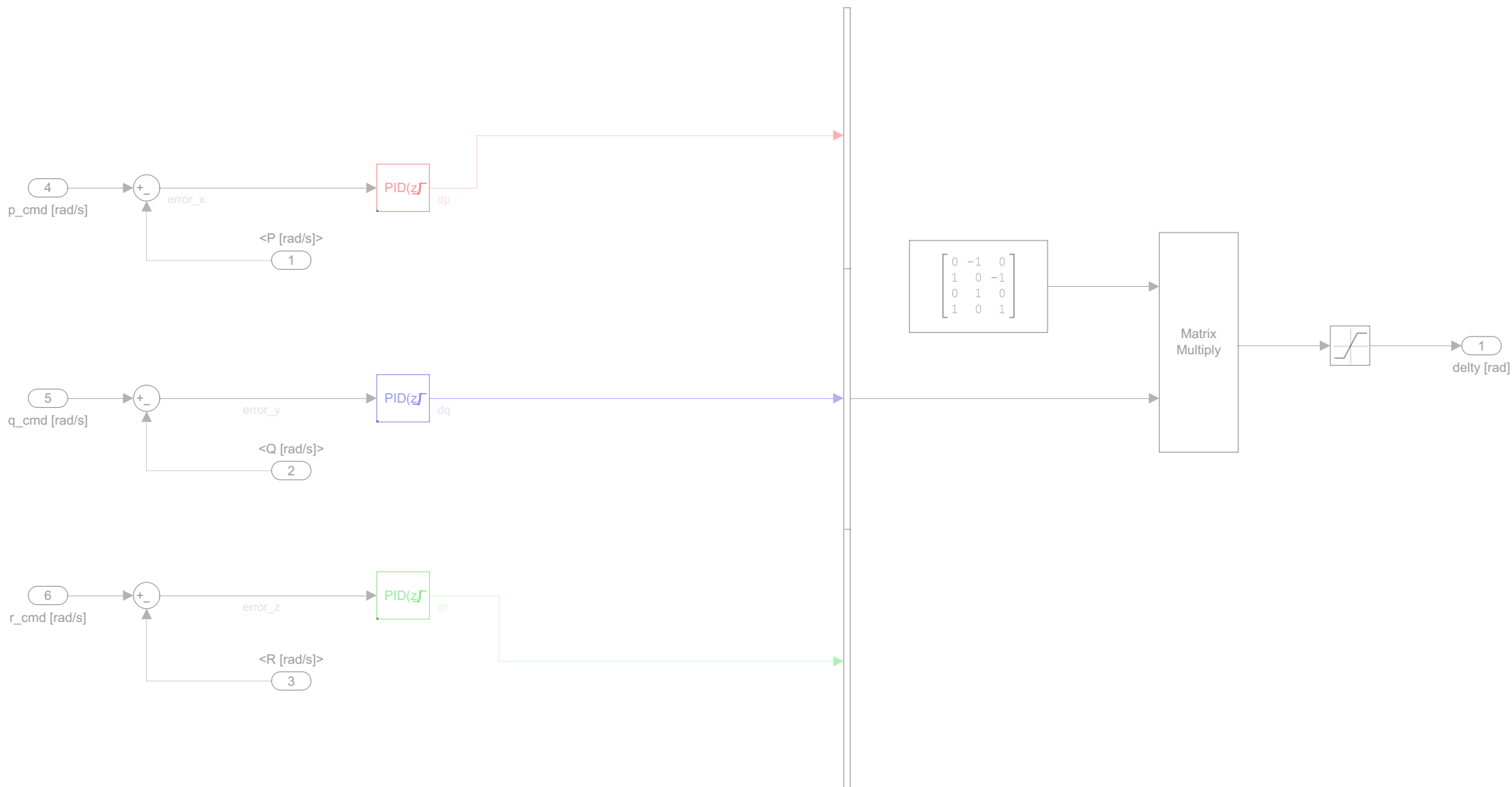


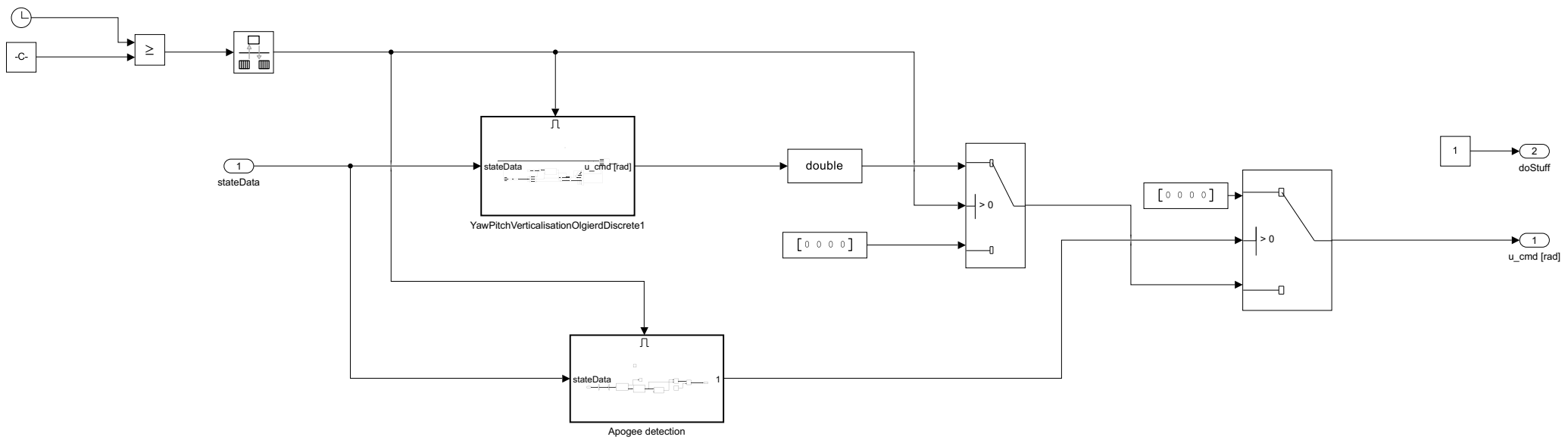


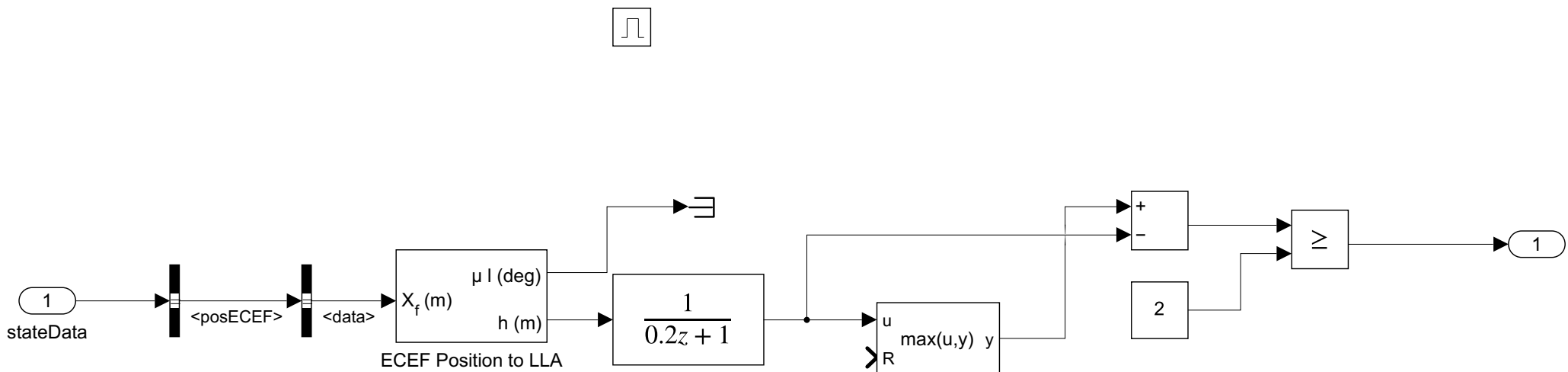


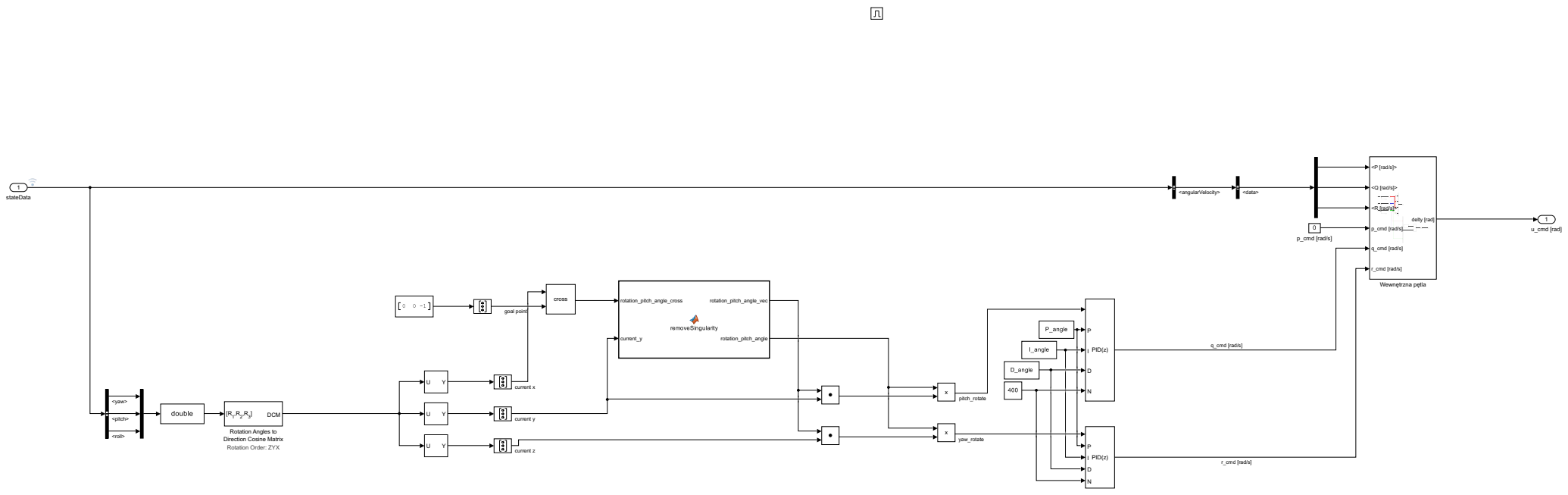


```
function [rotation_pitch_angle_vec, rotation_pitch_angle] = removeSingularity(rotation_pitch_angle_cross,current_y)
%#codegen
if norm(rotation_pitch_angle_cross) < 10^-6
    rotation_pitch_angle_vec = current_y;
    rotation_pitch_angle = 0;
else
    rotation_pitch_angle_vec = rotation_pitch_angle_cross / norm(rotation_pitch_angle_cross);
    rotation_pitch_angle = asin(norm(rotation_pitch_angle_cross));
end
```

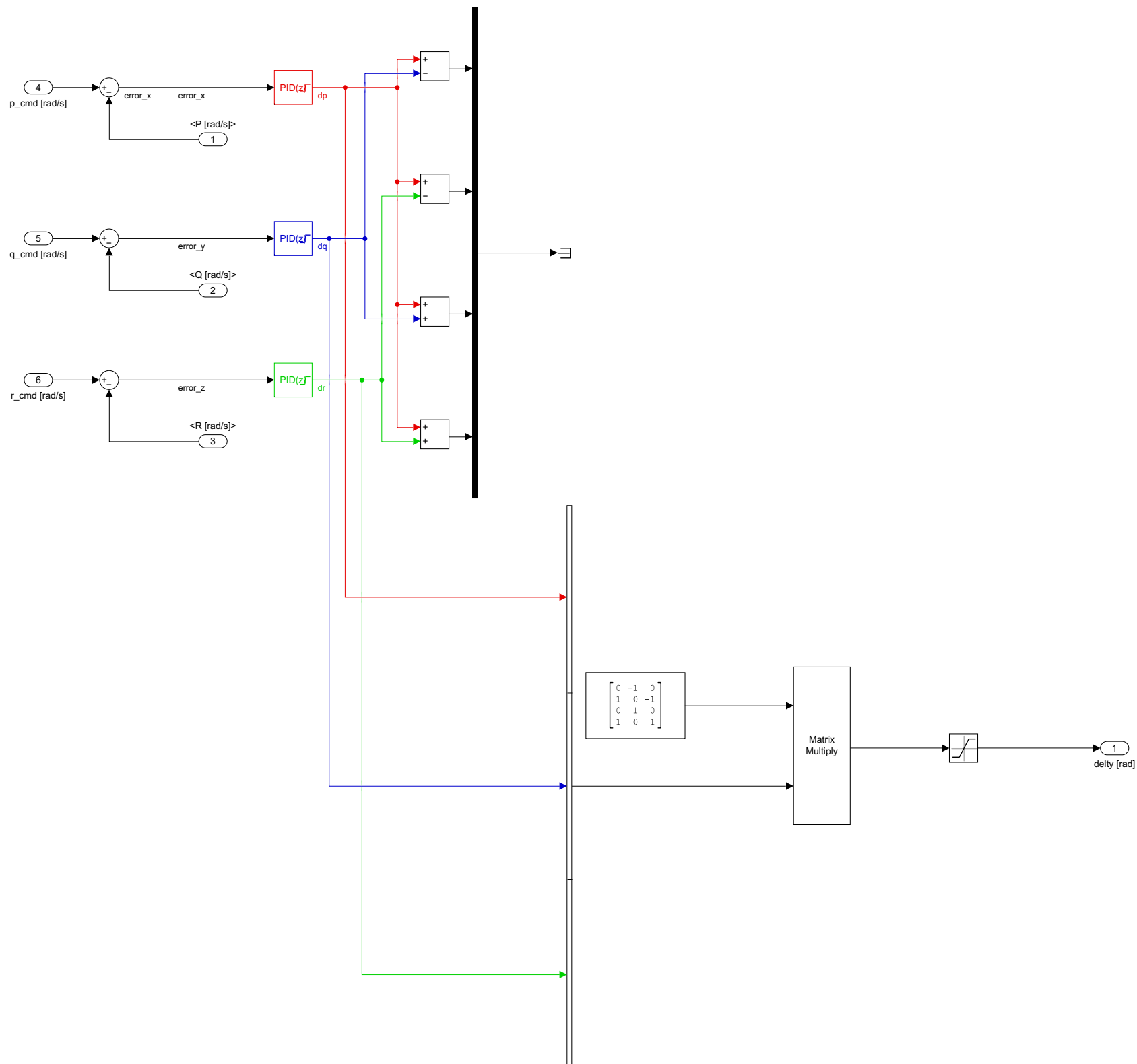


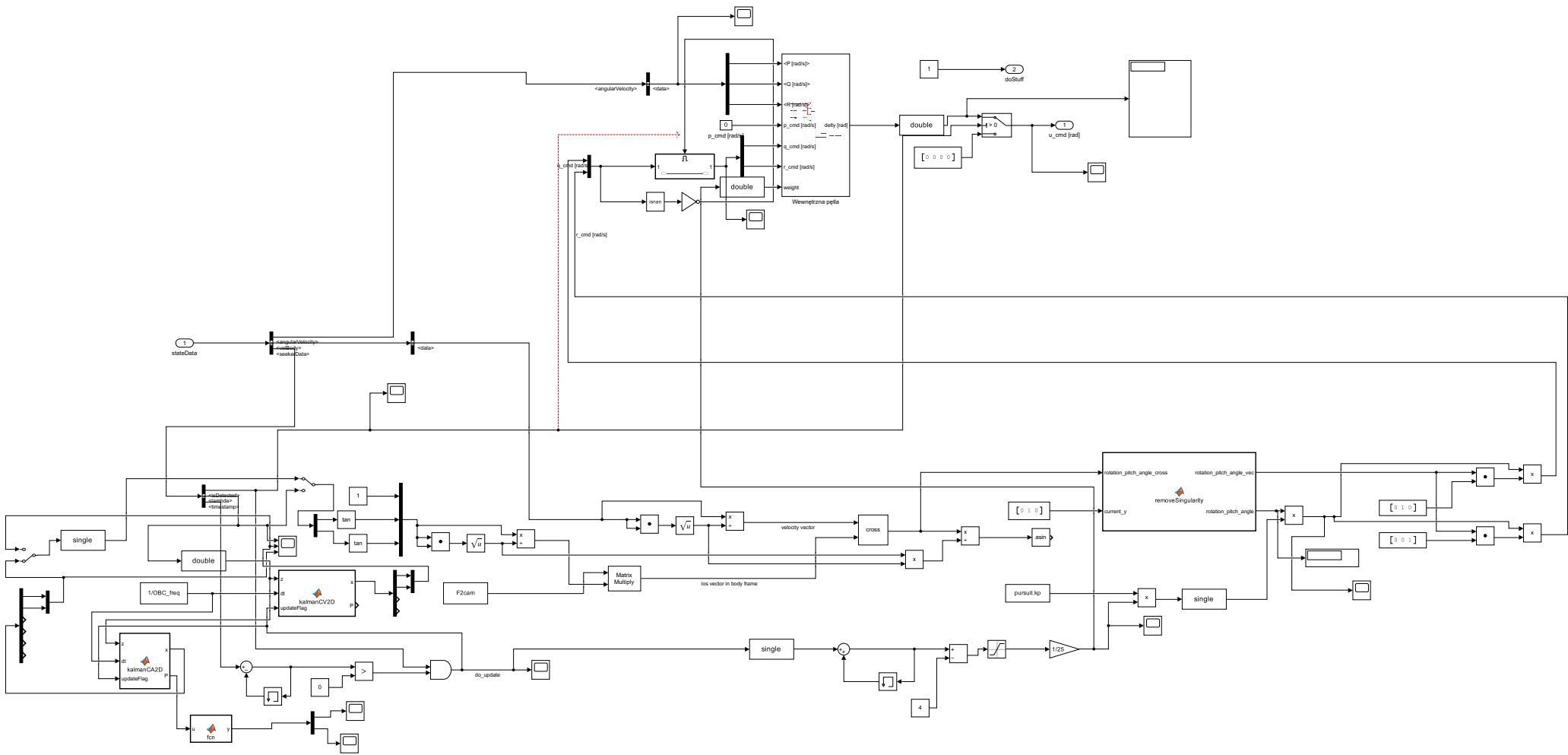






```
function [rotation_pitch_angle_vec, rotation_pitch_angle] = removeSingularity(rotation_pitch_angle_cross,current_y)
%#codegen
if norm(rotation_pitch_angle_cross) < 10^-6
    rotation_pitch_angle_vec = current_y;
    rotation_pitch_angle = 0;
else
    rotation_pitch_angle_vec = rotation_pitch_angle_cross / norm(rotation_pitch_angle_cross);
    rotation_pitch_angle = asin(norm(rotation_pitch_angle_cross));
end
```







1

1

```
function [rotation_pitch_angle_vec, rotation_pitch_angle] = removeSingularity(rotation_pitch_angle_cross,current_y)
%#codegen
if norm(rotation_pitch_angle_cross) < 10^-6
    rotation_pitch_angle_vec = current_y;
    rotation_pitch_angle = 0;
else
    rotation_pitch_angle_vec = rotation_pitch_angle_cross / norm(rotation_pitch_angle_cross);
    rotation_pitch_angle = asin(norm(rotation_pitch_angle_cross));
end
```

```

function [x, P] = kalmanCV2D(z, dt, updateFlag)
%KALMANCV2D Constant-velocity 2D Kalman filter, codegen (MATLAB Coder) friendly.
%
% [x, P] = kalmanCV2D(z, dt, updateFlag)
%
% Inputs:
%     z          - 2x1 position measurement [zx; zy] (double)
%     dt         - scalar time step (double, seconds)
%     updateFlag - logical/double scalar. If true/nonzero, the correction
%                   (measurement update) step runs using z. If false, only
%                   the prediction step runs and z is ignored. This lets
%                   you call the function every cycle even when a fresh
%                   measurement isn't available (e.g. sensor dropout).
%
% Outputs:
%     x - 4x1 state estimate [px; py; vx; vy]
%     P - 4x4 state covariance estimate
%
% State persists across calls via persistent variables, so this
% function behaves like a stateful filter object -- call it once per
% cycle with the latest measurement and flag.
%
% Design notes for codegen compatibility:
% - All arrays are fixed-size (4x1, 4x4, 2x1, 2x2) on every path.
% - No varargin/varargout, cell arrays, structs with dynamic fields,
%   or dynamic-size growth.
% - Uses mrdivide ('/') instead of inv() for the gain solve.
% - %#codegen pragma included; run this through MATLAB Coder as-is.
%#codegen

persistent x_est P_est

% ---- Initialization (runs once, first call) ----
if isempty(x_est)
    x_est = [0; 0; 0; 0]; % initial position (0,0), initial velocity (0,0)
end
if isempty(P_est)
    % Fairly confident in initial position, uncertain about velocity.
    P_est = diag([1, 1, 10, 10]);
end

% ---- Tunable noise parameters ----
q = 1.0; % process noise intensity (acceleration white-noise strength)
sigmaPos = 0.01; % measurement noise std dev (position, same units as z)

% ---- Model matrices (recomputed each call since dt may vary) ----
F = [1 0 dt 0;
     0 1 0 dt;
     0 0 1 0;
     0 0 0 1];

Q = q * [dt^4/4 0 dt^3/2 0;
        0 dt^4/4 0 dt^3/2;
        dt^3/2 0 dt^2 0;
        0 dt^3/2 0 dt^2];

H = [1 0 0 0;
     0 1 0 0];

```

```

R = sigmaPos^2 * eye(2);

% ---- Predict step (always runs) ----
x_pred = F * x_est;
P_pred = F * P_est * F' + Q;

% ---- Conditional update step ----
if updateFlag
    y = z - H * x_pred;           % innovation
    S = H * P_pred * H' + R;      % innovation covariance
    K = P_pred * H' / S;          % Kalman gain (mrdivide, codegen-friendly)
    x_est = x_pred + K * y;
    P_est = (eye(4) - K * H) * P_pred;
else
    x_est = x_pred;
    P_est = P_pred;
end

x = x_est;
P = P_est;

end

```

```

function [x, P] = kalmanCA2D(z, dt, updateFlag)
%KALMANCA2D Constant-acceleration 2D Kalman filter, codegen (MATLAB Coder) friendly.
%
% [x, P] = kalmanCA2D(z, dt, updateFlag)
%
% Inputs:
%     z          - 2x1 position measurement [zx; zy] (double)
%     dt          - scalar time step (double, seconds)
%     updateFlag - logical/double scalar. If true/nonzero, the correction
%                   (measurement update) step runs using z. If false, only
%                   the prediction step runs and z is ignored.
%
% Outputs:
%     x - 6x1 state estimate [px; py; vx; vy; ax; ay]
%     P - 6x6 state covariance estimate
%
% State persists across calls via persistent variables, so this
% function behaves like a stateful filter object -- call it once per
% cycle with the latest measurement and flag.
%
% Model: constant acceleration with white-noise jerk driving the
% acceleration states (the CA analog of the CV filter's white-noise
% acceleration model -- here the noise enters one derivative higher).
%
% Design notes for codegen compatibility:
%     - All arrays are fixed-size (6x1, 6x6, 2x1, 2x2) on every path.
%     - No varargin/varargout, cell arrays, structs with dynamic fields,
%       or dynamic-size growth.
%     - Q is built as an explicit literal (no blkdiag/interleaving calls)
%       so array sizes stay static and trivially inferable.
%     - Uses mrdivide ('/') instead of inv() for the gain solve.
%     - %#codegen pragma included; run this through MATLAB Coder as-is.

```

```

%#codegen

```

```

persistent x_est P_est

```

```

% ---- Initialization (runs once, first call) ----
if isempty(x_est)
    x_est = [0; 0; 0; 0; 0; 0]; % position (0,0), velocity (0,0), accel (0,0)
end
if isempty(P_est)
    % Confident in position, less confident in velocity, least confident
    % in acceleration -- same escalating-uncertainty pattern as the CV
    % filter, extended one state further.
    P_est = diag([1, 1, 10, 10, 100, 100]);
end

```

```

% ---- Tunable noise parameters (same names/values as the CV filter) ----
q          = 1.0; % process noise intensity (white-noise jerk strength)
sigmaPos = 0.01; % measurement noise std dev (position, same units as z)

```

```

% ---- Model matrices (recomputed each call since dt may vary) ----
F = [1 0 dt 0 0.5*dt^2 0;
     0 1 0 dt 0      0.5*dt^2;
     0 0 1 0 dt      0;
     0 0 0 1 0      dt;
     0 0 0 0 1      0;
     0 0 0 0 0      1];

```

```

% Discrete white-noise-jerk process noise, per axis:
% [dt^5/20 dt^4/8 dt^3/6;
%   dt^4/8 dt^3/3 dt^2/2;
%   dt^3/6 dt^2/2 dt ]
% interleaved into [px py vx vy ax ay] ordering, x/y axes decoupled.
Q = q * [dt^5/20 0 dt^4/8 0 dt^3/6 0;
         0 dt^5/20 0 dt^4/8 0 dt^3/6;
         dt^4/8 0 dt^3/3 0 dt^2/2 0;
         0 dt^4/8 0 dt^3/3 0 dt^2/2;
         dt^3/6 0 dt^2/2 0 dt 0;
         0 dt^3/6 0 dt^2/2 0 dt];

H = [1 0 0 0 0 0;
     0 1 0 0 0 0];

R = sigmaPos^2 * eye(2);

% ---- Predict step (always runs) ----
x_pred = F * x_est;
P_pred = F * P_est * F' + Q;

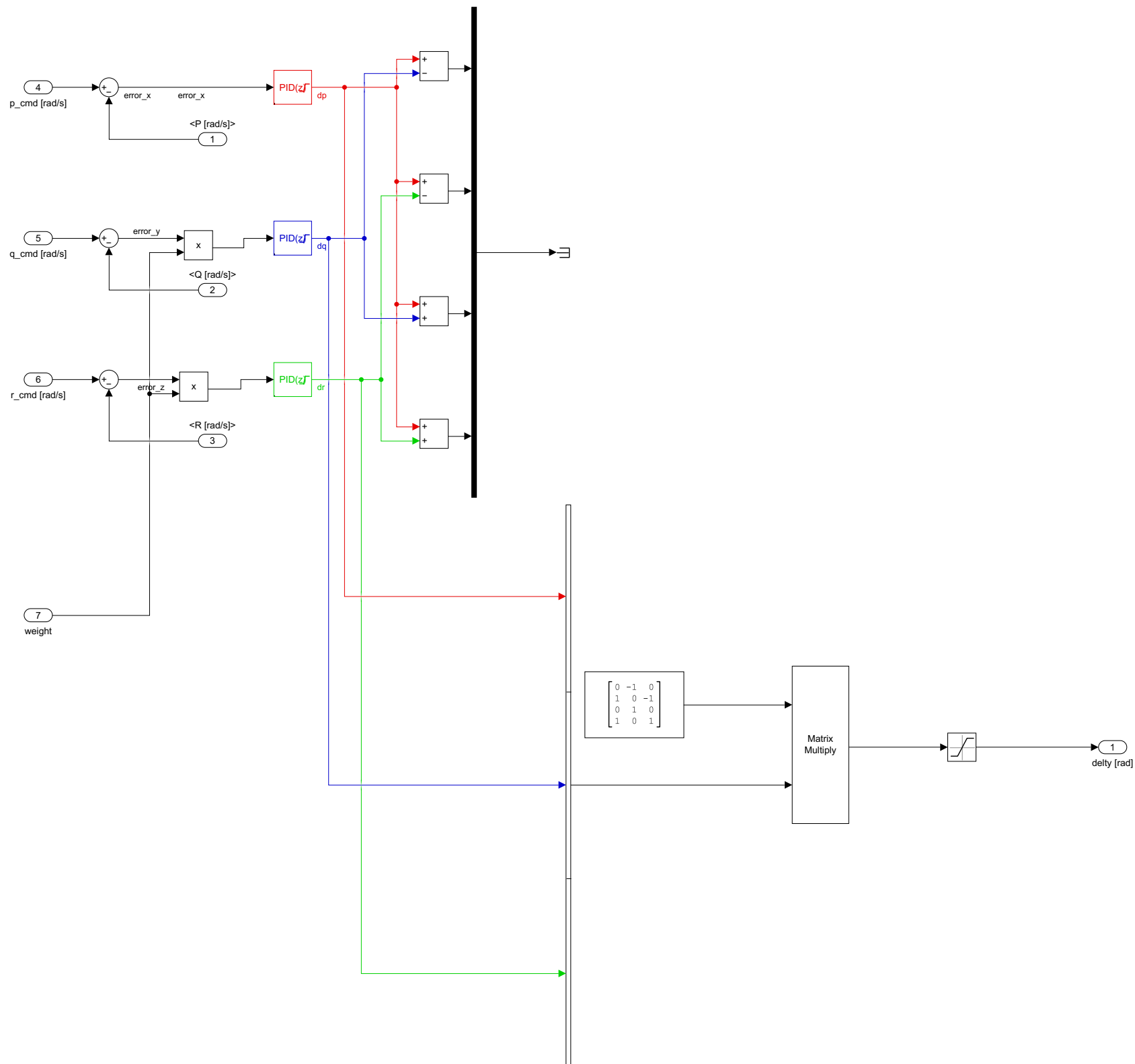
% ---- Conditional update step ----
if updateFlag
    y = z - H * x_pred; % innovation
    S = H * P_pred * H' + R; % innovation covariance
    K = P_pred * H' / S; % Kalman gain (mrdivide, codegen-friendly)
    x_est = x_pred + K * y;
    P_est = (eye(6) - K * H) * P_pred;
else
    x_est = x_pred;
    P_est = P_pred;
end

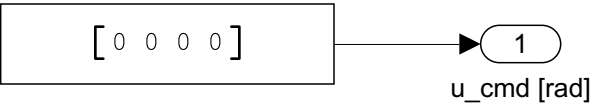
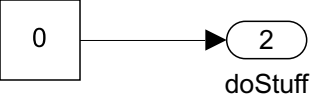
x = x_est;
P = P_est;

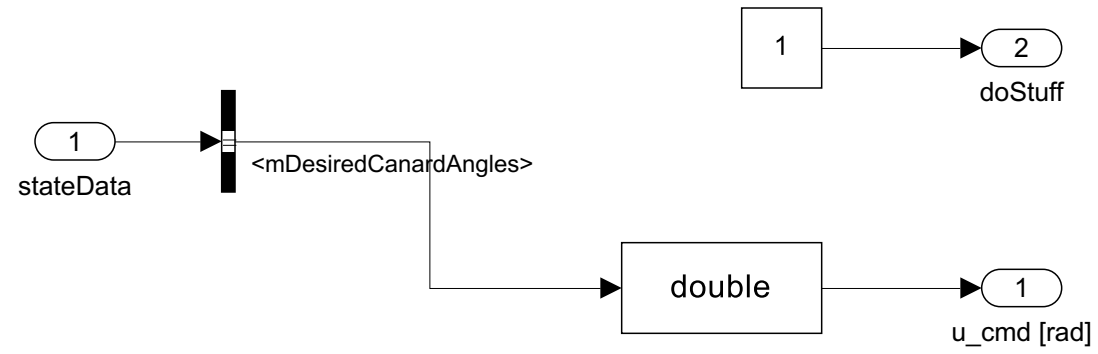
end

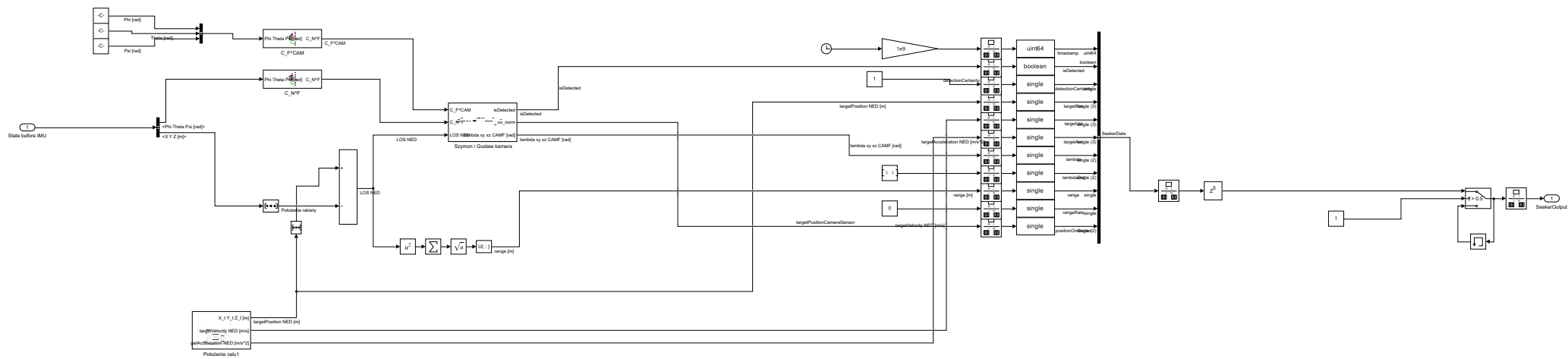
```

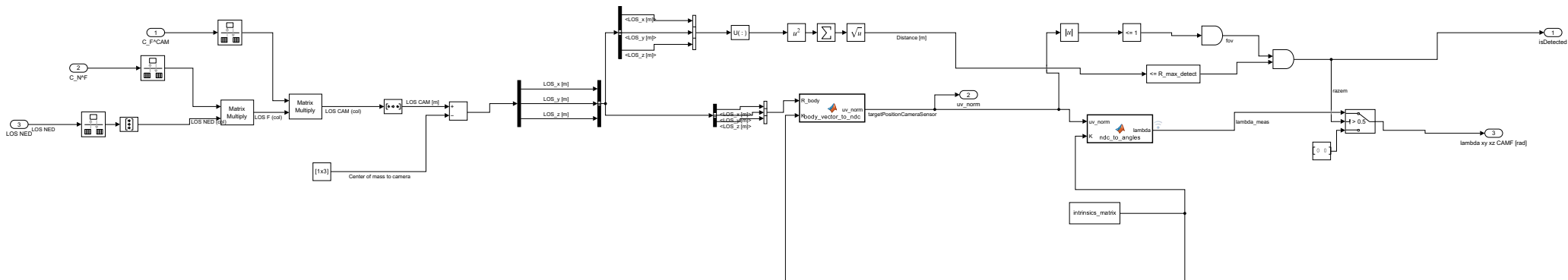
```
function y = fcn(u)
y = zeros(2,1);
y(1) = u(1,1);
y(2) = u(2,2);
```











```

function uv_norm = body_vector_to_ndc(R_body, K)

fx = K(1,1);
fy = K(2,2);
cx = K(1,3);
cy = K(2,3);

X_przod = R_body(1);
Y_prawo = R_body(2);
Z_dol   = R_body(3);

if X_przod < 0.001
    X_przod = 0.001;
end

u_norm = (fx * Y_prawo) / (cx * X_przod);
v_norm = (fy * Z_dol)   / (cy * X_przod);

uv_norm = [u_norm; v_norm];
end

```

```
function lambda = ndc_to_angles(uv_norm, K)
```

```
    u_norm = uv_norm(1);  
    v_norm = uv_norm(2);
```

```
    fx = K(1,1);  
    fy = K(2,2);  
    cx = K(1,3);  
    cy = K(2,3);
```

```
    lambda_x = atan(u_norm * (cx / fx));  
    lambda_y = atan(v_norm * (cy / fy));
```

```
    lambda = [lambda_x; lambda_y];  
end
```

1
Phi Theta Psi [rad]

<Phi [rad]>

cos

<Theta [rad]>

cos

<Psi [rad]>

cos

<Phi [rad]>

sin

<Theta [rad]>

sin

<Psi [rad]>

sin

Reshape

1
C_N*F

1
Phi Theta Psi [rad]

<Phi [rad]>

cos

<Theta [rad]>

cos

<Psi [rad]>

cos

<Phi [rad]>

sin

<Theta [rad]>

sin

<Psi [rad]>

sin

x

x

+

x

+

x

+

x

x

+

x

+

x

+

-1

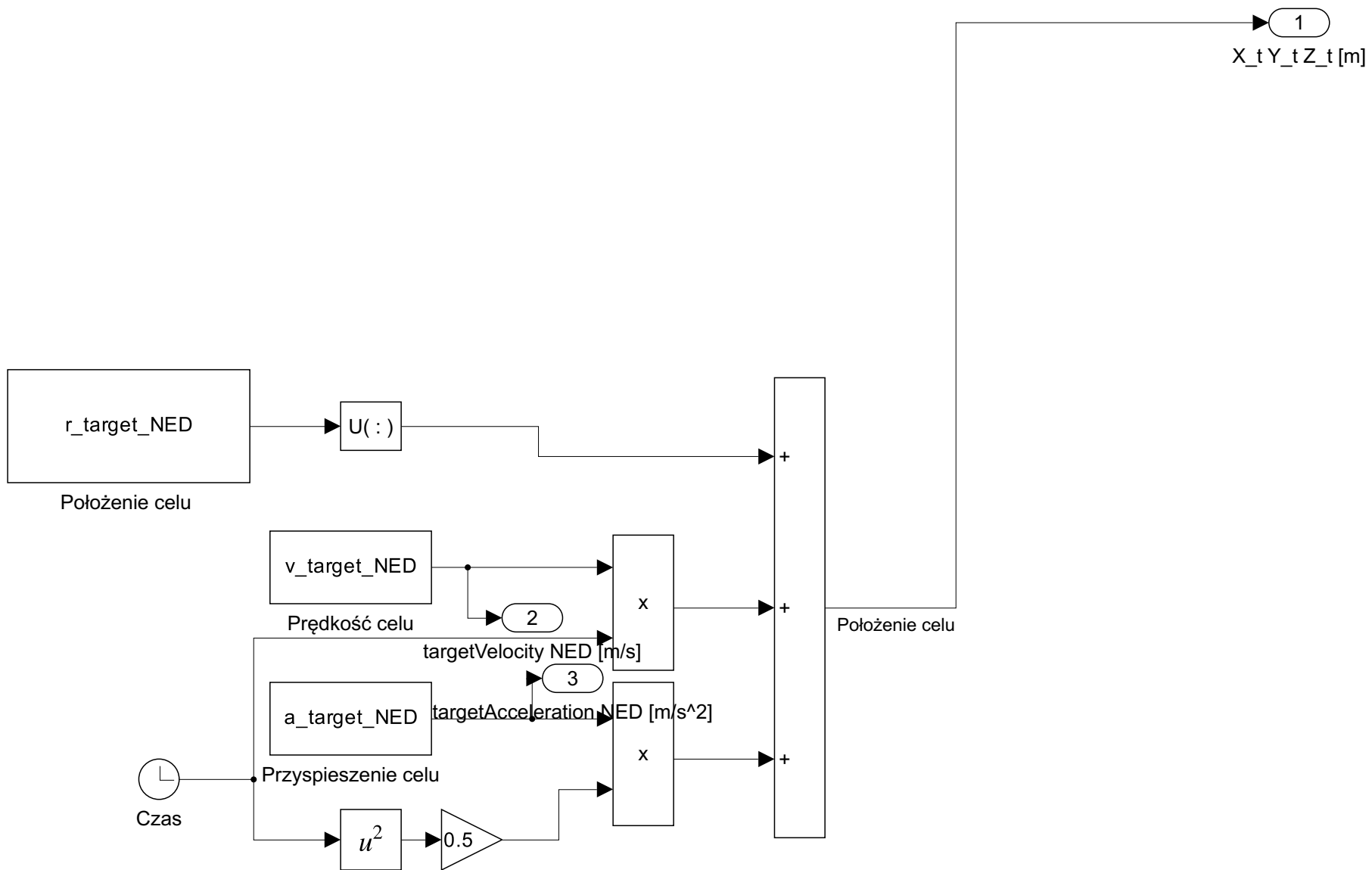
x

x

Reshape

1
C_N*F





1
stateDataInit

Bus

2
t [ns]

convert

:= timeStartup

3
U V W [m/s] BF

convert

:= velBody

7
a_x a_y a_z [m/s^2] BF

convert

:= linearAcceleration

6
Phi Theta Psi [rad]

convert

:= ypr

4
P Q R [rad/s] BF

convert

:= angularVelocity

5
X Y Z [m] ECEF

convert

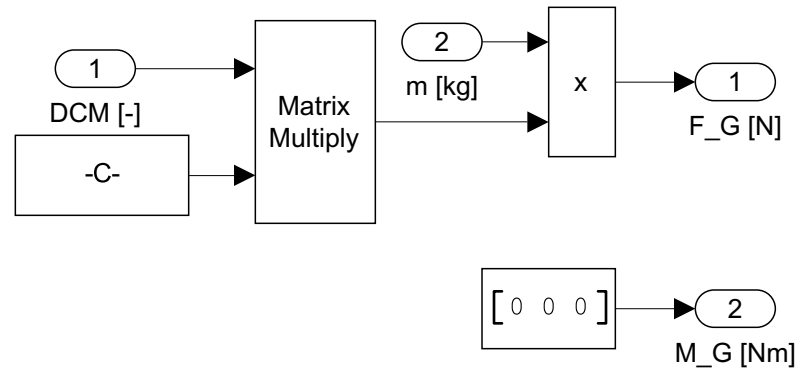
:= posECEF

8
seekerData

:= seekerData

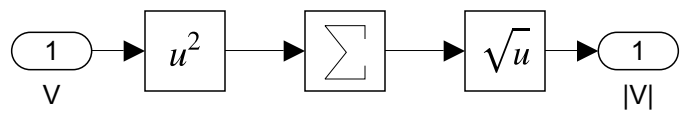
Bus → 1
stateData

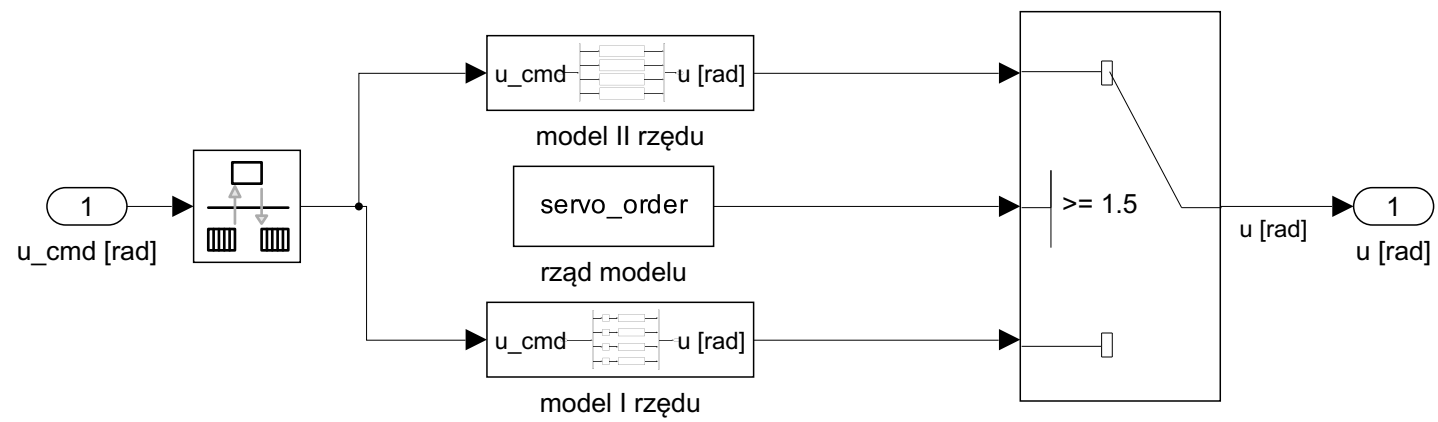


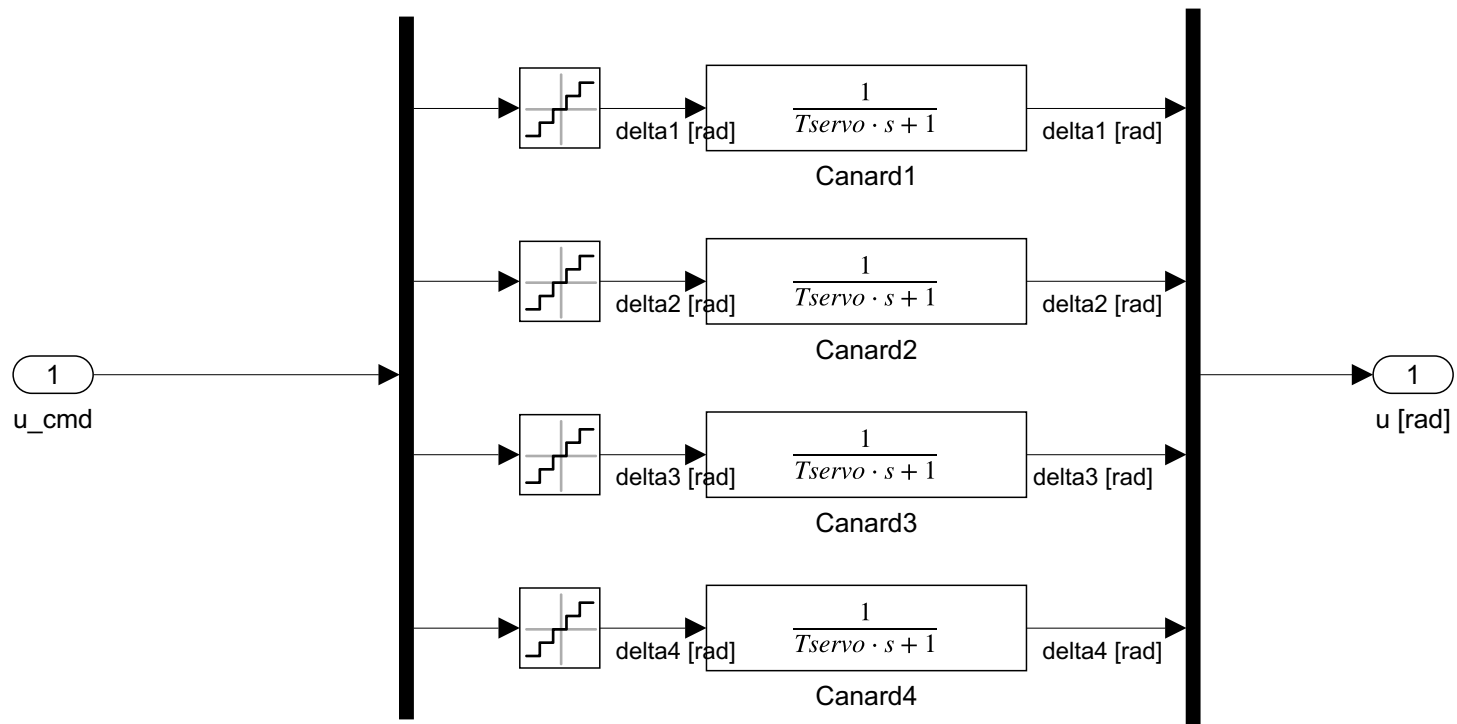


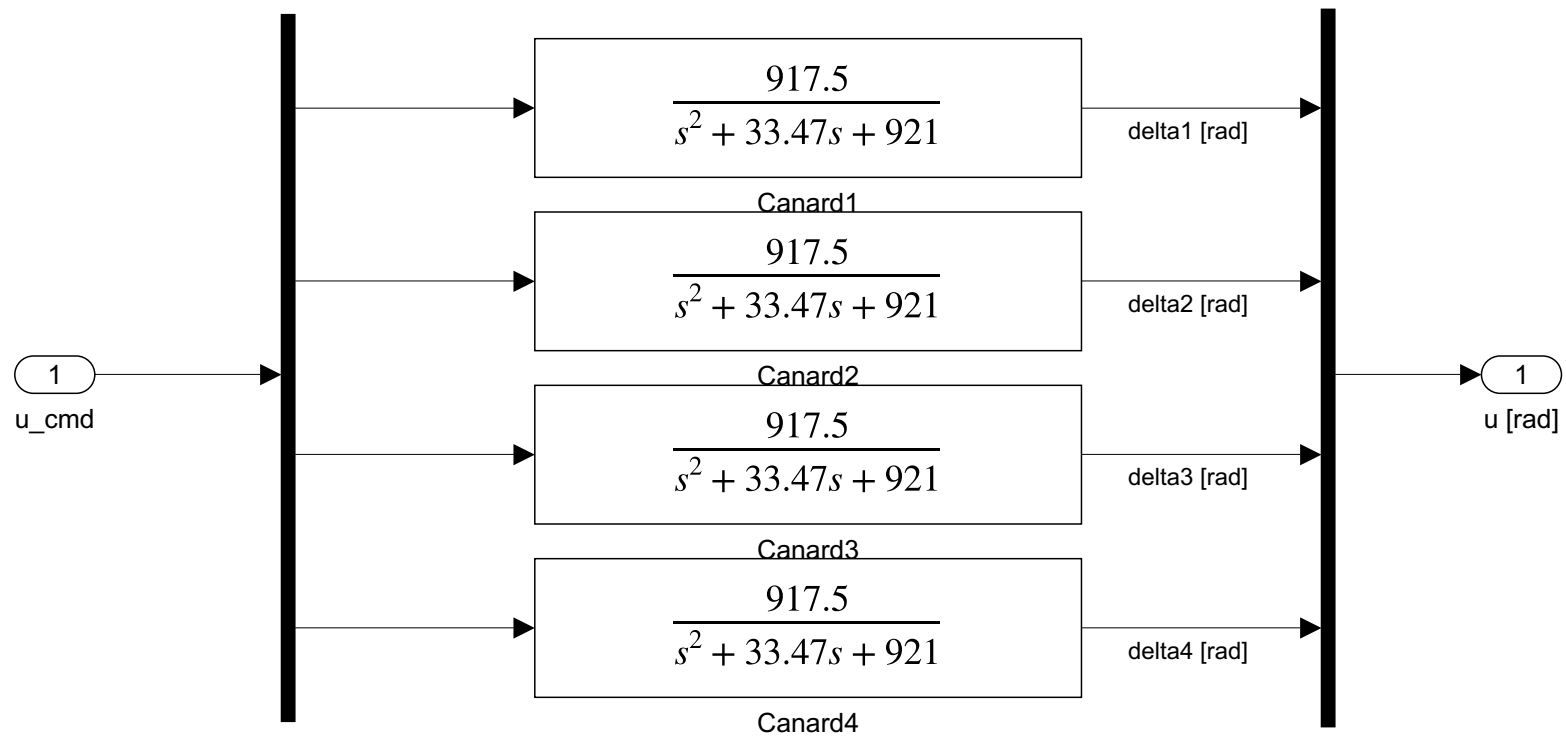


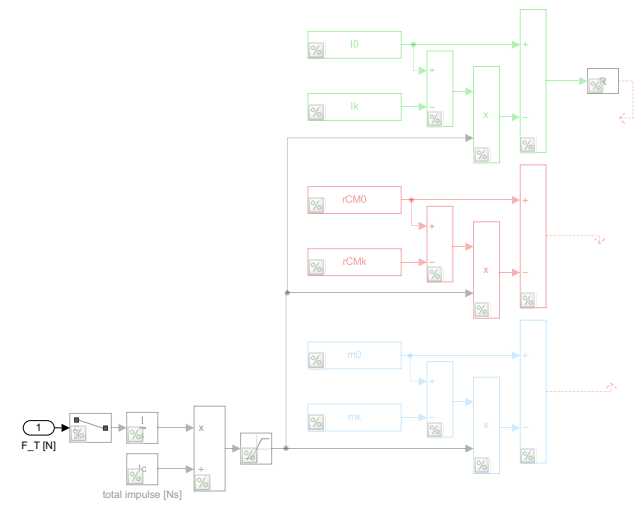


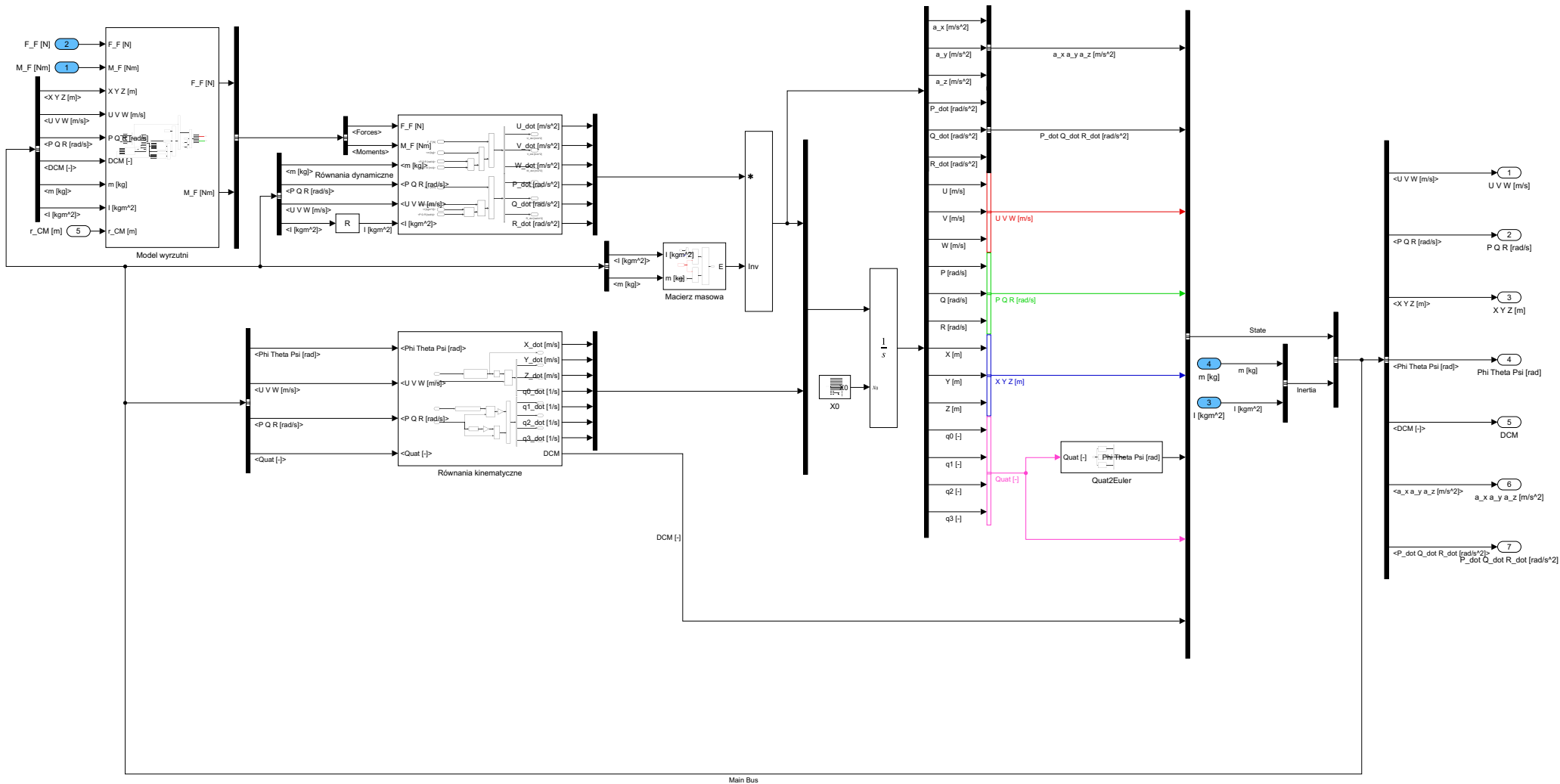


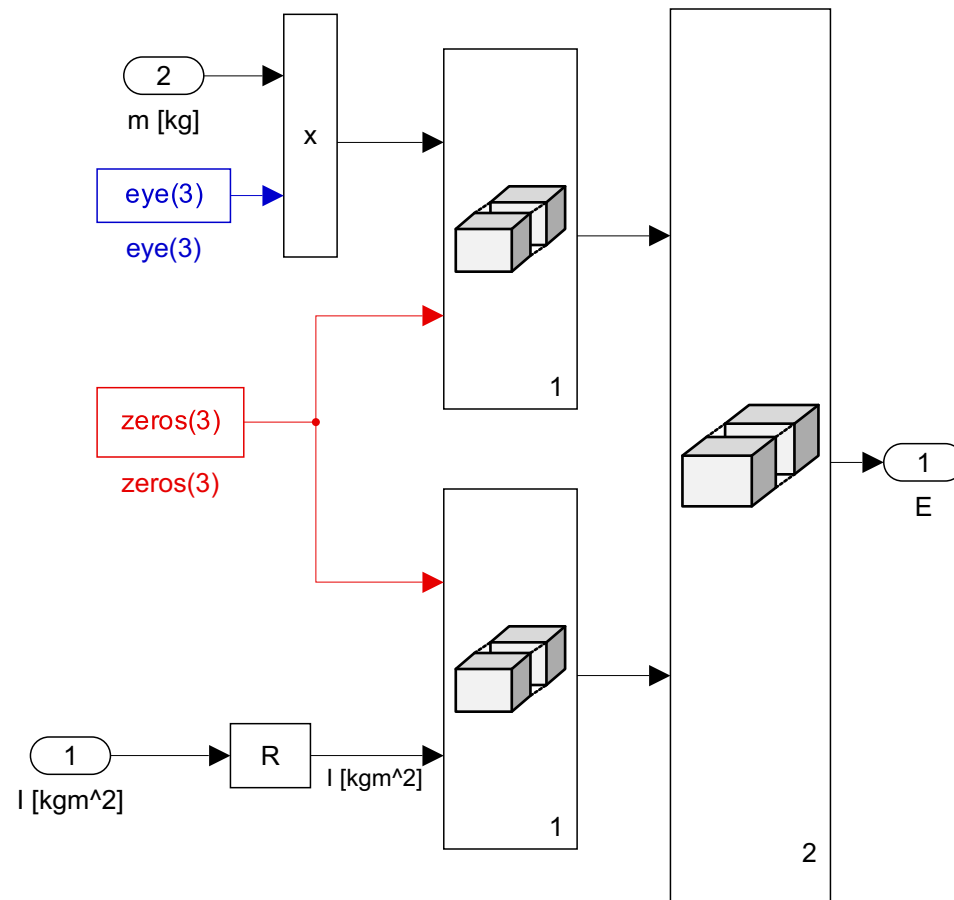


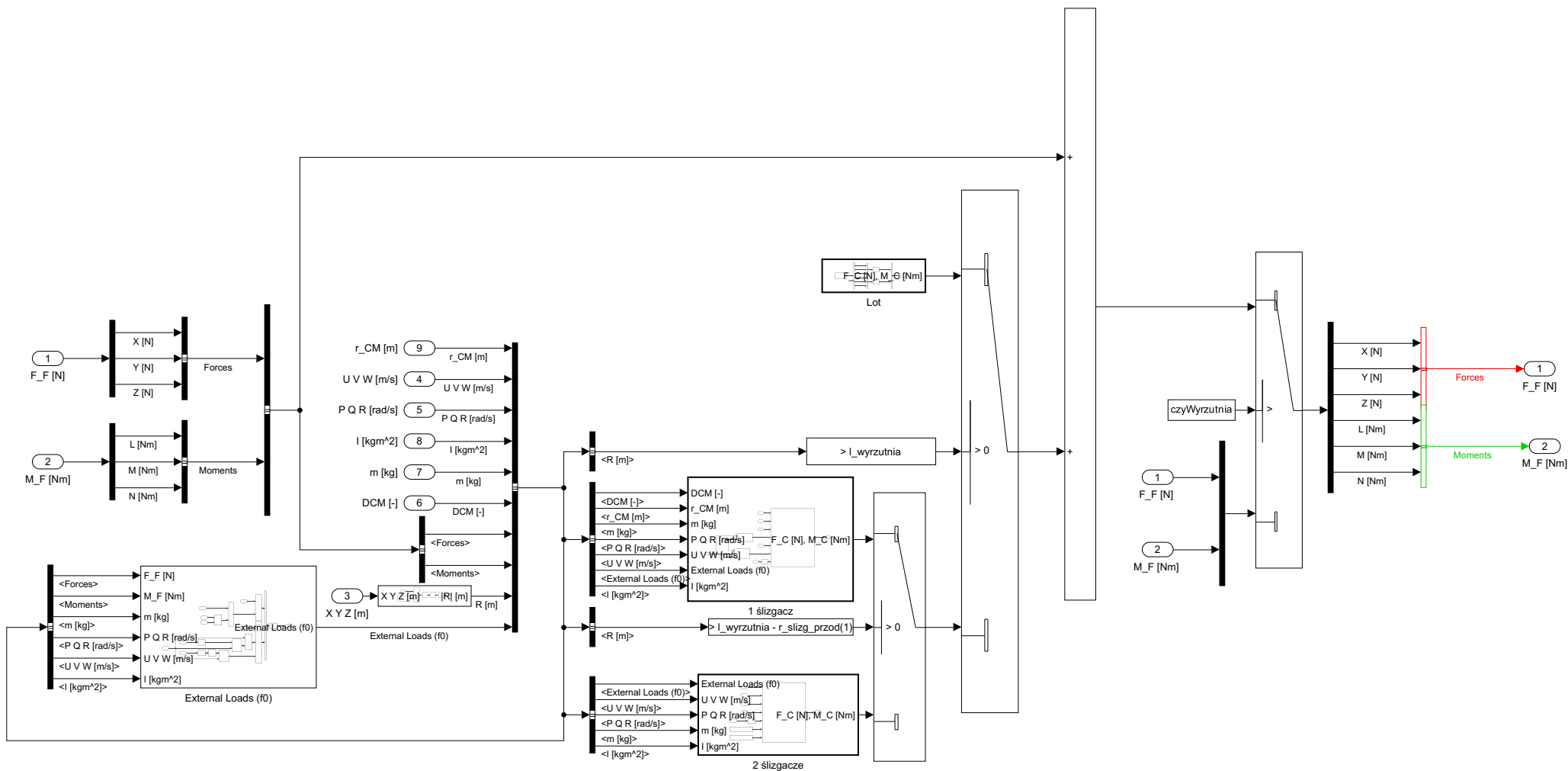


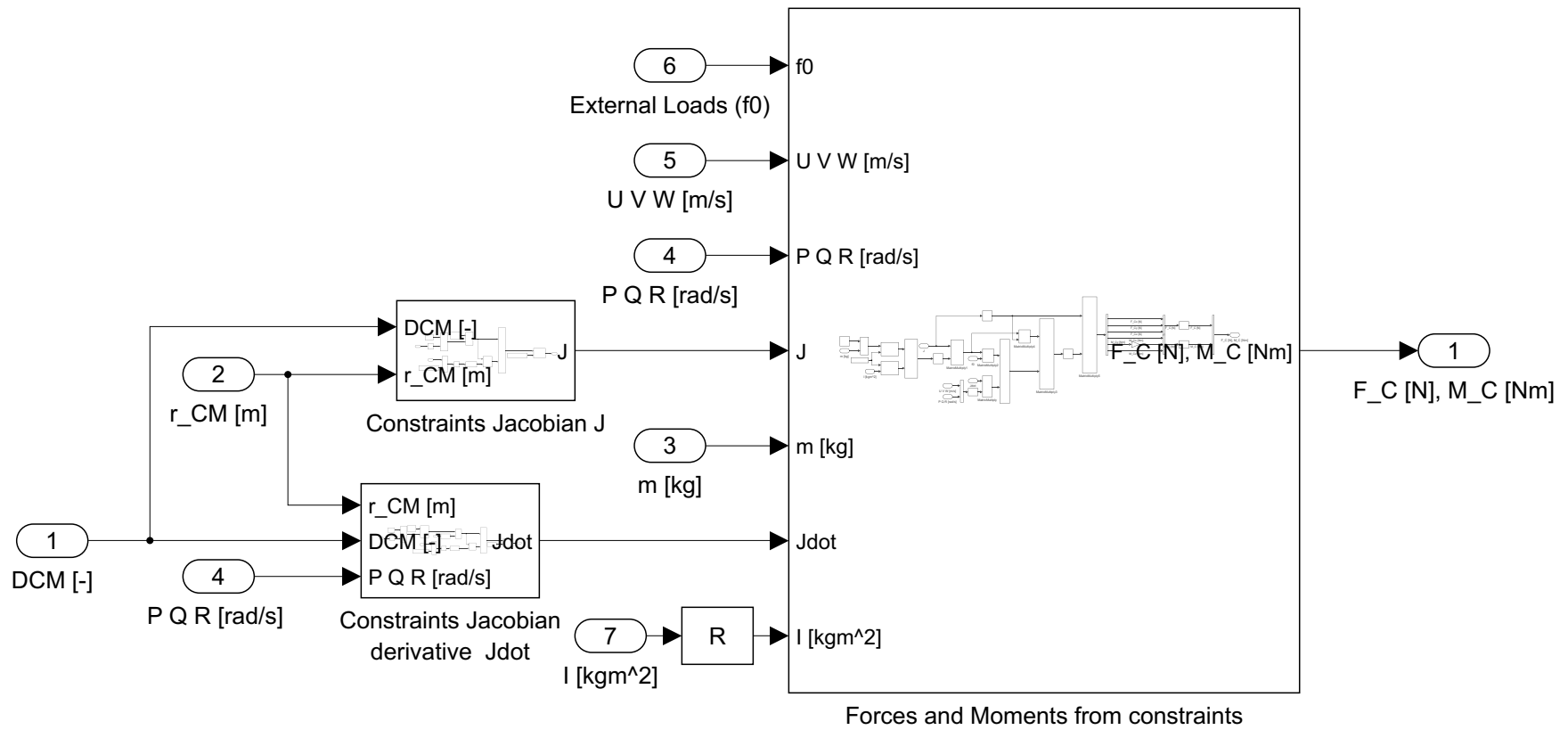




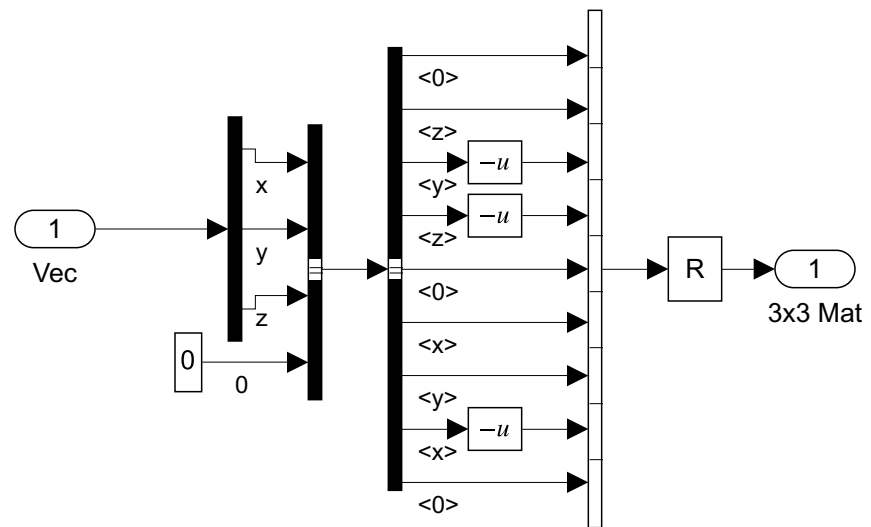


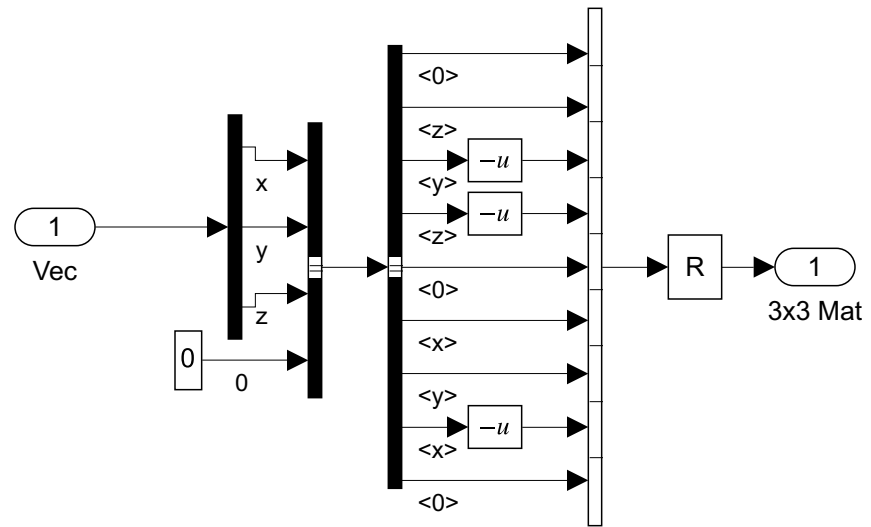


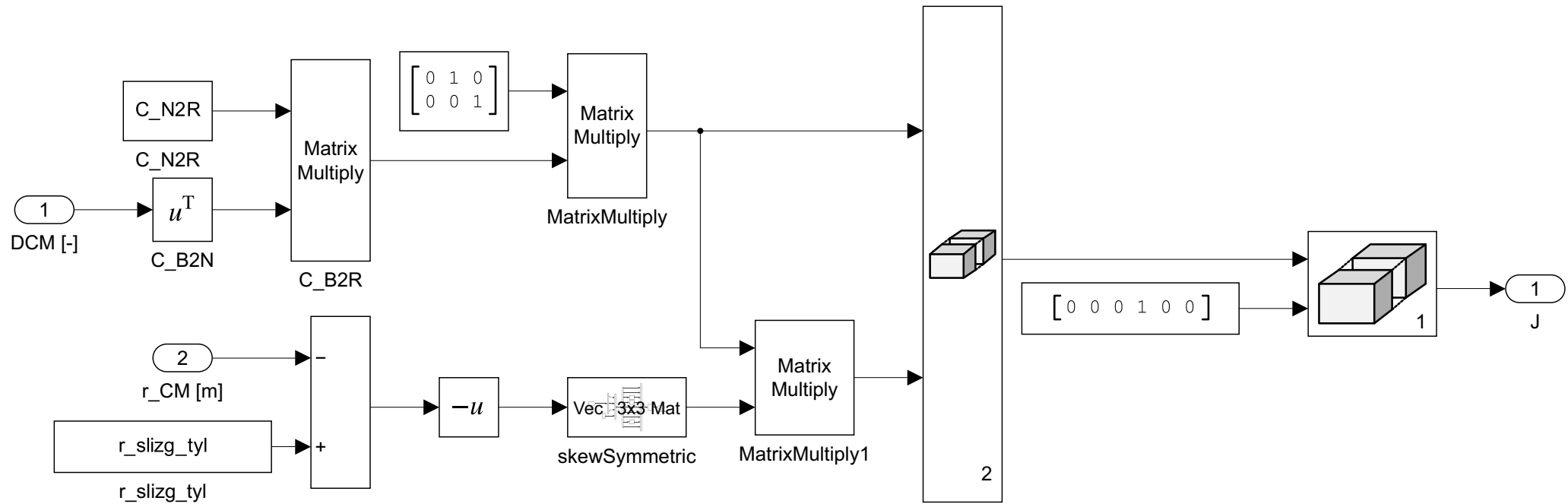


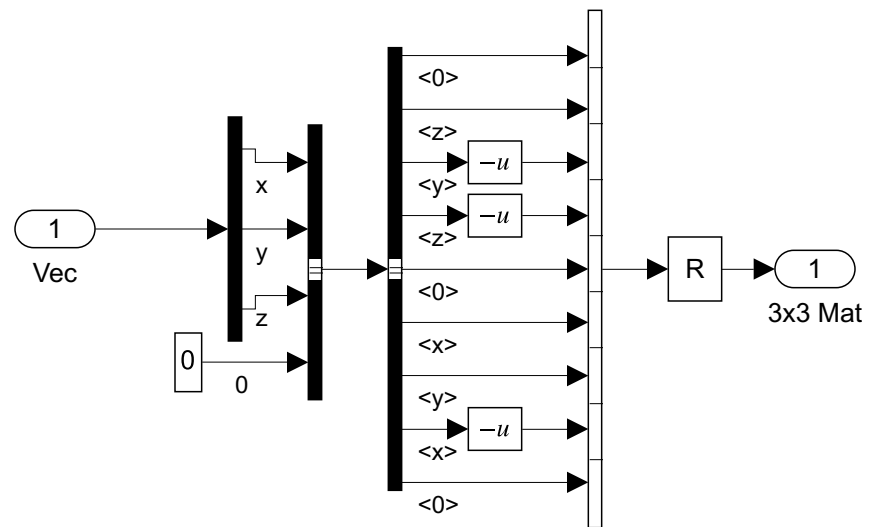


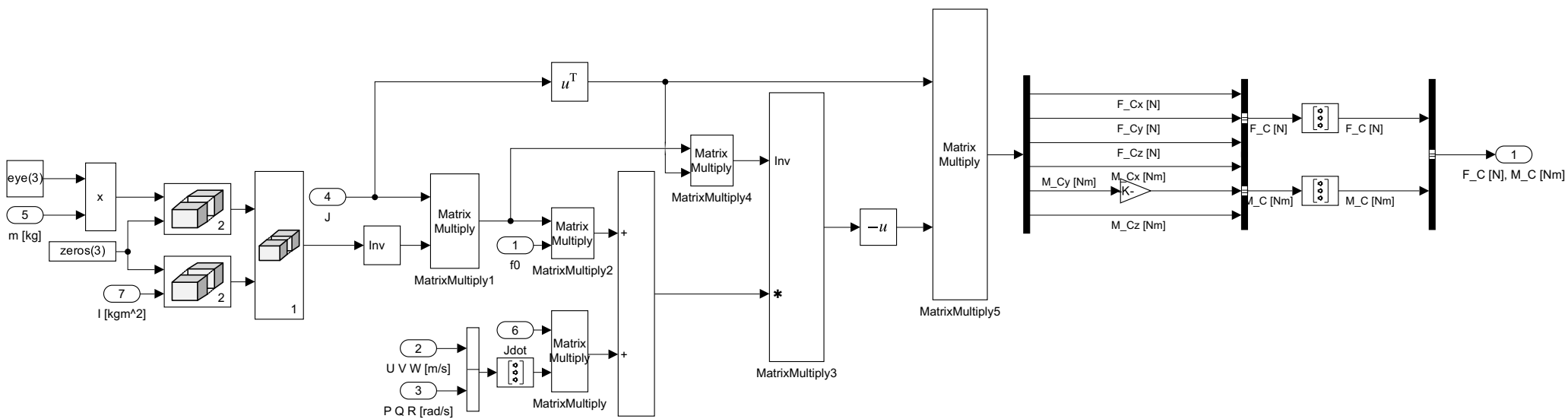


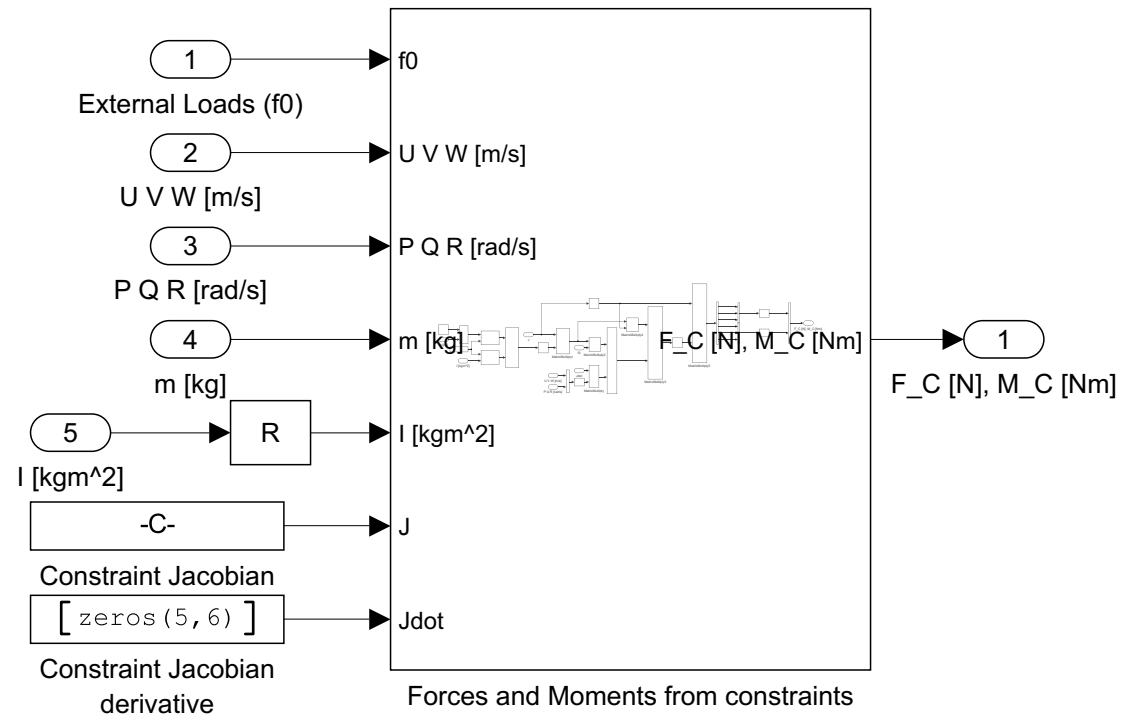


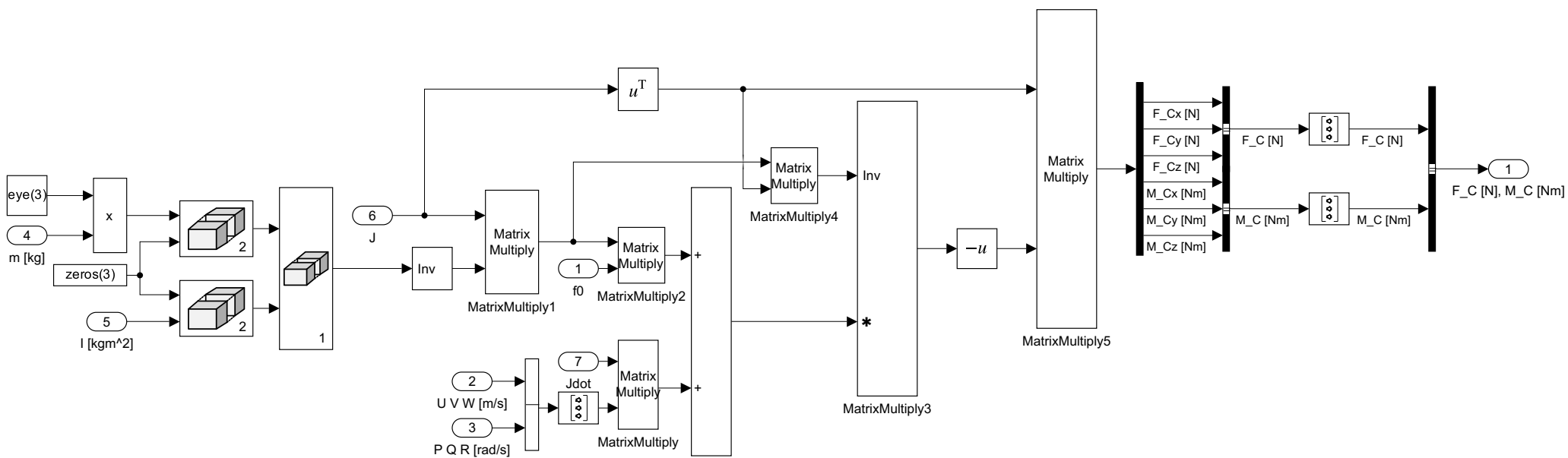


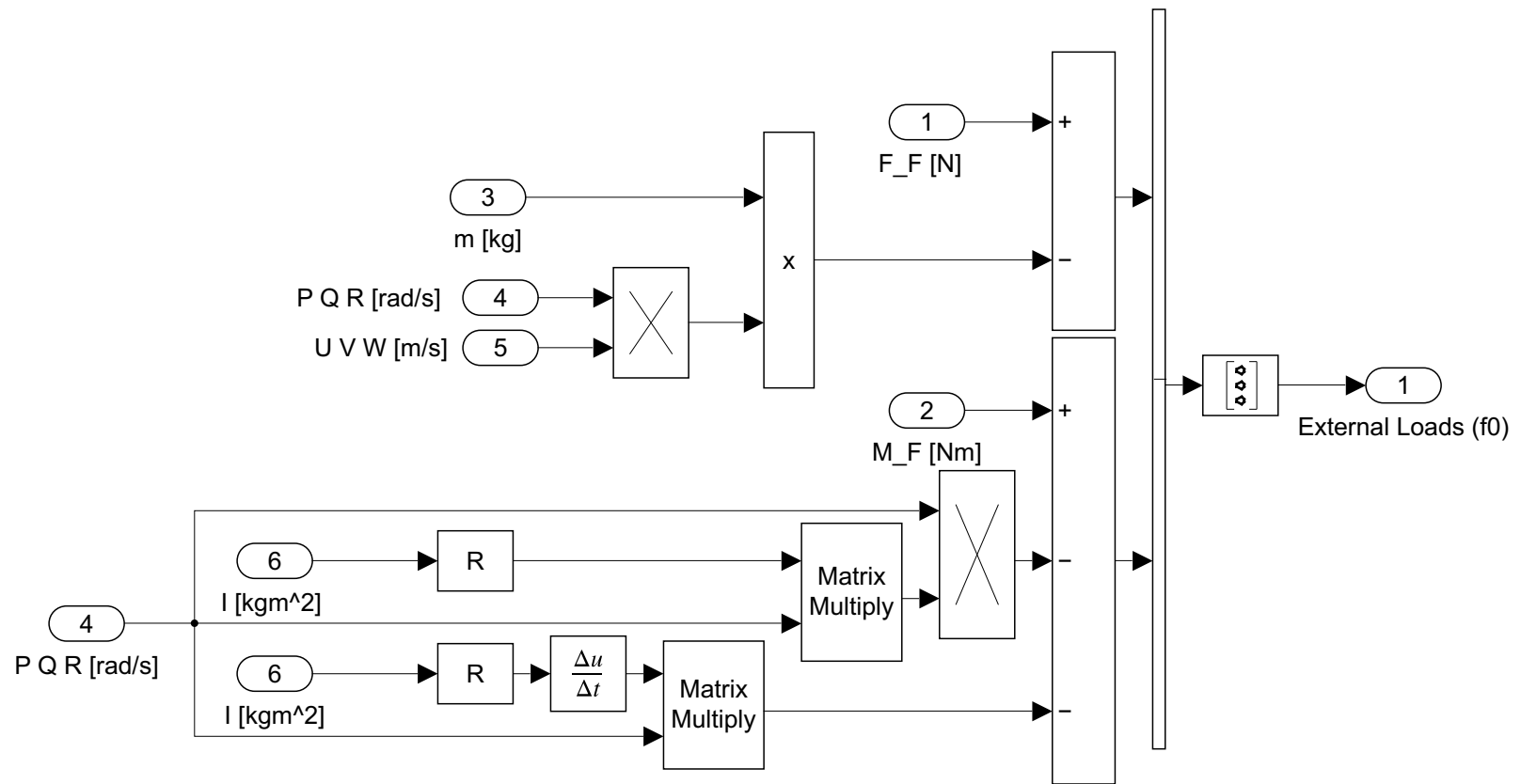


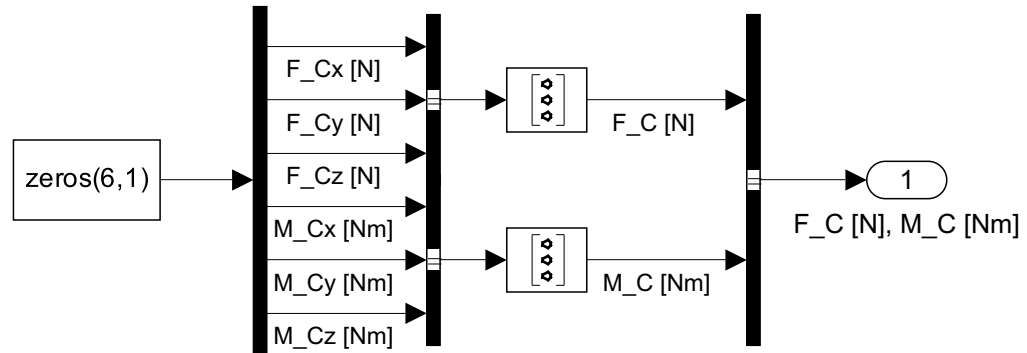


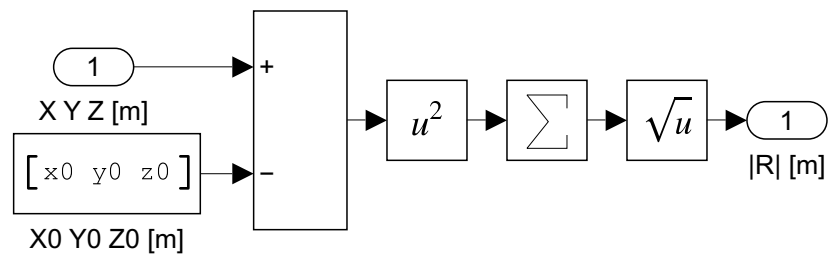


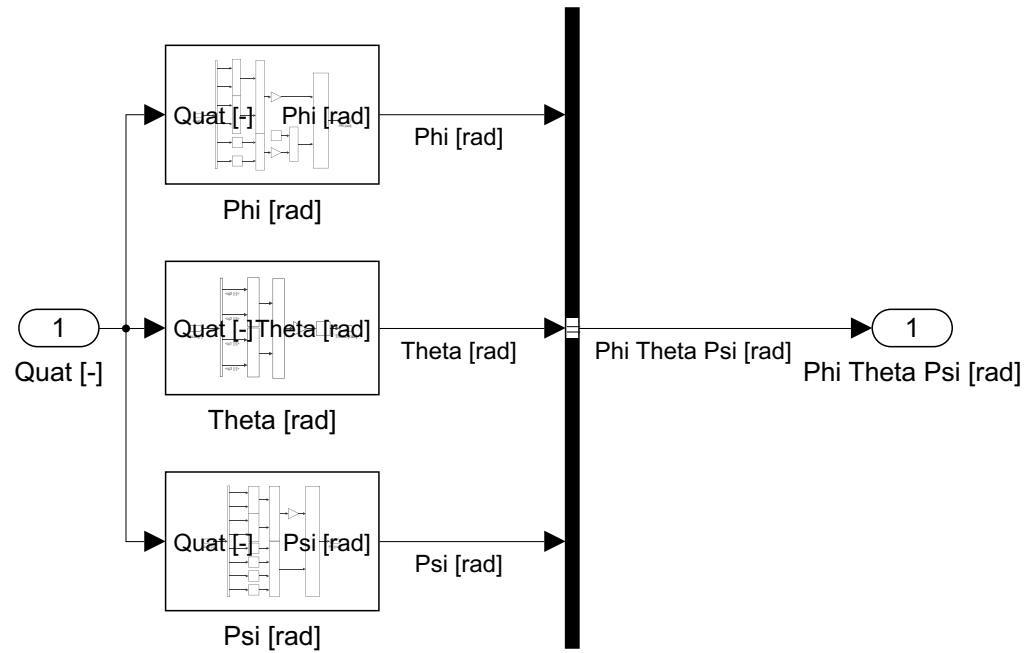


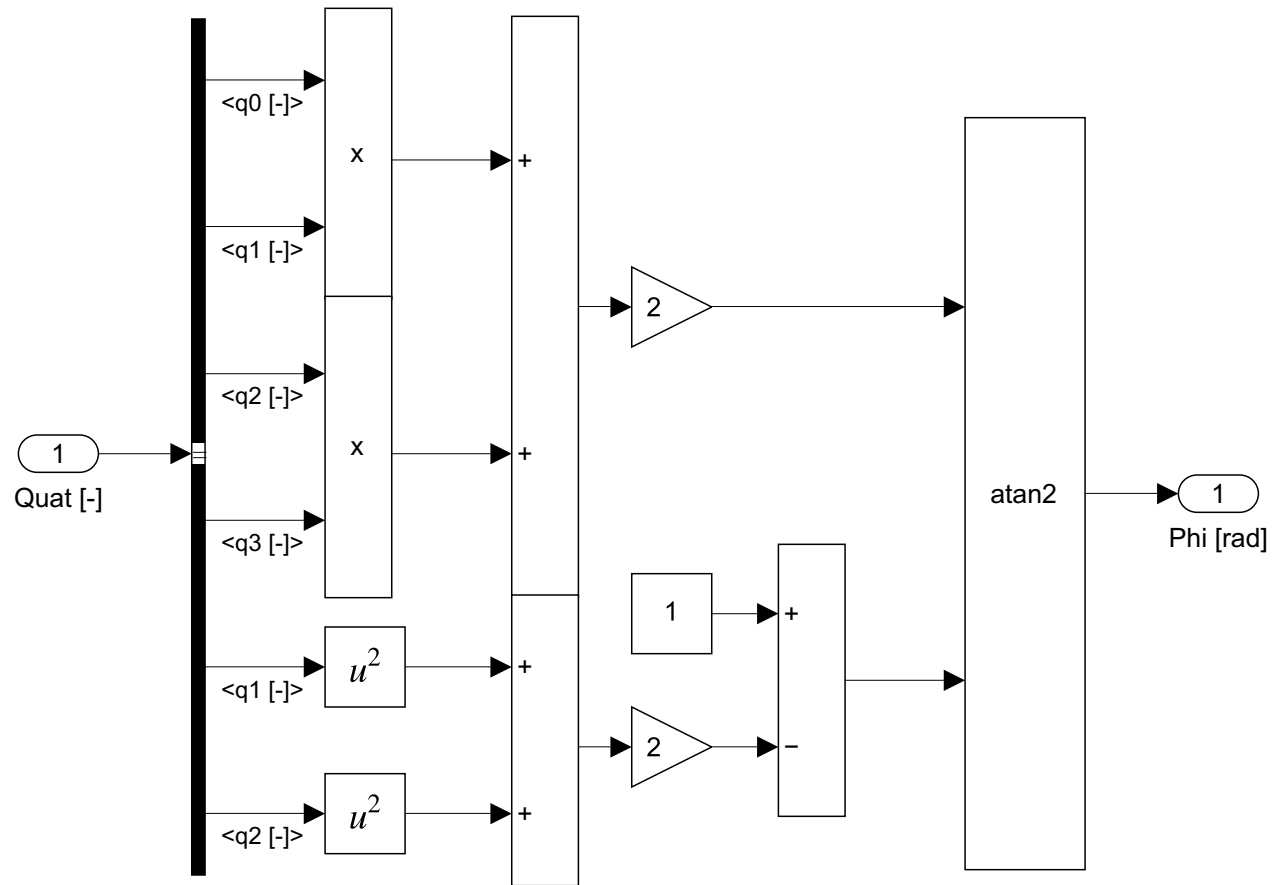


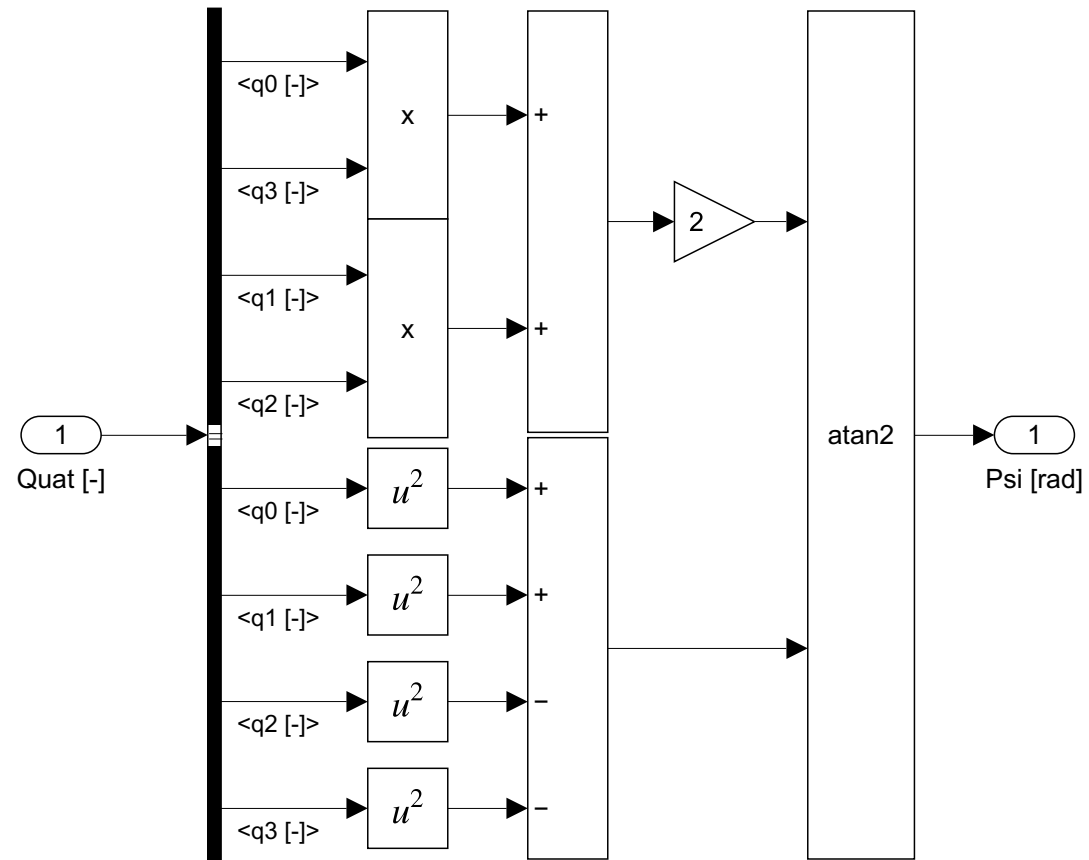


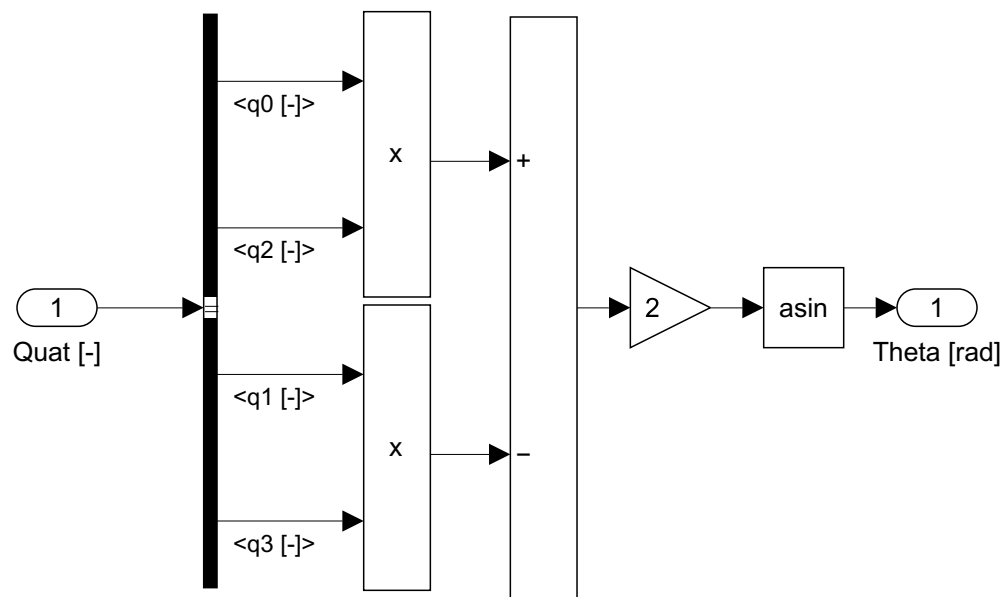


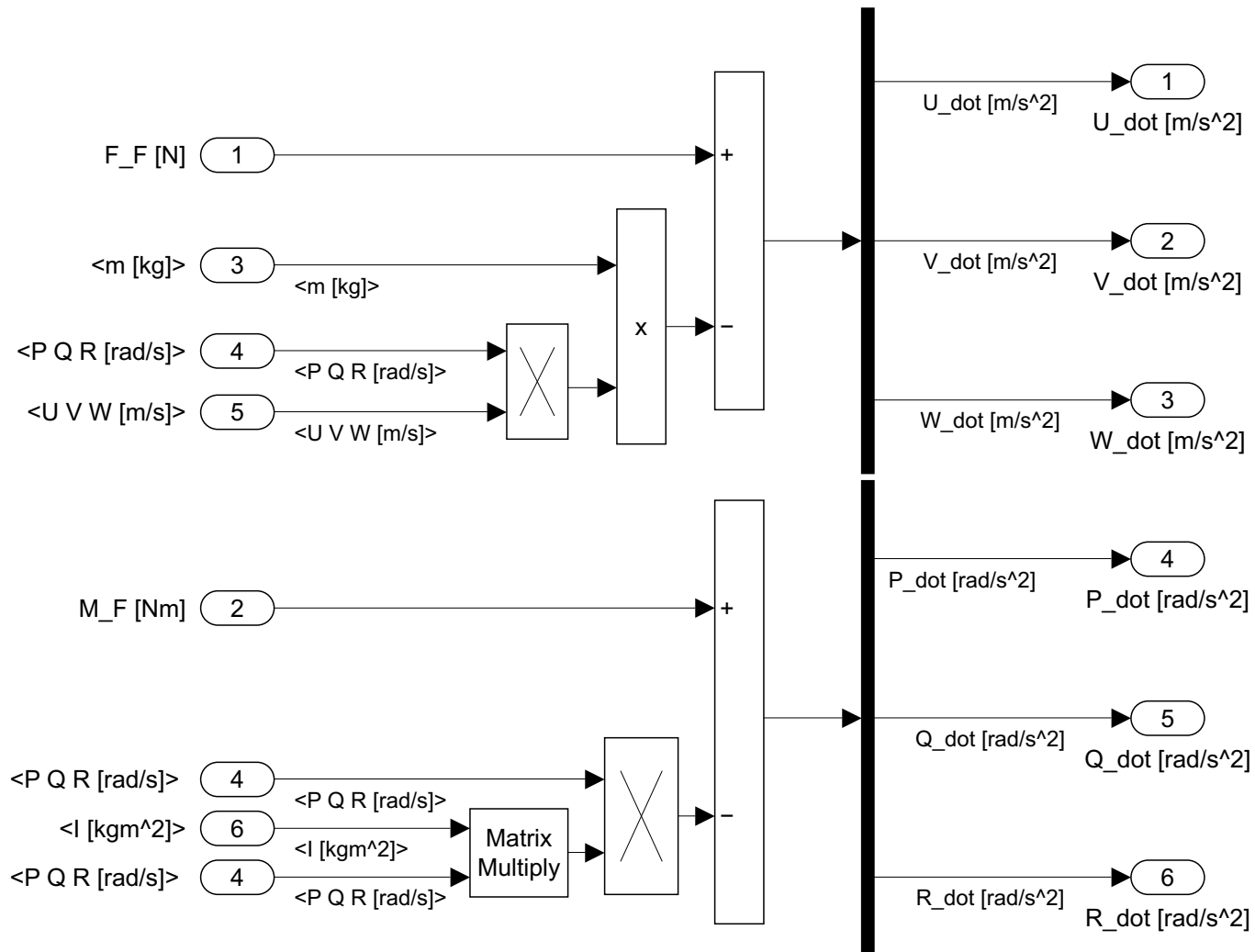


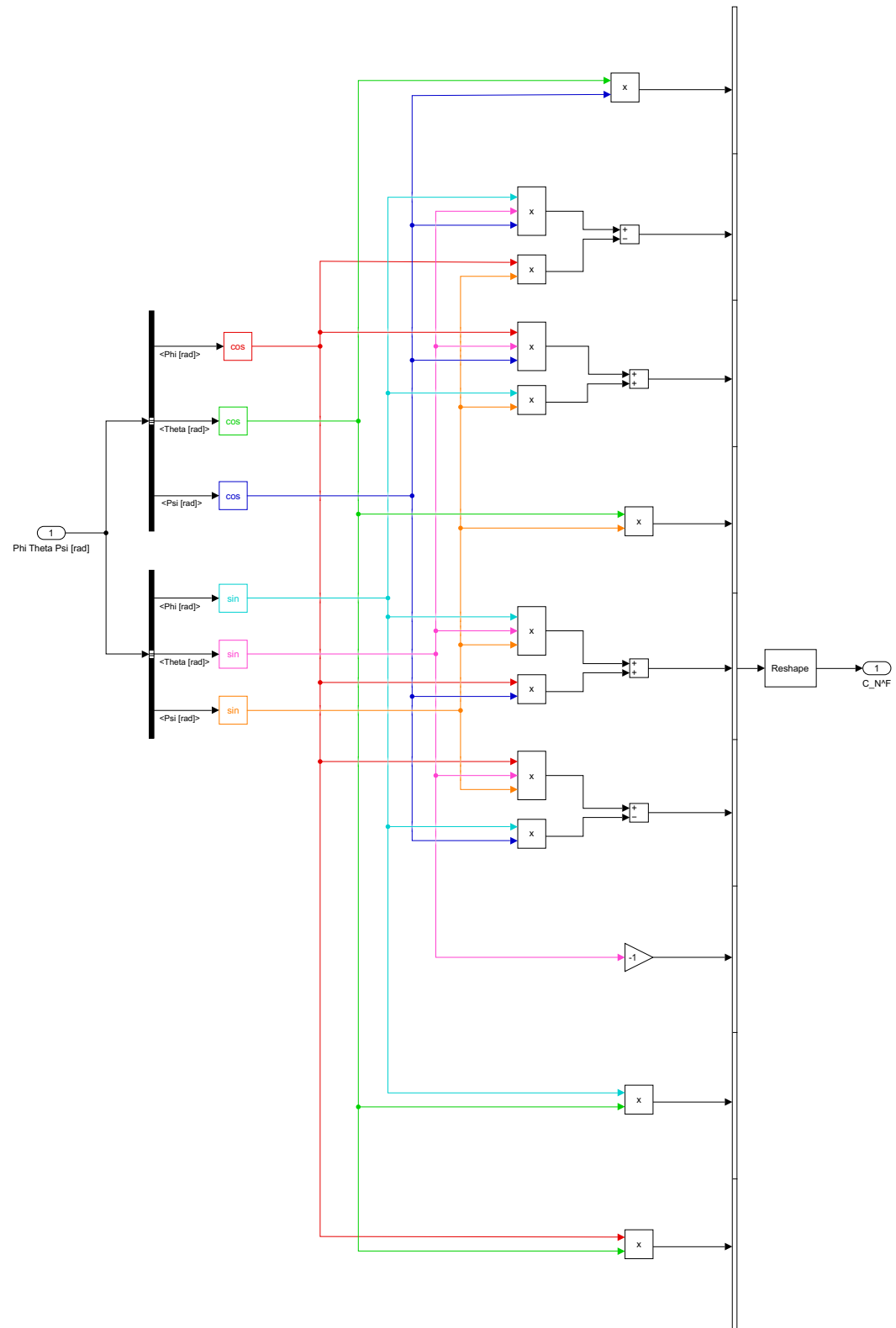


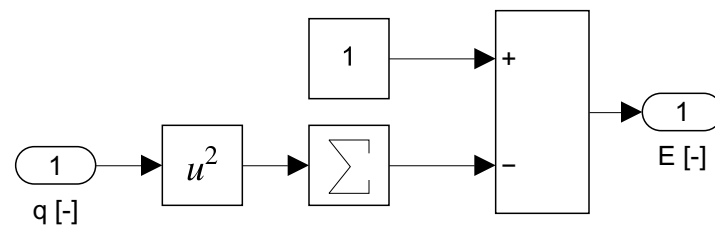


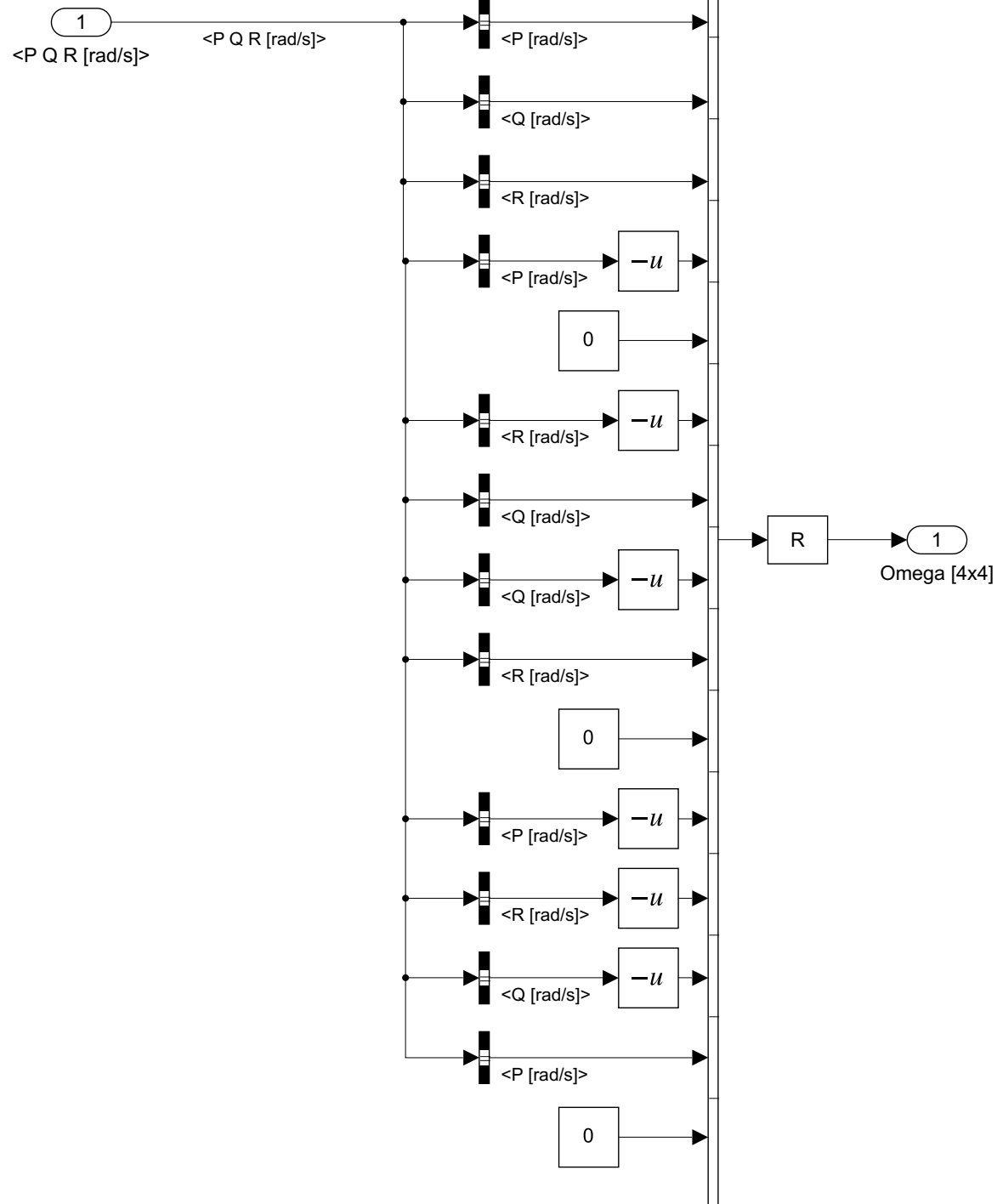


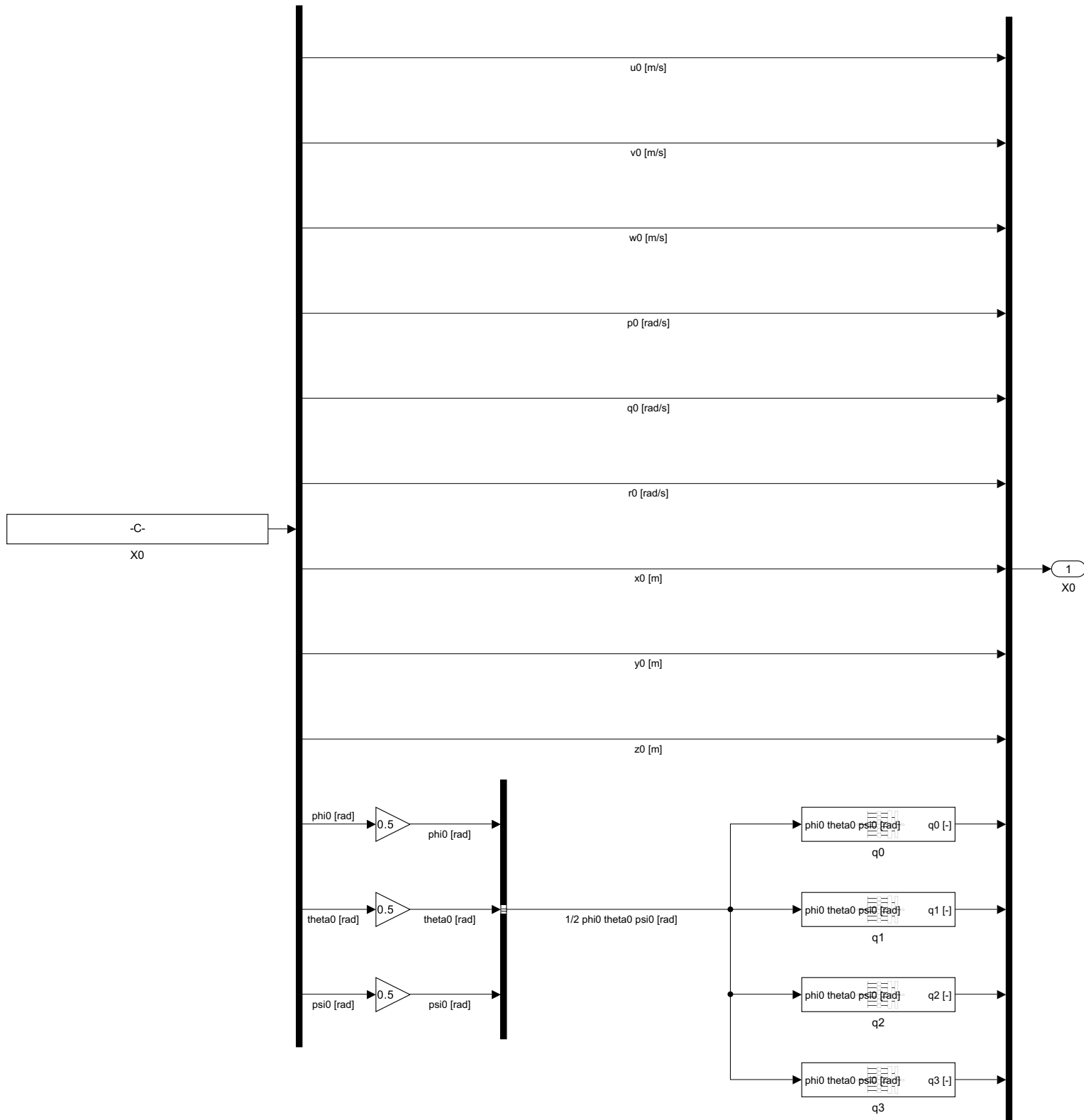


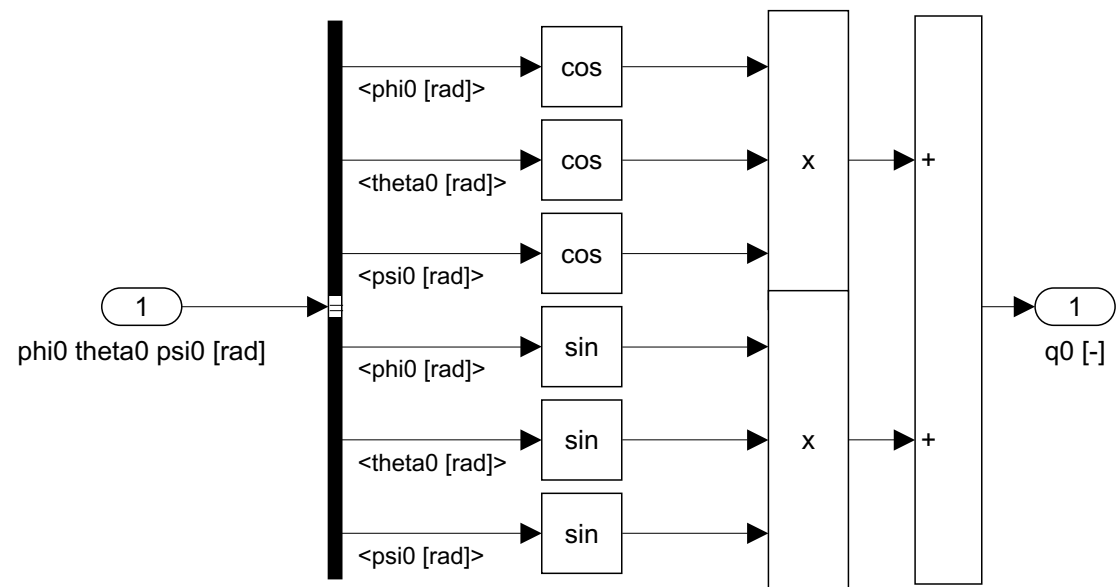


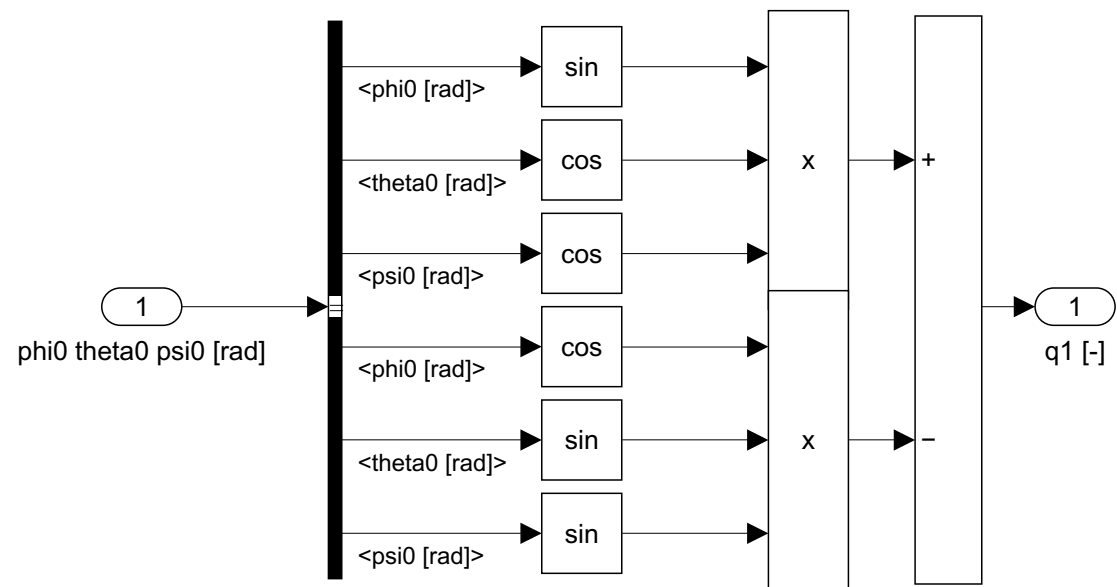


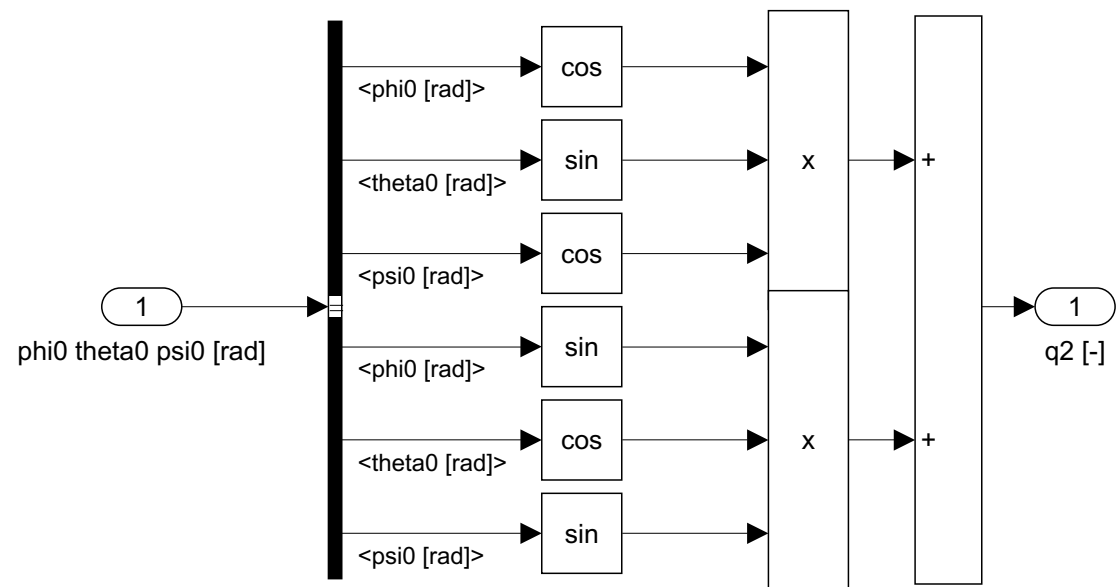


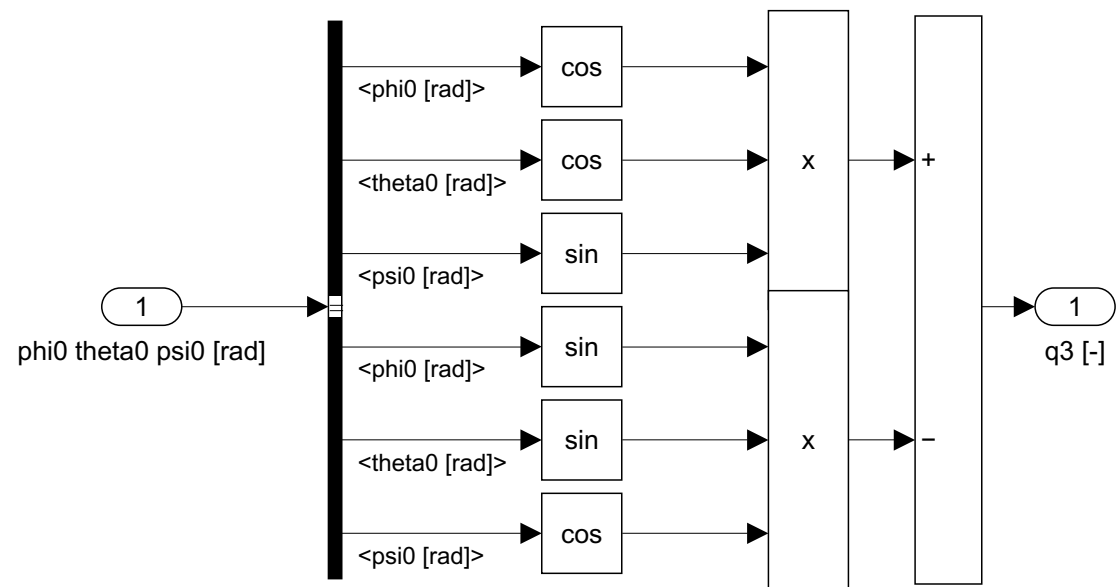


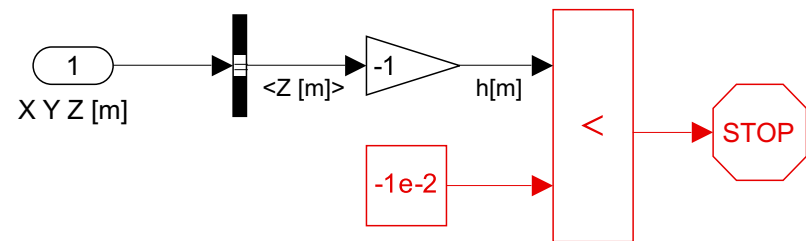


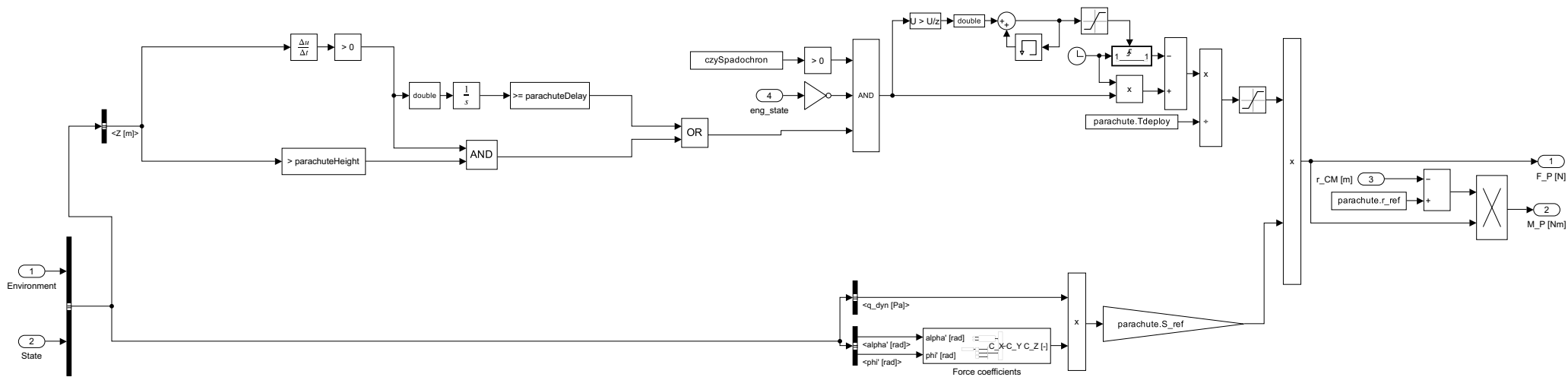


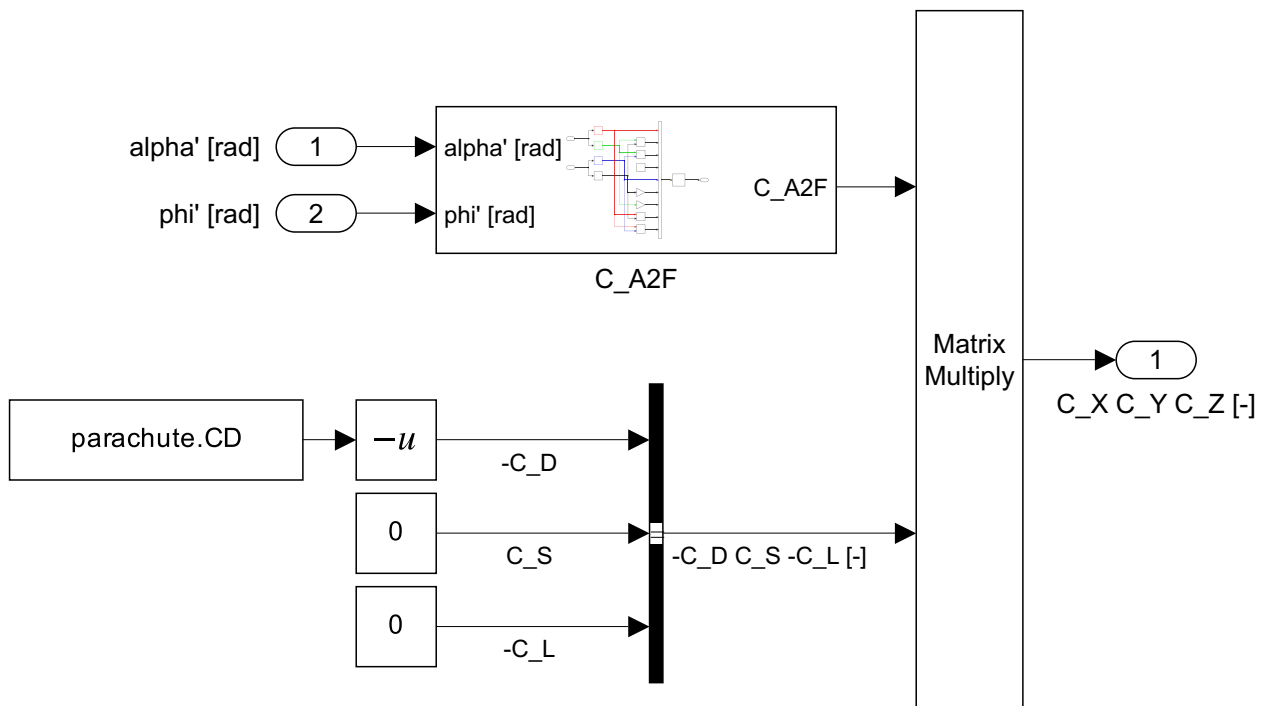










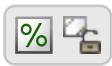
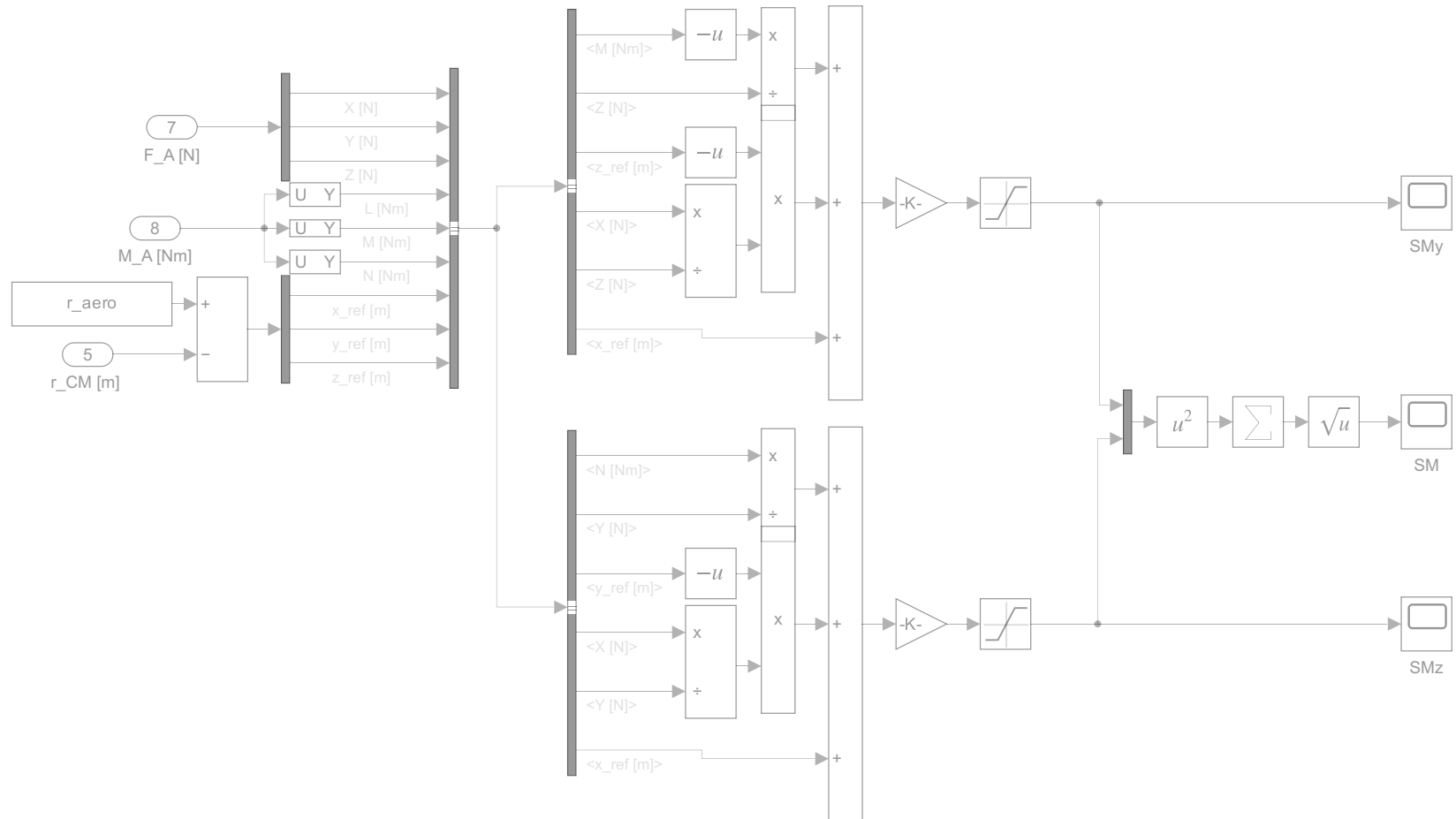
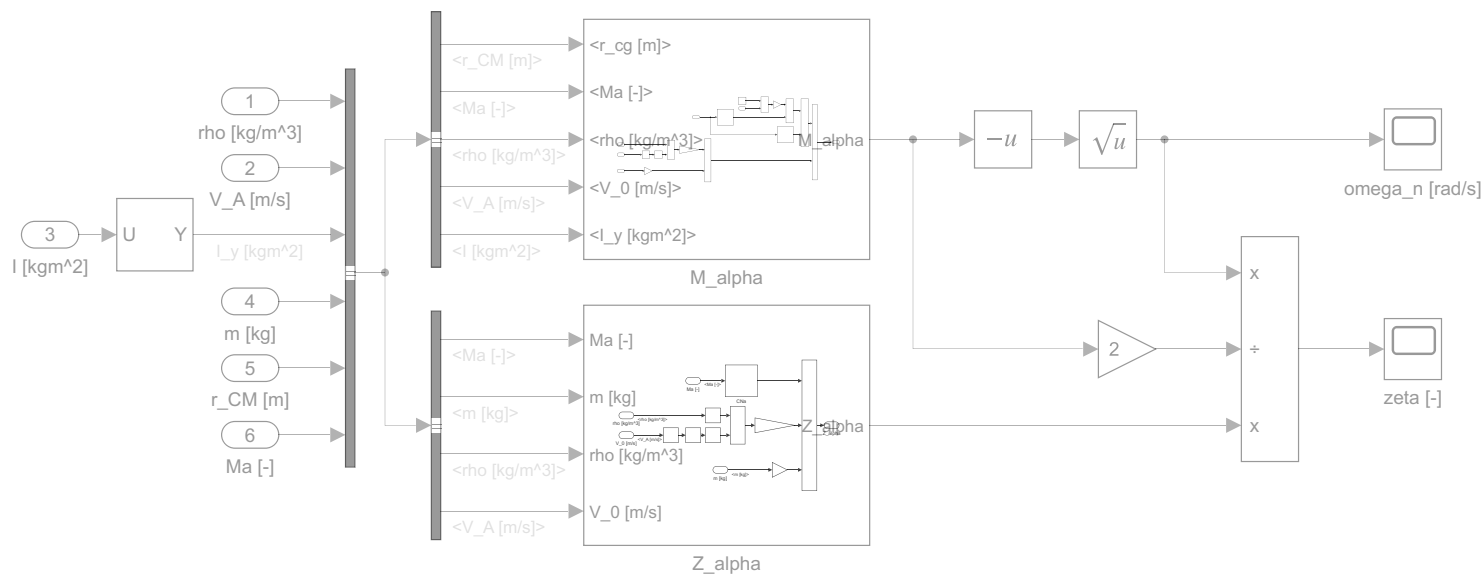


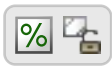
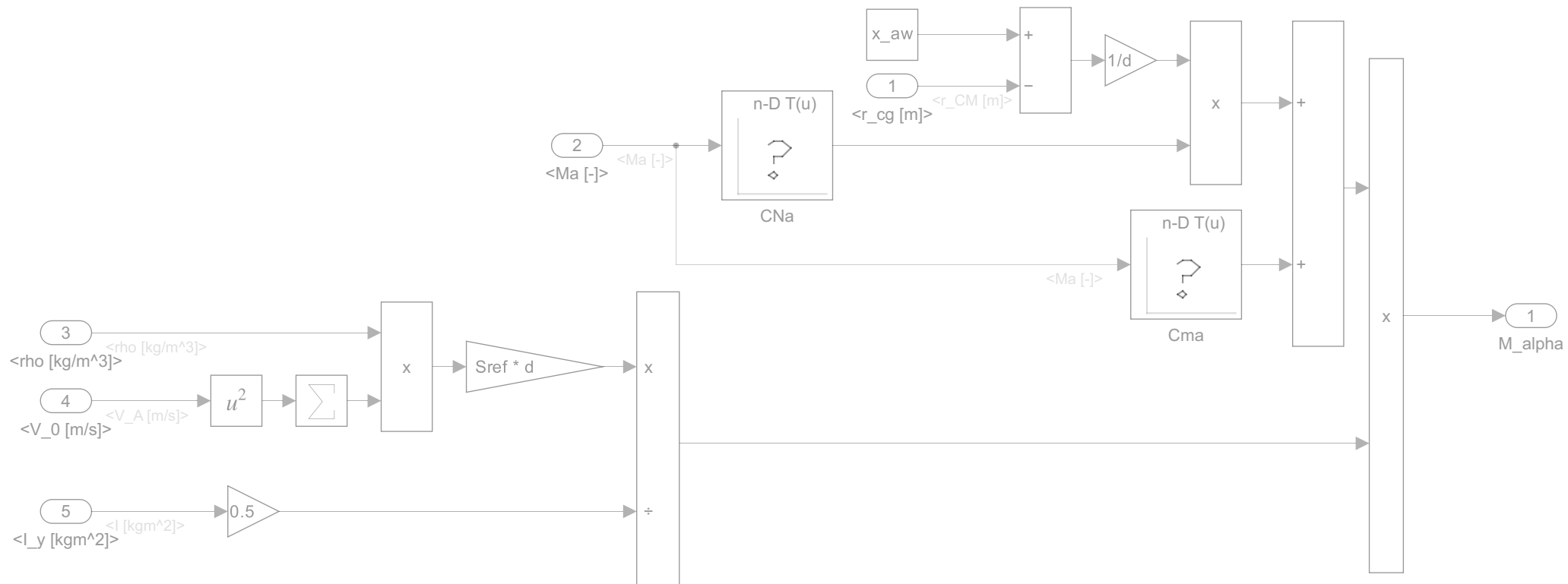


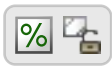
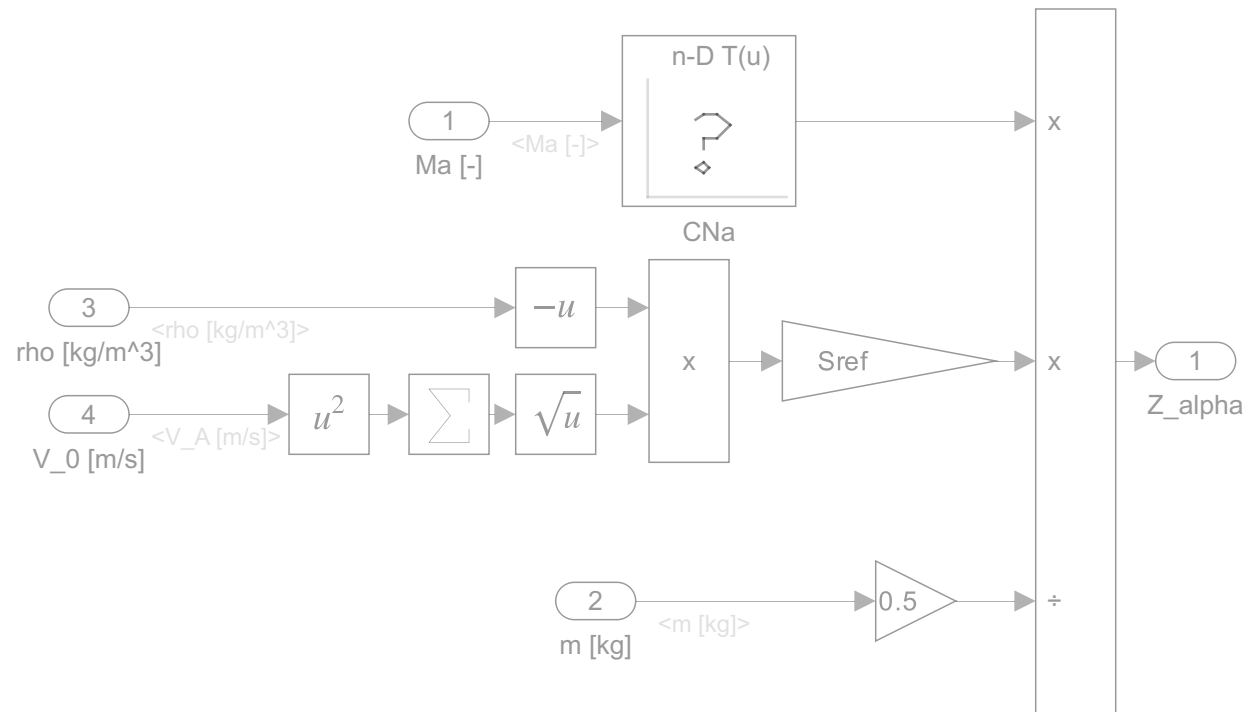


1

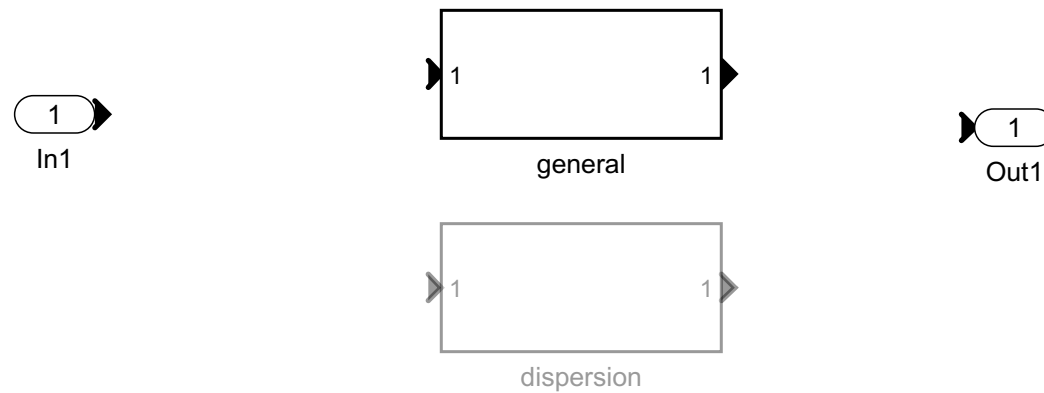
1



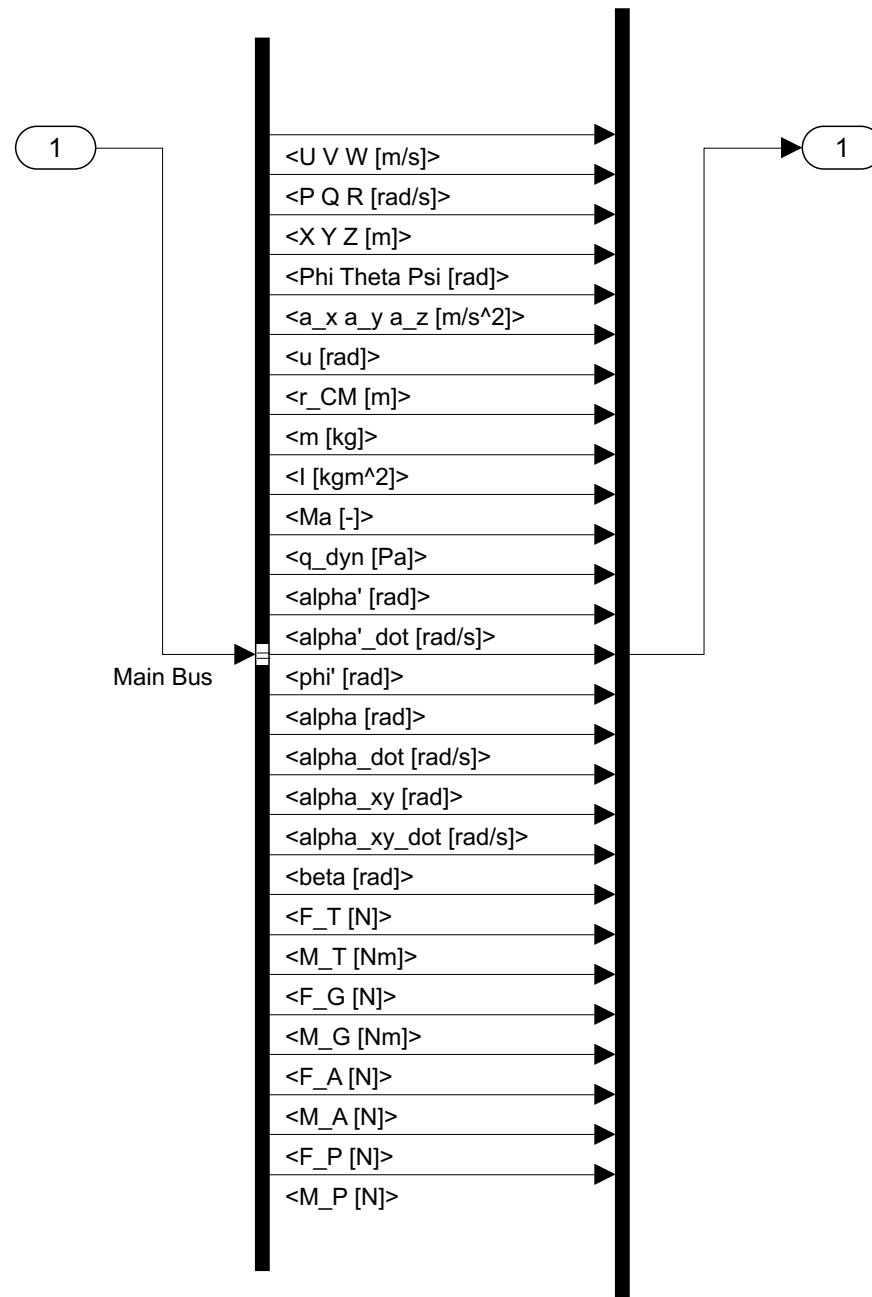




- 1) Add [Subsystem](#) or [Model](#) blocks as valid variant choices.
- 2) You cannot connect blocks at this level. At simulation, connectivity is automatically determined, based on the active variant and port name matching.

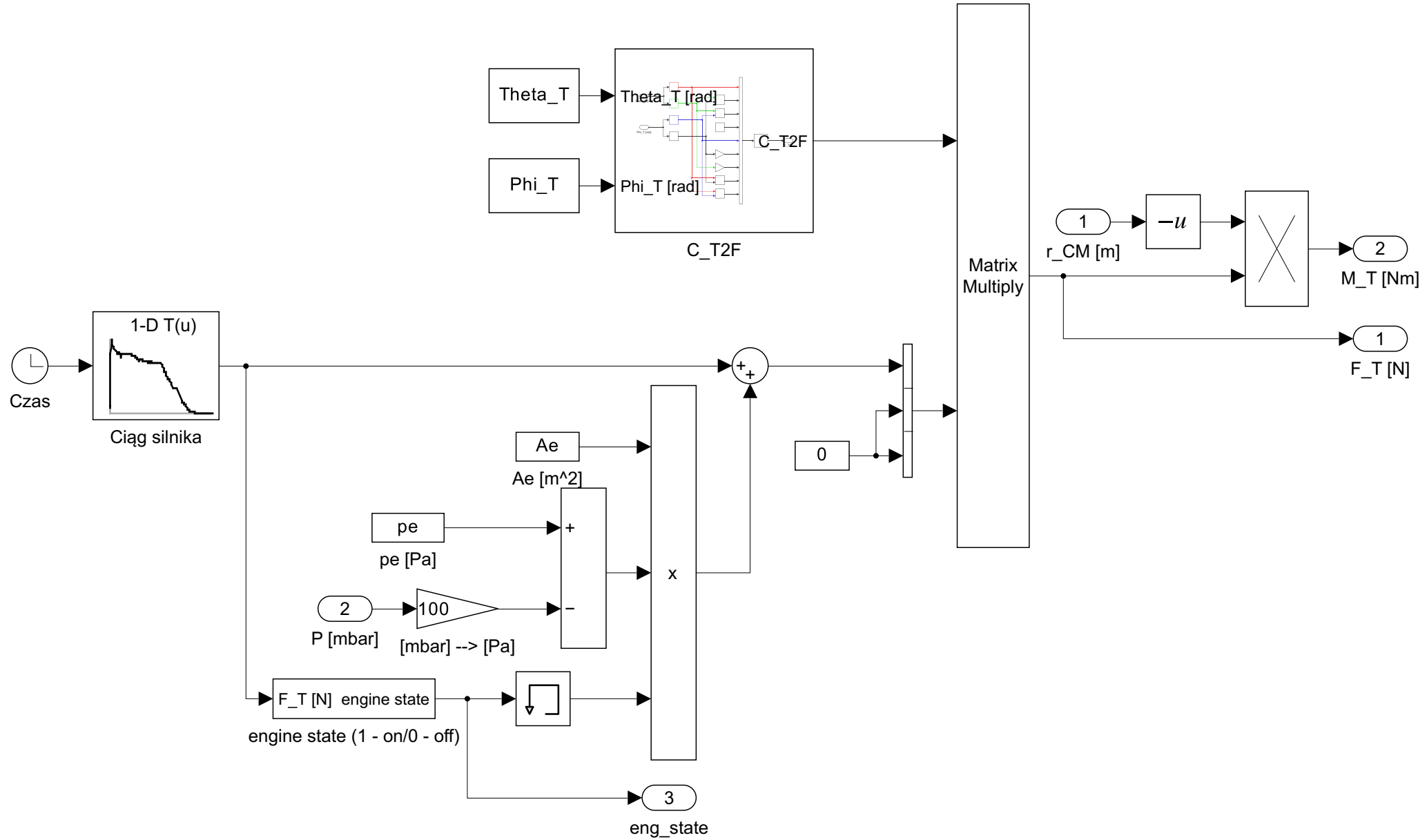


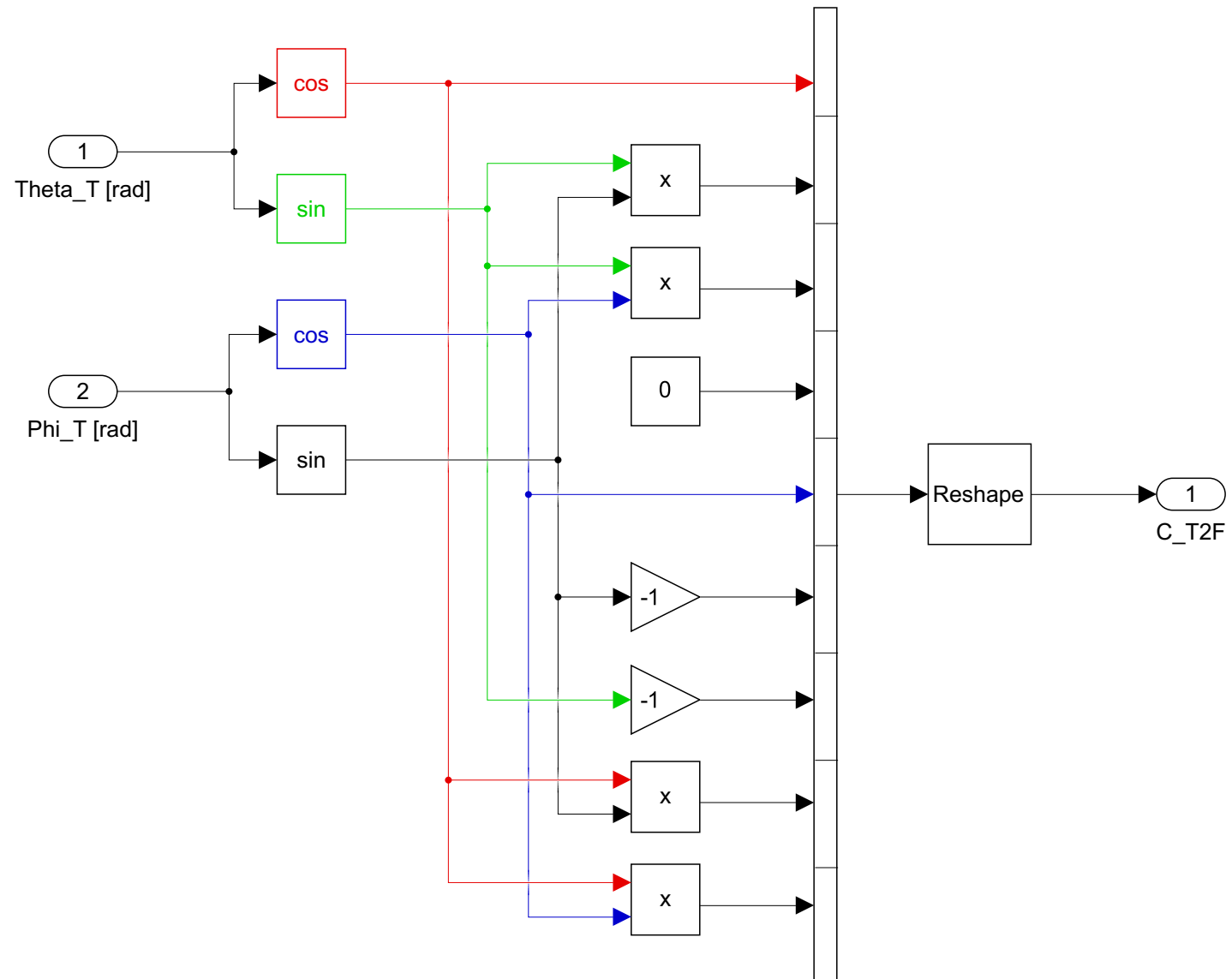


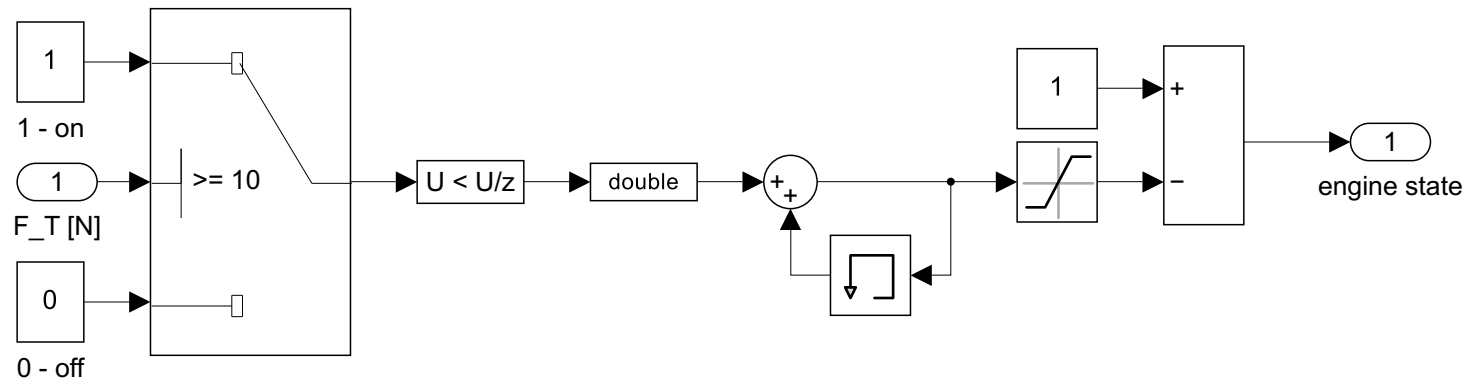


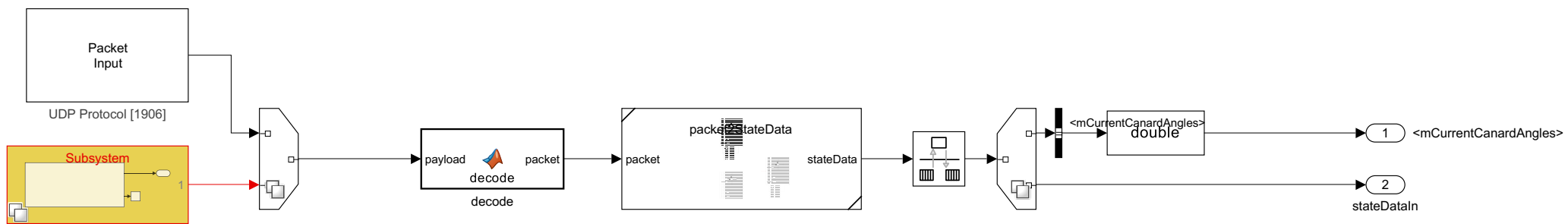
U | V | W | P | Q | R | X | Y | Z |
 Phi | Theta | Psi | a_x | a_y | a_z

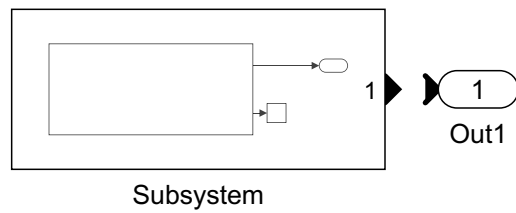


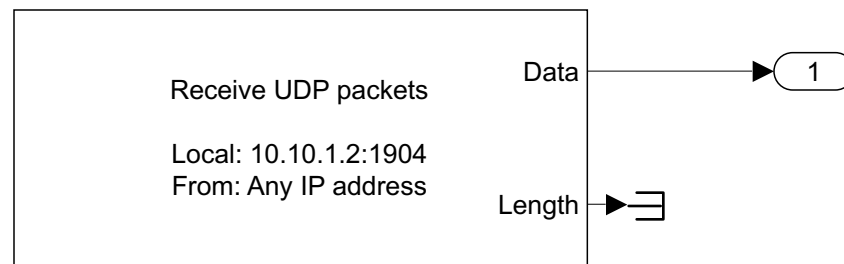


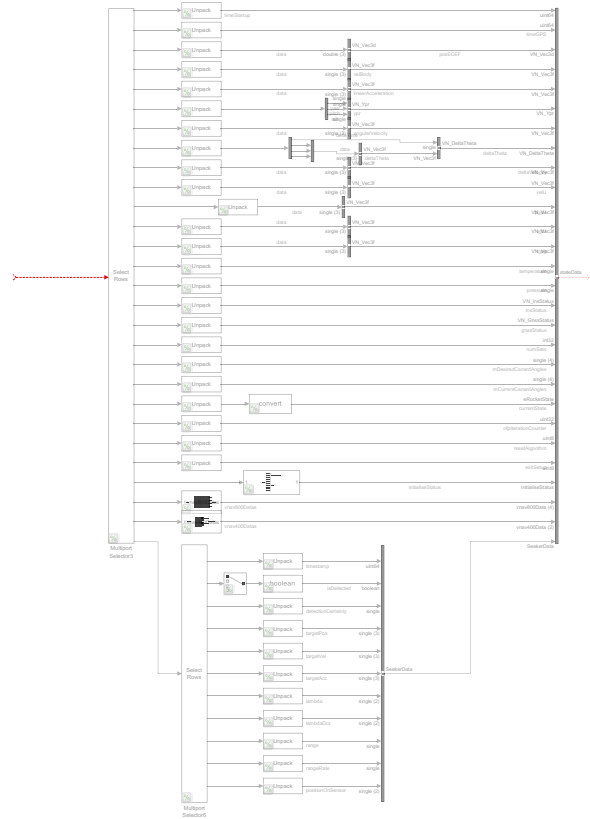
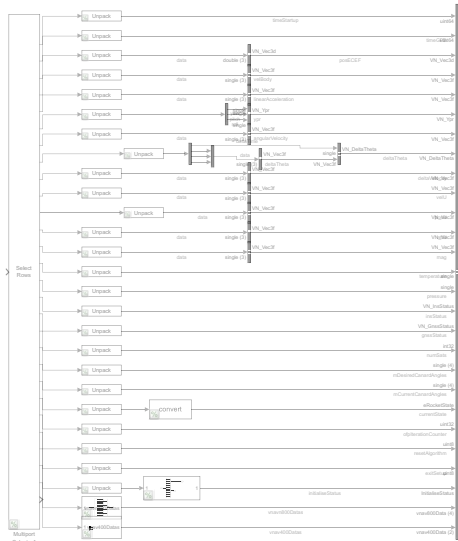
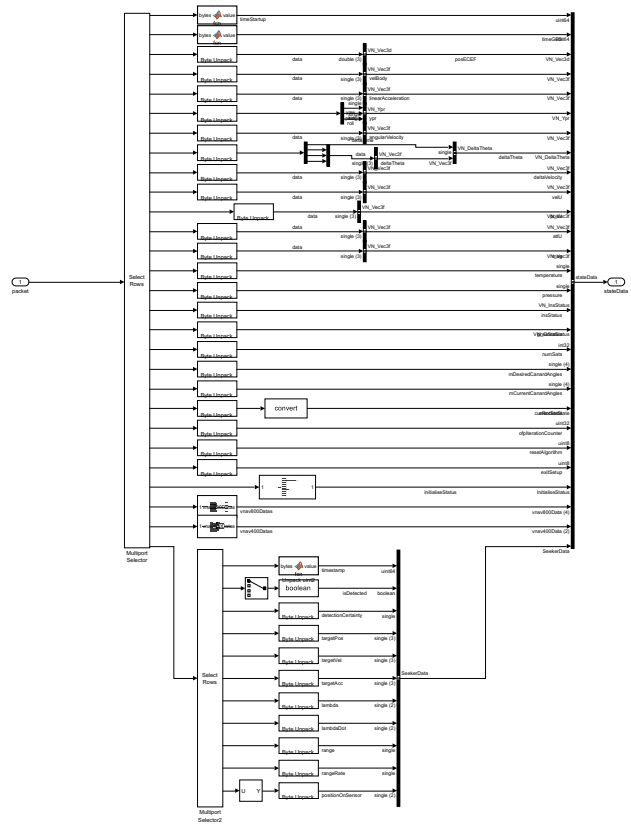


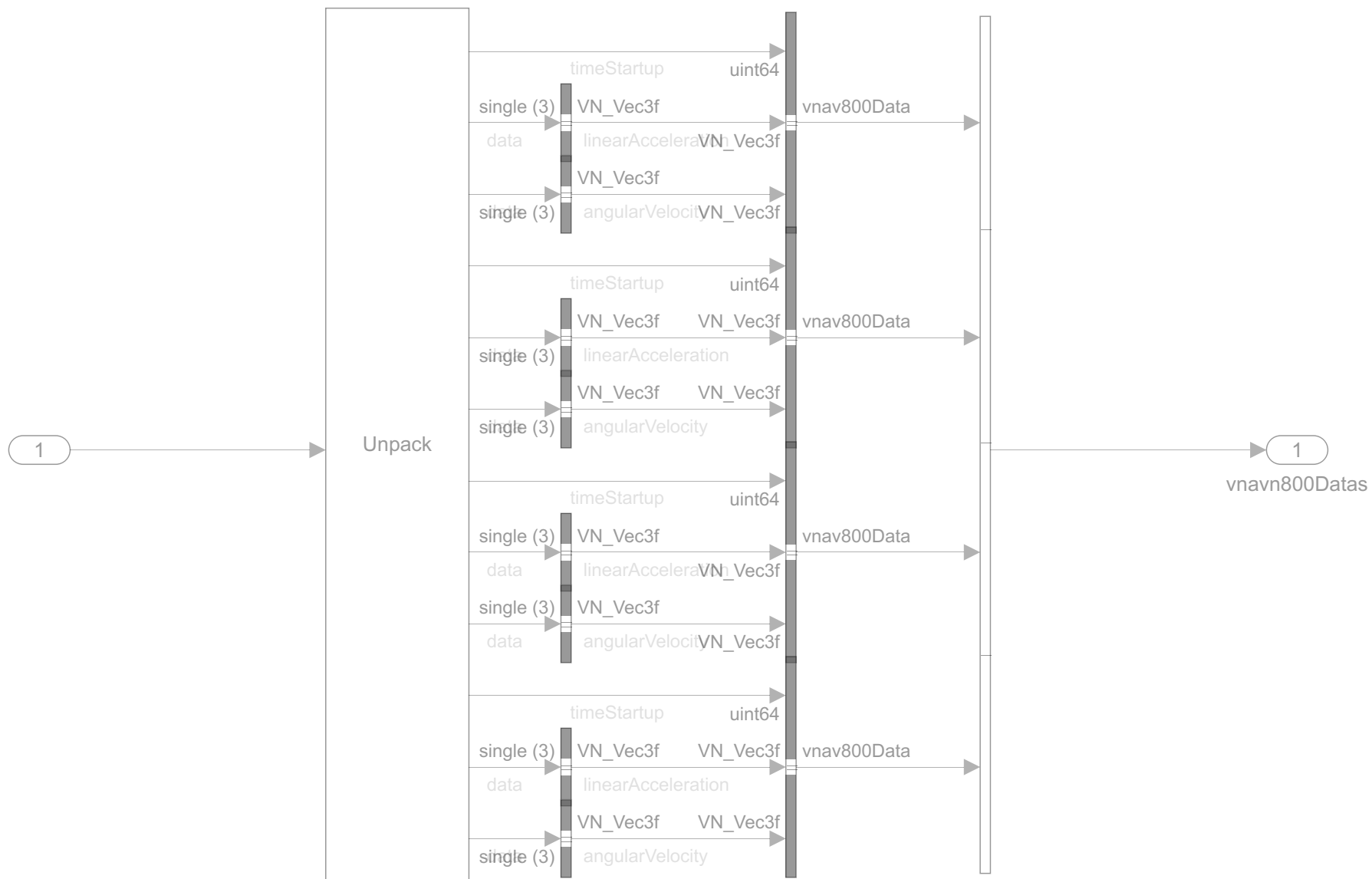


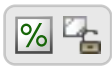
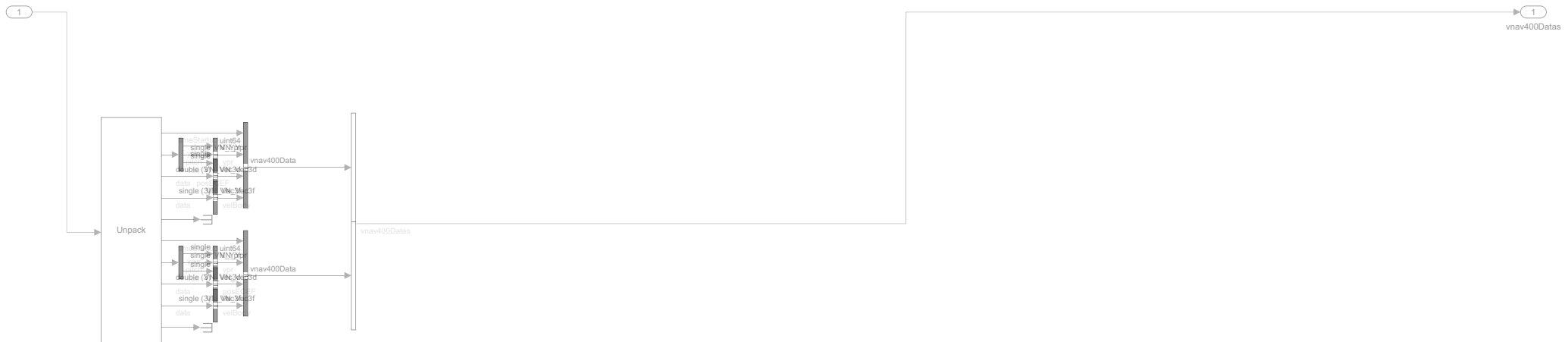


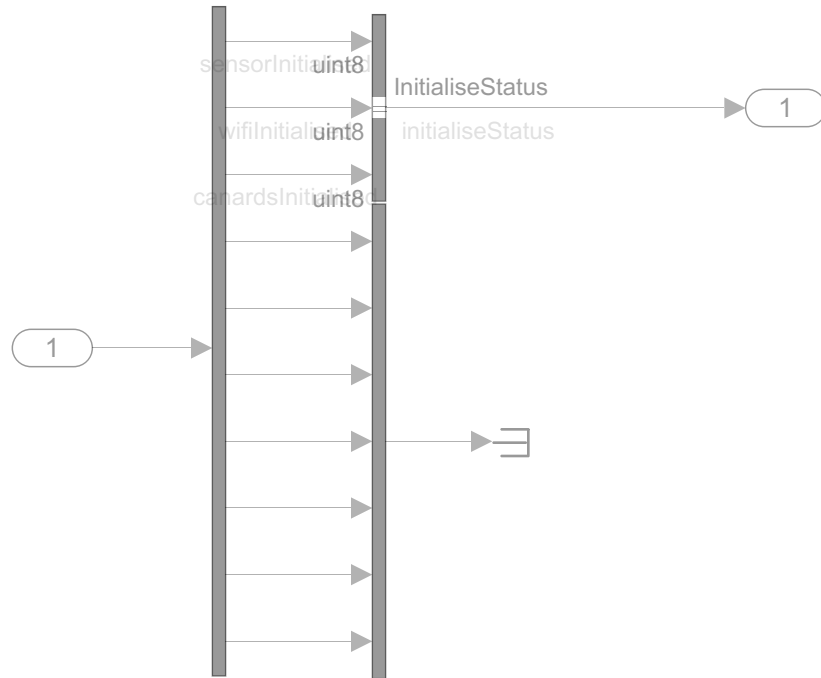


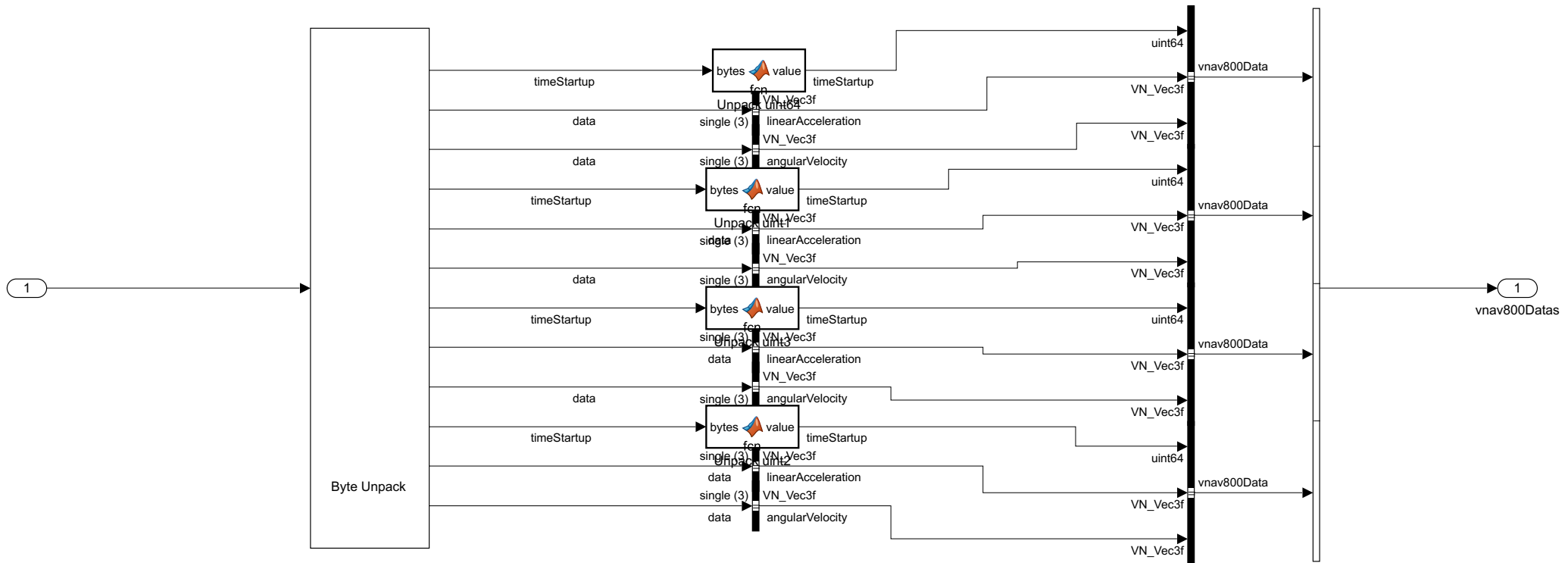










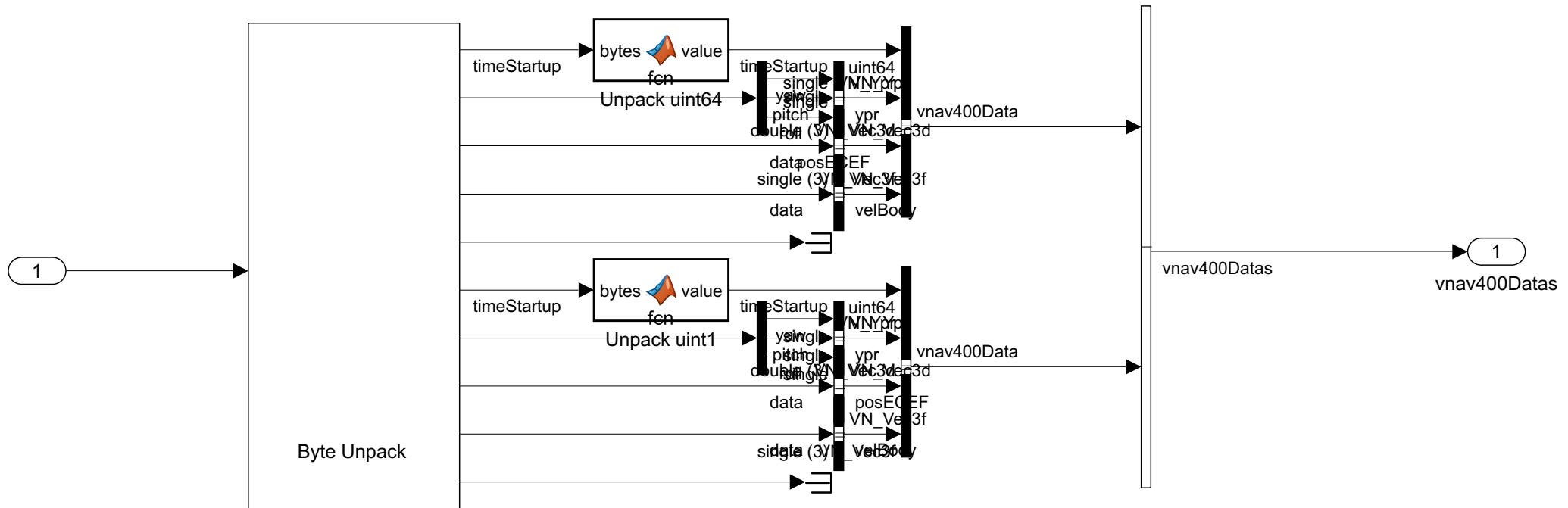


```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUInt64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUint64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUint64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUInt64(bytes);
```

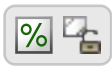
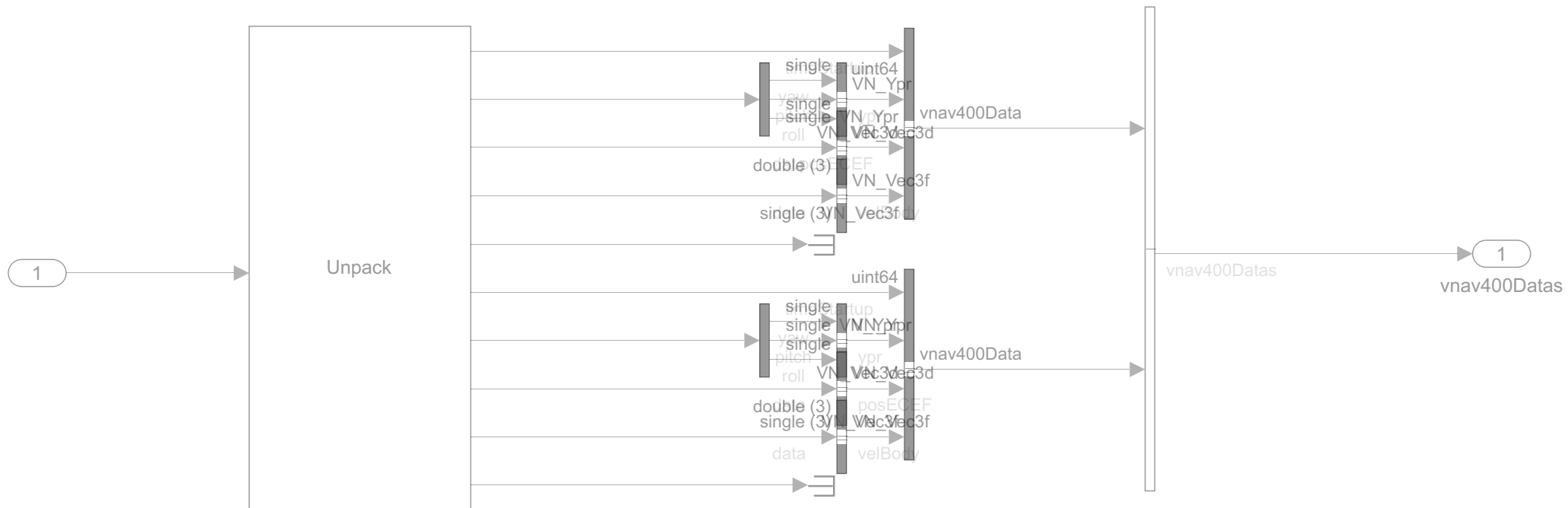


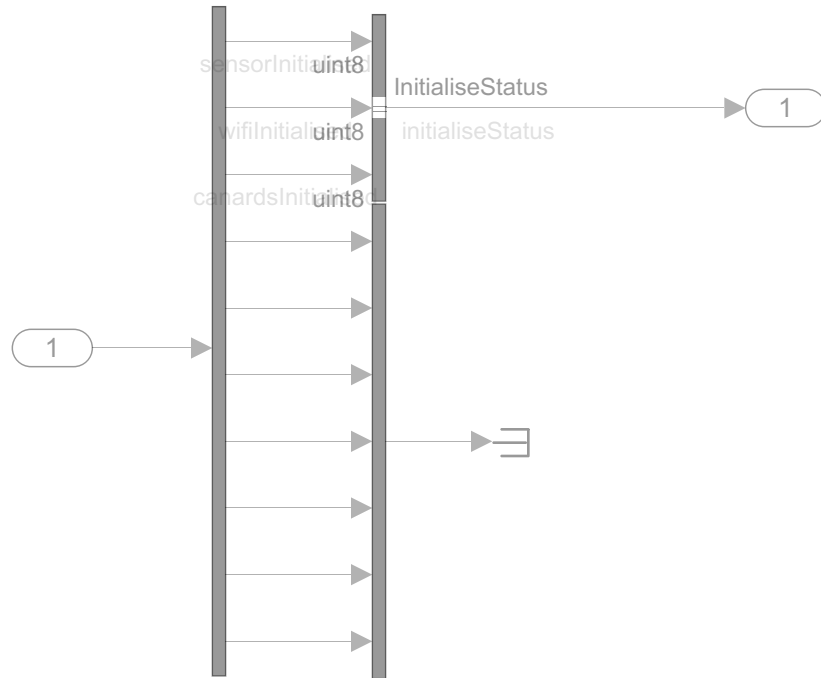
```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUInt64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUint64(bytes);
```









```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUInt64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUInt64(bytes);
```

```
function value = fcn(bytes)
%#codegen
value = uint64(0);
value(1) = unpackUint64(bytes);
```

```
function packet = decode(payload)
%#codegen
packet = decodePacket(payload);
end
```